

# Zero a Monero: Segunda Edição

## Um guia técnico para uma moeda digital privada; para principiantes, amadores e especialistas

## Tradução portuguesa em revisão, 3 de outubro de 2026

koe[1], Kurt M. Alonso[2], Sarang Noether[3]

Tradução de `slave_blocker`[4]

**Licença**: *Zero a Monero: Segunda Edição* é dedicada ao domínio público.

[1] ukoe@protonmail.com
[2] kurt@oktav.se
[3] sarang.noether@protonmail.com
[4] dollner@gmx.de

# Resumo

A criptografia usa construções matemáticas e computacionais para estabelecer propriedades de segurança. Muitos dos conceitos fundamentais necessários neste livro podem ser apresentados a partir de álgebra linear básica e ciência da computação.

Uma cripto-moeda usa criptografia para definir e transferir a posse de fundos. Para impedir que a mesma moeda seja gasta duas vezes ou que novas moedas sejam criadas arbitrariamente, as transacções são registadas numa lista de blocos (*blockchain*): um registo público e distribuído que terceiros podem verificar [95]. A verificabilidade pública não exige que remetentes, destinatários e montantes fiquem expostos: Monero oculta remetentes e destinatários segundo o protocolo CryptoNote [133], e os montantes mediante RingCT [104], permitindo ainda que a rede confirme a validade das transacções.

Para os leitores experientes: a rede principal usa criptografia sobre a curva Edwards25519 [7], endereços de utilização única, assinaturas em anel CLSAG, compromissos de Pedersen e provas de intervalo Bulletproof+. Cada anel tem 16 membros e as saídas incluem etiquetas de vista. RandomX protege a prova de trabalho; Dandelion++ intervém na difusão inicial de transacções, não nas regras de consenso. Este livro apresenta os fundamentos e a notação antes de descrever a sua composição no protocolo.

A segunda edição original descreve o protocolo v12. A rede principal actual de Monero usa o protocolo v16, estado conferido na revisão do código-fonte datada de 14 de Agosto de 2026 [89]. A edição portuguesa foi revista relativamente a esse estado. As construções de épocas anteriores permanecem quando ensinam uma derivação que o protocolo moderno herdou; nesses casos, são identificadas expressamente como históricas. As propostas que nunca foram implementadas são igualmente separadas das funcionalidades disponíveis.

# Índice

**1 Introdução** **1**

1.1 Objectivos . . . . . 2

1.2 Âmbito técnico desta edição . . . . . 2

1.3 Aos leitores . . . . . 3

1.4 Origens da cripto-moeda Monero . . . . . 4

1.4.1 Parte 1: “Essenciais” . . . . . 4

1.4.2 Parte 2: “Extensões” . . . . . 5

1.4.3 Conteúdo adicional . . . . . 5

1.5 Aviso . . . . . 5

1.6 A história de Zero a Monero . . . . . 6

1.7 Reconhecimentos . . . . . 6

**I Essenciais** **7**

**2 Conceitos Básicos** **8**

2.1 Algumas palavras sobre a notação . . . . . 8

2.2 Aritmética modular . . . 9
2.2.1 Adição e multiplicação modular . . . 10
2.2.2 Exponenciação modular . . . 11
2.2.3 Inversa multiplicativa modular . . . 12
2.2.4 Equações modulares . . . 12
2.3 Criptografia de curva elíptica . . . 13
2.3.1 O que são curvas elípticas? . . . 13
2.3.2 Criptografia de chave pública com curvas elípticas . . . 15
2.3.3 A troca de chaves tipo Diffie-Hellman com curvas elípticas . . . 16
2.3.4 Assinaturas tipo Schnorr e a transformada Fiat-Shamir . . . 16
2.3.5 Assinar mensagens . . . 19
2.4 Curva Edwards25519 . . . 21
2.4.1 Representação binária . . . 22
2.4.2 Compressão de ponto elíptico . . . 22
2.4.3 Algoritmo de assinatura EdDSA . . . 23
2.5 Operador binário XOR . . . 25

**3 Divisores: certificar uma multiplicação de pontos 27**

3.1 A pergunta que uma função pode responder . . . 27
3.2 Uma recta, três encontros e uma soma . . . 28
3.3 A função existe sobre a curva . . . 29
3.4 Um zero pode contar mais de uma vez . . . 30
3.5 O infinito entra na contabilidade . . . 31
3.6 O divisor: um registo de lugares e ordens . . . 32
3.7 Multiplicar funções soma os seus divisores . . . 33
3.8 Duas somas que convém nunca confundir . . . 34
3.9 Por que a ponte respeita a lei do grupo . . . 35
3.10 Construir o certificado de uma multiplicação . . . 36

3.11 O certificado completo em módulo dezassete . . . 37
3.12 Os dígitos substituem uma multiplicidade enorme . . . 38
3.13 O tamanho da função também faz parte da prova . . . 39
3.14 Avaliar uma função num divisor . . . 40
3.15 A contribuição local e o resultante elíptico . . . 41
3.16 Uma recta desafiada interroga a testemunha . . . 42
3.17 Dos produtos às somas: a derivada logarítmica . . . 43
3.18 Derivar ao longo da curva . . . 44
3.19 O que as restrições realmente verificam . . . 45
3.20 Da função ao certificado em conhecimento zero . . . 46

**4 Assinaturas tipo Schnorr avançadas** **47**

4.1 Igualdade do logaritmo discreto em várias bases . . . 47
4.2 Uma prova com múltiplas chaves privadas . . . 49
4.3 Assinaturas espontâneas anónimas de grupo . . . 51
4.4 Assinaturas em anel ligáveis de Adam Back . . . 53
4.5 Assinaturas em anel ligáveis com múltiplas camadas . . . 56
4.6 Assinaturas em anel ligáveis concisas . . . 59

**5 Endereços** **64**

5.1 Chaves . . . 64
5.2 Endereços ocultos . . . 66
5.2.1 Transacções de múltiplas saídas . . . 68
5.3 Sub-endereços . . . 69
5.3.1 Enviar para um sub-endereço . . . 70
5.4 Endereços integrados . . . 72
5.5 Endereços de multi-assinatura . . . 73

**6 Montantes ocultos** **74**

6.1 Compromissos . . . 74

6.2 Compromissos de Pedersen . . . 75

6.3 Montantes ocultos . . . 76

6.4 Introdução a RingCT . . . 77

6.5 Provas de intervalo . . . 78

6.6 Assinaturas em anel Borromean . . . 78

6.7 Bulletproofs . . . 79

6.7.1 Verificação . . . 84

6.8 Bulletproof+ . . . 91

6.8.1 O que fica e o que muda . . . 92

6.8.2 Dos tipos históricos ao Bulletproof+ . . . 92

6.8.3 Bulletproof+ na versão 16 . . . 93

6.8.4 Verificação . . . 96

**7 Transacções Confidenciais em Anel (RingCT)** **98**

7.1 Formato activo: `RCTTypeBulletproofPlus` . . . 99

7.1.1 Compromissos a montantes e taxas de transacção . . . 100

7.1.2 Dados de cada saída . . . 101

7.1.3 Uma prova Bulletproofs+ para todas as saídas . . . 101

7.1.4 Um anel: uma entrada real e quinze desvios . . . 102

7.1.5 Assinatura . . . 102

7.1.6 A mensagem que as CLSAG assinam . . . 103

7.1.7 Verificação . . . 104

7.1.8 Evitar o gasto duplo . . . 105

7.1.9 Maturação e tempo de desbloqueio . . . 105

7.1.10 A relação desconhecida $\gamma = \log_G H$ . . . 106

7.2 Resumo conceptual . . . 110

7.2.1 Requisitos de espaço . . . 111

**8 A Blockchain** **113**

8.1 Moedas digitais . . . 114

8.1.1 Versão de eventos comuns e distribuídos . . . 114

8.1.2 Lista de blocos simples . . . 115

8.2 Dificuldade . . . 115

8.2.1 Mineração de um bloco . . . 116

8.2.2 Velocidade de mineração . . . 116

8.2.3 Consenso: maior dificuldade cumulativa . . . 117

8.2.4 Minerar em Monero . . . 118

8.3 Quantidade monetária . . . 120

8.3.1 Recompensa de bloco . . . 120

8.3.2 Peso dinâmico de bloco . . . 122

8.3.3 Recompensa de bloco com multa . . . 123

8.3.4 Taxas dinâmicas: política, não consenso . . . 124

8.3.5 Cauda de emissão . . . 127

8.3.6 Transacção de mineiro: `RCTTypeNull` . . . 127

8.4 A estrutura da lista de blocos . . . 128

8.4.1 ID de transacção . . . 129

8.4.2 Árvore de Merkle . . . 130

8.4.3 Blocos . . . 132

**9 RandomX: função de prova de trabalho** **134**

9.1 Especialização de hardware numa função fixa . . . 135

9.2 Estrutura completa de uma avaliação . . . 135

9.3 Derivação e atraso da chave de época . . . 136

9.4 Cache e Dataset . . . 137

9.5 Modos completo e leve . . . 138

9.6 Compromissos entre memória e tempo . . . 139

9.7 Máquina virtual RandomX . . . 140
9.7.1 Codificação total das instruções . . . 141
9.8 Hierarquia do Scratchpad . . . 141
9.9 Paralelismo ao nível de instruções . . . 142
9.10 Encadeamento de oito programas . . . 143
9.10.1 Custo da filtragem de programas . . . 144
9.10.2 Variância do tempo acumulado . . . 144
9.11 Aritmética de vírgula flutuante . . . 145
9.12 Derivação completa do hash . . . 146
9.13 Condição de dificuldade e probabilidade de êxito . . . 147
9.14 Economia da produção de hashes . . . 148
9.15 Limites da resistência a ASIC . . . 148
9.16 Auditorias, versão e fronteira de consenso . . . 149
9.17 Resumo da avaliação RandomX . . . 150

**10 Dandelion++, propagação de transacções e poda de nós 152**
10.1 Propagação por inundação e exposição temporal . . . 153
10.2 Fases de encaminhamento e difusão . . . 154
10.3 Subgrafo de anonimato por época . . . 155
10.4 O comportamento implementado na rede pública . . . 155
10.4.1 Temporizadores de difusão e embargo . . . 156
10.5 Relação com Tor e I2P . . . 157
10.6 Modelo do adversário e limites . . . 158
10.7 Separação entre anonimato de origem, privacidade e consenso . . . 159
10.8 Poda da blockchain e sincronização de nós podados . . . 159
10.8.1 Dimensão da base podada . . . 160
10.8.2 Faixas, semente e janela recente . . . 160
10.8.3 Dados fornecidos por um nó podado . . . 161
10.8.4 Modos de sincronização . . . 161
10.8.5 Verificar um hash não verifica uma prova ausente . . . 162
10.8.6 Selecção de pares por faixa . . . 163
10.9 Garantias e limites . . . 163

**11 Provas de pertença à cadeia completa (FCMP++) 165**

11.1 Borromean, Bulletproofs e FCMP++ . . . 166

11.2 A declaração, antes da maquinaria . . . 167

11.3 Um compromisso não é uma prova de conhecimento . . . 168

11.4 A metade da árvore: conhecer uma folha e um caminho . . . 169

11.4.1 Uma folha dentro de um ramo . . . 170

11.4.2 Do ramo ao pai, e do pai à raiz . . . 171

11.5 A ponte pública $Q$ . . . 172

11.6 A metade SAL: a chave que autoriza o gasto . . . 172

11.6.1 Como $R$ identifica os candidatos extraídos . . . 173

11.7 O compromisso de montante atravessa a ponte . . . 177

11.8 A composição completa . . . 178

11.9 Construção da árvore pública . . . 179

11.9.1 Da saída aos escalares da primeira camada . . . 179

11.9.2 Subir, ordenar e escolher uma raiz . . . 180

11.9.3 Guardar, reconstruir e actualizar . . . 181

11.9.4 O caminho que a carteira entrega à prova . . . 182

11.10 Só agora: porque é que a árvore alterna duas curvas . . . 183

11.11 Generalized Bulletproofs conserva as referências . . . 184

11.11.1 Da escrita explícita ao produto comprimido . . . 184

11.12 Aplicação dos certificados de divisores . . . 185

11.12.1 Quantas funções racionais são construídas, e onde . . . 186

11.12.2 A rara falha de completude . . . 187

11.13 As quatro equações exactas do verificador SAL . . . 188

11.14 Da exposição ao código fixado . . . 189

11.14.1 A folha codificada, incluindo saídas históricas . . . 189

11.14.2 Achatamento, larguras e sinais . . . 190

11.14.3 Geradores e ordem de verificação . . . 190

11.15 Hipóteses, auditorias e estado de integração . . . 190

11.16 A frase inicial, agora demonstrada . . . 191

**12 Relações de logaritmo discreto entre geradores** **193**

12.1 Relações que envolvem montantes e saídas . . . . . 194

12.2 Consequências de relações entre as bases da SAL . . . . . 195

12.3 Análise das dez relações por pares . . . . . 196

12.4 Consequências para as provas de intervalo . . . . . 197

12.5 Consequências para Curve Trees, Selene e Helios . . . . . 198

12.6 Matriz de consequências . . . . . 199

## II Extensões 200

**13 Provas sobre transacções** **201**

13.1 Interface e alcance . . . . . 201

13.1.1 A prova V2 com duas bases . . . . . 203

13.2 Revelar a chave privada de transacção . . . . . 203

13.3 Prova de saída: `OutProofV2` . . . . . 203

13.4 Prova de entrada: `InProofV2` . . . . . 204

13.4.1 O estado gasto e a actividade de saída . . . . . 205

13.4.2 Âmbitos de vista previstos no Carrot . . . . . 205

13.5 Prova de gasto: `SpendProofV1` . . . . . 206

13.6 Prova de reserva: `ReserveProofV2` . . . . . 206

13.7 Afirmações de pagamento e identidade . . . . . 207

13.8 Construções não implementadas . . . . . 208

13.8.1 Provar que uma saída não corresponde a uma imagem de chave . . . . . 208

13.8.2 Provar a relação entre um endereço e um sub-endereço . . . . . 209

13.8.3 Limite de uma auditoria completa . . . . . 209

**14 Multi-assinaturas** **210**

14.1 Estado da implementação . . . 211

14.2 Modelo e objectivos de segurança . . . 211

14.3 Comunicação entre co-signatários . . . 212

14.4 Agregação das chaves de gasto . . . 213

14.4.1 A soma ingénua . . . 213

14.4.2 Coeficientes de agregação . . . 213

14.5 Construção de uma política M-de-N . . . 214

14.5.1 Chaves-base e chave de vista . . . 214

14.5.2 Segredos partilhados por subconjuntos . . . 215

14.5.3 Rondas e confirmação do endereço . . . 215

14.5.4 Exemplo 2-de-3 . . . 216

14.6 Recepção de saídas e imagens de chave . . . 216

14.7 Assinaturas de Schnorr com limiar . . . 217

14.7.1 Dois valores aleatórios por signatário . . . 218

14.8 Assinatura CLSAG distribuída . . . 218

14.9 Construção e assinatura da transacção . . . 219

14.9.1 Sincronização da carteira . . . 219

14.9.2 Proposta . . . 220

14.9.3 Respostas e submissão . . . 220

14.9.4 Comunicação simplificada . . . 220

14.10Sequência mínima de comandos . . . 221

14.11Políticas compostas não implementadas . . . 222

14.11.1 Agregação conceptual por conjuntos de chaves . . . 223

14.12Nota sobre multi-assinatura com FCMP++ . . . 224

14.12.1 As duas primitivas no ramo de referência . . . 224

14.13Resumo . . . 226

**15 Mercados garantidos por terceiros** **227**

15.1 Estado da implementação . . . 227

15.2 Modelo formal . . . 228

15.2.1 Fundos e resultados possíveis . . . 229

15.2.2 Identidade de uma ordem . . . 230

15.3 Configuração da carteira . . . 230

15.4 Financiamento . . . 231

15.5 Registo autenticado das mensagens . . . 231

15.6 Liquidação sem disputa . . . 232

15.7 Disputa e decisão . . . 233

15.7.1 Resultados concorrentes . . . 234

15.8 Prazos e disponibilidade . . . 234

15.9 Privacidade . . . 234

15.10 Relação com FCMP++ . . . 235

15.11 Requisitos mínimos de uma aplicação . . . 235

15.12 Resumo . . . 236

**16 Transacções conjuntas (`TxTangle`)** **237**

16.1 O núcleo algébrico . . . 238

16.2 Construção descentralizada histórica . . . 238

16.2.1 Canal de comunicação em grupo . . . 238

16.2.2 Cinco rondas de mensagens . . . 239

16.2.3 Chaves públicas de transacção e ataque de Janus . . . 240

16.2.4 Fraquezas . . . 240

16.3 TxTangle servido . . . 241

16.3.1 Comunicação através de I2P . . . 241

16.3.2 Operação como serviço . . . 242

16.4 Construção com administrador de confiança . . . 242

16.5 Avaliação face ao Monero actual . . . 243

**Bibliografia** 245

**Appendices** 254

**A Exemplo histórico: estrutura de transacção `RCTTypeBulletproof2`** 256

**B Conteúdo de bloco** 262

**C Bloco de génese** 265

**D Notação vectorial das Bulletproofs** 268

# CAPÍTULO 1

# Introdução

No mundo digital, copiar informação é trivial; alterá-la também. Para que uma moeda digital seja aceite, os seus utilizadores precisam de confiar em duas coisas: a quantidade total de moeda obedece às regras anunciadas e cada unidade só pode ser gasta uma vez. Quem recebe um pagamento tem, portanto, de poder verificar que as moedas não são contrafeitas nem foram já entregues a outra pessoa. Se não quisermos confiar essa tarefa a uma autoridade central, a oferta monetária e a história completa das transacções têm de ser verificáveis pelo público.

As cripto-moedas registam transacções numa lista de blocos (*blockchain*), uma base de dados distribuída cuja história é muito difícil de alterar sem que a rede o detecte [95]. Muitas listas de blocos expõem os intervenientes e os montantes para tornar simples a verificação colectiva. O preço é evidente: perde-se privacidade e, com ela, fungibilidade.[1]

Um utilizador de Bitcoin pode tentar ofuscar o percurso dos fundos com endereços intermédios [96]. Ainda assim, ferramentas de análise conseguem muitas vezes estabelecer ligações entre remetentes e destinatários [124, 14, 107, 24].

Monero adopta outra estratégia. Endereços de utilização única escondem o destinatário na lista de blocos; assinaturas em anel tornam ambígua a saída que foi efectivamente gasta; e compromissos criptográficos ocultam os montantes sem impedir a verificação da contabilidade.

[1] Um bem é **fungível** quando uma unidade pode substituir outra no uso ou no cumprimento de um contracto [76]. Numa lista de blocos transparente como a de Bitcoin, as moedas podem ser distinguidas pela sua história. Se essa história incluir actores considerados suspeitos, certas moedas podem ser tratadas como "manchadas" [74]; inversamente, houve quem pagasse um prémio por moedas recém-mineradas, sem história anterior [116].

O resultado é uma cripto-moeda com um elevado grau de privacidade e fungibilidade. Estas propriedades não eliminam a informação produzida pelo comportamento do utilizador ou disponível fora da lista de blocos, que pode permitir alguma análise.[2]

## 1.1 Objectivos

Monero foi lançado em 2014 e tem uma história continuada de desenvolvimento [127, 82]. Quando a segunda edição deste livro foi preparada, a comunidade e a adopção da moeda continuavam a aumentar [34].[3]

Documentar o sistema é difícil. Existe documentação prática no projecto `https://monerodocs.org/` e em *Mastering Monero* [123], mas partes importantes da construção teórica foram publicadas em textos incompletos, não revistos por pares ou entretanto ultrapassados. Para saber o que uma versão do programa faz, o código-fonte é a referência decisiva; para saber por que uma construção criptográfica é segura, continuam a ser indispensáveis os artigos e as provas que a fundamentam. [4]

Para leitores sem formação matemática, os fundamentos da criptografia de curva elíptica podem exigir conceitos preliminares que outras descrições pressupõem. Uma explicação anterior do funcionamento de Monero [105], por exemplo, não desenvolvia esses fundamentos, era incompleta e ficou obsoleta. Este livro introduz os conceitos necessários, revê os algoritmos criptográficos e reúne uma descrição construtiva, passo a passo, do funcionamento interno de Monero.

## 1.2 Âmbito técnico desta edição

A segunda edição original descreve o protocolo v12 e o programa Monero v0.15.x.x. A revisão portuguesa toma esse texto como ponto de partida, mas confere o estado corrente contra a rede principal e a revisão do código identificada no resumo inicial.[5]

Esta distinção evita três confusões frequentes:

- uma *regra de consenso activa* determina que blocos e transacções a rede principal aceita;

[2] Para um exemplo de heurísticas aplicadas às assinaturas em anel, veja-se [40].

[3] Naquele período, Monero ocupava o 14.º lugar por capitalização de mercado em 14 de Junho de 2018 e o 12.º em 5 de Janeiro de 2020; veja-se `https://coinmarketcap.com/`. Estes valores não são uma medida técnica nem uma afirmação sobre a posição actual.

[4] A série *Monero Building Blocks*, de Seguias [121], trata com cuidado as provas de segurança dos esquemas de assinatura. Tal como a primeira edição de *Zero a Monero* [2], concentra-se no protocolo v7.

[5] Foram conferidos, entre outros, `README.md`, `src/hardforks/hardforks.cpp`, `src/cryptonote_core/blockchain.cpp`, `src/cryptonote_protocol/levin_notify.cpp` e o directório `src/fcmp_pp/`. O repositório completo está em `https://github.com/monero-project/monero`.

- um *comportamento implementado* pode pertencer à carteira ou à rede entre nós sem fazer parte do consenso;
- *trabalho experimental* pode existir no código antes de estar completo, estável ou sequer programado para activação.

Na v16, as novas transacções usam CLSAG e Bulletproof+, anéis fixos de 16 membros e etiquetas de vista. RandomX é a prova de trabalho activa desde a v12. Dandelion++ é comportamento implementado para a difusão de transacções na rede pública, não uma regra de consenso. O ramo consultado contém componentes de FCMP++ e tipos associados a Carrot, mas não programa uma versão posterior à v16 na rede principal; são, por isso, trabalho experimental nesta edição.

O "protocolo" é o conjunto de regras contra o qual cada novo bloco é testado antes de entrar na lista. Inclui as regras das transacções, mas não se confunde com o campo de versão da própria transacção. Este último continua a identificar RingCT v2, embora várias regras concretas e o formato RingCT tenham mudado de facto ao longo das versões de consenso.

A actualização desta edição abrange todos os capítulos. As descrições do protocolo activo referem-se à v16; os formatos e esquemas de épocas anteriores são identificados expressamente como históricos quando ajudam a interpretar dados antigos ou a acompanhar uma derivação. A primeira edição, por sua vez, corresponde ao protocolo v7 e ao programa v0.12.x.x [2].

O código beneficia de revisão pública, mas isso não o torna infalível. O leitor é convidado a conferir as explicações nas fontes. Uma vulnerabilidade séria deve ser comunicada de forma responsável em `https://hackerone.com/monero`; uma correcção não sensível pode ser proposta no repositório do projecto. As propostas Triptych [102], RingCT 3.0 [144], Omniring [64] e Lelantus [53] pertencem ao contexto de investigação da segunda edição, não ao conjunto de regras activas aqui descrito.

## 1.3 Aos leitores

Prevemos que muitos leitores cheguem a este relatório com pouco ou nenhum contacto com matemática discreta, estruturas algébricas, criptografia ou listas de blocos. Procurámos dar pormenor suficiente para que um leitor atento possa aprender sem depender constantemente de pesquisa externa.[6]

Alguns pormenores matemáticos foram omitidos ou remetidos para notas de rodapé quando prejudicariam a clareza. Também não seguimos cada detalhe de implementação quando ele não é necessário para compreender o mecanismo. O objectivo é combinar criptografia matemática e implementação: apresentar as hipóteses formalmente e mostrar de forma concreta como os componentes se relacionam. [7]

[6] Uma introdução extensa à criptografia aplicada encontra-se em [15].

[7] Certas notas de rodapé antecipam capítulos posteriores. Foram pensadas para ganhar utilidade numa segunda leitura, quando os detalhes de implementação já têm onde assentar.

## 1.4 Origens da cripto-moeda Monero

Monero, inicialmente chamado BitMonero, foi criado em Abril de 2014 a partir da prova de conceito CryptoNote [127]. *Monero* significa "moeda" em esperanto; o plural é *moneroj*.

CryptoNote foi desenvolvido por várias pessoas. O texto de referência que o descreve foi publicado sob o pseudónimo Nicolas van Saberhagen, em Outubro de 2013 [133]. Propunha endereços de utilização única para ocultar o destinatário e assinaturas em anel para tornar ambíguo o remetente. Desde então, Monero reforçou a privacidade com montantes confidenciais, inspirados, entre outros, no trabalho de Greg Maxwell [69] e integrados em RingCT segundo as recomendações de Shen Noether [104]. As provas Bulletproof tornaram depois essa ocultação muito mais eficiente [18]; CLSAG reduziu o custo das assinaturas em anel [43], e Bulletproof+ sucedeu às provas Bulletproof nas transacções activas.

### 1.4.1 Parte 1: "Essenciais"

Começamos pelas ferramentas criptográficas necessárias. O capítulo 2 desenvolve aritmética modular, curvas elípticas, chaves, assinaturas de Schnorr e a notação usada no resto do livro. O capítulo 4 prolonga essa construção até às assinaturas em anel. No capítulo 5, vemos como os endereços controlam a recepção de fundos e por que uma saída não revela publicamente o destinatário.

O capítulo 6 introduz compromissos e provas de intervalo para ocultar montantes. Em conjunto, o capítulo 7 constrói uma transacção RingCT. O capítulo 8 abre por fim a lista de blocos: mineração, dificuldade, emissão, peso e taxas. O capítulo 9 segue uma tentativa de RandomX desde a semente de época até ao hash de prova de trabalho e explica por que programas aleatórios e memória abundante procuram reduzir a vantagem de máquinas especializadas. O capítulo 10 acompanha depois a vida de um nó: Dandelion++ afasta a origem de uma transacção do ponto onde começa a difusão pública, enquanto a poda e a sincronização podada distribuem pela rede a conservação da história sem as confundir com anonimato. O capítulo 11 analisa a alteração FCMP++ ainda em desenvolvimento: parte das provas de Schnorr e das árvores de curvas para explicar como uma entrada poderá pertencer privadamente ao conjunto de saídas da cadeia inteira. Como fecho dos "Essenciais", o capítulo 12 pergunta o que aconteceria se alguém descobrisse as relações supostamente desconhecidas entre os geradores: quando se perde só privacidade, quando se torna possível o gasto duplo e quando aparece inflação. Esta ordem fornece a base criptográfica para a parte seguinte; não transforma a proposta FCMP++ em regra activa da versão 16.

### 1.4.2 Parte 2: "Extensões"

Uma cripto-moeda é mais do que as suas regras de consenso. Esta parte explora aplicações e propostas, várias das quais não estão implementadas ou dependem de funcionalidades experimentais. Uma futura alteração do formato das transacções pode torná-las impraticáveis; cada capítulo deve, portanto, ser lido com o seu estatuto técnico em mente.

O capítulo 13 mostra que informação sobre uma transacção pode ser demonstrada a um observador. O capítulo 14 descreve a abordagem de multi-assinaturas da carteira; na implementação consultada, esta funcionalidade continua marcada como experimental e desactivada por omissão. O capítulo 15 usa multi-assinaturas para desenhar mercados com garantia. O capítulo 16 apresenta TxTangle, uma proposta original e descentralizada para reunir as transacções de várias pessoas numa só; o capítulo conserva a sua álgebra, mas deixa claro que o protocolo não foi implementado no Monero.

### 1.4.3 Conteúdo adicional

O apêndice A disseca uma transacção `RCTTypeBulletproof2`; é um exemplo histórico, não o formato que a v16 aceita para novas transacções. O apêndice B explica a estrutura de um bloco, incluindo o cabeçalho e a transacção de mineiro. O apêndice C fecha o percurso com o bloco génese de Monero. O apêndice D fixa a notação vectorial e deriva o termo $\delta(y, z)$ das Bulletproofs sem saltos algébricos. Em conjunto, ligam a teoria a objectos concretos da lista de blocos e às identidades que os verificam.

As notas de margem apontam para locais relevantes no código-fonte. Os caminhos modernos foram conferidos na revisão indicada no resumo; quando se conserva um caminho da v0.15.x.x para acompanhar uma construção histórica, o contexto assinala essa época. A história completa continua disponível em Git. Uma nota contém normalmente um caminho, como `src/ringct/rctOps.cpp`, e por vezes uma função, como `ecdhEncode()`. Um hífen num caminho estreito pode ser apenas uma quebra tipográfica; os qualificadores de *namespace*, como `Blockchain::`, são geralmente omitidos.

isto é útil!

## 1.5 Aviso

Os esquemas de assinatura, as aplicações de curvas elípticas e os detalhes da implementação aqui apresentados são descritivos. Quem pretenda construir um sistema real deve consultar as fontes primárias, as especificações técnicas e o código correspondente à versão que vai usar.

Um esquema de assinatura precisa de uma prova de segurança bem definida e de revisão competente. Vale aqui a velha regra: não invente a sua própria criptografia. Código que implemente

primitivas criptográficas deve ser revisto por especialistas e acumular um historial longo e fiável. As contribuições originais deste documento podem não ter recebido a mesma revisão nem testes; leia-as com o cuidado devido.

## 1.6 A história de Zero a Monero

*Zero a Monero* começou como expansão da tese de mestrado de Kurt Alonso, "Monero — Privacy in the Blockchain" [1], publicada em Maio de 2018. A primeira edição do livro apareceu em Junho desse ano [2].

Para a segunda edição, os autores melhoraram a introdução às assinaturas em anel (capítulo 4) e reorganizaram a explicação das transacções, acrescentando o capítulo de endereços (5). Modernizaram a comunicação dos montantes de saída (secção 6.3), substituíram as provas Borromean por Bulletproof no percurso corrente da época (secção 6.5), retiraram `RCTTypeFull` do formato corrente e actualizaram o peso dinâmico dos blocos e as taxas dos mineiros.

Foram ainda estudadas provas sobre transacções (capítulo 13), multi-assinaturas (capítulo 14), mercados com garantia (capítulo 15) e a proposta TxTangle (capítulo 16). Essa edição ficou alinhada com o protocolo v12 e o programa v0.15.x.x.[8]

## 1.7 Reconhecimentos

Texto original do autor "koe".

Este relatório não existiria sem a tese de mestrado de Kurt Alonso [1], a quem o autor deve profunda gratidão. Brandon "Surae Noether" Goodell e o investigador conhecido pelo pseudónimo "Sarang Noether", do Monero Research Lab, foram fontes constantes de conhecimento durante as duas edições. "moneromooo", um dos programadores mais prolíficos do projecto, identificou inúmeras vezes as partes relevantes do código. Agradecemos também a todos os contribuidores que responderam a perguntas e aos leitores que enviaram correcções e palavras de encorajamento.

[8] O código-fonte LaTeX das duas edições está em `https://github.com/UkoeHB/Monero-RCT-report`; a primeira edição encontra-se no ramo `ztm1`.

# Parte I

# Essenciais

# CAPÍTULO 2

# Conceitos Básicos

## 2.1 Algumas palavras sobre a notação

Um dos principais objectivos deste texto é reunir, corrigir e harmonizar a informação necessária para compreender os mecanismos internos de Monero. Ao mesmo tempo, queremos apresentar a matéria de forma construtiva: cada símbolo deve conservar o seu papel enquanto acrescentamos uma peça de cada vez.

Salvo indicação em contrário, adoptamos as seguintes convenções:

- letras minúsculas para inteiros, escalares, sequências de bits e valores simples;
- letras maiúsculas para pontos de curva elíptica e objectos compostos.

Tentamos também reservar os mesmos símbolos para os mesmos objectos. Um gerador de curva será normalmente $G$, a ordem do seu subgrupo será $l$, e um par de chaves privada e pública será escrito $k/K$, respectivamente.

A apresentação é deliberadamente *conceptual*. Um leitor de ciência da computação poderá sentir falta da representação exacta de certos objectos ou dos pormenores de uma implementação; um estudante de matemática poderá desejar mais álgebra abstracta. Daremos esses pormenores quando forem necessários ao raciocínio, sem os deixar esconder a construção principal.

Um objecto simples, como um inteiro ou uma sequência de caracteres, pode ser codificado como uma sequência de bits. A ordem dos bytes, ou *endianness*, é essencial para uma implementação

interoperável, mas raramente altera as ideias estudadas neste capítulo; quando importar, será indicada.

Um ponto de curva elíptica escreve-se habitualmente como um par $(x, y)$, mas isso não obriga a transmiti-lo como dois inteiros completos. Na secção 2.4.2 veremos como uma coordenada e um bit bastam para codificar os pontos que nos interessam. Sempre que contarmos bytes, assumiremos essa compressão.

As funções hash serão por vezes usadas sem declarar um algoritmo específico. Monero emprega uma variante de *Keccak*, cuja construção interna não é necessária para a teoria deste capítulo[1]. src/crypto/keccak.c

Chamaremos simplesmente *função hash* a uma função de dispersão criptográfica. Ela recebe uma mensagem $\mathfrak{m}$ de comprimento variável e devolve uma sequência $h$ de comprimento fixo. É determinística: a mesma entrada dá sempre a mesma saída. Pretende-se, contudo, que as saídas se pareçam com escolhas uniformes e imprevisíveis, que seja computacionalmente difícil recuperar uma entrada a partir da sua hash e que seja difícil encontrar duas entradas distintas com a mesma saída. O chamado *efeito avalanche* faz com que uma pequena alteração da entrada possa mudar muitos bits da saída.

Aplicaremos funções hash a inteiros, sequências de caracteres, pontos de curva e combinações desses objectos. Isso significa que primeiro se usa uma codificação binária inequívoca e, quando há vários objectos, uma concatenação inequívoca. Uma hash é originalmente uma sequência de bits; para a usar como escalar ou ponto de curva é ainda necessária uma conversão apropriada, que indicaremos quando for relevante.

## 2.2 Aritmética modular

A maior parte da criptografia moderna começa com aritmética modular. Fixemos um módulo $n > 1$. Para qualquer inteiro $a$, a operação

$$c = a \bmod n$$

devolve o único resto $c$ que satisfaz

$$a = qn + c, \qquad 0 \le c < n,$$

para algum inteiro $q$. O símbolo mod designa aqui uma operação. Quando queremos afirmar que dois inteiros deixam o mesmo resto, escrevemos antes a congruência $a \equiv c \pmod{n}$.

Os restos nunca são negativos. Em particular, se $0 < a < n$, então $(-a) \bmod n = n - a$. Nos casos de fronteira convém não abreviar: se $a \bmod n = 0$, também $(-a) \bmod n = 0$. A fórmula que cobre todos os casos é

$$(-a) \bmod n = \big(n - (a \bmod n)\big) \bmod n.$$

[1] O algoritmo Keccak serve de base ao standard NIST *SHA-3* [97].

### 2.2.1 Adição e multiplicação modular

Ao implementar estas operações, reduzir resultados intermédios evita inteiros desnecessariamente grandes. Suponhamos, por exemplo, que queremos calcular $(29 + 87) \bmod 99$, mas que não desejamos formar primeiro 116. Para restos $0 \leq a, b < n$, pomos $x = n - a$:

- se $b < x$, então $(a + b) \bmod n = a + b$;
- se $b \geq x$, então $(a + b) \bmod n = b - x$.

No exemplo, $x = 70$ e obtemos directamente $87 - 70 = 17$. A igualdade no segundo caso é importante: quando $a + b = n$, o resto é zero.

Podemos repetir esta adição segura para calcular uma multiplicação modular. O algoritmo chama-se *duplica-e-adiciona*. Por exemplo,

$$(7 \cdot 8) \bmod 9 = (8 + 8 + 8 + 8 + 8 + 8 + 8) \bmod 9 = 2.$$

Agrupando parcelas iguais,

$$(8 + 8) + (8 + 8) + (8 + 8) + 8$$

e depois

$$[(8 + 8) + (8 + 8)] + (8 + 8) + 8,$$

podemos reutilizar cada duplicação em vez de repetir todo o trabalho. [2]

Para calcular $c = (a \cdot b) \bmod n$, escrevemos o multiplicador $a$ em binário e percorremos os seus bits do menos para o mais significativo.

No nosso exemplo, seja $A = [0111]$, com os índices $3, 2, 1, 0$. [3] Assim, $A[0] = 1$ é o bit menos significativo. Inicializamos o acumulador com $r = 0$ e a parcela corrente com $s = b \bmod n$. Depois:

1. Para $i = 0, \ldots, |A| - 1$:
   (a) se $A[i] = 1$, faça $r = (r + s) \bmod n$;
   (b) faça $s = (s + s) \bmod n$.
2. O resultado é $c = r$.

Para $(7 \cdot 8) \bmod 9$, a sequência é:

1. $i = 0$

[2] A vantagem torna-se clara para multiplicadores grandes: a adição repetida exige um número de operações proporcional ao multiplicador, enquanto duplica-e-adiciona exige apenas um número proporcional ao seu comprimento em bits.

[3] Esta numeração é conhecida como "LSB 0", do inglês *least significant bit*: o bit menos significativo tem o índice zero. Usá-la-emos no resto do capítulo quando tornar a ordem dos bits mais clara.

   (a) $A[0] = 1$, portanto $r = (0 + 8) \bmod 9 = 8$;

   (b) $s = (8 + 8) \bmod 9 = 7$.

2. $i = 1$

   (a) $A[1] = 1$, portanto $r = (8 + 7) \bmod 9 = 6$;

   (b) $s = (7 + 7) \bmod 9 = 5$.

3. $i = 2$

   (a) $A[2] = 1$, portanto $r = (6 + 5) \bmod 9 = 2$;

   (b) $s = (5 + 5) \bmod 9 = 1$.

4. $i = 3$

   (a) $A[3] = 0$, portanto $r$ não muda;

   (b) $s = (1 + 1) \bmod 9 = 2$.

5. O resultado é $r = 2$.

### 2.2.2 Exponenciação modular

Tal como a multiplicação é adição repetida, a exponenciação é multiplicação repetida. Por exemplo,

$$8^7 \bmod 9 = (8 \cdot 8 \cdot 8 \cdot 8 \cdot 8 \cdot 8 \cdot 8) \bmod 9.$$

O análogo de duplica-e-adiciona chama-se *eleva-ao-quadrado-e-multiplica*. Para calcular $a^e \bmod n$, com $e \geq 0$, escrevemos $e$ em binário no vector $A$, do mesmo modo que acima, e inicializamos $r = 1 \bmod n$ e $m = a \bmod n$:

1. Para $i = 0, \ldots, |A| - 1$:

   (a) se $A[i] = 1$, faça $r = (r \cdot m) \bmod n$;

   (b) faça $m = (m \cdot m) \bmod n$.

2. O resultado é o valor final de $r$.

As condições iniciais tratam também o expoente zero: nenhum bit provoca uma multiplicação do acumulador e o resultado é $1 \bmod n$.

### 2.2.3 Inversa multiplicativa modular

Por vezes precisamos de dividir por $a$ em aritmética modular. Isso significa procurar uma inversa multiplicativa $a^{-1}$, isto é, um resto que, ao ser multiplicado por $a$, dê a identidade 1. Ao contrário do que sucede com números reais, nem todo o resto diferente de zero tem inversa para qualquer módulo.

Existe uma inversa módulo $n$ se, e somente se, $a$ e $n$ são relativamente primos, ou seja, se $\gcd(a, n) = 1$.[4] Nesse caso há um único resto $c$ tal que

$$\begin{aligned} c &= a^{-1} \bmod n, & 0 \le c < n, \\ ac &\equiv 1 \pmod{n}. \end{aligned}$$

Dois inteiros relativamente primos não têm qualquer divisor comum além de 1; por exemplo, a fracção $a/n$ já estaria na forma irredutível.

Quando $n$ é primo e $a \not\equiv 0 \pmod{n}$, podemos calcular a inversa com eleva-ao-quadrado-e-multiplica graças ao *pequeno teorema de Fermat*: [5]

$$\begin{aligned} a^{n-1} &\equiv 1 \pmod{n} \\ a \cdot a^{n-2} &\equiv 1 \pmod{n} \\ a^{n-2} &\equiv a^{-1} \pmod{n}. \end{aligned}$$

Para um módulo composto, e em geral de forma mais directa, o algoritmo estendido de Euclides [140] encontra a inversa sempre que $\gcd(a, n) = 1$; se o máximo divisor comum não for 1, confirma que a inversa não existe.

### 2.2.4 Equações modulares

As reduções modulares podem ser feitas antes ou depois de adições, subtracções e multiplicações. Mais precisamente, se $\circ \in \{+, -, \cdot\}$, então

$$(A \circ B) \bmod n = \big[(A \bmod n) \circ (B \bmod n)\big] \bmod n.$$

Por exemplo, para $c = (3 \cdot 4 \cdot 5) \bmod 9$, tomamos $A = 3 \cdot 4$, $B = 5$ e $n = 9$:

$$\begin{aligned} (3 \cdot 4 \cdot 5) \bmod 9 &= \big[(3 \cdot 4) \bmod 9\big] \cdot (5 \bmod 9) \bmod 9 \\ &= (3 \cdot 5) \bmod 9 \\ &= 6. \end{aligned}$$

Em particular, a subtracção pode ser tratada como a soma de um valor negativo:

$$\begin{aligned} (A - B) \bmod n &= (A + (-B)) \bmod n \\ &= \big[(A \bmod n) + ((-B) \bmod n)\big] \bmod n. \end{aligned}$$

[4] Na expressão $a \equiv b \pmod{n}$, dizemos que $a$ é *congruente* com $b$ módulo $n$. Isso equivale a $a \bmod n = b \bmod n$, ou a afirmar que $n$ divide $a - b$.

[5] Se $ac \equiv b \pmod{n}$ e $\gcd(a, n) = 1$, a congruência linear tem a solução única $c \equiv a^{-1}b \pmod{n}$. Só podemos voltar a multiplicar por $c^{-1}$ se também $c$ for invertível. Veja-se [141].

Uma potência com expoente inteiro não negativo obedece à mesma regra por ser uma sucessão de multiplicações. A divisão, porém, não pode ser introduzida sem mais: substituir uma divisão pela multiplicação por uma inversa só é válido quando essa inversa existe. Assim, uma expressão como

$$x = (a - bcd)^{-1}(ef + g^h) \bmod n$$

só está definida desta forma se $h \geq 0$ e $\gcd(a - bcd, n) = 1$. Esta condição será importante sempre que dividirmos num corpo finito.

## 2.3 Criptografia de curva elíptica

### 2.3.1 O que são curvas elípticas?

Quando $q$ é primo, podemos representar o corpo finito $\mathbb{F}_q$ pelo conjunto de restos $\{0, 1, \ldots, q-1\}$. A soma, a subtracção e a multiplicação são seguidas de redução módulo $q$; cada elemento não nulo tem ainda uma inversa multiplicativa. É precisamente esta última propriedade que faz do conjunto um *corpo*, e não apenas um conjunto de restos.

`fe`: field element

Por exemplo, em $\mathbb{F}_{29}$, escrevemos $17 + 20 = 8$, pois $(17 + 20) \bmod 29 = 8$, e $-13 = 16$, pois $(-13) \bmod 29 = 16$. Uma fracção como $u/v$ significa $u \cdot v^{-1}$ no corpo e só está definida quando $v \neq 0$.

Para $q > 3$, uma curva elíptica pode ser apresentada na forma de *Weierstrass* [47] como o conjunto dos pontos $(x, y)$ que satisfazem

$$y^2 = x^3 + ax + b, \qquad a, b, x, y \in \mathbb{F}_q,$$

juntamente com um elemento de identidade. Exige-se ainda que a curva não seja singular; nesta forma, isso equivale a $4a^3 + 27b^2 \not\equiv 0 \pmod{q}$.[6]

Monero trabalha com uma curva na forma de Edwards torcida (*Twisted Edwards*) [7]. Para parâmetros não nulos e distintos $a, d \in \mathbb{F}_q$, esta forma é

$$ax^2 + y^2 = 1 + dx^2y^2, \qquad a, d, x, y \in \mathbb{F}_q.$$

Usaremos esta representação daqui em diante. Além de admitir fórmulas de adição particularmente regulares, permite implementar certas primitivas com menos operações aritméticas [9].

Sejam $P_1 = (x_1, y_1)$ e $P_2 = (x_2, y_2)$ dois pontos da curva. A sua soma $P_3 = P_1 + P_2 = (x_3, y_3)$ é definida por[7]

$$x_3 = \frac{x_1y_2 + y_1x_2}{1 + dx_1x_2y_1y_2},$$
$$y_3 = \frac{y_1y_2 - ax_1x_2}{1 - dx_1x_2y_1y_2},$$

[6] A expressão $a \in \mathbb{F}$ significa que $a$ é um elemento do corpo finito $\mathbb{F}$.

[7] Por razões de eficiência, as implementações representam habitualmente os pontos em coordenadas projectivas, evitando assim inversões no corpo durante muitas operações [137].

src/crypto/crypto_ops_builder/ref10CommentedCombined/ge.h

onde todas as operações, incluindo as divisões, têm lugar em $\mathbb{F}_q$. Para os parâmetros usados mais adiante, esta lei de adição é completa: os denominadores não se anulam para pontos da curva. As mesmas fórmulas duplicam um ponto quando $P_1 = P_2$. O inverso aditivo de $P = (x, y)$ é $-P = (-x, y)$ [7]; portanto, $P_1 - P_2 = P_1 + (-P_2)$, e não uma duplicação. A coordenada $-x$ significa, naturalmente, $(-x) \bmod q$.

A soma permanece na curva. Com esta operação, os pontos formam um grupo abeliano: há um elemento de identidade, cada ponto tem um inverso, a soma é associativa e a ordem das parcelas não importa. Na forma de Edwards, a identidade é o ponto afim

$$0 = (0, 1).$$

Na forma de Weierstrass, o mesmo papel é desempenhado pelo chamado ponto no infinito.[8]

Somando um ponto $P$ repetidamente, obtemos o subgrupo cíclico que ele gera,

$$\langle P \rangle = \{0, P, 2P, \ldots, (u-1)P\}.$$

A sua ordem $u$ é o menor inteiro positivo para o qual $uP = 0$. Por exemplo, se $u = 5$, o subgrupo contém $0, P, 2P, 3P, 4P$, e $5P = 0$, donde $5P + P = P$.

Qualquer ponto gera, por definição, um subgrupo cíclico. Se a ordem desse subgrupo for prima, todos os seus pontos não nulos são também geradores. No exemplo de ordem 5, os múltiplos de $2P$ percorrem

$$(2P, 4P, 6P, 8P, 10P) = (2P, 4P, P, 3P, 0).$$

Isto muda quando a ordem é composta. Num subgrupo de ordem 6,

$$(2P, 4P, 6P, 8P, 10P, 12P) = (2P, 4P, 0, 2P, 4P, 0);$$

logo $2P$ gera apenas um subgrupo de ordem 3.

À quantidade total de pontos do grupo chamamos a *ordem da curva*, $N$. O elemento de identidade está incluído nessa contagem. Pelo teorema de Lagrange, a ordem de qualquer subgrupo divide $N$. No exemplo anterior, 3 divide 6, como seria necessário.

Conhecendo $N$, podemos encontrar a ordem $u$ de um ponto $P$:

1. Calcula-se $N$, por exemplo com o *algoritmo de Schoof*.
2. Enumeram-se os divisores positivos de $N$ por ordem crescente.
3. Para cada divisor $n$, calcula-se $nP$.
4. O primeiro $n$ para o qual $nP = 0$ é $u$.

As curvas escolhidas para criptografia têm frequentemente $N = hl$, onde $l$ é um primo grande e $h = N/l$ é um pequeno *cofactor*. [9] Escolhe-se então um ponto $G$ de ordem $l$ como gerador

[8] Uma definição concisa de grupo abeliano encontra-se em https://brilliant.org/wiki/abelian-group/.

[9] Um cofactor pequeno simplifica o controlo dos pequenos subgrupos, mas não permite ignorá-los; as verificações de subgrupo continuam a fazer parte da segurança de muitos protocolos [7].

do subgrupo primo. Cada ponto $P$ desse subgrupo tem uma representação $P = nG$ para um único escalar $n \in \{0, 1, \ldots, l-1\}$; o valor $n = 0$ corresponde à identidade.

Calcular a *multiplicação escalar* $nP$ é fácil: trata-se de somar $P$ a si próprio $n$ vezes, com $0P = 0$. O percurso inverso é muito diferente. Dados um ponto não nulo $P_2$, que nesse subgrupo primo é um gerador, e outro ponto $P_1$, pretende-se encontrar $n$ tal que

$$P_1 = nP_2.$$

Este é o *problema do logaritmo discreto* (PLD) em curvas elípticas. Para curvas e parâmetros bem escolhidos, não se conhece um algoritmo clássico que o resolva eficientemente. Nesse sentido, a multiplicação escalar funciona como uma operação de sentido único: fácil de calcular, mas impraticável de inverter com os recursos conhecidos. Mais adiante encontraremos um escalar $\gamma$ central nos compromissos de Monero, cujo valor ninguém deve conhecer e cuja recuperação exigiria resolver um PLD (secção 6.3).

Para não efectuar $n - 1$ adições, podemos adaptar o algoritmo duplica-e-adiciona da secção 2.2.1. Escrevemos um inteiro não negativo $n$ em binário no vector $A$, do bit menos significativo para o mais significativo, e queremos obter $R = nP$:

1. Inicialize $R = 0$, o elemento de identidade, e $S = P$.
2. Para $i = 0, \ldots, |A| - 1$:
   (a) se $A[i] = 1$, faça $R = R + S$;
   (b) faça $S = S + S$.
3. O resultado é o valor final de $R$.

Como $G$ tem ordem prima $l$, os escalares do seu subgrupo são elementos de $\mathbb{F}_l$. Podemos, portanto, reduzi-los módulo $l$: para qualquer inteiro $n$, $nG = (n \bmod l)G$. A aritmética entre esses escalares é feita em $\mathbb{F}_l$, enquanto as coordenadas dos pontos pertencem a $\mathbb{F}_q$. Esta distinção entre $l$ e $q$ acompanhar-nos-á no resto do livro.

### 2.3.2 Criptografia de chave pública com curvas elípticas

Escolhamos uniformemente um escalar $k$ em $\{1, \ldots, l-1\}$. Este escalar é a *chave privada*, também chamada chave secreta. A chave pública correspondente é o ponto

$$K = kG.$$

Usaremos de modo consistente uma minúscula para a chave privada $k$ e a maiúscula correspondente para a chave pública $K$. Qualquer pessoa pode conhecer $G$ e $K$, mas recuperar $k$ requer resolver o PLD no subgrupo de ordem $l$. Esta assimetria permite publicar $K$ sem publicar o segredo que a controla.

### 2.3.3 A troca de chaves tipo Diffie-Hellman com curvas elípticas

Alice e Bob podem usar uma troca de chaves de tipo *Diffie–Hellman* [31] para obter o mesmo segredo sem enviarem esse segredo pela rede:

1. Alice gera $(k_A, K_A)$ e Bob gera $(k_B, K_B)$. Trocam as chaves públicas e conservam as privadas. Numa implementação, cada um deve ainda validar a codificação e o subgrupo da chave pública que recebeu.

2. Alice calcula $S_A = k_A K_B$, e Bob calcula $S_B = k_B K_A$. Os dois pontos coincidem, pois
$$S_A = k_A K_B = k_A k_B G = k_B k_A G = k_B K_A = S_B.$$
Podemos, portanto, chamar $S$ a esse ponto partilhado.

3. Para obter material de chave, ambos codificam $S$ e aplicam uma função de derivação, que representaremos conceptualmente por $h = \mathcal{H}([S])$. Alice pode então cifrar uma mensagem $m$ com um esquema simétrico autenticado sob a chave derivada $h$, e Bob pode decifrá-la com o mesmo $h$. Uma aplicação real não deve confundir esta derivação com a simples soma de uma hash à mensagem.

Um observador passivo conhece $G$, $K_A$ e $K_B$, mas não os escalares privados. Calcular $S = k_A k_B G$ apenas a partir desses pontos é o *problema computacional de Diffie–Hellman* (PCDH), que se supõe difícil no grupo escolhido.[10] O PLD, por sua vez, impede a recuperação directa de $k_A$ ou $k_B$ a partir das chaves públicas.

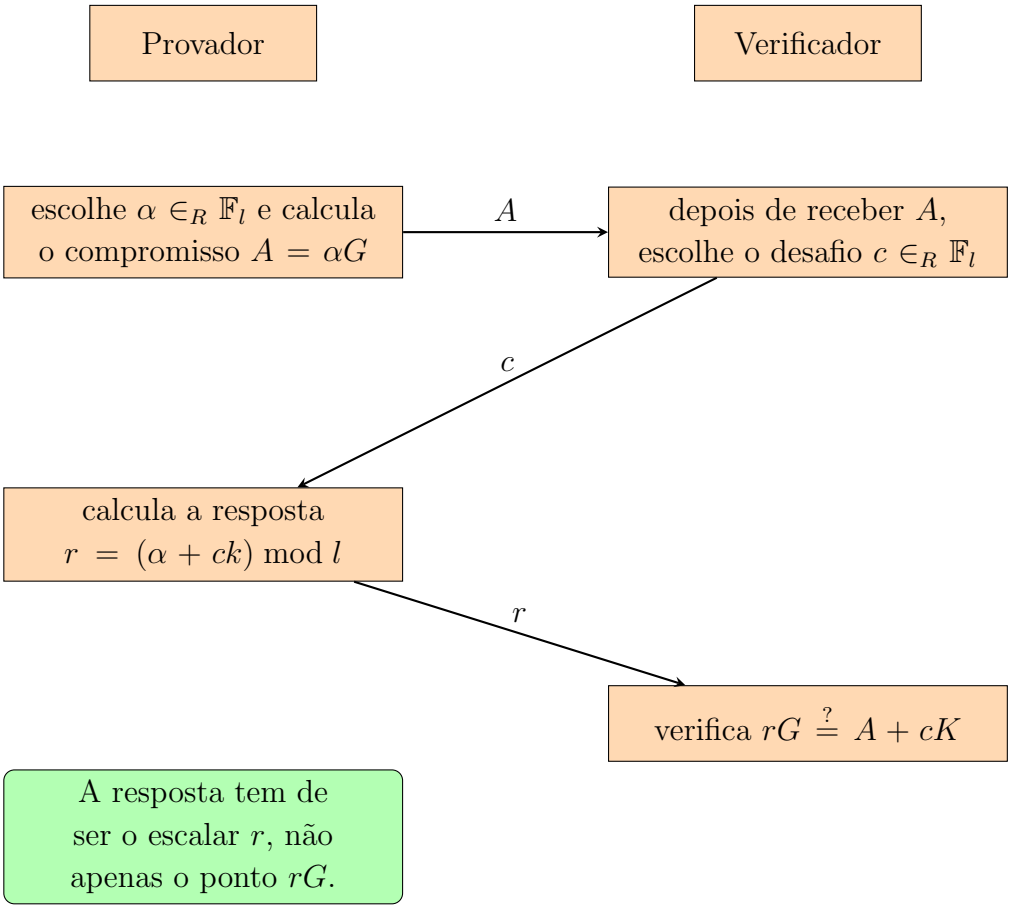
Esta troca, por si só, não autentica os interlocutores: um atacante activo pode substituir as duas chaves públicas e estabelecer um segredo diferente com cada lado. Quando a identidade de Alice ou Bob importa, as chaves trocadas têm de ser autenticadas, por exemplo com assinaturas.

### 2.3.4 Assinaturas tipo Schnorr e a transformada Fiat-Shamir

Na CRYPTO '89, Claus-Peter Schnorr apresentou um protocolo interactivo de identificação que se tornou fundamental [119]; Maurer generalizou esta família de provas em 2009 [67]. O protocolo permite a um provador demonstrar que conhece o escalar $k$ correspondente à chave pública $K = kG$, sem acrescentar ao verificador informação utilizável sobre $k$ [69].

Ambas as partes conhecem o grupo primo de ordem $l$, o gerador fixo $G$ e a chave pública $K$. Usaremos $\mathcal{H}_l$ para uma função hash cujo resultado é convertido num escalar de $\mathbb{F}_l$. A interacção tem três mensagens:

[10] A direcção da redução é importante: quem souber resolver o PLD pode recuperar $k_A$ e, portanto, resolver o PCDH. Logo, o PCDH não é mais difícil do que o PLD; a equivalência das dificuldades não está demonstrada em geral [138].




Um provador honesto é sempre aceite, porque

$$rG = (\alpha + ck)G = \alpha G + cK = A + cK.$$

O desafio só é escolhido depois de o compromisso $A$ ficar fixo. O registo completo da interacção (o *transcript*) contém as três mensagens $(A, c, r)$; sem $A$, a equação de verificação nem sequer fica determinada. Responder apenas com o ponto $rG$ seria inútil, pois qualquer pessoa poderia copiar o ponto já calculável $A + cK$. O trabalho do provador consiste em dar o seu logaritmo discreto $r$.

Se $\alpha$ for uniforme, então, para valores fixos de $c$ e $k$, a resposta $r$ também é uniforme [120]. Mais ainda, no caso de um verificador que escolhe honestamente um desafio uniforme, podemos simular um registo sem conhecer $k$: escolhemos $c, r \in_R \mathbb{F}_l$ e pomos $A = rG - cK$. O resultado tem a mesma distribuição que uma execução real. Esta é a propriedade de que a interacção não revela informação adicional sobre $k$ além da que já estava na chave pública $K$. Há, porém, uma condição decisiva: o provador não pode reutilizar $\alpha$. Se o mesmo compromisso $A$ receber desafios distintos $c \neq c'$ e produzir respostas válidas $r = \alpha + ck$ e $r' = \alpha + c'k$, qualquer observador extrai a chave: [11]

[11] Numa prova de segurança, um extractor pode executar o provador duas vezes a partir do mesmo estado posterior à escolha de $\alpha$, fornecendo um desafio diferente em cada execução. Esta técnica é designada por "rebobinagem" do provador.

$$k = (r - r')(c - c')^{-1} \bmod l.$$

A inversa existe porque $l$ é primo e $c - c' \not\equiv 0 \pmod{l}$. Esta possibilidade de extracção explica em que sentido uma resposta válida demonstra conhecimento, salvo a probabilidade negligível de adivinhar o desafio.

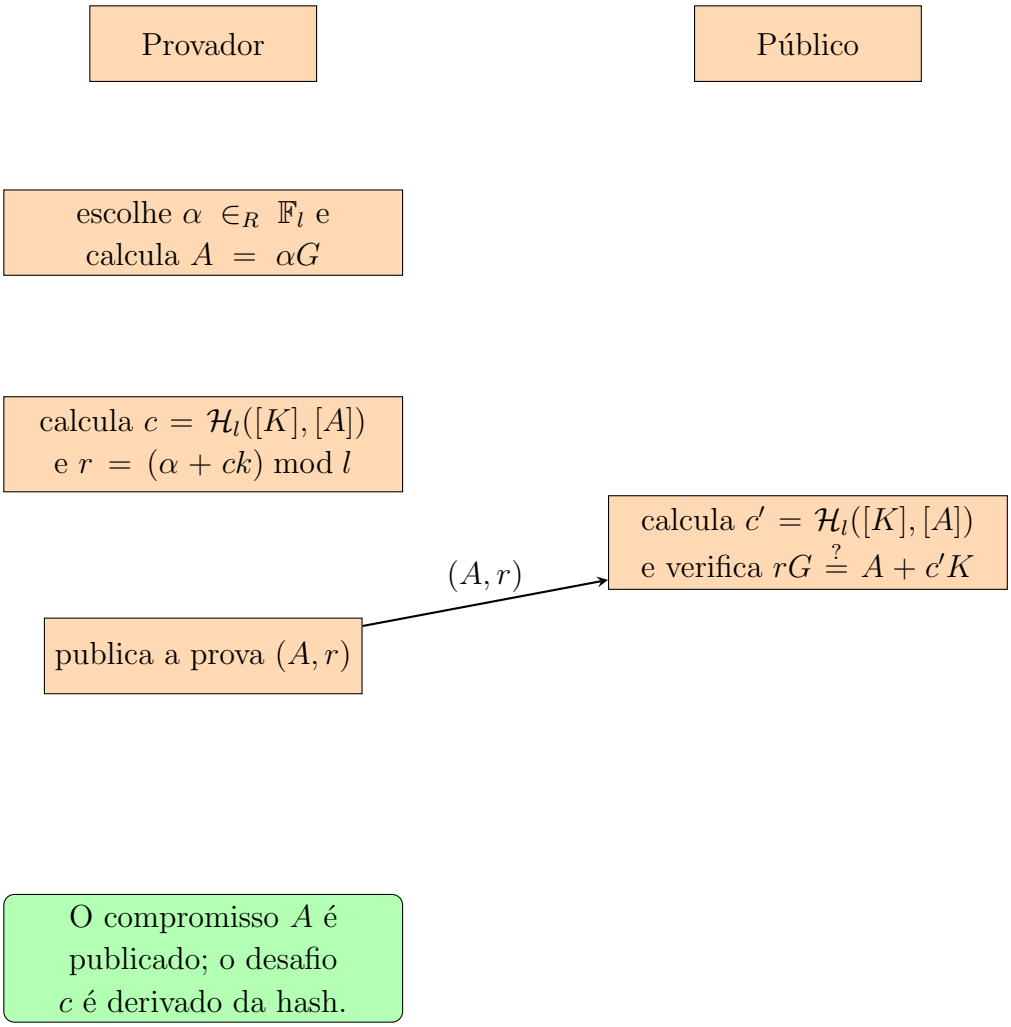
O atraso do desafio é essencial. Se conhecesse $c$ antes de fixar $A$, um impostor poderia escolher $r$ ao acaso e definir $A = rG - cK$; a verificação passaria sem que ele conhecesse $k$. A transformada de Fiat–Shamir [39] substitui o verificador por uma função hash que só pode ser avaliada depois de $A$ estar determinado. No modelo do oráculo aleatório, isso torna a prova não interactiva e publicamente verificável [69]. [12] [13] [14]

[12] Em criptografia, um oráculo é uma abstracção que responde a consultas segundo uma regra especificada. Um oráculo aleatório conserva uma resposta uniforme e independente para cada entrada nova [49].

[13] Uma função hash real é determinística. É a análise no modelo do oráculo aleatório que idealiza cada saída nova como uniforme; a conversão dessa saída num escalar deve ainda evitar ou limitar o viés de redução.

[14] O desafio deve ficar ligado à declaração que se prova. Inclui-se a chave pública $K$ na hash — o chamado prefixo de chave — e, se $G$ não for fixo para o protocolo, inclui-se também o gerador. Voltaremos a esta ideia nas secções 4.4 e 14.4.2.

**Prova não interactiva**




Já não há interacção: o provador calcula o desafio localmente e publica uma única prova. Qualquer pessoa pode repetir a hash e a verificação. A correcção continua a ser a mesma,

$$rG = (\alpha + ck)G = A + cK.$$

É importante distinguir o registo interactivo $(A, c, r)$ da prova não interactiva publicada $(A, r)$: nesta última, $c$ é reconstruído de forma determinística a partir de $K$ e $A$.

Os requisitos computacionais de uma prova incluem o espaço necessário para a guardar e o trabalho de verificação. Neste esquema, a prova contém um ponto de curva e um escalar; a declaração contém ainda a chave pública, outro ponto. A verificação exige uma hash e multiplicações escalares da curva.

src/ringct/rctOps.cpp

### 2.3.5 Assinar mensagens

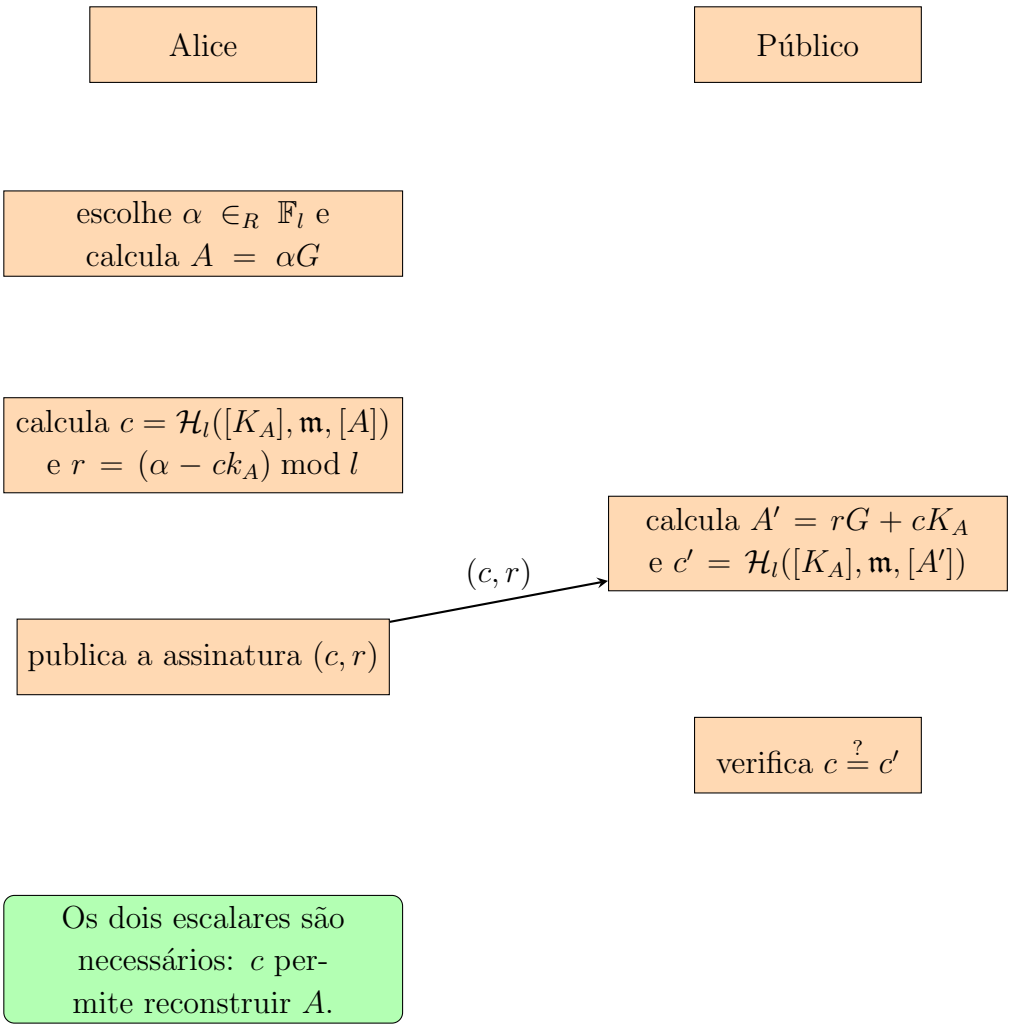
Um esquema de assinatura recebe uma mensagem de comprimento arbitrário e incorpora-a numa hash de comprimento fixo. Representaremos por $\mathfrak{m}$ a sequência de bytes que o esquema

recebe. Algumas variantes definem uma modalidade de pré-hash, na qual essa entrada já é a hash da mensagem; essa escolha faz parte do protocolo e não deve ficar implícita.

As assinaturas permitem ao destinatário verificar que uma mensagem foi autorizada pelo titular de uma chave privada e que o conteúdo assinado não foi alterado. ECDSA é um esquema comum; veja-se [54], ANSI X9.62 e [47].

O esquema seguinte é uma formulação da prova de Schnorr transformada, com uma convenção de sinais que permite publicar dois escalares $(c, r)$ em vez do ponto $A$. Pensar em assinaturas desta forma preparar-nos-á para as assinaturas em anel do capítulo seguinte.

Alice tem o par $(k_A, K_A)$. Para cada mensagem, escolhe um escalar secreto $\alpha$ novo e imprevisível; reutilizá-lo exporia $k_A$, exactamente como na prova interactiva.




Os dois escalares são necessários: $c$ permite reconstruir $A$.

Se $c = c'$, a assinatura é válida. A chave pública e a mensagem entram na hash, e as codificações devem ser inequívocas; um protocolo real inclui ainda um separador de domínio para não reutilizar o mesmo desafio noutro contexto.

**Funciona porque**

O verificador reconstrói exactamente o compromisso de Alice:

$$\begin{aligned} r &= \alpha - ck_A, \\ A' &= rG + cK_A \\ &= (\alpha - ck_A)G + cK_A \\ &= \alpha G - cK_A + cK_A \\ &= \alpha G = A. \end{aligned}$$

Logo,

$$c' = \mathcal{H}_l([K_A], \mathfrak{m}, [A']) = \mathcal{H}_l([K_A], \mathfrak{m}, [A]) = c.$$

Portanto, uma assinatura produzida por Alice passa a verificação. Esta álgebra demonstra a correcção, não basta por si só para provar a infalsificabilidade; essa propriedade depende da dificuldade do PLD, da segurança da hash, da escolha correcta de $\alpha$ e do modelo de segurança do esquema.

Requisitos:

Neste esquema, a assinatura guarda dois escalares. Para a verificar, são ainda necessários a mensagem, a chave pública de curva e os parâmetros do protocolo.

## 2.4 Curva Edwards25519

Monero usa a curva de Edwards torcida *Edwards25519*, que é birracionalmente equivalente à curva de Montgomery *Curve25519*. [15] As construções Curve25519, Edwards25519 e Ed25519 foram publicadas por Bernstein e colaboradores [5, 7, 8, 9]. Convém não confundir os nomes: Edwards25519 é a curva; Ed25519 é uma instância do esquema de assinatura EdDSA sobre essa curva, que estudaremos mais abaixo. [16]

A curva é definida sobre o corpo primo $\mathbb{F}_q$, com $q = 2^{255} - 19$, pela equação

$$-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2.$$

A fracção representa uma divisão em $\mathbb{F}_q$. Assim, na notação geral da secção 2.3.1, $a = -1$ e $d = -121665/121666 \bmod q$.

Os parâmetros foram escolhidos por um processo público e explicável. Essa propriedade, por vezes chamada rigidez, reduz a liberdade que um criador teria para procurar secretamente uma

src/crypto/crypto_ops_builder/ref10Comm-entedCombined/ge.h

src/wallet/ringdb.cpp

[15] Sem entrarmos nos pormenores, uma equivalência birracional relaciona pontos das duas curvas por funções racionais, com as excepções que essas funções necessariamente admitem.

[16] Bernstein desenvolveu também a cifra ChaCha [6, 99], usada pela implementação Monero [89] para cifrar certos dados sensíveis da carteira.

curva com uma fraqueza rara. [17]

O cuidado não é apenas teórico. O gerador Dual_EC_DRBG, que chegou a ser normalizado pelo NIST[18], ficou associado a um potencial mecanismo secreto de exploração baseado nos seus parâmetros elípticos [46]. A influência da NSA sobre esse processo de normalização tornou-se um exemplo dos riscos de concentrar escolhas criptográficas pouco transparentes [142]. Isto não implica que todos os standards do NIST sejam defeituosos; explica por que valorizamos parâmetros reproduzíveis e diversidade de análise. O projecto Edwards25519 foi também apresentado com atenção à ausência de restrições por patentes (veja-se [60]) e a operações eficientes [9]. O grupo de pontos de Edwards25519 tem ordem $N = hl$, com cofactor $h = 8 = 2^3$ e ordem prima

$$l = 7237005577332262213973186563042994240857116359379907606001950938285454250989.$$

O inteiro $l$ tem comprimento binário de 253 bits, pois $2^{252} < l < 2^{253}$; isso não significa que um escalar uniforme contenha 253 bits completos de entropia. A sua entropia é $\log_2 l$, muito próxima de 252 bits. Escalares e chaves privadas são habitualmente guardados em recipientes de 32 bytes, mas o tamanho da representação, o comprimento binário do módulo e a entropia são três grandezas distintas. A ordem total $8l$, por sua vez, é um inteiro de 256 bits e tem 77 algarismos decimais.

src/crypto/crypto_ops_builder/crypto_ops_builder/rctOps.h `curveOrder()`

## 2.4.1 Representação binária

O representante canónico de um elemento de $\mathbb{F}_{2^{255}-19}$ está entre 0 e $q-1$, pelo que cabe em 255 bits. A codificação usada por Edwards25519 ocupa 32 bytes em ordem *little-endian*; antes da compressão de pontos, o bit 255 desse recipiente está livre. Duas coordenadas completas ocupariam, portanto, 64 bytes.

Um escalar canónico em $\mathbb{F}_l$ também é guardado em 32 bytes, mas obedece ao limite diferente $0 \leq s < l$. Não devemos confundir uma sequência arbitrária de 32 bytes, um elemento do corpo de coordenadas, um escalar reduzido e uma codificação canónica: têm o mesmo tamanho exterior, não o mesmo domínio.

## 2.4.2 Compressão de ponto elíptico

Um ponto de Edwards25519 pode ser codificado nos mesmos 32 bytes que uma única coordenada. A ideia geral de compressão de pontos é anterior a esta curva [73]; a codificação de Edwards25519 é descrita em [8, 57].

[17] Uma curva pode não ter fraquezas públicas conhecidas e, ainda assim, ter sido seleccionada entre muitas candidatas por uma razão que o seu criador não revelou. Exigir uma derivação reproduzível dos parâmetros limita esse espaço de escolha. Edwards25519 é classificada como uma curva de geração totalmente explicada [118].

[18] National Institute of Standards and Technology, https://www.nist.gov/

Com $a = -1$, a equação da curva pode ser reorganizada como

$$x^2 = \frac{y^2 - 1}{dy^2 + 1}, \qquad d = -\frac{121665}{121666} \bmod q.$$

Para um $y$ válido, recuperar $x$ consiste em extrair uma raiz quadrada no corpo. Em regra há duas possibilidades, $x$ e $-x$; quando $x = 0$, elas coincidem. Como $q$ é ímpar, os representantes canónicos de $x \neq 0$ e $-x$ têm paridades opostas: por exemplo, módulo 5, os valores 3 e $-3 \bmod 5 = 2$ são um ímpar e um par. Um único bit escolhe, portanto, a raiz pretendida. Esse bit cabe na posição 255, que estava livre na codificação de $y$. Eis o procedimento conceptual para um ponto $(x, y)$:

**Codificar** Codifica-se $y$ canonicamente em 32 bytes *little-endian* e coloca-se no bit 255 a paridade do representante canónico de $x$: zero se $x$ é par, um se é ímpar. Chamemos $y'$ ao resultado.

src/crypto/crypto_ops_builder/ref10Comm-

**Descodificar**

entedCombined/ge_tobytes.c ge_frombytes.c

1. Copie-se o bit 255 de $y'$ para $b$, limpe-se esse bit e interpretem-se os restantes como $y$. Rejeita-se a codificação se $y \geq q$, pois não seria canónica.
2. Calcule-se $u = (y^2 - 1) \bmod q$ e $v = (dy^2 + 1) \bmod q$, de modo que $x^2 = u/v$ no corpo.
3. Calcule-se[19]
$$z = uv^3(uv^7)^{(q-5)/8} \bmod q.$$
   (a) Se $vz^2 \equiv u \pmod q$, ponha-se $x' = z$.
   (b) Se $vz^2 \equiv -u \pmod q$, ponha-se $x' = z\,2^{(q-1)/4} \bmod q$.
   (c) Se nenhuma congruência se verificar, $y'$ não codifica um ponto e deve ser rejeitado.
4. Se a paridade de $x'$ diferir de $b$, ponha-se $x = (-x') \bmod q$; caso contrário, $x = x'$. Rejeita-se ainda o caso $x = 0$ e $b = 1$, a codificação não canónica de "zero negativo".
5. O resultado é o ponto descomprimido $(x, y)$.

O gerador convencional do subgrupo de ordem $l$ é $G = (x, 4/5)$, com a raiz par de $x$ [8]. Por isso, a sua codificação usa $y = 4/5 \bmod q$ e bit de sinal zero.

### 2.4.3 Algoritmo de assinatura EdDSA

EdDSA designa uma família de assinaturas de estilo Schnorr. *Ed25519* é a instância que usa Edwards25519; não é o nome da curva. O artigo original apresentou-a como uma alternativa eficiente a ECDSA e relatou mais de 100 000 assinaturas por segundo no equipamento então

[19] Como $q = 2^{255} - 19 \equiv 5 \pmod 8$, os expoentes $(q-5)/8$ e $(q-1)/4$ são inteiros.

usado [8]. A especificação completa encontra-se na RFC 8032 [56]. Ed25519 deriva o valor efémero deterministicamente de um segredo e da mensagem, em vez de pedir ao sistema um novo número aleatório por assinatura. Isto evita uma classe importante de falhas de geradores aleatórios, embora não elimine a necessidade de proteger o segredo, a implementação e os cálculos contra falhas e canais laterais. O desenho procura, entre outros objectivos, evitar acessos a posições de memória dependentes de dados secretos e reduzir ataques temporais à cache [8].

A chave privada externa é uma semente $s$ de 32 bytes. Ed25519 calcula uma hash SHA-512 dessa semente. Os primeiros 32 bytes são podados: limpam-se os três bits menos significativos e o bit mais significativo, e fixa-se o segundo bit mais significativo. Interpretado em *little-endian*, o resultado é o inteiro secreto $a$; os 32 bytes restantes formam um prefixo secreto $p$. A chave pública é $A = aG$, transmitida na sua codificação comprimida $[A]$.

Estas grandezas voltam a ilustrar a diferença entre tamanho e entropia. Uma semente uniforme de 32 bytes tem 256 bits de entropia; o inteiro podado $a$ tem comprimento de 255 bits, mas apenas 251 posições variáveis, e não é uma codificação canónica de um escalar menor que $l$. Já os escalares da assinatura são reduzidos módulo $l$ e codificados em 32 bytes.

### Assinatura

Para a variante pura Ed25519, seja $\mathfrak{m}$ a mensagem exacta. A variante Ed25519ph, que recebe uma pré-hash, é um protocolo distinto e tem de ser seleccionada explicitamente [56]. Escrevendo $\mathcal{H}_l$ para SHA-512 seguida da conversão e redução módulo $l$:

1. Calcule-se o escalar efémero $\rho = \mathcal{H}_l(p, \mathfrak{m})$ e o ponto $R = \rho G$.
2. Calcule-se o desafio $e = \mathcal{H}_l([R], [A], \mathfrak{m})$.
3. Calcule-se $S = (\rho + ea) \bmod l$.
4. A assinatura é $([R], [S])$, um ponto comprimido e um escalar canónico.

### Verificação

A verificação procede da seguinte forma:

1. Descodifiquem-se $[A]$ e $[R]$ como pontos e $[S]$ como escalar. Rejeitem-se codificações inválidas ou não canónicas e valores $S \geq l$.
2. Calcule-se $e' = \mathcal{H}_l([R], [A], \mathfrak{m})$.

3. Aceite-se apenas se a equação de verificação da RFC 8032 se cumprir:
$$8SG \stackrel{?}{=} 8R + 8e'A.$$

O factor $8 = 2^3$ é o cofactor de Edwards25519 [8, 56]. Multiplicar um ponto arbitrário por 8 elimina a sua componente de pequena ordem; não demonstra que o ponto original já pertencia ao subgrupo primo. Em particular, a equação cofactorizada compara os dois lados depois de apagar possíveis componentes de torção. Uma verificação explícita $lA \stackrel{?}{=} 0$, acompanhada da rejeição da identidade, testa se a chave pública pertence ao subgrupo de ordem $l$. Não é sinónima de multiplicar toda a equação pelo cofactor. Protocolos e bibliotecas podem escolher regras de validação mais estritas, mas signatário e verificador têm de concordar exactamente nas mesmas regras. Esta distinção reaparecerá na secção 4.4.

**Funciona porque**

Para uma assinatura honesta, $A = aG$ e $R = \rho G$. Logo,
$$\begin{aligned} 8SG &= 8(\rho + ea)G \\ &= 8\rho G + 8e(aG) \\ &= 8R + 8eA. \end{aligned}$$
O verificador recompõe o mesmo $e$ a partir da mensagem, da chave e de $[R]$, pelo que a igualdade se verifica.

Requisitos:

A assinatura Ed25519 guarda um ponto comprimido de 32 bytes e um escalar de 32 bytes: 64 bytes no total. A chave pública comprimida ocupa mais 32 bytes, mas normalmente é fornecida separadamente da assinatura. A semente privada nunca é transmitida.

## 2.5 Operador binário XOR

O operador binário XOR será útil nas secções 5.4 e 6.3. Recebe dois bits e devolve 1 se exactamente um deles for 1 [143]. Eis a tabela de verdade:

| A | B | A ⊕ B |
|---|---|---|
| 1 | 1 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

Bit a bit, XOR é a adição módulo 2. Por exemplo,

$$\{1,1\} \oplus \{1,0\} = \{(1+1) \bmod 2, (1+0) \bmod 2\} = \{0,1\}.$$

Da tabela seguem três identidades simples:

$$A \oplus 0 = A, \qquad A \oplus A = 0, \qquad (A \oplus B) \oplus B = A.$$

Suponhamos que Alice quer cifrar um vector privado $V$ de $n$ bits. Ela escolhe uma chave $S$ também de $n$ bits e envia a Bob

$$C = V \oplus S.$$

Bob, que conhece $S$, recupera a mensagem porque

$$C \oplus S = (V \oplus S) \oplus S = V.$$

Para cada valor fixo de $C$, há $2^n$ pares $(V, S)$ que o produzem: a cada candidato $V$ corresponde exactamente $S = C \oplus V$. [20]

| | | | | |
|---|---|---|---|---|
| V | 0 | 1 | 1 | 0 |
| | ⊕ | ⊕ | ⊕ | ⊕ |
| S | 0 | 1 | 0 | 1 |
| C | 0 | 0 | 1 | 1 |

$C$

| | | | | |
|---|---|---|---|---|
| C | 0 | 0 | 1 | 1 |
| | ⊕ | ⊕ | ⊕ | ⊕ |
| S | 0 | 1 | 0 | 1 |
| V | 0 | 1 | 1 | 0 |

Isto só dá segredo perfeito, no sentido da teoria da informação, sob hipóteses fortes: $S$ tem de ser secreta, uniforme em $\{0,1\}^n$, independente de $V$ e usada uma única vez. Nessas condições, observar $C$ não altera a probabilidade de qualquer mensagem. Se a mesma chave for reutilizada, então $C_1 \oplus C_2 = V_1 \oplus V_2$, o que revela uma relação entre as mensagens. Uma chave enviesada, previsível ou mais curta também destrói a conclusão.

Quando $S$ é derivada de Diffie–Hellman ou de uma palavra-passe, a sua segurança é apenas computacional e depende da função de derivação e das hipóteses subjacentes. Além disso, XOR sozinho não autentica o texto cifrado: uma construção real usa normalmente uma cifra autenticada, como já antecipámos na secção 2.3.3.

[20] Outra aplicação de XOR é trocar dois registos sem um terceiro: $\{A, B\} \to \{A \oplus B, B\} \to \{A \oplus B, A\} \to \{B, A\}$.

# CAPÍTULO 3

# Divisores: certificar uma multiplicação de pontos

## 3.1 A pergunta que uma função pode responder

A afirmação $Q = nP$ tem um sentido conhecido: somar $P$ consigo mesmo $n$ vezes produz $Q$. Podemos confirmá-la por duplicações e adições. Que objecto poderia acompanhar $Q$ e certificar o resultado?

Uma função na curva pode desempenhar esse papel. A ponte está nos seus zeros e pólos: os lugares onde ocorrem, contados com multiplicidade, obedecem a uma relação na lei do grupo. Construiremos essa ligação desde uma recta até uma função que testemunha a multiplicação.

Para estudar essa ponte, usaremos uma curva não singular na forma curta de Weierstrass, sobre um corpo $\mathbb{F}$ de característica maior que três:

$$\mathbb{E}: \quad y^2 = x^3 + a_{\mathbb{E}}x + b_{\mathbb{E}}, \qquad 4a_{\mathbb{E}}^3 + 27b_{\mathbb{E}}^2 \neq 0. \tag{3.1}$$

Chamaremos $\mathcal{O}_\infty$ à identidade. Nesta apresentação, $-(x, y) = (x, -y)$. O inverso em coordenadas de Edwards, estabelecido no capítulo anterior, tem outra fórmula: a representação dos pontos importa.

O nosso exemplo será deliberadamente pequeno:

$$\mathbb{E}/\mathbb{F}_{17}: \quad y^2 = x^3 + 2x + 2, \qquad P = (5, 1), \qquad Q = (10, 6).$$

As contas deste exemplo serão módulo 17. Construiremos a função que certifica $Q = 3P$ e examinaremos o seu divisor. Um corpo pequeno permite conferir as contas; a solidez criptográfica exigirá corpos grandes.

## 3.2 Uma recta, três encontros e uma soma

Na forma de Weierstrass, a soma tem uma leitura geométrica. Uma recta passa por $A$ e $B$, encontra a curva num terceiro ponto $R$, e a soma $A + B$ é o inverso de $R$. Se os dois pontos coincidem, usa-se a tangente. Se são inversos, a recta é vertical e a soma é $\mathcal{O}_\infty$.

Para uma recta não vertical $y = mx + c$, a substituição na curva dá

$$x^3 - m^2x^2 + (a_\mathbb{E} - 2mc)x + b_\mathbb{E} - c^2 = 0.$$

O polinómio é cúbico. As suas raízes são as coordenadas dos três encontros, contados com multiplicidade, numa extensão do corpo se necessário. Pela soma das raízes, $x_A + x_B + x_R = m^2$; assim,

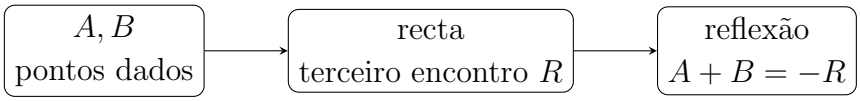
$$x_{A+B} = m^2 - x_A - x_B, \qquad y_{A+B} = m(x_A - x_{A+B}) - y_A.$$

São operações no corpo, mesmo quando a figura que as sugere é desenhada sobre os reais. Num corpo finito, a curva é um conjunto de pontos discretos; a equação de intersecção conserva o raciocínio algébrico.

No exemplo, a tangente em $P = (5, 1)$ tem declive

$$m = \frac{3 \cdot 5^2 + 2}{2 \cdot 1} = \frac{9}{2} = 13 \pmod{17}.$$

Logo, $2P = (6, 3)$. A recta por $P$ e $2P$ tem declive $(3-1)/(6-5) = 2$, e fornece $3P = (10, 6) = Q$.




A relação que interessa guardar é $A + B + R = \mathcal{O}_\infty$. Cada recta oferece, portanto, uma pequena soma nula. O certificado final reunirá estas relações e eliminará os pontos intermédios.

## 3.3 A função existe sobre a curva

A expressão $\lambda(x, y) = y - mx - c$ define uma função. Avaliá-la num ponto da curva dá $\lambda(A) = y_A - mx_A - c$. O resultado é zero exactamente quando $A$ pertence à recta. A recta desenhada no plano e a função restringida à curva têm papéis distintos: a primeira indica os encontros; a segunda permite multiplicar, dividir e comparar valores.

Na curva, as coordenadas não são independentes. A função $y^2$ é a mesma que $x^3 + a_{\mathbb{E}}x + b_{\mathbb{E}}$. Por isso, o anel das funções polinomiais na parte afim é

$$\mathbb{F}[\mathbb{E}] = \mathbb{F}[x, y]/(y^2 - x^3 - a_{\mathbb{E}}x - b_{\mathbb{E}}).$$

O quociente diz que identificamos expressões cuja diferença é um múltiplo da equação da curva. Reduzindo as potências de $y$, cada elemento admite um representante único

$$f(x, y) = f_0(x) + yf_1(x), \qquad f_0, f_1 \in \mathbb{F}[x]. \tag{3.2}$$

Por exemplo, $y^3 = y(x^3 + a_{\mathbb{E}}x + b_{\mathbb{E}})$. Esta redução tornará possível escrever uma função como uma lista finita de coeficientes, adequada a uma testemunha algébrica.

O corpo de funções $\mathbb{F}(\mathbb{E})$ é o corpo de fracções deste anel: os seus elementos são razões de funções polinomiais, com denominador não identicamente nulo. Uma razão pode não estar definida pela sua expressão num ponto particular; é preciso determinar se há cancelamento ou um pólo.

Há uma precaução própria dos corpos finitos. Consideramos funções como objectos algébricos na curva, e não apenas tabelas dos valores nos seus poucos pontos racionais. Um polinómio pode anular-se em todos esses pontos sem ser a função identicamente nula. Os zeros geométricos incluem também os pontos definidos sobre extensões do corpo.

**Verificação de compreensão.** Que função representa $y^2 - x^3 - 2x - 2$ no nosso exemplo? A função nula. Já $y - 2x + 9$ é uma função não nula, embora se anule em três encontros com a curva. Um certificado precisará sempre de uma função não nula.

## 3.4 Um zero pode contar mais de uma vez

Saber que $f(A) = 0$ ainda não diz quantas vezes esse zero conta. Num polinómio de uma variável, $(t - t_0)^2$ tem um zero duplo em $t_0$. Na curva, procede-se localmente: escolhe-se uma coordenada que se anule simplesmente no ponto e mede-se a primeira potência não nula da função.

Essa coordenada chama-se *parâmetro local*. Se $y_A \neq 0$, podemos usar $t = x - x_A$; a equação da curva determina uma expansão local de $y$ em potências de $t$. Uma função não nula escreve-se

$$f = t^m u, \qquad u(A) \neq 0.$$

O inteiro $m$ é a ordem de $f$ em $A$, escrita $\mathrm{ord}_A(f)$. Uma ordem positiva indica um zero; uma ordem negativa, um pólo; uma ordem nula, um valor local finito e não nulo.

A tangente elimina o termo linear da expansão. Num ponto comum, a sua função tem, por isso, um zero de ordem dois. Num ponto de inflexão, a ordem pode ser três. O número de pontos distintos e o número de intersecções contadas com multiplicidade deixam então de coincidir.

No nosso exemplo, a tangente é

$$\lambda_{P,P} = y - 13x + 13.$$

Substituir $y = 13x - 13$ na equação da curva produz o polinómio mónico

$$(x - 5)^2(x - 6).$$

Há duas ocorrências em $P = (5, 1)$ e uma em $-2P = (6, 14)$. A tangente não precisa de encontrar três pontos distintos para fornecer três zeros contados correctamente.

Se $y_A = 0$, a expressão $x - x_A$ já não é, em geral, um parâmetro local: neste caso, $y$ serve como parâmetro e $x - x_A$ tem ordem dois. É por isso que a ordem pertence à função na curva, e não apenas à aparência da sua expressão no plano.

**Verificação de compreensão.** Se $f$ tem um zero duplo e $g$ um zero simples no mesmo ponto, qual é a ordem de $f/g$? É um: localmente, $t^2u/(tv) = t(u/v)$. O cancelamento tem de respeitar as ordens.

## 3.5 O infinito entra na contabilidade

Uma função polinomial não tem pólos na parte afim da curva. Isso não impede que os tenha no ponto $\mathcal{O}_\infty$. Para ver esse ponto, homogeneizamos a equação:

$$Y^2Z = X^3 + a_{\mathbb{E}}XZ^2 + b_{\mathbb{E}}Z^3.$$

As coordenadas afins são $x = X/Z$, $y = Y/Z$. Quando $Z = 0$, a equação obriga a $X = 0$, ficando o único ponto $[0 : 1 : 0]$. É a identidade da curva nesta representação.

Na carta $Y \neq 0$, ponhamos $u = X/Y$ e $v = Z/Y$. A equação torna-se

$$v = u^3 + a_{\mathbb{E}}uv^2 + b_{\mathbb{E}}v^3.$$

Perto da identidade, $u$ é um parâmetro local e $v$ começa em ordem três. Como $x = u/v$ e $y = 1/v$, obtemos

$$\operatorname{ord}_{\mathcal{O}_\infty}(x) = -2, \qquad \operatorname{ord}_{\mathcal{O}_\infty}(y) = -3.$$

São ordens algébricas; não estamos a fazer tender números de um corpo finito para uma grandeza infinita.

A função vertical $x - x_A$ tem, portanto, um pólo duplo na identidade. A função de uma recta não vertical, $y - mx - c$, tem um pólo triplo: o termo $y$, de ordem $-3$, não pode ser cancelado por termos de ordens $-2$ e zero. Os dois zeros afins da vertical e os três zeros afins da recta equilibram esses pólos.

Também se explica a escolha dos pesos 2 para $x$ e 3 para $y$. No representante reduzido $f_0 + yf_1$, os monómios $x^i$ têm ordem de pólo $2i$, e os monómios $yx^j$, ordem $3 + 2j$. Um peso é par e o outro ímpar; os termos de maior peso não se cancelam entre as duas parcelas. Este peso mede a ordem do pólo no infinito.

O ponto no infinito é, pois, indispensável: deixa visível o saldo que uma função polinomial transporta para fora da parte afim.

## 3.6 O divisor: um registo de lugares e ordens

Podemos agora dar nome ao registo que temos vindo a preparar. Um *divisor* é uma soma formal finita

$$D = \sum_{A \in S} m_A [A], \qquad m_A \in \mathbb{Z}. \tag{3.3}$$

O símbolo $[A]$ guarda o lugar $A$; o coeficiente guarda uma multiplicidade inteira. Neste capítulo, os divisores serão escritos sobre os pontos geométricos, isto é, sobre o fecho algébrico do corpo. Quando todos os pontos usados são racionais, como no exemplo, as contas já se fazem sobre $\mathbb{F}$.

Somar divisores é somar coeficientes do mesmo símbolo. Assim,

$$2[A] + [B] - [A] = [A] + [B].$$

Esta operação não soma pontos na curva. Mesmo que $C = A + B$, o divisor $[A] + [B]$ conserva dois lugares, enquanto $[C]$ conserva um só. O grupo dos divisores é o grupo abeliano livre gerado pelos símbolos dos pontos; nenhuma relação de adição da curva é imposta nesta definição.

O *suporte* de $D$ é o conjunto dos lugares com coeficiente não nulo. O seu *grau* é

$$\deg D = \sum_{A} m_A.$$

Por exemplo, $2[P] + [-2P] - 3[\mathcal{O}_\infty]$ tem grau zero e, no nosso exemplo, suporte com três pontos distintos. Coeficientes negativos pertencem à definição de divisor; só quando o divisor provém de uma função significam efectivamente ordens de pólos.

Para uma função racional não nula, o divisor é

$$\operatorname{div}(f) = \sum_{A} \operatorname{ord}_A(f)[A]. \tag{3.4}$$

Chamamos *principal* a um divisor que tem esta forma. Uma constante não nula tem divisor zero. A função identicamente nula não tem divisor: não admite uma ordem finita nos seus pontos.

**Verificação de compreensão.** Quanto vale $[A] + [-A]$? Como divisor, continua a ser essa soma formal, salvo se os lugares coincidirem. A soma dos pontos $A + (-A)$, por sua vez, é a identidade. Os parênteses rectos preservam a distinção.

## 3.7 Multiplicar funções soma os seus divisores

Se localmente $f = t^m u$ e $g = t^n v$, com $u, v$ unidades, então $fg = t^{m+n} uv$. A ordem de um produto é a soma das ordens. Seguem-se as identidades

$$\operatorname{div}(fg) = \operatorname{div}(f) + \operatorname{div}(g), \qquad \operatorname{div}(f/g) = \operatorname{div}(f) - \operatorname{div}(g).$$

Esta é a primeira ferramenta de construção: relações pequenas podem ser reunidas por multiplicação; os lugares intermédios podem desaparecer por divisão.

Consideremos $C = A + B$, em posição genérica. A recta por $A, B$ tem terceiro encontro em $-C$. A vertical por $C$ passa também por $-C$. Os respectivos divisores são

$$\operatorname{div}(\lambda_{A,B}) = [A] + [B] + [-C] - 3[\mathcal{O}_\infty],$$
$$\operatorname{div}(v_C) = [C] + [-C] - 2[\mathcal{O}_\infty], \qquad v_C = x - x_C.$$

Ao dividir, o encontro $-C$ cancela-se e resta

$$h_{A,B} = \frac{\lambda_{A,B}}{v_C}, \qquad \operatorname{div}(h_{A,B}) = [A] + [B] - [C] - [\mathcal{O}_\infty]. \tag{3.5}$$

No caso genérico, os zeros são $A, B$ e os pólos são $C, \mathcal{O}_\infty$. Se houver coincidências, somam-se primeiro os coeficientes: a lista de zeros e pólos efectiva pode ser menor.

Para duplicar $P$, a mesma construção dá

$$\operatorname{div}(h_{P,P}) = 2[P] - [2P] - [\mathcal{O}_\infty].$$

A tangência já fornece o coeficiente dois; não se acrescenta uma segunda condição de avaliação num ponto diferente.

O divisor da razão tem grau zero. Esta igualdade reflecte um facto geral: numa curva projectiva não singular, toda a função racional não nula tem tantos zeros como pólos, contando multiplicidades. A condição é necessária para um divisor ser principal. Precisamos, contudo, de uma segunda condição que guarde a posição dos lugares, pois uma mera contagem perde a relação entre os pontos.

## 3.8 Duas somas que convém nunca confundir

A cada divisor podemos associar dois resultados. O primeiro é o inteiro $\deg D$. O segundo é um ponto, obtido aplicando a lei do grupo aos lugares com as suas multiplicidades:

$$\sigma(D) = \sum_A m_A A.$$

Coeficientes negativos mandam somar inversos; coeficientes positivos mandam somar repetidamente. O símbolo $D$ é o registo formal; $\sigma(D)$ é o resultado da soma dos pontos registados.

Para o divisor da recta por $A, B$, temos

$$\deg D = 3 - 3 = 0, \qquad \sigma(D) = A + B + (-A - B) = \mathcal{O}_\infty.$$

Para a vertical, a soma é $A + (-A) = \mathcal{O}_\infty$. Produtos e quocientes destas funções conservam as duas condições. Tanto o grau como $\sigma$ são aditivos.

O exemplo $D = [P] - [Q]$ mostra a diferença entre as condições. O grau é zero para quaisquer $P, Q$, mas $\sigma(D) = P - Q$ só é a identidade quando $P = Q$. Se $P \neq Q$, não existe uma função com um único zero simples em $P$ e um único pólo simples em $Q$. A curva elíptica impõe mais do que o equilíbrio do número de zeros e pólos.

O teorema que usaremos é precisamente

$$D \text{ principal} \quad \Longleftrightarrow \quad \deg D = 0 \ \text{ e } \ \sigma(D) = \mathcal{O}_\infty. \tag{3.6}$$

A formulação vale para divisores geométricos na curva elíptica. Para os divisores de pontos racionais considerados aqui, a função testemunha pode ser tomada sobre o corpo base. A construção por rectas que faremos fornece essa função directamente.

Este critério é a ponte procurada. Um registo formal cuja soma no grupo é zero pode ser o divisor de uma função; reciprocamente, o divisor de uma função tem soma nula. Para compreender a afirmação, falta explicar por que a geometria das rectas representa todas as classes de grau zero. Esse será o próximo passo.

## 3.9 Por que a ponte respeita a lei do grupo

Dois divisores dizem-se *linearmente equivalentes* quando a sua diferença é principal. Escrevemos $D \sim D'$. A função $h_{A,B}$ construída anteriormente demonstra

$$[A] + [B] \sim [A + B] + [\mathcal{O}_\infty].$$

Os divisores dos dois lados continuam diferentes. O sinal $\sim$ diz que existe uma função cuja contabilidade transforma um no outro.

Subtraindo duas cópias da identidade, obtemos

$$([A] - [\mathcal{O}_\infty]) + ([B] - [\mathcal{O}_\infty]) \sim [A + B] - [\mathcal{O}_\infty].$$

Esta relação deixa reduzir uma soma de diferenças a uma única diferença. As verticais permitem tratar os inversos, e a identidade não acrescenta nada. Para qualquer divisor de grau zero, a redução conduz a

$$D \sim [\sigma(D)] - [\mathcal{O}_\infty].$$

Em particular, se $\sigma(D) = \mathcal{O}_\infty$, o representante final é zero. As funções usadas nas reduções, multiplicadas ou divididas conforme o caso, fornecem uma função de divisor $D$.

Falta justificar a unicidade desse representante. Se $[S] - [\mathcal{O}_\infty]$ fosse principal com $S \neq \mathcal{O}_\infty$, a função correspondente teria um único pólo simples, na identidade, e nenhum pólo afim. Seria, pois, uma função polinomial na parte afim. Mas o representante $f_0 + yf_1$ mostra que uma função polinomial não constante tem ordem de pólo pelo menos dois: não há monómio de peso um. Tal função não existe. Isto completa a justificação do critério.

O conjunto de classes de divisores de grau zero chama-se $\mathrm{Pic}^0(\mathbb{E})$. A correspondência

$$S \longmapsto \text{classe de } [S] - [\mathcal{O}_\infty]$$

é um isomorfismo com o grupo da curva. O nome é útil, mas o mecanismo já está à vista: uma recta transporta a adição dos pontos para a equivalência dos divisores. É esse mecanismo que o certificado aproveitará.

## 3.10 Construir o certificado de uma multiplicação

Voltemos à relação $Q = nP$, com $n \geq 1$. O divisor

$$D_n = n[P] - [Q] - (n-1)[\mathcal{O}_\infty]$$

tem grau zero. A sua soma no grupo é $nP - Q$. Pelo critério, existe uma função com este divisor exactamente quando $Q = nP$.

A existência pode ser acompanhada passo a passo. Ponhamos $F_1 = 1$. Supondo $Q_k = kP$ e

$$\operatorname{div}(F_k) = k[P] - [Q_k] - (k-1)[\mathcal{O}_\infty],$$

a função da próxima adição satisfaz

$$\operatorname{div}(h_{Q_k,P}) = [Q_k] + [P] - [Q_{k+1}] - [\mathcal{O}_\infty].$$

Logo, $F_{k+1} = F_k h_{Q_k,P}$ tem o divisor desejado. O termo $-[Q_k]$ cancela-se com $[Q_k]$. A função final guarda a relação entre os extremos sem conservar os pontos intermédios no seu divisor.

Também podemos seguir uma cadeia de duplicações e adições. Se já temos $F_u, F_v$, então

$$F_{u+v} = F_u F_v h_{uP,vP}$$

tem divisor $(u+v)[P] - [(u+v)P] - (u+v-1)[\mathcal{O}_\infty]$. Para duplicar, usa-se $F_u^2 h_{uP,uP}$. A construção deixa assim acompanhar o algoritmo habitual de multiplicação escalar.

Para $n = 3$, a conta formal inteira cabe em duas linhas:

$$\operatorname{div}(h_{P,P}) = 2[P] - [2P] - [\mathcal{O}_\infty],$$

$$\operatorname{div}(h_{2P,P}) = [2P] + [P] - [3P] - [\mathcal{O}_\infty].$$

A soma dá $3[P] - [3P] - 2[\mathcal{O}_\infty]$.

Há aqui uma diferença de custos a conservar. O provador ainda calcula os pontos e constrói a função. O certificado abre uma possibilidade de verificação diferente; não elimina o trabalho necessário para obter a testemunha. Além disso, a ordem $n$ do zero em $P$ seria enorme para um escalar criptográfico. Teremos de escolher uma representação mais económica.

## 3.11 O certificado completo em módulo dezassete

No exemplo, já encontrámos $2P = (6, 3)$ e $Q = 3P = (10, 6)$. As duas rectas e as duas verticais são

$$\lambda_{P,P} = y - 13x + 13, \qquad v_{2P} = x - 6,$$
$$\lambda_{2P,P} = y - 2x + 9, \qquad v_Q = x - 10.$$

Assim,

$$F_3 = \frac{(y - 13x + 13)(y - 2x + 9)}{(x - 6)(x - 10)}$$

tem divisor $3[P] - [Q] - 2[\mathcal{O}_\infty]$. O cancelamento em $2P$ exprime-se nas ordens locais, mesmo que a fracção escrita pareça indeterminada nesse ponto.

Para preparar uma testemunha polinomial, multiplicamos por $v_Q$. A função $f = F_3 v_Q$ fica com divisor

$$\operatorname{div}(f) = 3[P] + [-Q] - 4[\mathcal{O}_\infty].$$

Reduzir o produto das rectas pela equação da curva dá

$$(y - 13x + 13)(y - 2x + 9) = (x - 6)\big(x^2 + 15x + 2y\big).$$

Podemos, portanto, tomar

$$f = x^2 + 15x + 2y.$$

O maior peso é quatro, vindo de $x^2$. A função tem um pólo de ordem quatro na identidade. Em $P$, a construção pelas rectas fornece um zero triplo. Em $-Q = (10, 11)$, fornece um zero simples. São quatro zeros: não há espaço para outros.

Uma substituição ordinária confirma $f(5, 1) = 0$ e $f(10, 11) = 0$, mas não mede, por si só, o zero triplo. Essa multiplicidade vem da tangente e do cancelamento local. Pelo contrário, $f(10, 6) = 7 \neq 0$: o zero adicional está em $-Q$, não em $Q$.

O certificado tem agora uma forma concreta: três coeficientes não nulos e uma ordem de pólo limitada. O divisor tem soma $3P - Q$, logo a existência desta função com estas ordens certifica a multiplicação. A próxima simplificação reduzirá também o número de ocorrências de $P$.

## 3.12 Os dígitos substituem uma multiplicidade enorme

Para um escalar $s_A$, escrevamos uma decomposição binária e preparemos uma tabela pública de pontos:

$$s_A = \sum_{i=0}^{\ell_A - 1} e_i 2^i, \qquad e_i \in \{0, 1\}, \qquad A_i = 2^i A. \tag{3.7}$$

A relação a certificar é

$$W = s_A A \quad \Longleftrightarrow \quad \sum_i e_i A_i - W = \mathcal{O}_\infty. \tag{3.8}$$

Em lugar de pôr $s_A$ ocorrências em $A$, usamos uma ocorrência em cada ponto seleccionado pela decomposição. O divisor correspondente é

$$D_{s_A} = \sum_i e_i [A_i] + [-W] - \left(1 + \sum_i e_i\right) [\mathcal{O}_\infty]. \tag{3.9}$$

O grau é zero, e a soma no grupo é $s_A A - W$. O número de zeros afins é, no máximo, $\ell_A + 1$, em vez de crescer com o valor inteiro do escalar. Como só há pólos no infinito, a função testemunha pode ser representada por um polinómio na curva.

No caso $3 = 1 + 2$, a tabela selecciona $P$ e $2P$. O divisor passa a ser $[P] + [2P] + [-Q] - 3[\mathcal{O}_\infty]$, e a própria recta $y - 2x + 9$ é uma testemunha. A função de peso quatro da página anterior era correcta; a decomposição permitiu uma de peso três.

Os bits precisam de ser realmente bits, por exemplo pela condição $e_i(e_i - 1) = 0$. A recomposição tem de usar os mesmos dígitos que o divisor. Se a representação deve ser canónica, impõe-se ainda o intervalo do escalar. As contas dos coeficientes usam o corpo base; as contas dos escalares seguem a ordem do subgrupo. Estes módulos não são intercambiáveis.

Se $W = \mathcal{O}_\infty$, o termo $[-W]$ junta-se ao termo do infinito; não se inventam coordenadas afins para a identidade. Se $s_A = 0$ e $W = \mathcal{O}_\infty$, o divisor é zero e uma constante não nula serve de testemunha. Casos de identidade exigem esse tratamento explícito nas representações e nos dispositivos de verificação.

## 3.13 O tamanho da função também faz parte da prova

Uma função pode anular-se numa lista qualquer de pontos se lhe concedermos grau suficiente. Por exemplo, o produto das verticais desses pontos anula-se também nos respectivos inversos. A mera existência de zeros nos lugares declarados não demonstra que a soma dos pontos seja a identidade. É preciso excluir zeros adicionais.

Se o divisor prescrito tem $d$ zeros afins, contando multiplicidades, uma função polinomial testemunha deve ter pólo de ordem $d$ no infinito. O representante reduzido impõe limites transparentes:

$$2\deg f_0 \le d, \qquad 3 + 2\deg f_1 \le d.$$

A segunda condição desaparece quando $f_1 = 0$. Estes limites dão uma ordem de pólo de, no máximo, $d$. Se já houver $d$ zeros prescritos com as multiplicidades certas, o equilíbrio do divisor obriga a que todos os zeros tenham sido contabilizados.

Esta observação fornece uma verificação exacta, embora potencialmente dispendiosa: confirmar que $f \neq 0$, confirmar o limite de peso e confirmar cada zero com a respectiva ordem. Para lugares repetidos, avaliar $f$ uma vez não basta. Verifica-se a expansão local ou condições de interpolação que expressem a multiplicidade. Em característica positiva, derivadas ordinárias de ordem elevada podem perder informação; coeficientes da expansão local evitam essa ambiguidade.

As funções $f_0 + y f_1$ de peso limitado formam um espaço vectorial de dimensão aproximadamente $d$, e exactamente $d$ para $d \ge 1$. Para $d = 3$, a base é $1, x, y$; para $d = 4$, é $1, x, x^2, y$. As condições de zeros constituem equações lineares nos coeficientes, quando os lugares e as condições locais são conhecidos.

No exemplo binário de $3P$, o peso três força uma recta. Uma recta por $P$ e $2P$ só admite como terceiro encontro $-Q$. O limite da função transforma a interpolação numa afirmação sobre a soma. Num protocolo, esta ligação entre limites, multiplicidades e pontos tem de ser conservada.

## 3.14 Avaliar uma função num divisor

Podemos substituir muitas avaliações individuais por uma identidade entre produtos. Para um divisor $D = \sum_A m_A[A]$, define-se

$$f(D) = \prod_A f(A)^{m_A},$$

desde que $f$ não tenha zeros nem pólos no suporte de $D$. Os expoentes negativos significam inversas multiplicativas. A expressão depende das multiplicidades, tal como o divisor.

A reciprocidade de Weil afirma, para funções racionais não nulas com divisores de suportes disjuntos,

$$f(\operatorname{div} g) = g(\operatorname{div} f).$$

Do lado esquerdo, avaliamos $f$ nos zeros e pólos de $g$. Do lado direito, avaliamos $g$ nos zeros e pólos de $f$. A identidade permite escolher $g$ simples e deslocar para essa função as numerosas avaliações da lista que queremos certificar.

Para visualizar a troca, suponhamos que os zeros de $f$ são $P_1, \ldots, P_d$ e que $g$ tem apenas três zeros. Avaliar uma função complicada em três lugares pode ser mais económico do que avaliá-la nos $d$ lugares. Do outro lado ficam $d$ avaliações de uma função simples. A reciprocidade não elimina essas parcelas; muda a sua natureza.

Há, contudo, um obstáculo que a fórmula curta não resolve. A nossa função $f$ é polinomial na curva, e queremos que $g$ seja a função de uma recta. Ambas têm um pólo em $\mathcal{O}_\infty$. Os suportes não são disjuntos. Substituir ingenuamente os valores no infinito produziria expressões sem sentido.

Precisamos de conservar a contribuição local nesse ponto antes de cancelar as ordens. A regularização da próxima página fará precisamente isso. Esta precaução é necessária para obter uma identidade exacta, com o sinal e os coeficientes correctos. Uma igualdade vaga de produtos esconderia a parte da contabilidade que já aprendemos a não omitir.

## 3.15 A contribuição local e o resultante elíptico

Se num lugar $A$ escrevemos $f = t^{m_f} u_f$ e $g = t^{m_g} u_g$, as unidades $u_f, u_g$ têm valores finitos e não nulos. O símbolo local guarda o valor depois do cancelamento das potências do parâmetro:

$$\{f, g\}_A = (-1)^{m_f m_g} \frac{u_f(A)^{m_g}}{u_g(A)^{m_f}}, \qquad \prod_A \{f, g\}_A = 1. \tag{3.10}$$

A segunda igualdade é a forma geral da reciprocidade. Quando os suportes são disjuntos, recupera a identidade da página anterior. Quando coincidem, permite tratar a contribuição comum sem avaliar um zero ou um pólo como se fosse um valor ordinário.

Para polinómios na curva, ponhamos $\deg_\infty(f) = -\operatorname{ord}_{\mathcal{O}_\infty}(f)$. O coeficiente do monómio de maior peso no representante reduzido chama-se $\operatorname{lc}(f)$. Escolhendo $t = x/y$ no infinito, os termos líderes de $x$ e $y$ são $t^{-2}$ e $t^{-3}$, com coeficiente um. Esta escolha torna explícita a correspondência com o peso elíptico.

Se os suportes finitos de $f, g$ forem disjuntos, o resultante elíptico admite as duas expressões

$$\begin{aligned} \operatorname{Res}_{\mathbb{E}}(f, g) &= \operatorname{lc}(g)^{\deg_\infty(f)} \prod_{\substack{g(A)=0 \\ A \neq \mathcal{O}_\infty}} f(A)^{\operatorname{ord}_A(g)} \\ &= (-1)^{\deg_\infty(f)\deg_\infty(g)} \operatorname{lc}(f)^{\deg_\infty(g)} \prod_{\substack{f(A)=0 \\ A \neq \mathcal{O}_\infty}} g(A)^{\operatorname{ord}_A(f)}. \end{aligned} \tag{3.11}$$

O sinal e os coeficientes líderes são a contribuição do infinito. Esta forma permite a troca de avaliações que procurávamos, respeitando o pólo comum. O resultante pode ainda ser definido quando há zeros comuns, mas as razões que usaremos exigem desafios fora desses casos degenerados.

É nesta identidade regularizada que assenta o método de certificação por avaliações desafiadas [33]. O teorema dos divisores certifica a soma; o resultante oferece uma maneira de testar a função testemunha contra a lista de pontos.

## 3.16 Uma recta desafiada interroga a testemunha

Depois de fixar a função testemunha, escolhemos a recta

$$\mathcal{L}(x, y) = y - c_{\text{declive}}x - c_{\text{corte}}, \qquad Z(x, y) = y - c_{\text{declive}}x. \tag{3.12}$$

Chamemos $B_0, B_1, B_2$ aos seus encontros com a curva. Para usar avaliações no corpo base, o procedimento de desafio tem de fornecer esses pontos no corpo. Uma construção possível escolhe dois pontos distintos, não inversos, e toma o terceiro encontro da recta por eles. Escolher dois coeficientes uniformes não garante que os três encontros sejam racionais.

Suponhamos encontros simples e nenhum zero comum finito com $f$. Se $\operatorname{div}(f) = \sum_j m_j[P_j] - d[\mathcal{O}_\infty]$, com $m_j > 0$ e $\sum_j m_j = d$, o resultante dá

$$\prod_{r=0}^{2} f(B_r) = (-1)^{3d} \operatorname{lc}(f)^3 \prod_j \mathcal{L}(P_j)^{m_j}.$$

Como $\mathcal{L}(P_j) = Z(P_j) - c_{\text{corte}}$, os sinais cancelam-se:

$$\prod_{r=0}^{2} f(B_r) = \operatorname{lc}(f)^3 \prod_j (c_{\text{corte}} - Z(P_j))^{m_j}.$$

A função trabalhosa aparece em três avaliações. A lista dos zeros aparece através de avaliações da função linear $Z$.

Por que escolher o declive depois de fixar a testemunha? Uma verificação que usasse só a coordenada $x$ confundiria $P$ e $-P$. Já $Z(P) - Z(P') = (y_P - y_{P'}) - c_{\text{declive}}(x_P - x_{P'})$ depende do declive. Para pontos distintos com abscissas diferentes, há um único declive que causa colisão; com abscissas iguais e ordenadas diferentes, não há nenhum. Um desafio imprevisível dificulta a substituição dos lugares.

Uma avaliação isolada não é uma equivalência determinística. Uma testemunha falsa pode coincidir num desafio particular. A solidez probabilística depende dos limites de grau e da distribuição dos desafios, escolhidos depois de a testemunha ficar vinculada. Esses elementos pertencem ao argumento.

## 3.17 Dos produtos às somas: a derivada logarítmica

Os produtos anteriores podem ser convertidos em somas de razões. Para uma função formal $H(c) \neq 0$, escrevemos

$$\frac{\mathrm{d}}{\mathrm{d}c} \log H(c) = \frac{H'(c)}{H(c)}.$$

Esta notação não invoca logaritmos reais num corpo finito. É uma abreviatura algébrica, cuja regra de produto segue da derivação formal: $(UV)'/(UV) = U'/U + V'/V$.

Mantendo fixo $c_{\text{declive}}$ e derivando em $c = c_{\text{corte}}$, os factores constantes do resultante desaparecem. Os encontros $B_r(c)$ movem-se formalmente com a recta. Obtemos

$$\sum_{r=0}^{2} \frac{\mathrm{d}}{\mathrm{d}c} \log f(B_r(c)) = \sum_{j} \frac{m_j}{c - Z(P_j)}. \tag{3.13}$$

Esta igualdade vale como identidade de funções racionais fora dos lugares onde os denominadores se anulam. Mesmo que os encontros individuais precisem de uma extensão para ser escritos como funções de $c$, a soma simétrica provém do resultante e está definida sobre o corpo base.

Para o divisor binário da multiplicação escalar, o lado direito é

$$\sum_{i} \frac{e_i}{c - Z(A_i)} + \frac{1}{c - Z(-W)},$$

quando $W$ é afim. Os dígitos entram linearmente; as parcelas da tabela pública podem ser preparadas fora do circuito. A parcela de $-W$ liga a identidade ao ponto resultante que se pretende certificar.

A derivada elimina também a escala global: substituir $f$ por $\alpha f$, com $\alpha \neq 0$, não altera $f'/f$. Isto é coerente com $\operatorname{div}(\alpha f) = \operatorname{div}(f)$. A função zero continua excluída, porque essas razões não existem para ela.

Em característica $p$, as multiplicidades da soma são lidas no corpo, logo módulo $p$. Por exemplo, a derivada de uma potência $p$-ésima é zero. Limites de grau muito inferiores à característica e restrições nos dígitos impedem que este fenómeno apague a informação pretendida. A derivada é uma ferramenta de verificação sob essas condições.

## 3.18 Derivar ao longo da curva

Para calcular o lado esquerdo, não basta derivar $f$ em $x$ mantendo $y$ fixo. O ponto permanece simultaneamente na curva e na recta. Escrevamos $B(c) = (x(c), y(c))$, com declive fixo $m = c_{\text{declive}}$. As equações dão

$$2y\frac{\mathrm{d}y}{\mathrm{d}c} = (3x^2 + a_{\mathbb{E}})\frac{\mathrm{d}x}{\mathrm{d}c}, \qquad \frac{\mathrm{d}y}{\mathrm{d}c} - m\frac{\mathrm{d}x}{\mathrm{d}c} = 1.$$

Resolvendo o sistema e escrevendo $\Delta = 3x^2 + a_{\mathbb{E}} - 2my$, obtemos, quando $\Delta \neq 0$,

$$\frac{\mathrm{d}x}{\mathrm{d}c} = \frac{2y}{\Delta}, \qquad \frac{\mathrm{d}y}{\mathrm{d}c} = \frac{3x^2 + a_{\mathbb{E}}}{\Delta}.$$

Se $f_x, f_y$ são as derivadas parciais do representante reduzido, a regra da cadeia fornece

$$\frac{\mathrm{d}}{\mathrm{d}c}\log f(B(c)) = \frac{2yf_x(B) + (3x^2 + a_{\mathbb{E}})f_y(B)}{f(B)(3x^2 + a_{\mathbb{E}} - 2my)}.$$

O numerador guarda a derivação ao longo da curva; o denominador inclui tanto o valor de $f$ como a condição de transversalidade da recta.

Podemos conferir a identidade em pequeno. Para $f = y - 2x + 9$, a recta $y = 6$ encontra a curva em $(0, 6), (7, 6), (10, 6)$. As três razões calculadas pela fórmula são, respectivamente, $14, 7, 2$. A soma é 6 em $\mathbb{F}_{17}$. Do lado dos zeros $P, 2P, -Q$, obtemos

$$\frac{1}{6-1} + \frac{1}{6-3} + \frac{1}{6-11} = 7 + 6 + 10 = 6 \pmod{17}.$$

A recta $y = 1$, pelo contrário, passa pelo zero $P$: aí uma razão teria denominador zero. Nem toda a recta serve para este teste.

**Verificação de compreensão.** Que condição geométrica corresponde a $\Delta = 0$, num ponto com $y \neq 0$? O declive da recta coincide com $(3x^2 + a_{\mathbb{E}})/(2y)$, o declive da tangente. Os encontros deixam de ser simples. As tangências eram úteis para construir a função; esta fórmula de avaliação desafiada exige evitá-las.

## 3.19 O que as restrições realmente verificam

Para cada encontro desafiado $B_r$, introduz-se a razão $o_r$ e impõe-se uma igualdade multiplicativa:

$$\begin{aligned} p^{(r)}_{\text{den}} o_r &= p^{(r)}_{\text{num}}, \\ p^{(r)}_{\text{num}} &= 2y_{B_r} f_x(B_r) + (3x^2_{B_r} + a_{\mathbb{E}}) f_y(B_r), \\ p^{(r)}_{\text{den}} &= f(B_r)(3x^2_{B_r} + a_{\mathbb{E}} - 2c_{\text{declive}} y_{B_r}). \end{aligned} \tag{3.14}$$

Esta restrição representa a divisão quando o denominador é não nulo. Se numerador e denominador forem ambos zero, aceita qualquer $o_r$. A multiplicação cruzada, sozinha, não demonstra a existência da razão.

Para um denominador secreto cuja inversa deve existir, pode impor-se explicitamente $du = 1$. Na parcela do ponto resultante,

$$d_W = c_{\text{corte}} + y_W + c_{\text{declive}} x_W, \qquad d_W d_W^{-1} = 1. \tag{3.15}$$

Para denominadores públicos, o verificador pode testar a não nulidade e calcular as inversas fora do circuito. Um protocolo que use apenas multiplicação cruzada em certas parcelas tem de incluir os casos degenerados na sua análise probabilística.

A identidade final compara $\sum_r o_r$ com a soma das parcelas prescritas pelo divisor. Porém, as avaliações de $f, f_x, f_y$ têm de vir dos mesmos coeficientes, já fixados, e os limites de peso têm de ser impostos. A variável $W$ deve satisfazer a equação da curva; os dígitos devem satisfazer as condições da decomposição. Valores soltos que satisfaçam uma igualdade final não bastam.

Nos argumentos do tipo Eagen, desafios imprevisíveis e limites de grau controlam a probabilidade de uma testemunha inválida passar pelos testes. A contagem de raízes de polinómios não nulos explica a economia: uma diferença algébrica de grau limitado só se pode anular num conjunto limitado de desafios. A distribuição concreta de rectas e as colisões de $Z$ entram igualmente nessa análise. A solidez pertence ao conjunto das condições, não a uma única igualdade de razões.

## 3.20 Da função ao certificado em conhecimento zero

Podemos finalmente reunir as peças. A afirmação é $W = s_A A$. O provador calcula $W$, decompõe $s_A$, constrói o divisor prescrito e encontra os coeficientes de uma função não nula com o peso permitido. A função é uma testemunha da soma nula; não precisa de ser publicada.

Para manter valores secretos, o protocolo exterior usa compromissos: objectos públicos que vinculam o provador a valores, permitindo que esses valores permaneçam ocultos. As suas construções e provas serão desenvolvidas nos capítulos seguintes. Aqui importa a ordem das operações: primeiro vincular pontos, dígitos e coeficientes; depois obter os desafios; por fim provar as restrições das avaliações e da identidade.

O conhecimento zero não resulta apenas da presença de divisores. Publicar os coeficientes pode revelar os zeros da função e, com eles, os pontos seleccionados. O sistema de prova envolvente é responsável por esconder a testemunha e por garantir que todas as restrições se referem aos mesmos valores comprometidos.

Uma normalização exclui a função nula, por exemplo fixando a um um coeficiente que se saiba ser não nulo. Multiplicar uma função por uma constante não nula preserva o divisor e a derivada logarítmica. Mas escolher de antemão um coeficiente específico pode excluir testemunhas verdadeiras quando esse coeficiente se anula. Uma estratégia de normalização deve tratar esse caso para assegurar a completude. O coeficiente fixado não é necessariamente o coeficiente líder de peso elíptico do resultante.

Em FCMP++, esta maquinaria certificará multiplicações ocultas nas aberturas de pontos e de ramos. O capítulo 11 identificará as relações concretas e a contagem das funções. A árvore fornece os pontos sobre os quais se fazem alegações; as funções testemunham as multiplicações necessárias para ligar as aberturas.

O fio matemático pode agora ser seguido sem saltos: a decomposição dá uma soma de pontos; essa soma determina um divisor de grau zero; a função testemunha que o divisor é principal; a reciprocidade liga a função aos lugares prescritos; as avaliações desafiadas tornam a verificação económica. Os limites e as condições de não nulidade conservam a validade de cada elo.

# CAPÍTULO 4

# Assinaturas tipo Schnorr avançadas

Entre as ferramentas do capítulo anterior, construímos uma prova de Schnorr a partir de uma só relação $K = kG$. Podemos agora ligar várias relações ao mesmo desafio, esconder a posição da relação verdadeira num anel e, por fim, tornar as assinaturas ligáveis. SAG e bLSAG servirão de antepassados conceptuais; MLSAG explica a evolução histórica de RingCT; CLSAG é a construção usada pelas transacções activas na versão 16 do protocolo. Esta ordem apresenta a extensão progressiva do mecanismo sem confundir história com consenso actual.

Em todo o capítulo trabalhamos no subgrupo de ordem prima $l$ e escrevemos $\mathbb{Z}_l$ para os escalares. As listas passadas a uma função hash têm uma codificação canónica e não ambígua: incluem a ordem e o número dos seus elementos. $\mathcal{H}_n$ designa uma função hash para escalares e $\mathcal{H}_p$, quando aparecer, uma função hash para pontos. São operações diferentes.

## 4.1 Igualdade do logaritmo discreto em várias bases

Suponhamos que Alice publica $K_1 = kJ_1$ e $K_2 = kJ_2$. O primeiro ponto pode ser uma chave pública $kG$, enquanto o segundo pode ser o segredo Diffie–Hellman $kR$ da secção 2.3.3. Alice quer provar duas coisas ao mesmo tempo: conhece o logaritmo discreto e esse logaritmo é o mesmo nas duas bases. Uma resposta Schnorr comum às duas relações faz precisamente esse trabalho.

## Declaração e testemunha

Seja $d \geq 1$. A declaração pública, escrita como listas ordenadas, é

$$\mathcal{J} = (J_1, \ldots, J_d), \qquad \mathcal{K} = (K_1, \ldots, K_d).$$

A testemunha privada é um único $k \in \mathbb{Z}_l$ tal que

$$K_i = kJ_i \qquad \text{para todo } i \in \{1, \ldots, d\}.$$

Cada $J_i$ e $K_i$ deve ter uma codificação válida, pertencer ao subgrupo de ordem $l$ e não ser a identidade. Sem estas condições, uma base nula poderia tornar uma das relações vazia de conteúdo.

A transformação Fiat–Shamir inclui ainda um rótulo de domínio e um contexto $x$. Numa assinatura, $x$ contém a mensagem $M$; numa prova destinada a outro protocolo, contém a sessão e os objectos a que a prova deve ficar presa.[1]

Assumimos que $\mathcal{H}_n$ transforma essa codificação em $\mathbb{Z}_l$. Não se exige que cada entrada tenha uma imagem diferente; uma função hash tem colisões em princípio. A análise usa-a como oráculo aleatório e a segurança computacional exige que seja impraticável encontrar colisões ou prever desafios.[2]

## Prova não interactiva

1. Escolhe-se $\alpha \in_R \mathbb{Z}_l$ e calculam-se os compromissos $A_i = \alpha J_i$, para $i = 1, \ldots, d$.
2. Calcula-se o desafio
$$c = \mathcal{H}_n(\mathsf{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A_1], \ldots, [A_d]).$$
3. Calcula-se a resposta
$$r = (\alpha - ck) \bmod l.$$
4. Publica-se a prova $(c, r)$. Os compromissos não precisam de ser publicados separadamente, porque o verificador os reconstrói.

## Verificação

O verificador rejeita primeiro pontos ou escalares mal codificados, pontos fora do subgrupo, identidades e dimensões incoerentes. Depois calcula

$$A'_i = rJ_i + cK_i, \qquad i = 1, \ldots, d,$$
$$c' = \mathcal{H}_n(\mathsf{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A'_1], \ldots, [A'_d])$$

e aceita apenas se $c' = c$.

[1] Uma prova deste género torna-se uma assinatura quando a mensagem $M$ é incluída no contexto do desafio.

[2] No código Monero, a operação correspondente calcula Keccak-256 e reduz os 256 bits módulo $l$. O escalar zero pertence legitimamente a $\mathbb{Z}_l$; a segurança não depende de excluir esse resultado, cuja ocorrência é de probabilidade desprezável.

### Funciona porque

Para uma prova honesta, cada reconstrução devolve o compromisso original:

$$\begin{aligned} A'_i &= rJ_i + cK_i \\ &= (\alpha - ck)J_i + c(kJ_i) \\ &= \alpha J_i = A_i. \end{aligned}$$

Logo o verificador recompõe exactamente a mesma entrada de $\mathcal{H}_n$, pelo que

$$\mathcal{H}_n(\mathsf{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A_1], \ldots, [A_d]) = \mathcal{H}_n(\mathsf{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A'_1], \ldots, [A'_d]),$$
$$c = c'.$$

Com $d = 1$, recuperamos a prova de Schnorr da secção 2.3.4. Para $d > 1$, usar o mesmo $\alpha$, o mesmo desafio e a mesma resposta liga as relações. No modelo do oráculo aleatório, duas transcrições aceitáveis com os mesmos compromissos e desafios distintos permitem extrair $k$; esta é a razão formal para falar em prova de conhecimento. A conclusão depende da dificuldade do logaritmo discreto no subgrupo e da validação dos pontos, não apenas da igualdade $c = c'$.

O valor $\alpha$ nunca pode ser reutilizado com outro desafio. De $r = \alpha - ck$ e $r' = \alpha - c'k$, um observador obtém

$$k = (r - r')(c' - c)^{-1} \pmod{l}.$$

Também por isso, qualquer geração determinística do valor efémero deve ficar presa a todo o contexto e a todas as chaves públicas.

## 4.2 Uma prova com múltiplas chaves privadas

A construção anterior reutilizava uma testemunha em várias bases. Podemos fazer o complemento: provar simultaneamente várias relações, cada qual com a sua testemunha, mas reuni-las num só desafio. É uma composição «E» de provas de Schnorr: a prova só é válida quando todas as relações o são.

### Declaração e testemunhas

Para $d \geq 1$, sejam

$$\mathcal{J} = (J_1, \ldots, J_d), \qquad \mathcal{K} = (K_1, \ldots, K_d).$$

A declaração pública afirma que existem testemunhas $(k_1, \ldots, k_d) \in \mathbb{Z}_l^d$ tais que

$$K_i = k_i J_i \qquad \text{para todo } i \in \{1, \ldots, d\}.$$

As bases podem repetir-se, e podem mesmo ser todas iguais a $G$.[3] Tal como antes, o verificador exige listas com a mesma dimensão, codificações canónicas, pontos não nulos no subgrupo de ordem $l$ e escalares canónicos. O contexto público $x$ inclui a mensagem ou a finalidade da prova.

[3] Bases repetidas seriam redundantes numa prova de igualdade de logaritmos, mas não aqui: por exemplo, $K_1 = k_1 G$ e $K_2 = k_2 G$ são duas relações distintas. A construção não afirma que os $k_i$ sejam diferentes.

## Prova não interactiva

1. Escolhem-se valores efémeros independentes $\alpha_i \in_R \mathbb{Z}_l$ e calculam-se
$$A_i = \alpha_i J_i, \qquad i = 1, \ldots, d.$$

2. Calcula-se o único desafio
$$c = \mathcal{H}_n(\mathsf{Schnorr\text{-}AND}, x, \mathcal{J}, \mathcal{K}, [A_1], \ldots, [A_d]).$$

3. Calcula-se uma resposta por testemunha:
$$r_i = (\alpha_i - c k_i) \bmod l, \qquad i = 1, \ldots, d.$$

4. Publica-se
$$(c, r_1, \ldots, r_d).$$

## Verificação

Depois de validar a declaração e a prova, o verificador reconstrói
$$A'_i = r_i J_i + c K_i, \qquad i = 1, \ldots, d,$$
$$c' = \mathcal{H}_n(\mathsf{Schnorr\text{-}AND}, x, \mathcal{J}, \mathcal{K}, [A'_1], \ldots, [A'_d])$$
e aceita apenas se $c' = c$.

## Funciona porque

Para cada índice,
$$\begin{aligned} r_i J_i + c K_i &= (\alpha_i - c k_i) J_i + c(k_i J_i) \\ &= \alpha_i J_i = A_i. \end{aligned}$$
Assim, todos os argumentos reconstruídos são iguais aos argumentos usados pelo provador. Abreviando por $\mathcal{T} = (\mathsf{Schnorr\text{-}AND}, x, \mathcal{J}, \mathcal{K})$ o prefixo comum da transcrição, obtemos
$$\begin{aligned} c' &= \mathcal{H}_n(\mathcal{T}, [A'_1], \ldots, [A'_d]) \\ &= \mathcal{H}_n(\mathcal{T}, [A_1], \ldots, [A_d]) = c. \end{aligned}$$

O desafio partilhado poupa $d - 1$ escalares em comparação com $d$ provas Fiat–Shamir independentes; não junta as testemunhas numa só. No modelo do oráculo aleatório, duas transcrições com os mesmos compromissos e desafios distintos permitem extrair cada
$$k_i = (r_i - r'_i)(c' - c)^{-1} \pmod{l}.$$
Por conseguinte, a alegação de conhecimento de todas as chaves depende da dificuldade do logaritmo discreto e de a transcrição ligar o domínio, o contexto, todas as bases, todas as chaves e todos os compromissos.

Cada $\alpha_i$ deve ser imprevisível e não pode reaparecer sob outro desafio: reutilizar apenas um deles basta para revelar o respectivo $k_i$. Índices trocados, listas truncadas ou uma codificação ambígua mudariam a declaração; por isso, dimensões e ordem são parte do que se autentica.

## 4.3 Assinaturas espontâneas anónimas de grupo

As assinaturas de grupo de Chaum e van Heyst recorriam a uma entidade de confiança para formar o grupo e, em certas variantes, para resolver disputas [25]. Uma assinatura em anel segue outra filosofia: o signatário escolhe espontaneamente um conjunto de chaves públicas e não precisa da autorização nem da colaboração dos respectivos donos. A construção foi introduzida por Rivest, Shamir e Tauman [117]; Liu, Wei e Wong desenvolveram depois a variante ligável LSAG [65]. Começaremos por SAG, sem ligação, porque o seu anel de desafios será reutilizado nas construções seguintes.

### Declaração, testemunha e índice

Seja $M$ a mensagem e seja

$$\mathcal{R} = (K_1, \ldots, K_n), \qquad n \geq 1,$$

um anel ordenado de chaves públicas distintas. O signatário conhece um índice secreto $\pi \in \{1, \ldots, n\}$ e a testemunha $k_\pi \in \mathbb{Z}_l$ que satisfaz

$$K_\pi = k_\pi G.$$

A declaração é uma disjunção: «conheço a chave privada de *algum* membro de $\mathcal{R}$». A posição $\pi$ não faz parte da assinatura. Todas as chaves devem ter codificações canónicas, ser pontos não nulos do subgrupo de ordem $l$, e a ordem e o comprimento do anel ficam presos à transcrição.

Para simplificar os índices, convencionamos que $c_{n+1} = c_1$. A função $\mathcal{H}_n$ recebe um rótulo de domínio, o anel inteiro, a mensagem e o compromisso da ronda. O chamado *prefixo de chave* é precisamente esta inclusão explícita de $\mathcal{R}$ no desafio; impede que a mesma transcrição seja destacada de um anel e apresentada como pertencendo a outro.

### Assinatura

1. Escolhe-se $\alpha \in_R \mathbb{Z}_l$. Para cada $i \neq \pi$, escolhe-se ainda uma resposta simulada $r_i \in_R \mathbb{Z}_l$.

2. Inicia-se a ronda posterior ao signatário:
$$c_{\pi+1} = \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [\alpha G]).$$

3. Percorrem-se, por ordem cíclica, $i = \pi+1, \pi+2, \ldots, n, 1, \ldots, \pi-1$. Em cada posição calcula-se
$$L_i = r_i G + c_i K_i,$$
$$c_{i+1} = \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [L_i]).$$

4. Fecha-se o anel com a única resposta que não foi simulada:
$$r_\pi = (\alpha - c_\pi k_\pi) \bmod l.$$

A assinatura é

$$\sigma = (c_1, r_1, \ldots, r_n).$$

O anel e a mensagem têm de acompanhar a verificação, mas podem estar noutro campo do protocolo; não é necessário repeti-los dentro desta lista de escalares.

## Verificação

O verificador confirma $n \geq 1$, o número exacto de respostas e a canonicidade de $c_1, r_1, \ldots, r_n$, além de validar todas as chaves do anel. Depois põe $\widehat{c}_1 = c_1$ e, para $i = 1, \ldots, n$, calcula

$$\widehat{L}_i = r_i G + \widehat{c}_i K_i,$$

$$\widehat{c}_{i+1} = \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [\widehat{L}_i]).$$

No último passo, $\widehat{c}_{n+1}$ é o desafio reconstruído para a primeira posição. Aceita-se se e só se

$$\widehat{c}_{n+1} = c_1.$$

Comparar o desafio final com o inicial, e não com o desafio da ronda anterior, é o que testa o fecho circular.

## Funciona porque

Nas posições simuladas, o verificador repete literalmente os cálculos do signatário. Na posição verdadeira,

$$\begin{aligned} \widehat{L}_\pi &= r_\pi G + c_\pi K_\pi \\ &= (\alpha - c_\pi k_\pi) G + c_\pi (k_\pi G) \\ &= \alpha G. \end{aligned}$$

Assim, também essa ronda devolve o compromisso inicial e a cadeia inteira fecha.

Vejamos o antigo exemplo de três membros, agora com os índices bem à vista. Se $\pi = 2$, o signatário escolhe $\alpha, r_1, r_3$ e calcula

$$\begin{aligned} c_3 &= \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [\alpha G]), \\ c_1 &= \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [r_3 G + c_3 K_3]), \\ c_2 &= \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [r_1 G + c_1 K_1]), \\ r_2 &= (\alpha - c_2 k_2) \bmod l. \end{aligned}$$

Como

$$r_2 G + c_2 K_2 = (\alpha - c_2 k_2) G + c_2 (k_2 G) = \alpha G,$$

o verificador obtém sucessivamente

$$\begin{aligned} \widehat{c}_2 &= \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [r_1 G + c_1 K_1]), \\ \widehat{c}_3 &= \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [r_2 G + \widehat{c}_2 K_2]), \\ \widehat{c}_1 &= \mathcal{H}_n(\mathsf{SAG}, \mathcal{R}, M, [r_3 G + \widehat{c}_3 K_3]) = c_1. \end{aligned}$$

### O que a assinatura garante

Sob a dificuldade do logaritmo discreto e no modelo do oráculo aleatório, a cadeia prova conhecimento de uma das chaves privadas sem revelar qual. A distribuição das respostas simuladas deve ser indistinguível da resposta verdadeira; valores efémeros enviesados, previsíveis ou reutilizados podem denunciar $\pi$ e, no caso de reutilização, revelar $k_\pi$. A infalsificabilidade exige ainda que mensagem, domínio, anel completo e compromissos sejam ligados ao hash e que chaves malformadas ou de pequena ordem sejam rejeitadas.

O valor $n$ é apenas um limite superior para a cardinalidade do conjunto de anonimidade que a criptografia proporciona. Se os membros forem escolhidos de maneira reconhecível ou se informação externa excluir alguns deles, a ambiguidade efectiva será menor. Para $n = 1$, SAG reduz-se essencialmente a Schnorr e não oferece anonimidade. Finalmente, SAG não é ligável: duas assinaturas válidas não revelam, por si só, se usaram a mesma chave. A imagem de chave da próxima secção acrescentará essa propriedade.

## 4.4 Assinaturas em anel ligáveis de Adam Back

SAG esconde qual membro assinou, mas permite repetir a mesma chave sem deixar um marcador público. Uma assinatura em anel *ligável* acrescenta esse marcador: a imagem de chave. O objectivo não é revelar o signatário; é permitir que qualquer pessoa reconheça uma segunda utilização da mesma chave de uso único.

LSAG [65] ligava assinaturas relativas ao mesmo anel ordenado. A variante que estudaremos, habitualmente chamada bLSAG, torna a imagem independente dos chamarizes escolhidos. A adaptação foi exposta em [104] a partir de uma observação de Adam Back [4] sobre a assinatura CryptoNote [133], por sua vez relacionada com o trabalho de Fujisaki e Suzuki [42]. É um antepassado conceptual: não é a assinatura usada pelas transacções Monero activas.

### Propriedades pretendidas

**Ambiguidade do signatário** Uma assinatura válida demonstra que alguém conhece a chave privada de um membro do anel, sem identificar a posição, salvo com probabilidade desprezável. [4]

**Ligação** Duas assinaturas válidas feitas com a mesma chave pública de uso único produzem a mesma imagem de chave, mesmo que as mensagens e os restantes membros dos anéis sejam diferentes. Reutilizar uma chave apenas como chamariz não cria uma ligação.

[4] O anel dá apenas um limite superior para o conjunto de anonimidade. Chamarizes escolhidos de forma reconhecível, informação temporal ou análise de outras transacções podem excluir membros; aumentar $n$ não transforma automaticamente todos os membros em candidatos igualmente plausíveis.

**Infalsificabilidade** Sem conhecer a chave privada de algum membro, um adversário não deve conseguir produzir uma assinatura válida, mesmo depois de escolher mensagens e anéis e observar assinaturas legítimas.

Estas propriedades são afirmações criptográficas no modelo do oráculo aleatório, sob as hipóteses indicadas adiante. Uma imagem repetida não prova, sozinha, propriedade civil, intenção nem validade de um gasto: é preciso validar a assinatura e todas as restantes regras do protocolo.

## Hash para ponto e imagem de chave

Além de $\mathcal{H}_n$, que devolve escalares, precisamos de

$$\mathcal{H}_p : \{0,1\}^* \longrightarrow \mathbb{G},$$

onde $\mathbb{G}$ é o subgrupo de ordem prima $l$. O resultado deve ser um ponto não nulo e a sua relação de logaritmo discreto com $G$ não deve ser conhecida. Não bastaria calcular $\mathcal{H}_n(K)G$: como esse multiplicador é público, qualquer observador obteria $\mathcal{H}_n(K)K$ sem conhecer a chave privada. [5]

Para o membro verdadeiro $K_\pi = k_\pi G$, define-se

$$I = k_\pi \mathcal{H}_p(K_\pi).$$

$I$ é a imagem de chave. Sendo determinística para a relação $(K_\pi, k_\pi)$, reaparece quando essa chave de uso único volta a assinar; como a base depende de $K_\pi$, chaves de uso único diferentes da mesma carteira não ficam linearmente ligadas.

## Declaração, testemunha e assinatura

Sejam a mensagem $M$, o anel ordenado $\mathcal{R} = (K_1, \ldots, K_n)$, com $n \geq 1$, o índice secreto $\pi \in \{1, \ldots, n\}$ e a testemunha $k_\pi$ tais que $K_\pi = k_\pi G$. Convencionamos novamente $c_{n+1} = c_1$.

1. Calcula-se a imagem $I = k_\pi \mathcal{H}_p(K_\pi)$.
2. Escolhem-se $\alpha \in_R \mathbb{Z}_l$ e $r_i \in_R \mathbb{Z}_l$ para todo $i \neq \pi$.
3. Inicia-se a ronda seguinte ao signatário:
$$c_{\pi+1} = \mathcal{H}_n(\mathsf{bLSAG}, \mathcal{R}, I, M, [\alpha G], [\alpha \mathcal{H}_p(K_\pi)]).$$
4. Para $i = \pi + 1, \ldots, n, 1, \ldots, \pi - 1$, calculam-se
$$\begin{aligned} L_i &= r_i G + c_i K_i, \\ R_i &= r_i \mathcal{H}_p(K_i) + c_i I, \\ c_{i+1} &= \mathcal{H}_n(\mathsf{bLSAG}, \mathcal{R}, I, M, [L_i], [R_i]). \end{aligned}$$

[5] A família Monero historicamente mapeia a codificação da chave para um ponto e limpa o cofactor; veja-se a descrição em [103] e a origem citada em [131]. Hash para ponto e hash para escalar não são operações permutáveis.

5. Fecha-se o anel com

$$r_\pi = (\alpha - c_\pi k_\pi) \bmod l.$$

A assinatura é

$$\sigma = (I, c_1, r_1, \ldots, r_n).$$

Incluímos $\mathcal{R}$ e $I$ no prefixo de cada desafio. A apresentação original de bLSAG omitia o anel nesse hash; o prefixo explícito é a práctica prudente para assinaturas deste género e evita depender de uma interpretação implícita da transcrição [59].

## Verificação

O verificador confirma as dimensões, valida todos os escalares e chaves públicas e exige

$$I \neq 0 \qquad \text{e} \qquad lI = 0.$$

Depois põe $\widehat{c}_1 = c_1$ e, para $i = 1, \ldots, n$, calcula

$$\begin{aligned}
\widehat{L}_i &= r_i G + \widehat{c}_i K_i, \\
\widehat{R}_i &= r_i \mathcal{H}_p(K_i) + \widehat{c}_i I, \\
\widehat{c}_{i+1} &= \mathcal{H}_n(\mathsf{bLSAG}, \mathcal{R}, I, M, [\widehat{L}_i], [\widehat{R}_i]).
\end{aligned}$$

Aceita apenas se

$$\widehat{c}_{n+1} = c_1.$$

A condição $lI = 0$, acompanhada da rejeição da identidade, prova que a imagem pertence ao subgrupo de ordem $l$. Isto não é o mesmo que multiplicar simplesmente a equação de verificação pelo cofactor. Se se aceitasse $I + T$, com $T$ um ponto de pequena ordem $t$, um atacante poderia explorar a componente de torção e procurar desafios com resíduos convenientes módulo $t$, criando variantes que escapassem à ligação. Para um desafio divisível por $t$, por exemplo,

`src/cryptonote_core/cryptonote_core.cpp check_tx_inputs_keyimages_domain()`

$$\begin{aligned}
c(I + T) &= cI + cT \\
&= cI.
\end{aligned}$$

A verificação de subgrupo elimina todas essas variantes. Esta falha existiu nos primeiros anos do Monero e foi corrigida antes de ser explorada, nas regras da versão 5 [41].

## Funciona porque

Na posição verdadeira, as duas reconstruções devolvem os compromissos de $\alpha$:

$$\begin{aligned}
\widehat{L}_\pi &= r_\pi G + c_\pi K_\pi \\
&= (\alpha - c_\pi k_\pi) G + c_\pi (k_\pi G) = \alpha G, \\
\widehat{R}_\pi &= r_\pi \mathcal{H}_p(K_\pi) + c_\pi I \\
&= (\alpha - c_\pi k_\pi) \mathcal{H}_p(K_\pi) + c_\pi k_\pi \mathcal{H}_p(K_\pi) \\
&= \alpha \mathcal{H}_p(K_\pi).
\end{aligned}$$

As outras rondas foram simuladas exactamente como serão verificadas; portanto a cadeia fecha no $c_1$ publicado.

### Ligação e hipóteses de segurança

Dadas duas assinaturas válidas

$$\sigma = (I, c_1, r_1, \ldots, r_n),$$
$$\sigma' = (I', c'_1, r'_1, \ldots, r'_{n'}),$$

o teste de ligação é simplesmente $I = I'$. Para uma mesma chave de uso único, ambas calculam

$$I = k_\pi \mathcal{H}_p(K_\pi),$$

logo a igualdade é inevitável. No sentido inverso, concluir que imagens iguais correspondem à mesma relação depende de a função hash para ponto se comportar como um oráculo aleatório no subgrupo, de serem impraticáveis colisões e logaritmos discretos e de todos os pontos terem sido validados.

Há uma hipótese distinta por trás da ambiguidade. Para testar se o candidato $K_i = k_i G$ corresponde à imagem $I = k_i \mathcal{H}_p(K_i)$, um observador teria de decidir se

$$(G, K_i, \mathcal{H}_p(K_i), I)$$

é um quádruplo Diffie–Hellman. A dificuldade deste problema decisional impede o teste. O PCDH pede antes *calcular* $I$, e o PLD pede recuperar $k_i$. Um algoritmo para o PLD resolveria também os dois problemas Diffie–Hellman, mas a mera dificuldade conhecida do PLD não demonstra, em geral, a dificuldade decisional ou computacional.

Mesmo quando duas assinaturas estão ligadas, a imagem não revela qual membro assinou. Se os dois anéis tiverem apenas uma chave em comum, porém, a intersecção denuncia essa chave; é uma perda de anonimidade causada pelo contexto dos anéis, não uma quebra da equação. A infalsificabilidade exige, além do logaritmo discreto, prefixos completos, separação de domínio e resistência do Fiat–Shamir no modelo do oráculo aleatório. Chaves públicas malformadas, pontos de torção, respostas não canónicas e comprimentos incoerentes têm de ser rejeitados antes do fecho da cadeia.

Como em Schnorr e SAG, reutilizar $\alpha$ sob desafios diferentes revela a testemunha. Uma fonte de aleatoriedade defeituosa pode também distinguir a posição verdadeira das respostas simuladas. A assinatura guarda $n + 1$ escalares e uma imagem de chave, além do anel público.

## 4.5 Assinaturas em anel ligáveis com múltiplas camadas

MLSAG generaliza bLSAG a uma matriz de chaves [104]. Foi uma peça central das primeiras transacções RingCT, pelo que continua a ser necessária para compreender dados históricos e a evolução do protocolo. Não é, contudo, a assinatura criada nem admitida para novas transacções na versão 16: CLSAG foi admitida na versão 13 e tornou-se obrigatória a partir da versão 14 na revisão Monero aqui consultada.

## Matriz, declaração e testemunhas

Seja $M$ a mensagem. Fixemos a orientação antes de escrever índices. Há $n \geq 2$ colunas, uma por membro do anel, e $m \geq 1$ linhas, uma por relação que o signatário deve provar. Escrevemos

$$\mathcal{R} = (K_{i,j})_{\substack{1 \leq i \leq n \\ 1 \leq j \leq m}}.$$

O primeiro índice $i$ escolhe a *coluna*; o segundo $j$, a *linha*. A declaração pública é que existe uma coluna secreta $\pi \in \{1, \ldots, n\}$ para a qual o signatário conhece todas as testemunhas

$$(k_{\pi,1}, \ldots, k_{\pi,m}) \in \mathbb{Z}_l^m, \qquad K_{\pi,j} = k_{\pi,j} G.$$

Portanto, o conjunto de anonimidade tem no máximo $n$ colunas, não $nm$ chaves independentes. Todas as relações verdadeiras partilham o mesmo índice $\pi$; se uma linha denunciar a coluna, denuncia-as a todas.

Na versão integralmente ligável que apresentamos, cada linha tem uma imagem

$$I_j = k_{\pi,j} \mathcal{H}_p(K_{\pi,j}), \qquad j = 1, \ldots, m.$$

Uma variante permite tornar ligáveis apenas $q \leq m$ linhas: essas linhas têm termos $R_{i,j}$ e imagens $I_j$; as restantes têm apenas termos $L_{i,j}$. A fórmula abaixo corresponde a $q = m$, que torna visível a estrutura sem esconder casos num índice adicional.

O verificador exige uma matriz rectangular $n \times m$, pontos canónicos, não nulos e no subgrupo de ordem $l$, imagens também não nulas no mesmo subgrupo e exactamente $nm$ respostas. Convencionamos $c_{n+1} = c_1$ e abreviamos o prefixo completo por

$$\Phi = (\mathsf{MLSAG}, n, m, \mathcal{R}, I_1, \ldots, I_m, M).$$

Esta codificação liga o domínio, as dimensões, cada posição da matriz, as imagens de chave e a mensagem.

## Assinatura

1. Calculam-se as imagens
$$I_j = k_{\pi,j} \mathcal{H}_p(K_{\pi,j}), \qquad j = 1, \ldots, m.$$

2. Escolhem-se valores independentes $\alpha_j \in_R \mathbb{Z}_l$, para todas as linhas, e respostas $r_{i,j} \in_R \mathbb{Z}_l$, para todas as posições com $i \neq \pi$.

3. Inicia-se a ronda seguinte à coluna signatária:
$$c_{\pi+1} = \mathcal{H}_n\big(\Phi, [\alpha_1 G], [\alpha_1 \mathcal{H}_p(K_{\pi,1})], \ldots, [\alpha_m G], [\alpha_m \mathcal{H}_p(K_{\pi,m})]\big).$$

4. Para as colunas $i = \pi + 1, \ldots, n, 1, \ldots, \pi - 1$, calculam-se, em cada linha,
$$L_{i,j} = r_{i,j} G + c_i K_{i,j},$$
$$R_{i,j} = r_{i,j} \mathcal{H}_p(K_{i,j}) + c_i I_j, \qquad j = 1, \ldots, m,$$
e encadeia-se
$$c_{i+1} = \mathcal{H}_n\big(\Phi, [L_{i,1}], [R_{i,1}], \ldots, [L_{i,m}], [R_{i,m}]\big).$$

5. Na coluna verdadeira, definem-se
$$r_{\pi,j} = (\alpha_j - c_\pi k_{\pi,j}) \bmod l, \qquad j = 1, \ldots, m.$$

A assinatura é
$$\sigma = (I_1, \ldots, I_m, c_1, r_{1,1}, \ldots, r_{1,m}, \ldots, r_{n,1}, \ldots, r_{n,m}).$$

## Verificação

Após as validações de pontos, escalares e dimensões, põe-se $\widehat{c}_1 = c_1$. Para cada coluna $i = 1, \ldots, n$ e linha $j = 1, \ldots, m$, calculam-se
$$\widehat{L}_{i,j} = r_{i,j}G + \widehat{c}_i K_{i,j},$$
$$\widehat{R}_{i,j} = r_{i,j}\mathcal{H}_p(K_{i,j}) + \widehat{c}_i I_j.$$
O desafio seguinte é
$$\widehat{c}_{i+1} = \mathcal{H}_n\big(\Phi, [\widehat{L}_{i,1}], [\widehat{R}_{i,1}], \ldots, [\widehat{L}_{i,m}], [\widehat{R}_{i,m}]\big).$$
Aceita-se apenas se o desafio depois da última coluna fechar a cadeia:
$$\widehat{c}_{n+1} = c_1.$$

## Funciona porque

Nas colunas simuladas, os valores reconstruídos são os que o signatário usou. Na coluna $\pi$, para cada $j$,
$$\begin{aligned}
\widehat{L}_{\pi,j} &= r_{\pi,j}G + c_\pi K_{\pi,j} \\
&= (\alpha_j - c_\pi k_{\pi,j})G + c_\pi(k_{\pi,j}G) = \alpha_j G, \\
\widehat{R}_{\pi,j} &= r_{\pi,j}\mathcal{H}_p(K_{\pi,j}) + c_\pi I_j \\
&= (\alpha_j - c_\pi k_{\pi,j})\mathcal{H}_p(K_{\pi,j}) + c_\pi k_{\pi,j}\mathcal{H}_p(K_{\pi,j}) \\
&= \alpha_j \mathcal{H}_p(K_{\pi,j}).
\end{aligned}$$
O verificador recompõe, linha a linha e na mesma ordem, o compromisso que iniciou $c_{\pi+1}$. Uma só resposta errada basta para alterar esse desafio e impedir o fecho.

## Ligação, anonimidade e infalsificabilidade

Se alguma testemunha $k_{\pi,j}$ for reutilizada para a mesma chave pública de uso único, a imagem $I_j$ reaparece e liga as assinaturas. Uma chave que surja apenas como chamariz não produz imagem e não cria ligação. Tal como em bLSAG, anéis cuja intersecção seja pequena podem transformar uma ligação numa forte pista sobre $\pi$; a criptografia não corrige um conjunto de anonimidade mal formado.

A infalsificabilidade exige conhecimento de *todas* as testemunhas da coluna escolhida, sob a dificuldade do PLD e no modelo do oráculo aleatório. A associação de uma imagem $I_j$ ao

candidato $K_{i,j}$ forma uma instância decisional de Diffie–Hellman $(G, K_{i,j}, \mathcal{H}_p(K_{i,j}), I_j)$; por isso, a ambiguidade requer que esse teste decisional seja difícil. Esta hipótese não deve ser confundida com o PCDH, que pede calcular o quarto ponto, nem com o PLD, que pede recuperar o escalar. Em grupos gerais, uma destas dificuldades não prova automaticamente as outras.

Imagens e chaves fora do subgrupo, identidades, matrizes não rectangulares, ordens de linhas divergentes, desafios ou respostas não canónicos e prefixos incompletos invalidam o argumento de segurança. Reutilizar $\alpha_j$ sob desafios diferentes revela $k_{\pi,j}$, mesmo que os outros valores efémeros sejam novos.

### Requisitos de espaço

Na variante com todas as linhas ligáveis, guardam-se $mn + 1$ escalares e $m$ imagens de chave; a matriz contém $mn$ chaves públicas. A assinatura cresce, portanto, com o produto das linhas pelas colunas. CLSAG conservará um único ciclo de $n$ respostas ao agregar as relações de cada coluna.

## 4.6 Assinaturas em anel ligáveis concisas

CLSAG, de *concise linkable spontaneous anonymous group signature*, conserva a coluna secreta de MLSAG mas agrega as relações dessa coluna antes de percorrer o anel [43]. Em vez de publicar uma resposta por linha e por coluna, publica uma resposta por coluna. É esta a assinatura em anel usada pelas transacções activas na versão 16 do Monero. [6]

### Chaves primárias e auxiliares

Seja $M$ a mensagem. Voltamos à matriz

$$\mathcal{R} = (K_{i,j})_{\substack{1 \le i \le n \\ 1 \le j \le m}},$$

com $n \ge 1$ colunas e $m \ge 1$ linhas. A linha $j = 1$ contém as chaves *primárias*; as linhas $j > 1$, as chaves *auxiliares*. O signatário conhece uma coluna secreta $\pi$ e todas as testemunhas

$$K_{\pi,j} = k_{\pi,j} G, \qquad j = 1, \ldots, m.$$

A ligação deve depender apenas da chave primária $k_{\pi,1}$; as relações auxiliares têm de ser provadas, mas não devem criar marcadores de gasto independentes.

Para cada coluna, define-se uma única base de imagem

$$H_i = \mathcal{H}_p(K_{i,1}).$$

[6] No código consultado, CLSAG é admitida desde a versão 13 do protocolo e obrigatória para novas transacções RingCT desde a versão 14. MLSAG continua necessária para interpretar formatos históricos, mas não é válida para uma nova transacção v16.

As imagens da coluna verdadeira são

$$I_j = k_{\pi,j} H_\pi, \qquad j = 1, \ldots, m.$$

$I_1$ é a imagem primária e será usada no teste de ligação; $I_2, \ldots, I_m$ são imagens auxiliares necessárias à agregação.

Todos os escalares, pontos, dimensões e codificações são validados como nas secções anteriores. Em particular, a imagem primária deve ser não nula e pertencer ao subgrupo de ordem $l$. O caso $n = 1$ é algebricamente válido, mas não oferece anonimidade; nas transacções v16 o anel tem exactamente 16 membros.

## Pesos de agregação

Não podemos somar as linhas com coeficientes fixos. Um atacante poderia escolher uma chave ou imagem auxiliar que cancelasse outra e obter um agregado legítimo a partir de componentes ilegítimos. Por isso, CLSAG deriva um peso independente para cada linha:

$$\mu_j = \mathcal{H}_n(T_j, m, n, \mathcal{R}, I_1, \ldots, I_m), \qquad j = 1, \ldots, m.$$

Os rótulos $T_j$ separam os domínios e toda a declaração, incluindo as imagens, fica presa a cada peso [43]. Definem-se então

$$W_i = \sum_{j=1}^{m} \mu_j K_{i,j},$$

$$\widetilde{W} = \sum_{j=1}^{m} \mu_j I_j,$$

$$w_\pi = \sum_{j=1}^{m} \mu_j k_{\pi,j}.$$

Na coluna verdadeira,

$$W_\pi = w_\pi G, \qquad \widetilde{W} = w_\pi H_\pi.$$

Assim, muitas relações transformam-se numa só relação bLSAG, sem perder a vinculação às componentes.

## Assinatura

Seja

$$\Psi = (T_c, m, n, \mathcal{R}, I_1, \ldots, I_m, M)$$

o prefixo das rondas, com um domínio $T_c$ distinto dos domínios de agregação. Convencionamos $c_{n+1} = c_1$.

1. Calculam-se $H_i$, as imagens $I_j$, os pesos $\mu_j$, as chaves agregadas $W_i$, a imagem agregada $\widetilde{W}$ e a testemunha agregada $w_\pi$.

2. Escolhem-se $\alpha \in_R \mathbb{Z}_l$ e $r_i \in_R \mathbb{Z}_l$ para todo $i \neq \pi$.

3. Inicia-se a ronda seguinte à coluna signatária:
$$c_{\pi+1} = \mathcal{H}_n(\Psi, [\alpha G], [\alpha H_\pi]).$$

4. Para $i = \pi+1, \ldots, n, 1, \ldots, \pi-1$, calculam-se
$$\begin{aligned} L_i &= r_i G + c_i W_i, \\ R_i &= r_i H_i + c_i \widetilde{W}, \\ c_{i+1} &= \mathcal{H}_n(\Psi, [L_i], [R_i]). \end{aligned}$$

5. Fecha-se a cadeia com
$$r_\pi = (\alpha - c_\pi w_\pi) \bmod l.$$

A assinatura conceptual é
$$\sigma = (I_1, \ldots, I_m, c_1, r_1, \ldots, r_n).$$

## Verificação

O verificador recompõe os mesmos pesos, $W_i$ e $\widetilde{W}$. Depois põe $\widehat{c}_1 = c_1$ e, para $i = 1, \ldots, n$, calcula
$$\begin{aligned} \widehat{L}_i &= r_i G + \widehat{c}_i W_i, \\ \widehat{R}_i &= r_i H_i + \widehat{c}_i \widetilde{W}, \\ \widehat{c}_{i+1} &= \mathcal{H}_n(\Psi, [\widehat{L}_i], [\widehat{R}_i]). \end{aligned}$$
Aceita apenas se
$$\widehat{c}_{n+1} = c_1.$$
O desafio que se compara é, uma vez mais, o valor obtido *depois* da última coluna; trocar $c_n$, $c_{n+1}$ e $c_1$ seria um erro de fronteira capaz de destruir o teste circular.

## Funciona porque

Na coluna verdadeira, a resposta agregada reconstrói ambos os compromissos:
$$\begin{aligned} \widehat{L}_\pi &= r_\pi G + c_\pi W_\pi \\ &= (\alpha - c_\pi w_\pi) G + c_\pi (w_\pi G) = \alpha G, \\ \widehat{R}_\pi &= r_\pi H_\pi + c_\pi \widetilde{W} \\ &= (\alpha - c_\pi w_\pi) H_\pi + c_\pi (w_\pi H_\pi) = \alpha H_\pi. \end{aligned}$$
As rondas simuladas coincidem por construção; portanto o desafio volta ao $c_1$ publicado.

Os pesos são parte da correcção, não apenas uma compressão. Como cada $\mu_j$ depende de todas as chaves e imagens, escolher uma componente para cancelar outra altera os próprios coeficientes. No modelo do oráculo aleatório, encontrar o ponto fixo necessário para esse ataque tem probabilidade desprezável.

## Especialização usada pelo Monero

src/ringct/
rctSigs.cpp
CLSAG_Gen()
verRctCLSAG
Simple()

Nas transacções Monero há duas linhas. A primeira contém chaves públicas de uso único

$$P_i = p_i G.$$

A segunda contém diferenças entre o compromisso candidato $C_i^{\mathrm{nz}}$ e um compromisso de compensação comum $C_{\mathrm{off}}$:

$$C_i = C_i^{\mathrm{nz}} - C_{\mathrm{off}} = z_i G.$$

Na coluna verdadeira, o signatário conhece $p = p_\pi$ e $z = z_\pi$. Com $H_i = \mathcal{H}_p(P_i)$, calcula

$$I = pH_\pi,$$

$$D = zH_\pi.$$

$I$ é a imagem primária. A assinatura serializa a imagem auxiliar como $\overline{D} = 8^{-1}D$; o verificador recupera $D = 8\overline{D}$. Esta multiplicação pelo cofactor coloca a imagem auxiliar no subgrupo primo e o verificador rejeita $D = 0$. A imagem $I$, que determina a ligação, é rejeitada se for nula ou se $lI \neq 0$.

Na implementação consultada, os pesos são exactamente

$$\mu_P = \mathcal{H}_n(\texttt{CLSAG_agg_0}, \boldsymbol{P}, \boldsymbol{C}^{\mathrm{nz}}, I, \overline{D}, C_{\mathrm{off}}),$$

$$\mu_C = \mathcal{H}_n(\texttt{CLSAG_agg_1}, \boldsymbol{P}, \boldsymbol{C}^{\mathrm{nz}}, I, \overline{D}, C_{\mathrm{off}}).$$

As duas relações agregadas são

$$W_i = \mu_P P_i + \mu_C (C_i^{\mathrm{nz}} - C_{\mathrm{off}}),$$

$$\widetilde{W} = \mu_P I + \mu_C D,$$

e a testemunha verdadeira é

$$w_\pi = \mu_P p + \mu_C z.$$

O hash de cada ronda usa o domínio `CLSAG_round` e liga, pela ordem serializada, todas as $P_i$, todos os $C_i^{\mathrm{nz}}$, $C_{\mathrm{off}}$, a mensagem da transacção e os dois compromissos $L_i, R_i$. $I$ e $\overline{D}$ ficam ligados às rondas através dos pesos $\mu_P, \mu_C$. Em notação compacta:

$$L_i = r_i G + c_i\{\mu_P P_i + \mu_C (C_i^{\mathrm{nz}} - C_{\mathrm{off}})\},$$

$$R_i = r_i H_i + c_i(\mu_P I + \mu_C D).$$

A assinatura serializada contém $r_1, \ldots, r_n$, $c_1$ e $\overline{D}$. $I$ já aparece como imagem da entrada da transacção e é reposto na estrutura antes da verificação; não é duplicado no corpo CLSAG. O verificador da revisão consultada rejeita anéis vazios, números errados de respostas, $c_1$ ou respostas não canónicos, $I = 0$, $8\overline{D} = 0$ e codificações de pontos inválidas; a validação da transacção exige ainda $lI = 0$. Cada desafio de ronda reconstruído tem também de ser não nulo.

## Ligação e hipóteses de segurança

O teste de ligação usa somente

$$I = k_{\pi,1}\mathcal{H}_p(K_{\pi,1}).$$

Se a chave primária de uso único assinar de novo, $I$ reaparece. As imagens auxiliares não entram nesse teste: existem para provar a consistência da agregação, e tratá-las como marcadores poderia criar ligações que o protocolo não pretende.

A infalsificabilidade e o conhecimento da coluna dependem da dificuldade do PLD, da segurança Fiat–Shamir no modelo do oráculo aleatório e de os pesos impedirem cancelamentos de chaves. A impossibilidade de calcular uma imagem a partir de $G, K, H$ sem a testemunha é uma hipótese do tipo PCDH; esconder a qual chave $K_i$ corresponde a $(H_i, I)$ exige a dificuldade do problema decisional de Diffie–Hellman. Resolver o PLD permitiria resolver os dois problemas Diffie–Hellman; o inverso, e a equivalência entre as respectivas dificuldades, não se seguem sem uma redução para o grupo usado.

A unicidade útil de $I$ exige ainda uma função hash para ponto resistente a colisões, bases no subgrupo correcto e rejeição de identidade e torção. Uma imagem repetida só estabelece ligação entre assinaturas válidas; não prova, isoladamente, que uma saída pertencia a alguém nem que todas as regras de valor de uma transacção foram cumpridas.

Reutilizar $\alpha$ em dois desafios revela a testemunha agregada $w_\pi$. Se a reutilização ocorrer sob conjuntos de pesos independentes, várias equações lineares podem ainda revelar as testemunhas individuais. Respostas previsíveis ou não uniformes podem denunciar $\pi$. Por isso, valor efémero, mensagem, matriz, imagens, dimensões, domínios e todos os prefixos devem ficar ligados à mesma execução.

## Requisitos de espaço

Na forma geral, CLSAG guarda $n + 1$ escalares e $m$ imagens, usando $nm$ chaves públicas. A redução de $mn + 1$ para $n + 1$ escalares é a vantagem concisa em relação a MLSAG. Na especialização Monero, a imagem primária acompanha a entrada e a assinatura serializa apenas a imagem auxiliar, além desses $n + 1$ escalares.

# CAPÍTULO 5

# Endereços

Um endereço de Monero é uma instrução pública que permite a um remetente criar uma nova saída para o destinatário. O endereço não é registado directamente na lista de blocos. A lista de blocos guarda essa saída sob uma chave pública de utilização única; quem conhecer a chave privada correspondente poderá depois usá-la como entrada de uma nova transacção. A mesma saída não pode ser gasta duas vezes.

Por isso, quando dizemos que um endereço tem um saldo, abreviamos uma ideia mais precisa: uma carteira encontrou saídas que consegue detectar para esse endereço, determinou os respectivos montantes e ainda não as marcou como gastas. A conservação dos valores e a ocultação dos montantes serão tratadas nos capítulos 6 e 7; aqui isolaremos o encaminhamento e a detecção das saídas.

Salvo indicação em contrário, todas as observações sobre a implementação neste capítulo referem-se à revisão do código Monero identificada no resumo inicial. A distinção será importante: algumas formas estão impostas pelo consenso da versão 16, ao passo que outras são convenções das carteiras para que remetentes e destinatários se entendam.

## 5.1 Chaves

Bob escolhe dois escalares privados não nulos do corpo escalar, $k_B^s, k_B^v \in \mathbb{Z}_l^*$, e publica os pontos

$$K_B^s = k_B^s G, \qquad K_B^v = k_B^v G.$$

Como na Secção 2.3.2, $G$ gera o subgrupo de ordem prima $l$. Chamaremos a $k_B^s$ *chave privada de gasto* e a $k_B^v$ *chave privada de vista*; os expoentes $s$ e $v$ indicam essas duas funções. O endereço principal é o par público $(K_B^s, K_B^v)$, nunca o par privado.

A separação permite entregar $k_B^v$, juntamente com o endereço público, a uma máquina de leitura ou a um auditor sem entregar $k_B^s$. Uma tal *carteira só de vista* consegue detectar saídas que lhe foram dirigidas, reconhecer o sub-endereço usado, descodificar os montantes RingCT e ler um ID de pagamento curto destinado a Bob. Não consegue calcular a chave privada de utilização única, criar a respectiva imagem de chave, autorizar um gasto nem assinar uma transacção. Sem imagens de chave importadas de uma carteira completa, também não consegue decidir com segurança se uma saída detectada já foi gasta. A capacidade de observação não confere autoridade de gasto.

As carteiras determinísticas da implementação consultada geram primeiro $k^s$ e derivam normalmente $k^v = \mathcal{H}_n(k^s)$. É uma convenção de recuperação da carteira, não uma equação exigida pelo consenso; também é possível importar dois segredos independentes. Em qualquer caso, os segredos devem ser escalares canónicos e os pontos públicos esperados pertencem ao subgrupo gerado por $G$.

## Codificação de um endereço principal

Para comunicar o par numa representação textual, Monero usa uma variante própria de Base58 [12, 27]. Na rede principal, a entrada binária de um endereço principal é

$$\text{varint}(18) \parallel K^s \parallel K^v,$$

pela mesma ordem em que os dois pontos aparecem na estrutura `account_public_address`. Calcula-se Keccak–256 sobre essa entrada, juntam-se os primeiros quatro bytes do resultado como soma de verificação e codifica-se tudo em blocos Base58. O resultado tem 95 caracteres e começa habitualmente por «4». O número 18, e não esse primeiro carácter isolado, é o prefixo que identifica simultaneamente a rede e o tipo de endereço.

Ao descodificar, a implementação exige Base58 bem formada, soma de verificação correcta, prefixo da rede e comprimento apropriados, e duas codificações canónicas de pontos da curva. A soma de quatro bytes detecta enganos de cópia; não autentica Bob, não cifra o endereço e não prova que alguém conhece as chaves privadas.

**Requisitos.** A rotina `check_key()` confirma que cada sequência de 32 bytes representa canonicamente um ponto da curva, mas não testa por si só se o ponto é não nulo e pertence ao subgrupo de ordem $l$. Uma carteira robusta deve exigir estas últimas condições para chaves de endereço, além de rejeitar escalares nulos ou não canónicos. As chaves produzidas pelo gerador normal satisfazem-nas por construção.

src/common/base58.cpp `encode_addr()`; src/cryptonote_basic/cryptonote_basic_impl.cpp

## 5.2 Endereços ocultos

Alice não escreve o endereço principal de Bob numa saída. Aplica-lhe antes uma troca semelhante a Diffie–Hellman e cria uma chave pública nova para cada saída [133]. O livro chamou historicamente a estes objectos *endereços ocultos*; diremos também, com maior precisão, *chaves de saída de utilização única*. Um observador pode ver a chave de saída, mas não o endereço público do qual ela foi derivada.

### Uma saída para um endereço principal

Isolemos a saída de índice $t = 0$ em que Alice paga 0,123 XMR a Bob. Uma transacção RingCT comum da versão 16 tem pelo menos duas saídas, mas a construção de cada uma pode ser estudada separadamente. Alice conhece $(K_B^s, K_B^v)$ e procede assim:

1. Gera, de modo uniforme, um escalar efémero não nulo $r \in_R \mathbb{Z}_l^*$. A função usada pela implementação rejeita zero e produz uma representação escalar canónica.

2. Calcula a *chave pública de transacção* ordinária
$$R = rG$$
e a derivação partilhada
$$\mathcal{D}_B = 8rK_B^v.$$
O factor 8 é o cofactor da curva. Faz parte da rotina `generate_key_derivation()` e não deve ser omitido.

3. Liga a derivação à posição serializada da saída: src/crypto/crypto.cpp `generate_key_derivation()`, `derivation_to_scalar()` e `derive_public_key()`
$$h_0 = \mathcal{H}_n\big(\mathcal{D}_B, \mathrm{varint}(0)\big),$$
$$K_0^o = h_0 G + K_B^s.$$
A função $\mathcal{H}_n$ reduz o hash módulo $l$; o índice é codificado como inteiro de comprimento variável, não como o carácter «0».

4. Coloca $K_0^o$ no alvo da saída e, segundo a convenção das carteiras, coloca $R$ no campo `extra`. Na versão 16 o alvo leva ainda a etiqueta de vista que definiremos abaixo. O compromisso que oculta o montante será explicado na Secção 6.3.

Ao percorrer a lista de blocos, Bob lê $R$ e calcula
$$\mathcal{D}'_B = 8k_B^v R.$$
As duas partes obtêm exactamente os mesmos bytes de derivação, pois
$$\mathcal{D}'_B = 8k_B^v(rG) = 8r(k_B^v G) = 8rK_B^v = \mathcal{D}_B.$$
Para o índice $t = 0$, Bob recompõe $h_0$ e subtrai
$$K_0^o - h_0 G = K_B^s.$$

Uma carteira guarda as chaves públicas de gasto que reconhece numa tabela; a igualdade coloca esta saída na conta principal de Bob. Repetirá o teste para os índices e chaves públicas de transacção apropriados.

**Funciona porque** a chave privada correspondente é

$$k_0^o = h_0 + k_B^s \pmod{l},$$
$$K_0^o = (h_0 + k_B^s)G = k_0^o G.$$

Alice conhece $r$, $\mathcal{D}_B$, $h_0$ e $K_0^o$, mas não conhece $k_B^s$ e portanto não obtém $k_0^o$. Uma carteira só de vista conhece $k_B^v$ e reconhece $K_B^s$, mas continua sem a parcela $k_B^s$. Só a carteira completa calcula $k_0^o$, a imagem de chave e a assinatura necessárias para gastar. A detecção é uma conclusão local da carteira, não uma prova pública de propriedade; veremos a autorização do gasto no Capítulo 7.

## Cofactor, codificações e pontos malformados

Se as duas chaves públicas provêm de escalares sobre $G$, todos os pontos acima já estão no subgrupo de ordem $l$. A multiplicação por 8 tem uma função mais geral: elimina a componente de torção de qualquer ponto Edwards descodificável antes de este ser usado como derivação. Não transforma, porém, uma chave maliciosa numa chave autenticada. Um ponto de baixa ordem pode resultar na identidade depois dessa multiplicação, e uma sequência que nem sequer codifique canonicamente um ponto faz `generate_key_derivation()` falhar.

As carteiras devem, pois, rejeitar ou saltar chaves públicas de transacção malformadas e exigir chaves de endereço não nulas no subgrupo primo. Também devem evitar o caso negligível $k_t^o = 0$, regenerando o segredo efémero se necessário. A implementação gera $r$ uniformemente em $\mathbb{Z}_l^*$, e os seus pontos honestos têm codificação canónica.

Há aqui uma fronteira de protocolo fácil de perder. O consenso valida que a chave pública de cada saída é uma codificação de ponto admissível e, na versão 16, que o alvo tem a variante com etiqueta. Não prova a relação entre $K_t^o$, $R$ e qualquer endereço, nem exige que uma chave pública de transacção semanticamente válida exista em `extra`; esses são contractos de interoperabilidade das carteiras. A rotina de consenso `check_key()` também não impõe, isoladamente, pertença ao subgrupo primo. Uma transacção deliberadamente defeituosa pode assim ser aceite pelo consenso e, ainda assim, criar uma saída que a carteira pretendida não consiga encontrar ou gastar.

## Etiquetas de vista

Testar $K_t^o - h_t G$ contra uma grande tabela de chaves públicas custa mais do que comparar um byte. Para cada derivação e índice, o remetente da implementação consultada calcula

$$v_t = \text{primeiroByte}\big(\mathcal{H}\big(\texttt{view_tag} \parallel \mathcal{D}_B \parallel \text{varint}(t)\big)\big)\,,$$

onde $\mathcal{H}$ é Keccak–256 sem redução escalar, e inclui $v_t$ junto de $K_t^o$. O sal ASCII de oito bytes `view_tag` separa este uso dos restantes hashes.

Ao examinar uma saída, Bob calcula primeiro o seu próprio $v_t$. Se os bytes diferirem, a carteira salta logo a derivação pública completa. Se coincidirem, a saída é apenas uma *candidata*: a carteira ainda tem de subtrair $h_tG$ e encontrar a chave de gasto resultante na sua tabela. Para saídas alheias, e sob o comportamento pseudo-aleatório do hash, o filtro elimina em média 255 de cada 256; cerca de uma em 256 passa como falso positivo e recebe o teste completo.

src/crypto/crypto.cpp `derive_view_tag()`;

Na versão 15 houve um período de transição em que uma transacção podia usar alvos antigos sem etiqueta ou alvos etiquetados, desde que todas as suas saídas tivessem o mesmo tipo. Na versão 16, congelada para esta edição, cada saída tem obrigatoriamente a variante `txout_to_tagged_key` e uma etiqueta de exactamente um byte. Saídas históricas sem etiqueta continuam a ser lidas pelo caminho de varrimento completo.

src/cryptonote_basic/cryptonote_format_utils.cpp

O consenso verifica a presença do byte, mas não consegue verificar se ele foi derivado do segredo partilhado. Uma etiqueta errada pode fazer uma carteira ignorar uma saída que, pela equação de $K_t^o$, lhe pertenceria. Logo, a etiqueta é uma optimização de varrimento, não prova de propriedade, não autorização de gasto e não mecanismo independente de privacidade. Um observador sem $r$ nem $k_B^v$ continua sem poder calcular $\mathcal{D}_B$; quem recebe a chave privada de vista calcula tanto a etiqueta como o teste completo. Para revelar partes escolhidas do histórico sem entregar essa chave, recorreremos ao Capítulo 13.

### 5.2.1 Transacções de múltiplas saídas

A transacção corrente costuma pagar ao destinatário e devolver troco ao remetente. Para transacções RingCT não mineiras, a versão 16 exige por consenso pelo menos duas saídas. A forma canónica de Bulletproof+ usada nessa versão agrega no máximo 16; por isso, uma transacção corrente tem $2 \le p \le 16$ saídas. As razões de valor e a prova de intervalo ficam para o capítulo seguinte.

src/cryptonote_core/cryptonote_tx_utils.cpp `construct_tx_with_tx_key()`

Antes de criar as chaves, a carteira remetente baralha os destinos. O *índice de saída* $t \in \{0, \ldots, p-1\}$ é a posição final no vector serializado `vout`, e é essa posição que todos os cálculos devem usar. Para uma transacção cujos destinatários têm endereços principais, um único segredo fresco $r \in_R \mathbb{Z}_l^*$ e $R = rG$ bastam. Para a saída $t$, destinada a $(K_t^s, K_t^v)$, definimos

$$\begin{aligned} \mathcal{D}_t &= 8rK_t^v = 8k_t^v R, \\ h_t &= \mathcal{H}_n\big(\mathcal{D}_t, \mathrm{varint}(t)\big), \\ K_t^o &= h_t G + K_t^s = (h_t + k_t^s)G, \\ k_t^o &= h_t + k_t^s \pmod{l}. \end{aligned}$$

Mesmo que Alice pague duas vezes ao mesmo endereço na mesma transacção, os índices diferentes entram no hash e produzem chaves de saída distintas, salvo colisão de probabilidade

negligível. A etiqueta de vista de cada saída usa a mesma $\mathcal{D}_t$ e o mesmo varint$(t)$. O destinatário recompõe ambos a partir de $8k_t^v R$; uma carteira completa soma depois $k_t^s$ para obter a testemunha privada.

É correcto partilhar $r$ entre estas saídas da *mesma* transacção. Não se deve reutilizá-lo noutra transacção: $R$ repetiria imediatamente, ligando as duas construções, e repetiria também as derivações para os mesmos destinatários. Se houver sub-endereços, a escolha de $R$ e o número de segredos efémeros mudam; veremos já esse caso na Secção 5.3.

## 5.3 Sub-endereços

Um endereço principal pode originar muitos sub-endereços. Todos conduzem à mesma carteira, mas Bob pode entregar um diferente a cada cliente, factura ou serviço e, quando a saída for detectada, saber qual deles foi usado. Ao contrário de publicar repetidamente o endereço principal, isto evita que um observador ligue essas identidades apenas por igualdade textual [101].

Os sub-endereços entraram no software associado à versão 7 do protocolo [58]. A carteira mantém uma tabela das respectivas chaves públicas de gasto: construir essa tabela requer tempo e memória proporcionais ao número de sub-endereços, mas cada saída candidata pode ser classificada por uma consulta à tabela, sem uma nova passagem completa pela lista de blocos para cada endereço.

### Derivação e codificação

O índice de uma conta e de um sub-endereço é o par

$$\iota = (j, i) \in \{0, \ldots, 2^{32} - 1\} \times \{0, \ldots, 2^{32} - 1\}.$$

A implementação reserva $\iota = (0, 0)$ para o endereço principal. Para outro par, codifica $j$ e $i$ como inteiros de 32 bits em ordem *little-endian* e define

$$T_{\text{sub}} = \texttt{SubAddr} \parallel \texttt{0x00},$$

uma sequência de oito bytes: os sete caracteres ASCII e o byte NUL final. A partir das chaves principais de Bob calcula

$$\begin{aligned} m_\iota &= \mathcal{H}_n\big(T_{\text{sub}} \parallel k_B^v \parallel \text{LE32}(j) \parallel \text{LE32}(i)\big), \\ k_B^{s,\iota} &= k_B^s + m_\iota \pmod{l}, \\ K_B^{s,\iota} &= K_B^s + m_\iota G = k_B^{s,\iota} G, \\ K_B^{v,\iota} &= k_B^v K_B^{s,\iota}. \end{aligned}$$

O sub-endereço é o par $(K_B^{s,\iota}, K_B^{v,\iota})$.

src/device/device_default.cpp `get_subaddress_secret_key()` e `get_subaddress()`

Não se introduz aqui uma nova chave privada $k_B^{v,\iota}$ com o papel da chave de vista principal. Uma carteira completa conhece ambos os factores e sabe que $K_B^{v,\iota} = (k_B^v k_B^{s,\iota})G$. Mas não é esse

produto que o algoritmo de varrimento usa: a chave de vista da conta continua a ser $k_B^v$, agora aplicada como escalar a $K_B^{s,\iota}$. Esta distinção é precisamente o que permite classificar todos os sub-endereços num só varrimento.

Uma carteira só de vista que possua $k_B^v$ e $K_B^s$ consegue recompor $m_\iota$, $K_B^{s,\iota}$ e $K_B^{v,\iota}$, e preencher a tabela de detecção. Não consegue obter $k_B^{s,\iota} = k_B^s + m_\iota$, porque lhe falta $k_B^s$. Por sua vez, quem recebe apenas um sub-endereço público não aprende o endereço principal. Testar se um sub-endereço e um endereço principal partilham a mesma chave de vista exigiria distinguir a relação Diffie–Hellman $(G, K_B^v, K_B^{s,\iota}, K_B^{v,\iota})$; a privacidade assenta, portanto, nas hipóteses de dificuldade usuais do grupo, não em segredo da fórmula. Divulgar $k_B^v$ permite fazer o teste e liga todos os sub-endereços da conta. Reutilizar literalmente o mesmo sub-endereço também o liga por igualdade.

Na rede principal, a codificação Base58 usa o prefixo numérico 42 seguido de $K_B^{s,\iota}$ e $K_B^{v,\iota}$, e a mesma soma de verificação de quatro bytes. Tem 95 caracteres e começa habitualmente por «8». Tal como no endereço principal, o carácter visível não substitui a validação do prefixo, do comprimento, da soma e dos pontos. Os pontos produzidos honestamente estão no subgrupo primo; uma carteira deve rejeitar identidades e pontos fora desse subgrupo. O caso $k_B^s + m_\iota = 0 \pmod{l}$ tem probabilidade negligível, mas uma implementação defensiva não deve emitir o endereço degenerado.

### 5.3.1 Enviar para um sub-endereço

Suponhamos que Alice paga a saída de índice $t$ ao sub-endereço $(K_B^{s,\iota}, K_B^{v,\iota})$. Escolhe $r \in_R \mathbb{Z}_l^*$ e, no caso simples em que esse é o único endereço distinto entre os destinatários que não são troco, publica

$$R = rK_B^{s,\iota}$$

em vez de $rG$. A derivação e a chave da saída são

$$\begin{aligned} \mathcal{D}_{B,t} &= 8rK_B^{v,\iota}, \\ h_t &= \mathcal{H}_n\big(\mathcal{D}_{B,t}, \mathrm{varint}(t)\big), \\ K_t^o &= h_t G + K_B^{s,\iota}. \end{aligned}$$

A etiqueta de vista é calculada sobre esta mesma $\mathcal{D}_{B,t}$ e o mesmo índice.

Bob usa a chave privada de vista principal:

$$\begin{aligned} 8k_B^v R &= 8k_B^v(rK_B^{s,\iota}) \\ &= 8r(k_B^v K_B^{s,\iota}) = 8rK_B^{v,\iota} = \mathcal{D}_{B,t}. \end{aligned}$$

Depois do teste da etiqueta, subtrai $h_t G$ de $K_t^o$. O resultado $K_B^{s,\iota}$ identifica a linha da tabela e, portanto, o índice $\iota$. A carteira completa pode ainda formar

$$\begin{aligned} k_t^o &= h_t + k_B^{s,\iota} = h_t + k_B^s + m_\iota \pmod{l}, \\ K_t^o &= k_t^o G. \end{aligned}$$

Alice conhece $r$, as duas chaves públicas e $h_t$, mas não conhece $k_B^{s,\iota}$ e portanto não obtém $k_t^o$. A carteira só de vista chega à identificação, mas não a esse segredo.

src/crypto/crypto.cpp `derive_subaddress_public_key()`; src/cryptonote_core/cryptonote_tx_utils.cpp

### Chave principal e chaves adicionais

Uma só fórmula não descreve todos os lotes de destinatários. Na revisão consultada, a carteira classifica os *endereços distintos que não são de troco* e escolhe as chaves públicas de transacção assim:

- Se todos forem endereços principais, usa uma chave principal $R = rG$, sem chaves adicionais.
- Se houver exactamente um sub-endereço distinto e nenhum endereço principal entre esses destinatários, usa $R = rK^{s,\iota}$, sem chaves adicionais. Várias saídas para esse mesmo sub-endereço podem partilhar a derivação; os seus índices continuam a produzir $h_t$ distintos.
- Se misturar endereços principais e sub-endereços, ou se houver mais de um sub-endereço distinto, mantém uma chave principal $R = rG$ e cria um segredo fresco $r_t \in_R \mathbb{Z}_l^*$ para cada posição. A chave adicional alinhada com uma saída de endereço principal é $R_t = r_t G$, embora essa saída continue a usar $R$ na derivação. Para uma saída de sub-endereço que não seja troco, é $R_t = r_t K_t^{s,\iota}$, e a derivação usa $R_t$.

As chaves adicionais aparecem em `extra` num vector cuja posição corresponde ao índice final de `vout`. A posição reservada ao troco pode não ser usada para a sua derivação: como o remetente conhece a própria chave de vista, a implementação deriva o troco da chave principal. Esse pormenor não autoriza deslocar as restantes posições.

Numa saída de endereço principal, o segredo partilhado continua a ser $8rK_t^v = 8k_t^v R$. Numa saída de sub-endereço que use $R_t$, é $8r_t K_t^{v,\iota} = 8k_t^v R_t$. É a derivação correspondente que entra em $h_t$ e na etiqueta de vista da saída.

Ao varrer, a carteira tenta primeiro a derivação da chave principal e, quando existe, a chave adicional da mesma posição $t$. Uma chave malformada não produz uma derivação válida: o caminho de varrimento afectado falha ou salta o candidato. Quando um auxiliar interno coloca a identidade como valor de recurso, isso marca a falha e não torna válida a chave original. Cada etiqueta de vista tem de ser calculada com a derivação que realmente criou a sua saída. Nem o consenso confirma que o vector está alinhado, nem prova as relações $R = rG$, $R_t = r_t G$ ou $R_t = r_t K^{s,\iota}$: estas são convenções necessárias à descoberta pelas carteiras.

### O limite de Janus

O teste de recepção acima confirma que a chave de gasto recuperada pertence à tabela de Bob; não confirma publicamente que as duas metades do endereço fornecido a Alice foram usadas como um par. Suponhamos que um remetente malicioso suspeita da relação entre $(D_1, C_1) = (K_B^{s,1}, k_B^v D_1)$ e $(D_2, C_2) = (K_B^{s,2}, k_B^v D_2)$. Pode escolher $r$ e publicar

$$R = rD_2, \qquad K_t^o = \mathcal{H}_n\big(8rC_2, \mathrm{varint}(t)\big)G + D_1.$$

Bob calcula $8k_B^v R = 8rC_2$, recupera $D_1$ e associa a saída ao primeiro sub-endereço. Se depois revelar ao remetente que recebeu esse pagamento, a resposta confirma que $D_1$ e $D_2$ usam a mesma chave de vista: é o chamado ataque de Janus [35].

Isto não entrega ao atacante a chave privada nem lhe permite gastar a saída; é uma possibilidade de ligação provocada activamente. Evitar confirmações externas automáticas reduz o oráculo, mas uma defesa criptográfica completa exige um contracto adicional entre remetente e carteira receptora. As propostas incluem publicar material que permita à carteira completa verificar que a chave de vista e a chave de gasto usadas pelo remetente pertencem ao mesmo sub-endereço [100]. Tal ligação não é hoje uma regra de consenso, pelo que uma mitigação de carteira deve declarar exactamente o que verifica e como trata transacções antigas.

## 5.4 Endereços integrados

Um endereço integrado junta um endereço *principal* a um ID de pagamento curto escolhido pelo destinatário. Serve, por exemplo, para Bob atribuir um identificador de oito bytes a uma factura e reconhecer esse identificador quando Alice paga. Não cria outro conjunto de chaves e não é uma variante de sub-endereço.

Na rede principal, a entrada Base58 é

$$\text{varint}(19) \parallel K_B^s \parallel K_B^v \parallel \text{ID}_8,$$

seguida da soma de verificação Keccak de quatro bytes. O texto tem 106 caracteres e começa habitualmente por «4». A descodificação exige exactamente oito bytes de ID. O prefixo integrado só é aceite com o par de um endereço principal; não existe uma codificação integrada de sub-endereço [92].

Estes endereços continuam suportados na revisão consultada por motivos de compatibilidade. Não devem confundir-se com os antigos IDs de pagamento autónomos de 32 bytes, escritos em claro em `extra`: a carteira corrente recusa criá-los e ignora-os em blocos a partir da versão 12, conservando apenas caminhos de leitura histórica. Um `extra` transporta no máximo um ID curto cifrado, sem indicar a que saída pertence. Por isso, a construção corrente requer uma única chave de vista de destinatário não troco a quem o associar; um pagamento agrupado para destinatários distintos não oferece uma atribuição inequívoca.

### Codificar

Se $r$ é o segredo efémero da chave pública principal $R = rG$, Alice forma para o endereço integrado de Bob

$$\mathcal{D}_B = 8rK_B^v,$$

e calcula Keccak–256 sobre os 33 bytes $\mathcal{D}_B \parallel \texttt{0x8d}$. Sejam os primeiros oito bytes

$$q = \text{primeiros8}(\mathcal{H}(\mathcal{D}_B \parallel \texttt{0x8d}))\,.$$

O valor colocado no nonce de `extra` é

$$c = \mathrm{XOR}(\mathrm{ID}_8, q).$$

Aqui não se aplica $\mathcal{H}_n$, não há redução módulo $l$ e não entra qualquer índice de saída. O byte `0x8d`, de valor decimal 141, separa este hash dos hashes das chaves de saída e das etiquetas de vista.

src/device/device_default.cpp `encrypt_payment_id()`

**Descodificar**

Com a chave pública principal $R = rG$, Bob recompõe

$$\mathcal{D}'_B = 8k^v_B R = \mathcal{D}_B, \qquad \mathrm{ID}_8 = \mathrm{XOR}(c, \mathrm{primeiros8}(\mathcal{H}(\mathcal{D}'_B \parallel \texttt{0x8d}))) .$$

A igualdade pressupõe, como nas saídas, que $R$ corresponde ao segredo efémero usado por Alice. A operação XOR é a da Secção 2.5.

Esta transformação apenas oculta o ID a quem não conhece a derivação. Não o autentica, não o liga por consenso a uma saída, não prova que Bob recebeu o pagamento e não impede Alice de escolher qualquer texto cifrado. A presença do campo também distingue estruturalmente a transacção. Para reduzir essa distinção, o construtor da carteira tenta inserir um ID curto nulo cifrado em certas transacções comuns sem ID explícito, quando há no máximo dois destinos e uma única chave de vista aplicável. Trata-se de comportamento de carteira, não de uma regra de consenso; ao descodificá-lo obtêm-se oito bytes nulos.

## 5.5 Endereços de multi-assinatura

Uma carteira de multi-assinatura faz várias partes cooperarem para controlar as chaves descritas neste capítulo. Não existe, no entanto, um prefixo Base58, um tipo de endereço ou uma espécie de saída de consenso reservados para «multi-assinatura»: o endereço partilhado contém o par público habitual e o remetente cria uma chave de saída de utilização única habitual. A diferença está em como os participantes formam e usam os segredos da carteira.

Na revisão de implementação fixada para este capítulo, o suporte da carteira é marcado como experimental, vem desactivado e a interface de linha de comandos exige activá-lo explicitamente com `set enable-multisig-experimental 1`; a interface RPC apresenta igualmente um aviso de perigo. Isto é estado do software, não uma restrição de consenso. O Capítulo 14 desenvolve o protocolo e as suas hipóteses operacionais.

# CAPÍTULO 6

# Montantes ocultos

O capítulo anterior separou a chave de saída de utilização única do endereço público que lhe deu origem. Falta ocultar o número que essa saída representa. Uma transacção RingCT publica um *compromisso* para cada montante: todos podem verificar relações entre compromissos, mas só quem possui a derivação partilhada apropriada recupera o montante e a máscara da saída.

Um compromisso não basta, porém, para definir dinheiro válido. Os seus escalares vivem módulo $l$, pelo que um valor modular enorme poderia fazer de «montante negativo». Veremos por isso duas camadas distintas: compromissos de Pedersen ocultam os montantes e preservam somas; provas de intervalo obrigam cada montante de saída a pertencer a $[0, 2^{64} - 1]$.

Salvo indicação em contrário, as referências à implementação neste capítulo dizem respeito à revisão do código Monero identificada no resumo inicial.

## 6.1 Compromissos

Um esquema de compromisso permite a Alice fixar uma mensagem sem a revelar de imediato. Mais tarde, Alice apresenta uma *abertura*; Bob verifica que é a mensagem originalmente fixada. Queremos duas propriedades:

**Ocultação.** Antes da abertura, o compromisso não deve revelar a mensagem.

**Vinculação.** Depois de publicado o compromisso, Alice não deve conseguir abri-lo como duas mensagens diferentes.

Para comprometer uma mensagem $m$, Alice gera um sal aleatório longo $s$ e publica

$$h = \mathcal{H}\big(s \parallel m\big).$$

Na abertura, revela $(s, m)$. Bob repete o hash e compara-o com $h$. O sal impede a enumeração antecipada quando o domínio de mensagens é pequeno. Neste compromisso por hash, a ocultação depende do sal e a vinculação depende, em particular, de ser impraticável encontrar colisões. Esta construção não preserva a adição das mensagens comprometidas.

## 6.2 Compromissos de Pedersen

Para somar montantes ocultos, empregamos um compromisso de Pedersen [110]. Sejam $G$ e $H$ pontos não nulos do subgrupo de ordem prima $l$. O grupo é cíclico, portanto existe matematicamente um escalar $\gamma$ tal que

$$H = \gamma G.$$

Aqui «geradores independentes» não quer dizer independência linear: quer dizer que ninguém conhece essa relação de logaritmo discreto. O valor fixo de $H$ usado pelo Monero foi derivado historicamente pela interpretação de Keccak–256 da codificação de $G$ como um ponto comprimido, seguida de multiplicação pelo cofactor 8. Essa construção específica não deve ser reutilizada como função hash-para-ponto genérica.

src/ringct/rctTypes.h, `H`; tests/unit_tests/ringct.cpp, `HPow2`

Para uma máscara $x \in \mathbb{Z}_l$ e um montante escalar $b \in \mathbb{Z}_l$, definimos

$$C(x, b) = xG + bH. \tag{6.1}$$

O par $(x, b)$ é uma abertura do ponto $C$. Os escalares canónicos usados pela implementação ocupam 32 bytes em ordem *little-endian* e representam inteiros menores que $l$. Um montante Monero é um inteiro sem sinal de 64 bits, colocado nos oito bytes menos significativos de um escalar; os outros 24 bytes são nulos. Os pontos têm a codificação comprimida de Edwards de 32 bytes.

**Ocultação.** Se $x$ for uniforme em $\mathbb{Z}_l$, então, para qualquer $b$, o ponto $xG + bH$ é uniforme no subgrupo. O esquema abstracto tem assim ocultação perfeita: mesmo cálculo ilimitado não escolhe uma abertura preferida entre todas as possibilidades [68, 120]. Na carteira, as máscaras de saída são derivadas de um segredo Diffie–Hellman por um hash reduzido; para um observador, a sua pseudo-aleatoriedade é uma garantia computacional, não uma fonte literal de aleatoriedade perfeita.

src/ringct/rctTypes.cpp, `d2h()`; src/ringct/rctOps.cpp, `genC()`

**Vinculação.** Se Alice encontrasse duas aberturas distintas $(x, b) \neq (x', b')$ para o mesmo ponto, obteria

$$(x - x')G = (b' - b)H.$$

Salvo o caso trivial em que ambas as diferenças são nulas, isso revelaria a relação entre $G$ e $H$. A vinculação é portanto *computacional*, sob a dificuldade do logaritmo discreto; não é «vinculação perfeita». Quem conhecesse $\gamma$ poderia escolher qualquer $b'$ e calcular $x' = x + (b - b')\gamma$, quebrando-a.

**Homomorfismo aditivo.** A aritmética dos escalares é módulo $l$, e

$$C(x, a) + C(y, b) = C(x + y, a + b), \tag{6.2}$$

$$C(x, a) - C(y, b) = C(x - y, a - b). \tag{6.3}$$

Isto deixa comparar somas ocultas. Não significa que a igualdade pública de dois compromissos revele a igualdade dos montantes: máscaras diferentes podem compensar montantes diferentes. Para provar uma relação entre montantes é preciso também ligar ou cancelar as máscaras de maneira verificável.

## 6.3 Montantes ocultos

Consideremos a saída $t$, de montante $b_t$, enviada a Bob. A Secção 5.2 definiu a derivação partilhada $\mathcal{D}_B$ e o escalar dependente do índice

$$h_t = \mathcal{H}_n\big(\mathcal{D}_B \,\|\, \mathrm{varint}(t)\big).$$

Para um endereço principal ordinário, $\mathcal{D}_B = 8rK_B^v$ do lado de Alice e $\mathcal{D}_B = 8k_B^v R$ do lado de Bob. Sub-endereços, troco e chaves adicionais escolhem a chave pública de transacção apropriada como descrito no capítulo anterior; depois de obtido $h_t$, o tratamento do montante é o mesmo.

A carteira calcula a máscara de compromisso

$$y_t = \mathcal{H}_n\big(\texttt{commitment_mask} \,\|\, h_t\big) \tag{6.4}$$

e publica o compromisso normal

$$C_t^b = C(y_t, b_t) = y_t G + b_t H. \tag{6.5}$$

Aqui $y_t$ é um escalar, não um ponto. A redução do hash produz a sua codificação módulo $l$; a construção não rejeita explicitamente a máscara nula, cuja ocorrência tem probabilidade desprezável no modelo do hash.

O montante propriamente dito cabe em oito bytes. Seja

$$q_t = \mathrm{primeiros8}(\mathcal{H}(\texttt{amount} \,\|\, h_t)) \,.$$

Alice coloca no campo `ecdhInfo.amount`

$$e_t = \mathrm{LE8}(b_t) \oplus q_t, \tag{6.6}$$

onde $\oplus$ é o XOR da Secção 2.5. Desde o formato `RCTTypeBulletproof2`, incluindo CLSAG e Bulletproof+, só estes oito bytes são serializados; a máscara não é transmitida porque ambos os lados a recompõem pela Equação (6.4). O formato anterior, mais volumoso, guardava 32 bytes de montante e uma máscara codificada; a versão 10 introduziu a forma compacta sob o nome histórico `RCTTypeBulletproof2` [89].

Bob calcula o mesmo $h_t$, recupera

$$\mathrm{LE8}(b_t) = e_t \oplus q_t$$

src/device/device_default.cpp, `generate_output_ephemeral_keys()`; src/ringct/rctOps.cpp, `genCommitmentMask()` e `ecdhEncode()`

e recompõe $y_t$. A carteira exige que as representações escalares obtidas sejam canónicas, que os 24 bytes superiores do montante sejam nulos e que

$$y_t G + b_t H = C_t^b.$$

Uma chave de vista permite assim detectar a saída, descodificar o montante e confirmar a abertura; não permite gastá-la. O XOR não oferece segurança de teoria da informação nem autenticação autónoma: a confidencialidade depende computacionalmente do segredo partilhado e do hash, enquanto o compromisso público fornece a verificação de consistência da abertura. src/ringct/rctSigs.cpp, `decodeRct()`; src/ringct/rctTypes.h, `serialize_rctsig_base()`

## 6.4 Introdução a RingCT

Uma saída reúne objectos com funções diferentes. A chave de saída de utilização única liga a saída à autorização de gasto; $C_t^b$ oculta o montante; e $e_t$ permite à carteira certa recuperar esse montante. Nenhum destes objectos, isoladamente, oferece anonimato completo nem prova que a transacção está autorizada.

Suponhamos que a transacção gasta $m$ saídas anteriores de montantes $a_1, \ldots, a_m$, cria $p$ saídas de montantes $b_0, \ldots, b_{p-1}$ e declara a taxa pública $f$. A conservação desejada é

$$\sum_{j=1}^{m} a_j = \sum_{t=0}^{p-1} b_t + f. \tag{6.7}$$

O verdadeiro compromisso gasto pela entrada $j$ tem a forma

$$C_j^a = x_j G + a_j H.$$

Publicá-lo como a entrada verdadeira denunciaria o membro escolhido no anel. Por isso, o remetente cria um *pseudo-compromisso de saída*

$$\widetilde{C}_j^a = x'_j G + a_j H.$$

Para o membro verdadeiro,

$$C_j^a - \widetilde{C}_j^a = (x_j - x'_j) G. \tag{6.8}$$

O Capítulo 7 mostrará como a CLSAG prova, sem revelar o membro, conhecimento da chave de gasto e desta diferença de máscaras. Um pseudo-compromisso não é, só por ser publicado, uma prova de propriedade ou de igualdade de montantes. src/ringct/rctSigs.cpp, `genRctSimple()` e `verRctSemanticsSimple()`

Os compromissos das novas saídas são os da Equação (6.5). Para os primeiros $m-1$ pseudo-compromissos, a implementação escolhe máscaras independentes e uniformes em $\mathbb{Z}_l^*$. A última é determinada por

$$x'_m = \sum_{t=0}^{p-1} y_t - \sum_{j=1}^{m-1} x'_j \pmod{l}. \tag{6.9}$$

Logo, $\sum_j x'_j = \sum_t y_t$, e o verificador testa a equação de pontos

$$\boxed{\sum_{j=1}^{m} \widetilde{C}_j^a = \sum_{t=0}^{p-1} C_t^b + fH.} \tag{6.10}$$

Ao expandi-la, o componente em $G$ cancela por construção e sobra a Equação (6.7), interpretada módulo $l$. O público vê os compromissos e $f$, mas não as suas aberturas. Esta verificação prova o balanço dos compromissos; a CLSAG trata da autorização e da escolha anónima da entrada, e as provas de intervalo que se seguem excluem montantes de saída modulares ilegítimos.

## 6.5 Provas de intervalo

A Equação (6.10) é uma igualdade no subgrupo de ordem $l$. Sem outra condição, ela aceita escalares que não são montantes Monero. Por exemplo, tomemos entradas de 6 e 5 unidades e saídas formalmente iguais a 21 e $-10$. No corpo escalar,

$$(6+5) - \big(21 + (l-10)\big) = -l \equiv 0 \pmod{l}.$$

Se as máscaras também se cancelassem, a equação de compromissos passaria, apesar de $l-10$ não ser um montante admissível. O problema não é que a curva tenha inteiros negativos; é que a mesma classe modular admite representantes que dariam a aparência de criar valor.

Uma prova de intervalo exclui esses representantes modulares. Para cada compromisso de saída $C_t^b$, a afirmação exacta é

$$\exists\, y_t \in \mathbb{Z}_l, \quad \exists\, b_t \in \{0, \ldots, 2^{64}-1\} : \quad C_t^b = y_t G + b_t H. \tag{6.11}$$

O montante $b_t$ e a máscara $y_t$ são as testemunhas privadas; o compromisso, os geradores e a prova são públicos. Os extremos são inclusivos: há exactamente $2^{64}$ valores possíveis, todos representáveis nos oito bytes usados pela carteira.

Esta prova não identifica o destinatário, não prova propriedade, não autoriza o gasto, não impede uma dupla despesa e não verifica por si só o balanço da transacção. Prova apenas a relação de intervalo para os compromissos que entram no seu traslado. A validação RingCT combina-a depois com a equação de balanço e com as assinaturas das entradas.

## 6.6 Assinaturas em anel Borromean

As primeiras transacções RingCT provaram a Equação (6.11) com assinaturas em anel Borromean [69, 104]. Esta construção permanece no código para validar a história da lista de blocos, mas deixou de ser permitida em novas transacções depois da versão 8 do protocolo. É portanto um formato histórico, não a prova usada pela versão 16.

src/cryptonote_core/blockchain.cpp, `check_tx_outputs()`; src/ringct/rctSigs.cpp, `proveRange()` e `verRange()`

Vale a pena perceber a ideia, porque nela se vê directamente o intervalo. Escrevemos os 64 bits de $b$ como

$$b = \sum_{i=0}^{63} b_i 2^i, \qquad b_i \in \{0,1\}.$$

O provador escolhe máscaras $x_i \in \mathbb{Z}_l^*$, forma

$$C_i = x_i G + b_i 2^i H \tag{6.12}$$

e obtém

$$C(x,b) = \sum_{i=0}^{63} C_i, \qquad x = \sum_{i=0}^{63} x_i \pmod{l}. \tag{6.13}$$

Para cada índice, publica-se o anel de dois pontos

$$\big(C_i,\ C_i - 2^i H\big).$$

Se $b_i = 0$, o provador conhece $x_i$ tal que $C_i = x_i G$. Se $b_i = 1$, conhece o mesmo $x_i$ tal que $C_i - 2^i H = x_i G$. Uma assinatura em anel esconde qual destes dois casos foi usado; a composição Borromean liga os 64 anéis numa só assinatura.

**Funciona porque** uma assinatura válida demonstra conhecimento do logaritmo discreto, na base $G$, de um ponto em cada par. Sob a vinculação do compromisso e a solidez da assinatura, isso obriga cada $b_i$ a ser um bit. A soma pública dos $C_i$ liga os 64 bits ao compromisso original, pelo que $0 \le b \le 2^{64} - 1$.

As testemunhas privadas são $(b_0, \ldots, b_{63})$ e $(x_0, \ldots, x_{63})$. Os dados públicos são $C$, os 64 pontos $C_i$, os pontos fixos $2^i H$ e a assinatura. O percurso legado codifica cada ponto e escalar em 32 bytes; descodifica os $C_i$ como pontos canónicos da curva e reduz a Keccak–256 a escalares quando encadeia os desafios. Não existe nesse percurso um teste autónomo de pertença de cada $C_i$ ao subgrupo primo, nem existe no traslado um prefixo textual de separação de domínio. São regras históricas de compatibilidade que não devemos transportar para um protocolo novo.

O custo de armazenamento explica a substituição. Para uma única saída, o objecto histórico contém 64 pontos $C_i$, dois vectores de 64 respostas e um desafio: $64+64+64+1 = 193$ objectos de 32 bytes, isto é, 6176 bytes antes de pequenos custos de serialização. Além disso, existe uma prova separada para cada saída. As Bulletproofs seguintes comprimem a relação e agregam várias saídas.

## 6.7 Bulletproofs

*Sou mais rápido quando me mexo.*
*Sundance*

**Em português: provas de bala.**

À partida nem se percebe como é que alguém chega a descobrir o algoritmo anterior, o das assinaturas Borromean. Também não se explicaram todos os vectores de ataque contra essas assinaturas, nem se, para o seu custo de comunicação, eram a forma mais compacta possível. O leitor que pergunta «como é que se chega a tal coisa?» ou «não se pode forjar isto?» ficou sem a história da descoberta.

O que se segue é ainda mais complicado. Não se promete aqui inventar a prova de novo; promete-se algo mais honesto: seguir a bala enquanto ela se mexe, passo a passo, até se perceber por que acerta no alvo. Assim se vê por que o método funciona, que peças impedem a falsificação e onde uma implementação mal feita poderia estragar a magia. Esta é a Bulletproof clássica de

Bünz *et al.* [18, 135], apresentada primeiro para um único montante. A agregação e a passagem para Bulletproof+ virão depois; não se atropela esta batalha com a seguinte.

Fixe-se $n = 64$. Se $v$ está entre 0 e $2^{64} - 1$, passa-se primeiro $v$ para a sua representação binária:

$$\underline{a}_L \leftarrow \text{bits}(v), \qquad v = \langle \underline{a}_L, \underline{2}^n \rangle, \qquad \underline{2}^n = (2^0, 2^1, \ldots, 2^{n-1}).$$

Depois impõem-se dois constrangimentos adicionais:

$$\begin{aligned} v &= \langle \underline{a}_L, \underline{2}^n \rangle \\ \underline{a}_R &= \underline{a}_L - \underline{1} \\ \underline{a}_L \circ \underline{a}_R &= \underline{0}. \end{aligned}$$

O segundo diz que cada coordenada é mesmo um bit: $a_{L,i}(a_{L,i} - 1) = 0$. Agora faz-se o seguinte:

$$\begin{aligned} v &= \langle \underline{a}_L, \underline{2}^n \rangle \\ 0 &= \langle \underline{a}_L - \underline{1} - \underline{a}_R, \underline{y}^n \rangle \\ 0 &= \langle \underline{a}_L \circ \underline{a}_R, \underline{y}^n \rangle. \end{aligned}$$

Aqui entra a primeira pequena faísca. As afirmações vectoriais e as duas igualdades escalares são equivalentes, excepto com probabilidade negligenciável de falhar. Daqui em diante abreviaremos esta ressalva por *enp*: excepto com probabilidade negligenciável. Com $\underline{y}^n = (y^0, y^1, \ldots, y^{n-1})$, qualquer vector $\underline{c} = (c_0, c_1, \ldots, c_{n-1})$ define o polinómio

$$F(y) = c_0 + c_1 y + c_2 y^2 + \cdots + c_{n-1} y^{n-1}.$$

Se $\underline{c} \neq \underline{0}$, este polinómio não nulo tem no máximo $n - 1$ raízes em $\mathbb{Z}_l$. Como o provador já se comprometeu aos vectores antes de o traslado produzir $y$, só pode esperar que o desafio calhe numa dessas raízes. Para $n = 64$, são no máximo 63 valores bons entre os $l$ valores possíveis: a probabilidade de uma relação falsa sobreviver é no máximo $63/l < 2^{-246}$. É isto que quer dizer *enp*; não é impossível por decreto, é tão improvável que se trata como impossível. Se $\underline{c} = \underline{0}$, pelo contrário, $F$ é o polinómio zero e aceita todos os $y$. É precisamente essa a resposta honesta.

Continua-se a representar a mesma coisa de outra forma:

$$\begin{aligned} vz^2 &= \langle \underline{a}_L, \underline{2}^n \rangle z^2 \\ 0z &= \langle \underline{a}_L - \underline{1} - \underline{a}_R, \underline{y}^n \rangle z \\ 0 &= \langle \underline{a}_L \circ \underline{a}_R, \underline{y}^n \rangle \end{aligned}$$

Depois de muita manipulação, $\underline{a}_L$ e $\underline{a}_R$ acabam em lados opostos do mesmo produto interno:

$$z^2 v + \delta(y, z) = \langle \underline{a}_L - z\underline{1}, \underline{y}^n \circ (\underline{a}_R + z\underline{1}) + z^2 \underline{2}^n \rangle. \tag{6.14}$$

$$\delta(y, z) = (z - z^2)\langle \underline{1}, \underline{y}^n \rangle - z^3 \langle \underline{1}, \underline{2}^n \rangle.$$

Para ver cada passo até estas equações, veja-se o Apêndice D. À primeira vista, o que se conseguiu foi representar as três regras iniciais de uma forma altamente elaborada. Mas já há movimento: os 64 constrangimentos cabem agora num produto interno. O provador não pode enviar os vectores do lado direito da Equação (6.14), senão o verificador fica a saber os bits de $v$.

Há portanto que ofuscar $\underline{a}_L$ e $\underline{a}_R$. Por agora, $x$ é a indeterminada formal do polinómio; só depois de $T_1, T_2$ entrarem no traslado se tornará no desafio concreto:

$$\begin{array}{ll} (\underline{a}_L + x\underline{\dot{s}}_L) - z\underline{1} & *^5 \\ \underline{y}^n \circ (\underline{a}_R + x\underline{\dot{s}}_R + z\underline{1}) + z^2\underline{2}^n & *^6 \,. \end{array}$$

Note-se que se alcançou agora um polinómio diferente. Em geral,

$$z^2 v + \delta(y,z) \not\equiv \langle \underline{a}_L + x\underline{\dot{s}}_L - z\underline{1},\, \underline{y}^n \circ (\underline{a}_R + x\underline{\dot{s}}_R + z\underline{1}) + z^2\underline{2}^n \rangle.$$

Define-se então o que já se tinha conseguido como $\underline{l}_0 = \underline{a}_L - \underline{1}z$

$\underline{r}_0 = \underline{y}^n \circ (\underline{a}_R + \underline{1}z) + z^2\underline{2}^n$

ou seja:

$$z^2 v + \delta(y,z) = \langle \underline{l}_0, \underline{r}_0 \rangle = t_0$$

e os factores ofuscantes como $\underline{l}_1 = \underline{\dot{s}}_L$
$\underline{r}_1 = \underline{y}^n \circ \underline{\dot{s}}_R$

Forçam-se então novas igualdades: $*^2, *^3$ $\underline{l}(x) = \underline{l}_0 + \underline{l}_1 x$
$\underline{r}(x) = \underline{r}_0 + \underline{r}_1 x$

Finalmente: $*^4$

$$t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle = t_0 + t_1 x + t_2 x^2$$

Obtém-se assim um polinómio cujo termo constante $t_0 = z^2 v + \delta(y,z)$ está ligado ao compromisso $V$. O provador enviará compromissos e respostas que formam um circuito aritmético: em conjunto, eles obrigam $v$ a satisfazer o domínio requerido sem mostrar os seus bits.

Calculam-se também os coeficientes de $t(x)$: $*^1$

$$t(x) = \langle (\underline{l}_0 + \underline{l}_1 x), (\underline{r}_0 + \underline{r}_1 x) \rangle = \langle \underline{l}_0, \underline{r}_0 \rangle + \langle \underline{l}_0, \underline{r}_1 x \rangle + \langle \underline{l}_1 x, \underline{r}_0 \rangle + \langle \underline{l}_1 x, \underline{r}_1 x \rangle$$

$t_0 = \langle \underline{l}_0, \underline{r}_0 \rangle$

$t_1 x = \langle \underline{l}_0, \underline{r}_1 x \rangle + \langle \underline{l}_1 x, \underline{r}_0 \rangle = \langle \underline{l}_0, \underline{r}_1 \rangle x + \langle \underline{l}_1, \underline{r}_0 \rangle x = (\langle \underline{l}_0, \underline{r}_1 \rangle + \langle \underline{l}_1, \underline{r}_0 \rangle) x$
$t_2 x^2 = \langle \underline{l}_1, \underline{r}_1 \rangle x^2$

$G$, $H$, $\underline{G}$, $\underline{H}$ ${}^{0}\perp[init]$
$V = \gamma G + vH$ ${}^{1}\perp_U(V)$
$\underline{a}_L \leftarrow \text{bits}(v)$
$\underline{a}_R = \underline{a}_L - \underline{1}$
$\dot{\alpha} \leftarrow R()$
$A = \langle \underline{a}_L, \underline{G} \rangle + \langle \underline{a}_R, \underline{H} \rangle + \dot{\alpha} G$ ${}^{2}\perp_U(A)$
$\dot{\rho} \leftarrow R()$
$\underline{\dot{s}}_L \leftarrow R()$
$\underline{\dot{s}}_R \leftarrow R()$
$S = \langle \underline{\dot{s}}_L, \underline{G} \rangle + \langle \underline{\dot{s}}_R, \underline{H} \rangle + \dot{\rho} G$ ${}^{3}\perp_U(S)$
$y \leftarrow^{4} \perp_C(),\ y^{-1}$
$z \leftarrow^{5} \perp_C()$

$$(t_0, t_1, t_2)*^{1}$$

$\dot{\tau_1} \leftarrow R()$
$\dot{\tau_2} \leftarrow R()$
$T_1 = t_1 H + \dot{\tau_1} G$ ${}^{6}\perp_U(T_1)$
$T_2 = t_2 H + \dot{\tau_2} G$ ${}^{7}\perp_U(T_2)$
$x \leftarrow^{8} \perp_C()$
$\tau_x = \dot{\tau_1} x + \dot{\tau_2} x^2 + \gamma z^2$ ${}^{9}\perp_U(\tau_x)$
$\mu = x\dot{\rho} + \dot{\alpha}$ ${}^{10}\perp_U(\mu)$

$$(\underline{l}(x) = \underline{l}_0 + \underline{l}_1 x)*^{2}$$
$$(\underline{r}(x) = \underline{r}_0 + \underline{r}_1 x)*^{3}$$

$$t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle = (t_0 + t_1 x + t_2 x^2)*^{4}$$

${}^{11}\perp_U(t(x))$

$x_{ip} \leftarrow^{12} \perp_C()$
$U = x_{ip} H$

$IP \leftarrow \{\underline{G}, \underline{H'}, U, \underline{l}(x), \underline{r}(x)\}$

O símbolo $\perp$ é o traslado: a memória sequencial da conversa. O operador $\perp_U(\cdot)$ absorve uma mensagem e renova o estado; $\perp_C()$ extrai desse estado um desafio escalar e substitui o estado por esse desafio. Assim, duas chamadas consecutivas fazem avançar o traslado em vez de devolverem o mesmo escalar. O $j$ em ${}^{j}\perp$ marca o instante do traslado, para que provador e verificador voltem a colher o mesmo desafio sem o terem combinado fora da prova.

O diagrama mostra a ordem pedagógica das dependências, não os limites exactos das concatenações na implementação histórica de Monero. Escrevendo $\mathcal{H}_n$ para Keccak–256 seguido

de redução módulo $l$, essa implementação calcula

$$\begin{aligned} h_0 &= \mathcal{H}_n(V_0 \parallel \cdots \parallel V_{p-1}), \\ y &= \mathcal{H}_n(h_0 \parallel A \parallel S), & z &= \mathcal{H}_n(y), \\ x &= \mathcal{H}_n(z \parallel z \parallel T_1 \parallel T_2), \\ x_{ip} &= \mathcal{H}_n(x \parallel x \parallel \tau_x \parallel \mu \parallel t), \\ w_j &= \mathcal{H}_n(w_{j-1} \parallel L_j \parallel R_j), & w_0 &= x_{ip}. \end{aligned}$$

Nestes hashes, $V_j = 8^{-1}C_j$ é a representação interna reconstruída a partir do compromisso normal $C_j$; nas equações algébricas, $V$ continua a designar o ponto matemático já restaurado.

Vectores são sublinhados: $\underline{a}$ contém escalares e $\underline{G}$ pontos da curva elíptica. Um ponto sobre uma letra, como em $\dot{\alpha}$ ou $\underline{\dot{s}}_L$, marca aqui aleatoriedade fresca escolhida por $R()$. Esses segredos não são necessariamente enviados; aparecem escondidos nos compromissos e combinados nas respostas. Já $\underline{G}, \underline{H}$ são geradores públicos determinísticos, que o verificador pode obter sozinho. Nesta apresentação de uma saída, cada vector tem 64 elementos. O argumento de produto interno reduzirá essa dimensão a 1 em $\log_2 64 = 6$ dobras.

Antes das dobras, o provador constrói os dois circuitos aritméticos seguintes. O primeiro amarra o valor à Equação (6.14); o segundo amarra os vectores sem os revelar:

$$\begin{array}{ccccccccc} t(x)H & = & z^2 vH & + & \delta(y,z)H & + & t_1 xH & + & t_2 x^2 H \\ + & & + & & + & & + & & + \\ \tau_x G & = & \gamma z^2 G & + & 0G & + & \dot{\tau}_1 xG & + & \dot{\tau}_2 x^2 G \\ \| & & \| & & \| & & \| & & \| \\ & = & \underbrace{z^2 V + \delta(y,z)H}_{\text{público}} & + & xT_1 & + & x^2 T_2 & & \end{array}$$

As mentes ávidas dizem logo: «Ah e tal, mas $T_1$ e $T_2$ podiam ser qualquer coisa; há ali variáveis aleatórias por todo o lado. Como é que isto diz alguma coisa sobre $v$?» A resposta está no movimento. $T_1$ e $T_2$ comprometem o provador aos coeficientes $t_1, t_2$ antes de o traslado lançar $x$. Depois, uma única avaliação tem de satisfazer simultaneamente $t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle$ e a equação do compromisso de valor. Os factores aleatórios escondem $t_0$, e portanto $v$, do verificador; não dão ao provador liberdade para mudar o polinómio depois de ver $x$. Aqui «público» significa a estrutura formal: $y, z, x$ são recompostos pelo verificador a partir do mesmo traslado.

$$\begin{array}{ccccccccc} \langle \underline{l}(x), \underline{G} \rangle & = & \langle \underline{a}_L, \underline{G} \rangle & + & x\langle \underline{\dot{s}}_L, \underline{G} \rangle & + & \langle -z\underline{1}, \underline{G} \rangle & & *^5 \\ + & & + & & + & & + & & \\ \langle \underline{r}(x), \underline{H'} \rangle & = & \langle \underline{a}_R, \underline{H} \rangle & + & x\langle \underline{\dot{s}}_R, \underline{H} \rangle & + & \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle & & *^6 \\ + & & + & & + & & \| & & \\ \mu G & = & \dot{\alpha} G & + & x\dot{\rho} G & & & & \\ \| & & \| & & \| & & & & \\ & = & A & + & xS & + & \underbrace{\langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle}_{\text{público}} & & \end{array}$$

Em que

$$\underline{H'} = \underline{y}^{-n} \circ \underline{H} \quad \Longleftrightarrow \quad \underline{H} = \underline{y}^{n} \circ \underline{H'}.$$

Esta é uma das partes mais elegantes. Quando se compromete a $\underline{a}_R$ e $\underline{\dot{s}}_R$, o provador ainda não possui $\underline{y}^n$: $y$ só nasce depois do compromisso $S$. A troca para $\underline{H'}$ introduz depois essas potências de forma implícita. Aquilo que parecia chegar tarde entra no circuito pela mudança dos geradores.

Imagine-se que o provador envia:

$t(x)$, $\tau_x$, $\underline{l}(x)$,$\underline{r}(x)$, $\mu$, $T_1$, $T_2$, $A$, $S$

ao verificador. Mesmo recebendo $\underline{l}(x)$ e $\underline{r}(x)$, ele não conseguiria extrair $t_0$, por causa dos factores ofuscantes $\underline{\dot{s}}_L$ e $\underline{\dot{s}}_R$. Já a seguir nem esses dois vectores terão de atravessar a rede.

### 6.7.1 Verificação

Se recebesse simplesmente os dois vectores de 64 elementos, o verificador já conseguiria conferir os dois circuitos. Mas não saltemos por cima da ponte: são mesmo *duas* igualdades.

Do primeiro circuito, o compromisso ao valor, vem

$$t(x)H + \tau_x G = z^2 V + \delta(y,z)H + xT_1 + x^2 T_2. \tag{6.15}$$

Do segundo circuito, o compromisso aos dois vectores, vem

$$\begin{aligned} &\langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H'} \rangle + \mu G \\ &\quad = A + xS + \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned} \tag{6.16}$$

A regra é escolar: se $a = b$ e $c = d$, então $a + c = b + d$. Somando os lados esquerdos das duas igualdades, e fazendo o mesmo aos lados direitos, obtemos

$$\begin{aligned} &t(x)H + \tau_x G + \langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H'} \rangle + \mu G \\ &= z^2 V + \delta(y,z)H + xT_1 + x^2 T_2 + A + xS \\ &\quad + \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned}$$

Esta linha é simplesmente a soma das Equações (6.15) e (6.16). Mas enviar $\underline{l}(x), \underline{r}(x)$ seria carregar 128 escalares.

*A sorte é que:* há uma porta mais estreita.

Depois de $t(x)$ entrar no traslado, ambos os lados extraem $x_{ip}$ e calculam $U = x_{ip}H$. A construção do polinómio dá então a seta decisiva:

$$t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle \quad \stackrel{\times U}{\Longrightarrow} \quad \underbrace{\langle \underline{l}(x), \underline{r}(x) \rangle U}_{\text{vectores}} = \underbrace{\underline{t(x)U}}_{\text{escalar}}.$$

Acrescentemos esse mesmo ponto aos dois lados da igualdade grande:

$$\begin{aligned} &t(x)H + \tau_x G + \underbrace{\langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H'} \rangle + \langle \underline{l}(x), \underline{r}(x) \rangle U}_{P_k} + \mu G \\ &= t(x)U + z^2 V + \delta(y,z)H + xT_1 + x^2 T_2 + A + xS \\ &\quad + \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned}$$

Ponhamos $k = \log_2 64 = 6$. O argumento de produto interno dobra o termo sob a chaveta até restarem os escalares $a, b$:

$$\begin{aligned} P_k &= \langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H'} \rangle + \langle \underline{l}(x), \underline{r}(x) \rangle U \\ &= P_0 - \sum_{j=1}^{k} \left( u_j^2 L_j + u_j^{-2} R_j \right). \end{aligned} \tag{6.17}$$

Substituir a chaveta $P_k$ por esta expressão dá finalmente

$$\begin{aligned} & t(x)H + \tau_x G + P_0 - \sum_{j=1}^{k} \left( u_j^2 L_j + u_j^{-2} R_j \right) + \mu G \\ &= t(x)U + z^2 V + \delta(y,z)H + xT_1 + x^2 T_2 + A + xS \\ &\quad + \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned}$$

pow!

Nem os $u_j$ atravessam a rede: ambos os lados os colhem do traslado. Para uma saída enviam-se $2 \cdot 6 + 9 = 21$ objectos de 32 bytes:

$$t(x), \tau_x, a, b, \mu, T_1, T_2, A, S, \quad L_1, \ldots, L_6, \quad R_1, \ldots, R_6.$$

São 672 bytes, antes dos pequenos prefixos de serialização, em vez dos 6176 bytes da prova Borromean de uma saída. É aqui que a bala começa realmente a ganhar velocidade.

### Mais detalhes do provador

Aqui continua o processo do provador no protocolo do produto interno IP. Começa-se com vectores de dimensão $2^k$, onde $k = 6$, e com

$$P_k = \langle \underline{l}_k(x), \underline{G}_k \rangle + \langle \underline{r}_k(x), \underline{H'}_k \rangle + \langle \underline{l}_k(x), \underline{r}_k(x) \rangle U.$$

Aqui $isto^{\rightarrow}$ significa a primeira metade do vector e $isto^{\leftarrow}$ a segunda. Enquanto $k > 0$, o provador cruza as metades:

$$L_k = \langle \underline{l}_k^{\rightarrow}(x), \underline{G}_k^{\leftarrow} \rangle + \langle \underline{r}_k^{\leftarrow}(x), \underline{H'}_k^{\rightarrow} \rangle + \langle \underline{l}_k^{\rightarrow}(x), \underline{r}_k^{\leftarrow}(x) \rangle U,$$
$$R_k = \langle \underline{l}_k^{\leftarrow}(x), \underline{G}_k^{\rightarrow} \rangle + \langle \underline{r}_k^{\rightarrow}(x), \underline{H'}_k^{\leftarrow} \rangle + \langle \underline{l}_k^{\leftarrow}(x), \underline{r}_k^{\rightarrow}(x) \rangle U.$$

$^{13}\perp_U(L_k)$
$^{14}\perp_U(R_k)$

$$u_k \leftarrow^{15} \perp_C(), \qquad u_k \in \mathbb{Z}_l^*.$$

Os números 13, 14 e 15 marcam apenas a primeira ronda. Na ronda seguinte o traslado continua a contar; não recomeça. Se algum desafio for zero, não existe $u_k^{-1}$: o provador recomeça e o verificador rejeita.

Agora dobram-se testemunhas e geradores:

$$\underline{l}_{k-1}(x) = u_k \underline{l}_k^{\rightarrow}(x) + u_k^{-1} \underline{l}_k^{\leftarrow}(x),$$
$$\underline{r}_{k-1}(x) = u_k^{-1} \underline{r}_k^{\rightarrow}(x) + u_k \underline{r}_k^{\leftarrow}(x),$$
$$\underline{G}_{k-1} = u_k^{-1} \underline{G}_k^{\rightarrow} + u_k \underline{G}_k^{\leftarrow},$$
$$\underline{H'}_{k-1} = u_k \underline{H'}_k^{\rightarrow} + u_k^{-1} \underline{H'}_k^{\leftarrow}.$$

Depois faz-se $k \leftarrow k - 1$. Quando $k = 0$, os dois vectores de testemunhas já são os escalares $a$ e $b$.

Passamos agora à magia em volta de $P_k$. Para chegar ao compromisso dobrado $P_{k-1}$, substituem-se exactamente as quatro dobras:

$$\begin{aligned} P_{k-1} = {} & \left\langle u_k \underline{l}_k^{\rightarrow} + u_k^{-1} \underline{l}_k^{\leftarrow}, u_k^{-1} \underline{G}_k^{\rightarrow} + u_k \underline{G}_k^{\leftarrow} \right\rangle \\ & + \left\langle u_k^{-1} \underline{r}_k^{\rightarrow} + u_k \underline{r}_k^{\leftarrow}, u_k \underline{H'}_k^{\rightarrow} + u_k^{-1} \underline{H'}_k^{\leftarrow} \right\rangle \\ & + \left\langle u_k \underline{l}_k^{\rightarrow} + u_k^{-1} \underline{l}_k^{\leftarrow}, u_k^{-1} \underline{r}_k^{\rightarrow} + u_k \underline{r}_k^{\leftarrow} \right\rangle U. \end{aligned}$$

O $u_k$ de cada termo diagonal encontra o seu inverso e desaparece. Os termos cruzados não desaparecem: ficam marcados por $u_k^2$ ou $u_k^{-2}$.

Agora ponhamo-los lado a lado, como peças de um fecho éclair:

$$
\begin{array}{ll}
 & P_{k-1} = \\
\langle \underline{l}_k^{\rightarrow}(x), \underline{G}_k^{\rightarrow} \rangle & +u_k^2 \langle \underline{l}_k^{\rightarrow}(x), \underline{G}_k^{\leftarrow} \rangle \\
+\langle \underline{l}_k^{\leftarrow}(x), \underline{G}_k^{\leftarrow} \rangle & +u_k^{-2} \langle \underline{l}_k^{\leftarrow}(x), \underline{G}_k^{\rightarrow} \rangle \\
+\langle \underline{r}_k^{\rightarrow}(x), \underline{H'}_k^{\rightarrow} \rangle & +u_k^{-2} \langle \underline{r}_k^{\rightarrow}(x), \underline{H'}_k^{\leftarrow} \rangle \\
+\langle \underline{r}_k^{\leftarrow}(x), \underline{H'}_k^{\leftarrow} \rangle & +u_k^2 \langle \underline{r}_k^{\leftarrow}(x), \underline{H'}_k^{\rightarrow} \rangle \\
+\langle \underline{l}_k^{\rightarrow}(x), \underline{r}_k^{\rightarrow}(x) \rangle U & +u_k^2 \langle \underline{l}_k^{\rightarrow}(x), \underline{r}_k^{\leftarrow}(x) \rangle U \\
\underbrace{+\langle \underline{l}_k^{\leftarrow}(x), \underline{r}_k^{\leftarrow}(x) \rangle U}_{P_k} & \underbrace{+u_k^{-2} \langle \underline{l}_k^{\leftarrow}(x), \underline{r}_k^{\rightarrow}(x) \rangle U}_{u_k^2 L_k + u_k^{-2} R_k}
\end{array}
.
$$

clik!

E como tal possibilita-se a seguinte recursão:

$$
\begin{array}{c}
P_k = P_{k-1} - \left( u_k^2 L_k + u_k^{-2} R_k \right) \\
\downarrow \\
P_{k-1} = P_{k-2} - \left( u_{k-1}^2 L_{k-1} + u_{k-1}^{-2} R_{k-1} \right) \\
\downarrow \\
\vdots \\
\downarrow \\
P_1 = P_0 - \left( u_1^2 L_1 + u_1^{-2} R_1 \right).
\end{array}
$$

Logo,

$$
P_k = P_0 - \sum_{j=1}^{k} \left( u_j^2 L_j + u_j^{-2} R_j \right), \tag{6.18}
$$

$$
P_0 = a^0 \underline{G} + b^0 \underline{H'} + abU. \tag{6.19}
$$

A floresta de 64 coordenadas acabou em dois escalares, $a, b$, e dois pontos finais que o verificador consegue reconstruir. A Equação (6.17), prometida acima, já não é um truque por explicar.

### O verificador e o fim da batalha

Os escalares finais $a, b$ têm de ser transmitidos. Os pontos finais ${}^0\underline{G}$ e ${}^0\underline{H'}$, porém, são reconstruídos pelo verificador. É aqui que as setas mostram todo o caminho. Para não esconder a mecânica atrás de reticências, imagine-se um vector inicial com oito pontos e escreva-se $G_{q,i}$ para o elemento $i$ de $\underline{G}_q$:

$$\begin{bmatrix} G_{3,0} \\ G_{3,1} \\ G_{3,2} \\ G_{3,3} \\ G_{3,4} \\ G_{3,5} \\ G_{3,6} \\ G_{3,7} \end{bmatrix} \xrightarrow{u_3} \begin{bmatrix} G_{2,0} = u_3^{-1}G_{3,0} + u_3 G_{3,4} \\ G_{2,1} = u_3^{-1}G_{3,1} + u_3 G_{3,5} \\ G_{2,2} = u_3^{-1}G_{3,2} + u_3 G_{3,6} \\ G_{2,3} = u_3^{-1}G_{3,3} + u_3 G_{3,7} \end{bmatrix}.$$

$$\begin{bmatrix} G_{1,0} = u_2^{-1}G_{2,0} + u_2 G_{2,2} \\ G_{1,1} = u_2^{-1}G_{2,1} + u_2 G_{2,3} \end{bmatrix} \xleftarrow{u_1} \left[ {}^0\underline{G} = u_1^{-1}G_{1,0} + u_1 G_{1,1} \right]$$

Cada seta corta o vector ao meio e volta a cosê-lo com um desafio e o seu inverso. Nenhuma coordenada desaparece; muda apenas o peso com que chega ao último ponto.

Façamos o jogo das setas. Cada $G_{3,i}$ começa com uma mochila vazia. Ao passar por uma seta, guarda nela o factor escrito por cima. As setas abaixo seguem a *contribuição* de cada ponto; não afirmam que, por exemplo, $G_{3,0} = G_{2,0}$, pois $G_{2,0}$ recebe também $G_{3,4}$.

$$
\begin{aligned}
&G_{3,0} \xrightarrow{u_3^{-1}} G_{2,0} \xrightarrow{u_2^{-1}} G_{1,0} \xrightarrow{u_1^{-1}} {}^{0}\underline{G}, && \phi_0 = u_3^{-1}u_2^{-1}u_1^{-1}, && 0 = 000_2,\\
&G_{3,1} \xrightarrow{u_3^{-1}} G_{2,1} \xrightarrow{u_2^{-1}} G_{1,1} \xrightarrow{u_1} {}^{0}\underline{G}, && \phi_1 = u_3^{-1}u_2^{-1}u_1, && 1 = 001_2,\\
&G_{3,2} \xrightarrow{u_3^{-1}} G_{2,2} \xrightarrow{u_2} G_{1,0} \xrightarrow{u_1^{-1}} {}^{0}\underline{G}, && \phi_2 = u_3^{-1}u_2u_1^{-1}, && 2 = 010_2,\\
&G_{3,3} \xrightarrow{u_3^{-1}} G_{2,3} \xrightarrow{u_2} G_{1,1} \xrightarrow{u_1} {}^{0}\underline{G}, && \phi_3 = u_3^{-1}u_2u_1, && 3 = 011_2,\\
&G_{3,4} \xrightarrow{u_3} G_{2,0} \xrightarrow{u_2^{-1}} G_{1,0} \xrightarrow{u_1^{-1}} {}^{0}\underline{G}, && \phi_4 = u_3u_2^{-1}u_1^{-1}, && 4 = 100_2,\\
&G_{3,5} \xrightarrow{u_3} G_{2,1} \xrightarrow{u_2^{-1}} G_{1,1} \xrightarrow{u_1} {}^{0}\underline{G}, && \phi_5 = u_3u_2^{-1}u_1, && 5 = 101_2,\\
&G_{3,6} \xrightarrow{u_3} G_{2,2} \xrightarrow{u_2} G_{1,0} \xrightarrow{u_1^{-1}} {}^{0}\underline{G}, && \phi_6 = u_3u_2u_1^{-1}, && 6 = 110_2,\\
&G_{3,7} \xrightarrow{u_3} G_{2,3} \xrightarrow{u_2} G_{1,1} \xrightarrow{u_1} {}^{0}\underline{G}, && \phi_7 = u_3u_2u_1, && 7 = 111_2.
\end{aligned}
$$

O padrão já se vê sem pedir licença: lendo os bits da esquerda para a direita, o primeiro escolhe o expoente de $u_3$, o segundo o de $u_2$ e o terceiro o de $u_1$; 0 escolhe $-1$, 1 escolhe $+1$. Escrevendo a regra com $m = 1$ a partir do bit menos significativo, a seta geral é

$$
G_{k,i} \longrightarrow G_{k,i} \prod_{m=1}^{k} u_m^{b(i,m)}, \qquad b(i,m) = \begin{cases} -1, & \text{se o } m\text{-ésimo bit de } i \text{ for 0,} \\ +1, & \text{se o } m\text{-ésimo bit de } i \text{ for 1,} \end{cases} \tag{6.20}
$$

e o verificador forma o vector $\underline{\phi}$ juntando o conteúdo das oito mochilas:

$$
\phi_i = \prod_{m=1}^{k} u_m^{b(i,m)}, \qquad {}^{0}\underline{G} = \langle \underline{\phi}, \underline{G}_k \rangle.
$$

Para $\underline{H'}$, cada seta troca o sinal do expoente. O espelho fica visível nesta pequena tabela:

| | bit 0 | bit 1 |
|---|---|---|
| $\underline{G}$ | $u_m^{-1}$ | $u_m$ |
| $\underline{H'}$ | $u_m$ | $u_m^{-1}$ |

$$
\Longrightarrow \qquad \varphi_i = \phi_i^{-1}.
$$

Portanto,

$$
\varphi_i = \prod_{m=1}^{k} u_m^{-b(i,m)}, \qquad {}^{0}\underline{H'} = \langle \underline{\varphi}, \underline{H'}_k \rangle.
$$

Este sinal contrário é essencial. A versão anterior escrevia os dois padrões com o mesmo sinal, o que estava incorrecto.

O verificador refaz agora toda a viagem do traslado:

$$\begin{array}{ll} G, H, \underline{G}, \underline{H} & {}^0\!\perp[init] \\ V & {}^1\!\perp_U(V) \\ A & {}^2\!\perp_U(A) \\ S & {}^3\!\perp_U(S) \\ y \leftarrow^4 \perp_C(), \quad y^{-1}, & z \leftarrow^5 \perp_C(). \end{array}$$

Com esses desafios calcula

$$\delta(y,z) = (z - z^2)\langle \underline{1}, \underline{y}^n \rangle - z^3 \langle \underline{1}, \underline{2}^n \rangle.$$

Depois continua:

$$\begin{array}{ll} T_1 & {}^6\!\perp_U(T_1) \\ T_2 & {}^7\!\perp_U(T_2) \\ x \leftarrow^8 \perp_C() & \\ \tau_x & {}^9\!\perp_U(\tau_x) \\ \mu & {}^{10}\!\perp_U(\mu) \\ t(x) & {}^{11}\!\perp_U(t(x)) \\ x_{ip} \leftarrow^{12} \perp_C(), \qquad U = x_{ip} H. & \end{array}$$

Finalmente, para $j = k, k-1, \ldots, 1$, absorve $L_j$, absorve $R_j$ e extrai $u_j$. Os estados 13, 14 e 15 são a primeira destas rondas; os números continuam a avançar nas cinco seguintes. Assim o verificador não recebe $y, z, x, x_{ip}, u_1, \ldots, u_k$: reencontra-os.

Agora está armado para a verificação final. Defina-se

$$\Sigma_{\text{IP}} = \sum_{j=1}^{k} \left( u_j^2 L_j + u_j^{-2} R_j \right).$$

Não saltemos logo para a última linha: devolvamos ao fim da batalha as equações que mostram como cada peça cai no seu lugar. A primeira equação de verificação, depois de substituir os dois vectores por $P_k - t(x)U$, é

$$\begin{aligned} & t(x)H + \tau_x G + P_k - t(x)U + \mu G \\ &= z^2 V + \delta(y,z)H + xT_1 + x^2 T_2 + A + xS \\ &\quad + \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned} \tag{6.21}$$

Substituam-se agora as Equações (6.18) e (6.19). A floresta dobrada reaparece só nos seus dois escalares finais:

$$\begin{aligned} & t(x)H - t(x)U + (\tau_x + \mu)G + a^0 \underline{G} + b^0 \underline{H'} + abU - \Sigma_{\text{IP}} \\ &= z^2 V + \delta(y,z)H + xT_1 + x^2 T_2 + A + xS \\ &\quad + \langle z\underline{y}^n + z^2 \underline{2}^n, \underline{H'} \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned} \tag{6.22}$$

Passando tudo para o lado direito, sem esconder nenhum sinal,

$$\begin{aligned}\mathcal{O} = {} & z^2 V + xT_1 + x^2 T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\mathrm{IP}} \\ & + [\delta(y,z) - t(x)]H + [t(x) - ab]U \\ & + \langle z\underline{y}^n + z^2\underline{2}^n, \underline{H'}\rangle + \langle -z\underline{1}, \underline{G}\rangle \\ & - a^0\underline{G} - b^0\underline{H'}. \end{aligned} \tag{6.23}$$

O verificador reconstruiu ${}^0\underline{G} = \langle \underline{\phi}, \underline{G}\rangle$ e ${}^0\underline{H'} = \langle \underline{\varphi}, \underline{H'}\rangle$. Logo, as duas últimas linhas juntam-se aos produtos internos públicos:

$$\begin{aligned}\mathcal{O} = {} & z^2 V + xT_1 + x^2 T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\mathrm{IP}} \\ & + [\delta(y,z) - t(x)]H + [t(x) - ab]U \\ & + \langle -z\underline{1} - a\underline{\phi}, \underline{G}\rangle \\ & + \langle z\underline{y}^n + z^2\underline{2}^n - b\underline{\varphi}, \underline{H'}\rangle. \end{aligned} \tag{6.24}$$

Finalmente usa-se $U = x_{ip}H$. Só agora se chega à equação compacta que a implementação pode verificar como uma soma multiescalar:

$$\begin{aligned}\mathcal{O} = {} & z^2 V + xT_1 + x^2 T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\mathrm{IP}} \\ & + \big[\delta(y,z) - t(x) + (t(x) - ab)x_{ip}\big]H \\ & + \langle -z\underline{1} - a\underline{\phi}, \underline{G}\rangle \\ & + \langle z\underline{y}^n + z^2\underline{2}^n - b\underline{\varphi}, \underline{H'}\rangle. \end{aligned} \tag{6.25}$$

E, como $\underline{H'} = \underline{y}^{-n} \circ \underline{H}$, a mesma batalha pode terminar toda escrita nos geradores originais:

$$\begin{aligned}\mathcal{O} = (0,1) = {} & z^2 V + xT_1 + x^2 T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\mathrm{IP}} \\ & + \big[\delta(y,z) - t(x) + (t(x) - ab)x_{ip}\big]H \\ & + \big\langle z\underline{1} + \underline{y}^{-n} \circ (z^2\underline{2}^n - b\underline{\varphi}), \underline{H}\big\rangle \\ & + \langle -z\underline{1} - a\underline{\phi}, \underline{G}\rangle. \end{aligned} \tag{6.26}$$

bang!

Se esta soma multiescalar dá a identidade, as codificações são canónicas e todos os desafios que precisam de inverso são não nulos, o verificador aceita. Sob a dificuldade do logaritmo discreto e o modelo Fiat–Shamir, uma prova forjada de que $0 \le v \le 2^{64} - 1$ passa apenas com probabilidade negligenciável. Os bits, a máscara $\gamma$ e os vectores intermédios nunca saíram do provador. As setas mexeram-se; a prova ficou mais pequena; Sundance tinha razão.

## 6.8 Bulletproof+

A Bulletproof clássica já teve o seu palco, as suas setas e o seu fim da batalha. Bulletproof+ não apaga nada disso. É a batalha seguinte: prova a mesma afirmação sobre os montantes, conserva a ideia de dobrar vectores com pares $L_j, R_j$, mas muda a coreografia que liga o compromisso inicial ao fecho.

### 6.8.1 O que fica e o que muda

**Fica.** O compromisso de Pedersen, a decomposição em bits, os desafios de Fiat–Shamir colhidos de um traslado, a agregação de vários montantes e as $\log_2(64M)$ rondas que publicam pares $L_j, R_j$.

**Muda.** Desaparecem do objecto transmitido

$$S,\ T_1,\ T_2,\ \tau_x,\ \mu,\ t,\ a,\ b.$$

Em seu lugar aparecem $A_1, B, r_1, s_1, d_1$, com novas equações de fecho, um traslado separado por domínio e uma equação de verificação própria. As setas continuam a dobrar vectores; já não se pode, porém, pegar na Equação (6.26) e trocar apenas os nomes.

As duas formas serializadas tornam a diferença visível:

$$\Pi_{\mathrm{BP}} = (A, S, T_1, T_2, \tau_x, \mu, \boldsymbol{L}, \boldsymbol{R}, a, b, t),$$
$$\Pi_{+} = (A, A_1, B, r_1, s_1, d_1, \boldsymbol{L}, \boldsymbol{R}).$$

Para a mesma dimensão, Bulletproof+ retira três objectos de 32 bytes. A agregação, note-se, não nasceu com o sinal +: a Bulletproof clássica já agregava montantes. O ganho estrutural de Bulletproof+ está no fecho mais curto e na verificação reorganizada.

A ideia algébrica continua a começar em bits. Para $p$ montantes, ponhamos

$$N = 64, \qquad M = 2^{\lceil \log_2 p \rceil}, \qquad n = MN,$$

com $M = 1$ quando $p = 1$. Concatenam-se os bits dos $p$ montantes num vector $\boldsymbol{a}_L \in \mathbb{Z}_l^n$; as $M - p$ posições de montante restantes são preenchidas implicitamente com zeros. Definimos

$$\boldsymbol{a}_R = \boldsymbol{a}_L - \mathbf{1}.$$

As duas relações

$$b_j = \left\langle \boldsymbol{a}_L^{(j)}, \mathbf{2}^{64} \right\rangle, \tag{6.27}$$

$$\boldsymbol{a}_L \circ \boldsymbol{a}_R = \mathbf{0} \tag{6.28}$$

dizem, respectivamente, que cada valor é a soma dos seus bits e que cada coordenada é 0 ou 1. Aqui $\mathbf{2}^{64} = (1, 2, \ldots, 2^{63})$, $\circ$ é o produto coordenada a coordenada e $\boldsymbol{a}_L^{(j)}$ é o bloco de 64 coordenadas do montante $j$, para $0 \le j < p$.

O provador compromete-se com estes vectores sem os revelar. Cada dobra publica dois pontos, pelo que são necessárias $\log_2(64M)$ rondas em vez de 64 compromissos por saída. A verificação continua a ter trabalho proporcional a $64M$: a agregação poupa sobretudo espaço, não transforma toda a verificação num cálculo logarítmico.

### 6.8.2 Dos tipos históricos ao Bulletproof+

Há três nomes parecidos que convém não misturar:

`RCTTypeBulletproof.` A versão 8 admitiu a prova Bulletproof original. O objecto serializado contém

$$\Pi_{\mathrm{BP}} = (A, S, T_1, T_2, \tau_x, \mu, \boldsymbol{L}, \boldsymbol{R}, a, b, t).$$

A versão 9 deixou de admitir as provas Borromean em novas transacções.

`RCTTypeBulletproof2.` A versão 10 admitiu este tipo e a versão 11 tornou-o obrigatório em lugar do tipo anterior. O sufixo «2» não designa Bulletproof+: a prova continua a usar exactamente a estrutura $\Pi_{\mathrm{BP}}$ e o mesmo verificador. A revisão reduziu os dados ECDH por saída para o montante cifrado de oito bytes e passou a derivar a máscara, como vimos na Secção 6.3.

`RCTTypeBulletproofPlus.` A versão 15 admitiu Bulletproof+; a versão 16 proibiu os tipos que ainda usam a prova Bulletproof anterior. Na rede principal, a versão 16 começou no bloco 2 689 608. Este é o tipo criado para novas transacções na implementação congelada.

Os leitores históricos encontrarão ainda `RCTTypeCLSAG`: esse tipo já usava CLSAG para autorizar entradas, mas conservava a estrutura de prova $\Pi_{\mathrm{BP}}$. O código continua a analisar e a verificar todos estes formatos para ler blocos antigos. Essa compatibilidade não os torna escolhas válidas para construir uma transacção da versão 16.

src/cryptonote_core/blockchain.cpp, `check_tx_outputs()`; src/hardforks/hardforks.cpp

Nas equações da secção clássica usámos os pontos matemáticos já restaurados. No objecto Monero serializado, $A, S, T_1, T_2, \boldsymbol{L}, \boldsymbol{R}$ da Bulletproof clássica eram guardados multiplicados por $8^{-1}$; o verificador volta a multiplicá-los por 8. Bulletproof+ conserva esta convenção para $A, A_1, B, \boldsymbol{L}, \boldsymbol{R}$. É uma representação de implementação, não uma seta nova na álgebra.

Para o mesmo $M$, a parte formada por objectos de 32 bytes ocupa

$$|\Pi_{\mathrm{BP}}| = 32\big(9 + 2\log_2(64M)\big), \tag{6.29}$$

$$|\Pi_{+}| = 32\big(6 + 2\log_2(64M)\big). \tag{6.30}$$

Não se contam aqui os pequenos prefixos de comprimento da serialização. Bulletproof+ retira três objectos, ou 96 bytes: para $M = 2$, são 736 contra 640 bytes; para $M = 16$, 928 contra 832 bytes. A prova Borromean, recordemos, ocupava 6176 bytes *por saída*. Estes números são comparações de formato; não afirmam, sem uma medição separada, que um verificador seja tantas vezes mais rápido do que outro.

src/ringct/rctTypes.h, `Bulletproof` e `BulletproofPlus`; src/cryptonote_basic/cryptonote_format_utils.cpp, `get_pruned_transaction_weight()`

### 6.8.3 Bulletproof+ na versão 16

Para não sobrecarregar a notação anterior, chamemos $v_j$ ao montante e $\gamma_j$ à máscara de compromisso nesta subsecção:

$$C_j = \gamma_j G + v_j H.$$

Uma Bulletproof+ agregada prova, para todos os seus $p$ compromissos,

$$\exists\,(\gamma_j, v_j)_{j=0}^{p-1} : \quad C_j = \gamma_j G + v_j H \quad \wedge \quad 0 \le v_j < 2^{64}. \tag{6.31}$$

As aberturas $(\gamma_j, v_j)$ são as testemunhas. Os compromissos $(C_0, \ldots, C_{p-1})$, os geradores, o traslado e

$$\Pi_+ = (A, A_1, B, r_1, s_1, d_1, \boldsymbol{L}, \boldsymbol{R})$$

são públicos. O vector de compromissos internos, chamado $V$ no código, não é serializado dentro da prova: é reconstruído a partir de `outPk.mask`.

### Agregação e preenchimento

Uma transacção RingCT canónica deste tipo contém exactamente uma Bulletproof+, e essa prova cobre todos os seus compromissos de saída. O formato admite $1 \leq p \leq 16$; a regra geral da versão 16 exige separadamente pelo menos duas saídas numa transacção comum. O valor interno $M$ é a menor potência de dois não inferior a $p$:

| $p$ | 1 | 2 | 3–4 | 5–8 | 9–16 |
|---|---|---|---|---|---|
| $M$ | 1 | 2 | 4 | 8 | 16 |
| $\lvert\boldsymbol{L}\rvert = \lvert\boldsymbol{R}\rvert$ | 6 | 7 | 8 | 9 | 10 |

Os montantes fictícios usados para completar $M$ são zeros implícitos; não criam saídas nem compromissos serializados.

Durante a análise da transacção, o tamanho de $\boldsymbol{L}, \boldsymbol{R}$ tem de corresponder precisamente a esse $M$, o número reconstruído de $V_j$ tem de ser $p$, e o limite é 16. Assim, uma prova para capacidade 16 não é uma codificação canónica de apenas duas saídas.

src/cryptonote_core/tx_verification_utils.cpp, `is_canonical_bulletproof_plus_layout()`; src/ringct/rctTypes.cpp, `n_bulletproof_plus_amounts()`

### As duas representações do compromisso

A estrutura de transacção guarda o compromisso normal $C_j$. O provador, porém, constrói

$$V_j = 8^{-1} C_j = (8^{-1}\gamma_j)G + (8^{-1}v_j)H, \tag{6.32}$$

onde $8^{-1}$ é o inverso do cofactor no corpo $\mathbb{Z}_l$. Na criação, `bulletproof_plus_PROVE()` devolve estes $V_j$; `genRctSimple()` multiplica-os por 8 antes de os colocar em `outPk.mask`. Na leitura acontece o inverso: o analisador multiplica cada compromisso de `outPk` por $8^{-1}$ para restaurar $V_j$.

Os pontos $A, A_1, B, \boldsymbol{L}, \boldsymbol{R}$ são igualmente produzidos e serializados na representação dividida por 8. Antes da equação final, o verificador multiplica todos esses pontos e cada $V_j$ por 8. Isso elimina qualquer componente de torsão e deixa os pontos usados na multiplicação multiescalar no subgrupo de ordem prima. Não se deve, pois, multiplicar `outPk.mask` por 8 mais uma vez: ele já é $C_j$.

src/ringct/rctSigs.cpp, `proveRangeBulletproofPlus()` e `genRctSimple()`; src/cryptonote_basic/cryptonote_format_utils.cpp, `expand_transaction_1()`

### Geradores e traslado

Além de $G$ e $H$, a prova usa vectores públicos $\boldsymbol{G} = (G_0, \ldots, G_{n-1})$ e $\boldsymbol{H} = (H_0, \ldots, H_{n-1})$. A implementação deriva cada ponto dos bytes de $H$, da cadeia ASCII `bulletproof_plus`, da

codificação `varint` do índice e de uma função hash-para-ponto com limpeza pelo cofactor. Uma derivação que produzisse a identidade seria rejeitada. A segurança pressupõe que não se conhecem relações de logaritmo discreto úteis entre estes geradores. Na notação do artigo Bulletproof+, os papéis dos geradores de valor e de máscara aparecem trocados: no código Monero, $H$ leva o montante e $G$ leva a máscara.

O traslado começa num ponto fixo derivado da cadeia `bulletproof_plus_transcript`. Escrevendo $\mathcal{H}_n$ para Keccak–256 seguido de redução módulo $l$, a ordem é:

$$T_1 = \mathcal{H}_n(T_0 \parallel \mathcal{H}_n(V_0 \parallel \cdots \parallel V_{p-1}))\,,$$

$$y = \mathcal{H}_n(T_1 \parallel A), \qquad z = \mathcal{H}_n(y), \tag{6.33}$$

$$T^{(0)} = z, \qquad u_k = T^{(k+1)} = \mathcal{H}_n\big(T^{(k)} \parallel L_k \parallel R_k\big)\,, \tag{6.34}$$

$$e = \mathcal{H}_n\big(T^{(\log_2 n)} \parallel A_1 \parallel B\big)\,. \tag{6.35}$$

A cadeia inicial separa este protocolo de outros usos do hash. Todos os desafios são escalares reduzidos; o provador recomeça e o verificador rejeita se $y, z, u_k$ ou $e$ forem zero.

src/cryptonote_config.h, `HASH_KEY_BULLETPROOF_PLUS_*`; src/ringct/bulletproofs_plus.cc, `get_exponent()` e `transcript_update()`

### Mais detalhes do provador

Definamos o compromisso vectorial

$$\langle \boldsymbol{a}, \boldsymbol{G}\rangle + \langle \boldsymbol{b}, \boldsymbol{H}\rangle = \sum_{i=0}^{n-1} a_i G_i + \sum_{i=0}^{n-1} b_i H_i.$$

O primeiro ponto enviado é

$$A = 8^{-1}(\langle \boldsymbol{a}_L, \boldsymbol{G}\rangle + \langle \boldsymbol{a}_R, \boldsymbol{H}\rangle + \alpha G)\,, \tag{6.36}$$

com $\alpha \in_R \mathbb{Z}_l^*$. Depois dos desafios $y, z$, o código constrói a janela

$$d_{jN+i} = z^{\,2(j+1)} 2^i, \quad 0 \le j < M, \quad 0 \le i < N, \tag{6.37}$$

e os vectores

$$\boldsymbol{a}' = \boldsymbol{a}_L - z\mathbf{1},$$

$$b'_i = a_{R,i} + z + d_i\, y^{\,n-i}. \tag{6.38}$$

A máscara acumulada que liga estes vectores aos compromissos é

$$\alpha' = \alpha + y^{\,n+1} \sum_{j=0}^{p-1} z^{\,2(j+1)} \gamma_j. \tag{6.39}$$

Todas estas igualdades são em $\mathbb{Z}_l$.

O argumento de produto interno dobra $(\boldsymbol{a}', \boldsymbol{b}', \boldsymbol{G}, \boldsymbol{H})$ em cada desafio $u_k$. Cada ronda envia $L_k, R_k$, inclui duas máscaras aleatórias novas e reduz sucessivamente a dimensão até 1. Chamemos $a^\star, b^\star$ aos escalares finais, $G^\star, H^\star$ aos geradores finais e $\alpha^\star$ à máscara acumulada depois de todas as dobras. Com escalares aleatórios não nulos $r, s, d, \eta$, o fecho calculado pelo

código é

$$A_1 = 8^{-1}(rG^\star + sH^\star + dG + y(rb^\star + sa^\star)H)\,, \tag{6.40}$$

$$B = 8^{-1}(\eta G + rysH)\,, \tag{6.41}$$

$$r_1 = r + ea^\star, \qquad s_1 = s + eb^\star, \tag{6.42}$$

$$d_1 = \eta + ed + e^2\alpha^\star. \tag{6.43}$$

Na criação pela carteira, as máscaras do compromisso são derivadas como na Equação (6.4); $\alpha$, as máscaras das rondas e $r, s, d, \eta$ vêm do gerador de escalares uniformes não nulos. Esta aleatoriedade é necessária para ocultar os bits e as aberturas. A ocultação de Pedersen perfeita, a propriedade de conhecimento da prova e a heurística Fiat–Shamir são afirmações distintas; a solidez e o conhecimento de Bulletproof+ são garantias computacionais sob as hipóteses do protocolo, não uma consequência da ocultação perfeita por si só.

### 6.8.4 Verificação

O verificador não recebe os bits nem as máscaras. Recompõe $V_j$ e todos os desafios do traslado das Equações (6.33)– (6.35). Recompõe também as potências de $y, z, u_k$ e os geradores dobrados. As respostas $r_1, s_1, d_1$ substituem os escalares finais nas Equações (6.40)–(6.43). Depois de reunir termos, toda a prova se reduz a uma multiplicação multiescalar que tem de dar a identidade $\mathcal{O}$.

A implementação pode verificar várias provas desta família numa só equação. Se $\mathcal{F}_q$ é a combinação de pontos que deveria ser $\mathcal{O}$ para a prova $q$, escolhe pesos independentes $w_q \in_R \mathbb{Z}_l^*$ e testa

$$\sum_q w_q \mathcal{F}_q = \mathcal{O}. \tag{6.44}$$

Os pesos impedem, excepto com probabilidade desprezável da ordem de $1/l$, que erros de provas diferentes se cancelem de propósito. Isto é *verificação em lote* de várias provas; não é a agregação de vários montantes dentro de uma prova.

No percurso de consenso congelado, `verRctSemanticsSimple()` reúne separadamente as Bulletproof+ e as Bulletproof antigas recebidas no conjunto de transacções que lhe foi passado. Um suplemento de transacções de bloco pode assim ser verificado em lote; a validação isolada de uma transacção chama o mesmo verificador com uma única prova. Não afirmamos que toda a verificação aconteça sempre «por bloco».

src/ringct/rctSigs.cpp, `verRctSemanticsSimple()`; src/cryptonote_core/tx_verification_utils.cpp, `ver_mixed_rct_semantics()`

#### O verificador e o fim da batalha

Antes da equação final, há condições de aceitação que não devem desaparecer atrás da álgebra:

- $r_1, s_1, d_1$ têm de ser codificações escalares canónicas, isto é, menores que $l$. Os desafios produzidos por hash são reduzidos módulo $l$ e têm de ser não nulos.

- $A, A_1, B, V_j, L_k, R_k$ têm de descodificar como pontos comprimidos canónicos sobre a curva. Uma codificação não canónica ou que não representa um ponto provoca rejeição; as excepções de descodificação são convertidas em `false` pelo invólucro RingCT.
- Os pontos fornecidos pela prova são multiplicados pelo cofactor 8 antes da multiplicação multiescalar. Não há, contudo, uma regra separada que rejeite toda ocorrência da identidade: uma identidade honesta é extremamente improvável nos pontos aleatórios, mas o verificador decide-a pela estrutura e pela equação completa. Os geradores derivados são explicitamente não nulos.
- A prova tem de ser não vazia, os vectores $\boldsymbol{L}, \boldsymbol{R}$ têm o mesmo comprimento exacto e esse comprimento tem de concordar com o número de compromissos e com o limite de 16. A transacção RingCT deste tipo tem exactamente uma prova canónica.
- A equação de lote, a correspondência entre prova e `outPk`, a Equação (6.10) e as CLSAG são testes distintos; todos têm de passar.

Chegámos assim à afirmação prometida: para cada compromisso ligado ao traslado de $\Pi_+$, o provador conhece uma abertura cujo montante está em $[0, 2^{64}-1]$. Continuam públicos o número de saídas, os compromissos, o formato e tamanho da prova e a taxa da transacção. Continuam fora desta afirmação a identidade do destinatário, a propriedade da saída, a ausência de dupla despesa, a autorização das entradas e a validade completa da transacção. O próximo capítulo juntará essas peças sem alterar o significado limitado, mas essencial, da prova de intervalo.

src/ringct/bulletproofs_plus.cc, `bulletproof_plus_VERIFY()`; src/ringct/rctOps.cpp, `scalarmult8()`; src/crypto/crypto-ops.c, `ge_frombytes_vartime()`

# CAPÍTULO 7

# Transacções Confidenciais em Anel (RingCT)

Os capítulos 5 e 6 definiram os componentes necessários a uma transacção RingCT. Cada saída contém uma chave de utilização única, uma etiqueta de vista, um montante cifrado e um compromisso. Uma prova Bulletproofs+ demonstra que os montantes de saída pertencem ao intervalo de 64 bits. Cada entrada oculta a saída gasta entre quinze desvios; a CLSAG demonstra conhecimento da chave de gasto e da diferença correcta de máscaras na mesma posição do anel. A imagem de chave permite detectar gastos repetidos, e os pseudo-compromissos das entradas equilibram os compromissos das saídas e a taxa.

É necessário verificar quatro propriedades: o remetente conhece uma chave de gasto em cada anel; o montante da saída real coincide com o usado no pseudo-compromisso; as saídas não criam valor; e a assinatura vincula à autorização os bytes exactos da transacção, incluindo as saídas, a taxa, os anéis e as provas.

As respostas vêm de três construções que já conhecemos:

- os endereços ocultos e as imagens de chave do capítulo 5;
- os compromissos, pseudo-compromissos e provas Bulletproofs+ do capítulo 6;
- a assinatura CLSAG da secção 4.6.

Este capítulo relaciona esses componentes com os bytes que a versão 16 aceita. As afirmações acerca da implementação referem-se à revisão do código Monero identificada no resumo inicial.

Nesse estado, a versão 16 activou-se na rede principal à altura 2 689 608 e é a última versão agendada. Esta descrição é verificável para essa revisão e não afirma que o protocolo permanecerá inalterado.[1]

## 7.1 Formato activo: `RCTTypeBulletproofPlus`

Uma transacção ordinária criada pelo programa de referência na versão 16 tem versão de serialização 2 e tipo RingCT '6', isto é, `RCTTypeBulletproofPlus`. Usa uma CLSAG por entrada, anéis de 16 membros e uma prova Bulletproofs+ agregada para todas as saídas. Tem entre duas e dezasseis saídas. A transacção de mineiro é a excepção importante: tem tipo '0' (`RCTTypeNull`), montantes visíveis, nenhuma CLSAG e nenhuma prova de domínio. A rotina que constrói o bloco cria actualmente uma saída de mineiro; o consenso não contém, porém, uma regra isolada de "exactamente uma saída": verifica a entrada de geração, o tipo, o desbloqueio, os alvos e a soma da recompensa.[2]

O nome merece ser lido com cuidado. *Confidencial* quer dizer que os montantes ficam escondidos em compromissos. *Em anel* quer dizer que a autorização aponta para um conjunto de saídas possíveis, não para uma saída declarada. Nenhuma destas duas propriedades esconde tudo: a taxa, o número de entradas, o número de saídas, os membros de cada anel, as imagens de chave e o campo `extra` continuam públicos.

### Formato histórico: `RCTTypeBulletproof2`

O tipo '4', `RCTTypeBulletproof2`, pertence à história do protocolo. Usava Bulletproofs e MLSAG, e era o tipo activo na versão 12 descrita pela edição anterior deste relatório. O apêndice A conserva uma transacção real desse período como exemplo histórico para a leitura dos bytes, mas não representa uma nova transacção v16. Entre os dois extremos existiu ainda o tipo '5', `RCTTypeCLSAG`, que já trocava MLSAG por CLSAG mas ainda não usava Bulletproofs+. Os tipos '1'–'5' continuam necessários para validar a história da cadeia; na versão 16 não são permitidos em novas transacções ordinárias.[3]

[1] Veja-se `src/hardforks/hardforks.cpp` para a agenda da rede principal e `src/cryptonote_config.h` para as constantes de cada versão.

[2] A admissão dos tipos por versão está em `src/cryptonote_core/blockchain.cpp`; a enumeração dos tipos, em `src/ringct/rctTypes.h`; e a escolha feita pela carteira, em `src/wallet/wallet2.cpp` e `src/ringct/rctSigs.cpp`.

[3] As transições estão codificadas em `src/cryptonote_core/blockchain.cpp`: CLSAG é admitida desde v13, Bulletproofs+ desde v15, e v16 deixa de admitir os Bulletproofs anteriores em novas transacções.

### 7.1.1 Compromissos a montantes e taxas de transacção

Um remetente recebeu $m$ saídas com montantes $a_1, \ldots, a_m$, endereços ocultos $P_{\pi_1,1}, \ldots, P_{\pi_m,m}$ e compromissos

$$C^a_{\pi_j,j} = x_j G + a_j H, \qquad j \in \{1, \ldots, m\}.$$

Ele conhece as chaves privadas de gasto $p_j$ tais que $P_{\pi_j,j} = p_j G$, os montantes $a_j$ e as máscaras $x_j$.

Pretende criar $p$ saídas com montantes $b_0, \ldots, b_{p-1}$ e pagar uma taxa $f$, todos em unidades atómicas, de modo que

$$\sum_{j=1}^{m} a_j = \sum_{t=0}^{p-1} b_t + f. \tag{7.1}$$

A taxa $f$ é pública e serializada como um inteiro de comprimento variável. O mínimo de taxa que os nós costumam exigir para propagar uma transacção é política da carteira e da *mempool*, não esta equação de consenso: um bloco pode conter uma transacção de taxa inferior, desde que esta seja válida nos restantes aspectos. O valor declarado continua, claro, a entrar no equilíbrio e na recompensa do mineiro. Veja-se a secção 8.3.4.

Para cada saída nova a carteira deriva, como na secção 6.3, uma máscara determinística

$$y_t = \mathcal{H}_n(\text{“}\texttt{commitment_mask}\text{”} \,\|\, h_t)$$

e publica o compromisso

$$C^b_t = y_t G + b_t H.$$

Para cada entrada cria também um pseudo-compromisso

$$\widetilde{C}^a_j = x'_j G + a_j H.$$

As primeiras $m-1$ pseudo-máscaras $x'_j$ são escolhidas ao acaso. A última é determinada pela condição de balanço:

$$x'_m = \sum_{t=0}^{p-1} y_t - \sum_{j=1}^{m-1} x'_j \pmod{l}.$$

Por homomorfismo dos compromissos, a equação

$$\sum_{j=1}^{m} \widetilde{C}^a_j \stackrel{?}{=} \sum_{t=0}^{p-1} C^b_t + fH \tag{7.2}$$

verifica ao mesmo tempo o equilíbrio dos montantes e o das máscaras, sem revelar nenhum $a_j$ ou $b_t$. A própria igualdade não prova que o autor possui as entradas. É a CLSAG que prende cada pseudo-compromisso à saída real de um anel.

O programa de referência baralha as saídas antes de lhes atribuir os índices $t$ e ordena as entradas por imagem de chave. São escolhas de construção da carteira; o consenso recebe apenas a ordem final.

### 7.1.2 Dados de cada saída

Para a saída $t$, o remetente escolhe a chave de transacção apropriada e calcula a derivação partilhada $D_t$, o escalar

$$h_t = \mathcal{H}_n(D_t \,\|\, \text{varint}(t))$$

e o endereço oculto do destinatário

$$P_t^b = h_t G + K_t^s.$$

Na versão 16 o alvo serializado é etiquetado: contém $P_t^b$ e o primeiro byte de

$$\mathcal{H}(\text{“}\texttt{view_tag}\text{”} \,\|\, D_t \,\|\, \text{varint}(t)).$$

A etiqueta deixa a carteira rejeitar quase todas as saídas alheias antes de fazer o trabalho mais caro; não é uma prova de propriedade.

O montante cifrado contém apenas oito bytes:

$$\text{LE8}(b_t) \oplus \mathcal{H}(\text{“}\texttt{amount}\text{”} \,\|\, h_t)[0 \ldots 7].$$

A máscara não é transmitida cifrada ao lado do montante. O destinatário volta a calcular $y_t$ pela fórmula anterior e confirma que o compromisso publicado é $C_t^b = y_t G + b_t H$. Esta confirmação é importante: decifrar oito bytes com aspecto razoável não basta para possuir uma saída. [4]

Uma transacção publica uma chave principal de transacção em `extra`. Quando há sub-endereços, pode publicar ainda uma lista de chaves adicionais, uma por saída, seguindo exactamente os casos da secção 5.2.1. Essas chaves permitem ao destinatário obter $D_t$; não demonstram ao consenso que certo endereço foi o destinatário pretendido.

### 7.1.3 Uma prova Bulletproofs+ para todas as saídas

Os compromissos $C_0^b, \ldots, C_{p-1}^b$ entram numa única prova Bulletproofs+ que mostra

$$b_t \in [0, 2^{64} - 1] \qquad \text{para } 0 \le t < p.$$

Na transacção ordinária v16, $2 \le p \le 16$. A prova é preenchida internamente até

$$M = 2^{\lceil \log_2 p \rceil},$$

e o verificador exige precisamente o menor $M$ que cobre as saídas; não se pode escolher uma prova maior só para mudar o aspecto dos bytes.

O objecto serializado é

$$\Pi_{\text{BPP+}} = (A, A_1, B, r_1, s_1, d_1, \mathbf{L}, \mathbf{R}).$$

Os compromissos $V_t$ não são serializados dentro da prova: o nó reconstrói-os a partir dos $C_t^b$ que já estão na base RingCT. Aplica-se a seguinte convenção de cofactor. `outPk` guarda o compromisso normal $C_t^b$, enquanto a prova usa $V_t = 8^{-1} C_t^b$. Do mesmo modo, $A, A_1, B, L_i, R_i$ são serializados divididos por 8 e o verificador multiplica-os por 8. Esta codificação não altera

[4] A derivação da saída e do *view tag* está em `src/device/device_default.cpp`; a máscara e o XOR de oito bytes, em `src/ringct/rctOps.cpp`; e a forma serializada, em `src/ringct/rctTypes.h`.

o montante comprometido. A prova e a verificação completas estão na secção 6.8. Os compromissos históricos do anel, os compromissos de saída em `outPk` e os pseudo-compromissos são serializados na representação normal; não se lhes aplica esta divisão por 8. [5]

### 7.1.4 Um anel: uma entrada real e quinze desvios

Para cada entrada $j$, a transacção contém 16 índices de saídas históricas. Depois de converter os *offsets* relativos em índices absolutos, o nó obtém o anel público

$$\mathcal{R}_j = \left\{(P_{i,j}, C^a_{i,j})\right\}_{i=0}^{15}. \tag{7.3}$$

Um desses pares, no índice secreto $\pi_j$, é a saída que o remetente possui; os outros quinze são desvios. Os índices absolutos aparecem por ordem crescente e só o primeiro é guardado em absoluto. Por exemplo, $\{7, 11, 15, 20\}$ torna-se $\{7, 4, 4, 5\}$. A repetição de um índice dentro do mesmo anel produz um *offset* relativo zero e é rejeitada.

Na versão 16 todas as entradas RingCT têm exactamente quinze desvios, isto é, dezasseis membros, e todas têm o mesmo tamanho de anel. Isto é consenso. A escolha de *quais* saídas ocupam os quinze lugares é comportamento da carteira. O programa de referência insere a saída real e amostra candidatos históricos com a sua `gamma_picker`: parte de uma distribuição gamma de forma 19,28 e escala 1/1,61, corrige a janela de maturação, estima um índice pela densidade recente de saídas e escolhe ao acaso uma saída dentro do bloco encontrado.[6]

Quem verifica conhece todos os pares de $\mathcal{R}_j$, mas não recebe esses pontos repetidos dentro da assinatura. Recebe os *offsets* e vai buscá-los à cadeia. Antes de verificar a CLSAG confirma que as referências existem, têm tipos admissíveis, já maturaram e estão desbloqueadas.

### 7.1.5 Assinatura

O pseudo-compromisso da entrada $j$ é subtraído a cada compromisso do anel:

$$Q_{i,j} = C^a_{i,j} - \widetilde{C}^a_j.$$

Para os desvios $i \neq \pi_j$, $Q_{i,j}$ é apenas um ponto para o qual o signatário não conhece necessariamente logaritmo em base $G$. Na coluna real, os montantes cancelam:

$$\begin{aligned} Q_{\pi_j,j} &= (x_j G + a_j H) - (x'_j G + a_j H) \\ &= (x_j - x'_j)G = z_j G. \end{aligned} \tag{7.4}$$

Assim, para assinar a entrada $j$, o remetente conhece dois testemunhos na mesma coluna secreta:

$$\begin{aligned} p_j &= \log_G(P_{\pi_j,j}), \\ z_j &= \log_G(Q_{\pi_j,j}) = x_j - x'_j \pmod{l}. \end{aligned}$$

[5] Comparem-se `src/ringct/rctSigs.cpp`, `src/ringct/rctTypes.cpp`, `src/cryptonote_basic/cryptonote_format_utils.cpp` e `src/ringct/bulletproofs_plus.cc`.

[6] Veja-se `src/wallet/wallet2.cpp`, em particular `gamma_picker::pick()` e `wallet2::get_outs()`. Esta distribuição é uma heurística de privacidade da carteira congelada, não uma regra de validade. Também não se deve chamar aos desvios "saídas não gastas": a cadeia não marca uma saída RingCT como gasta, apenas regista imagens de chave.

A CLSAG comprime estas duas relações numa só assinatura, sem revelar $\pi_j$. As imagens ligáveis são

$$I_j = p_j \mathcal{H}_p(P_{\pi_j,j}),$$
$$D_j = z_j \mathcal{H}_p(P_{\pi_j,j}).$$

$I_j$ é guardada na entrada, antes da parte RingCT. A assinatura CLSAG serializa, para um anel v16, dezasseis respostas $s_{0,j}, \ldots, s_{15,j}$, o desafio inicial $c_{1,j}$ e

$$\bar{D}_j = 8^{-1} D_j.$$

Não serializa $I_j$ uma segunda vez. Os pesos de agregação, o ciclo de desafios e as equações completas estão na secção 4.6 e não são repetidos aqui.

Em suma, a *declaração pública* de uma CLSAG desta transacção contém $\mathfrak{m}$, os dezasseis pares $(P_{i,j}, C^a_{i,j})$, $\widetilde{C}^a_j$, $I_j$ e $\bar{D}_j$. Os *testemunhos privados* são o índice $\pi_j$, a chave de utilização única $p_j$ e a diferença de máscaras $z_j = x_j - x'_j$. Os *hashes* de agregação incluem ambas as filas, as imagens e o pseudo-compromisso; o ciclo de desafios usa os dois testemunhos na mesma coluna. Esta coincidência de índice vincula a chave de gasto à diferença correcta de compromissos.

Para $m$ entradas, a assinatura total é

$$\tau = \{\sigma_1(\mathfrak{m}), \ldots, \sigma_m(\mathfrak{m})\}.$$

Cada entrada tem o seu próprio anel e o seu próprio índice real. Todas as CLSAG assinam a mesma mensagem $\mathfrak{m}$, mas cada uma prende também o seu pseudo-compromisso à sua assinatura. A transacção só passa se todas passarem.

### 7.1.6 A mensagem que as CLSAG assinam

As CLSAG dependem de toda a transacção, mas essa afirmação não descreve os bytes com precisão suficiente. Na revisão consultada, a mensagem anterior às CLSAG é

$$\mathfrak{m} = \mathcal{H}(h_0 \,\|\, h_1 \,\|\, h_2), \tag{7.5}$$

onde:

$h_0$ é o *hash* do prefixo serializado: versão, `unlock_time`, entradas (montantes visíveis zero, *offsets* e imagens de chave), saídas (montantes visíveis zero, tipo de alvo, endereços ocultos e etiquetas de vista) e os bytes exactos de `extra`;

$h_1$ é o *hash* da base RingCT serializada: tipo '6', taxa, os oito bytes cifrados de cada montante e os compromissos normais de saída em `outPk`;

$h_2$ é o *hash* da prova Bulletproofs+ pela ordem $A, A_1, B, r_1, s_1, d_1, L_0, \ldots, L_q, R_0, \ldots, R_q$.

Os corpos CLSAG ficam necessariamente de fora, pois ainda estão a ser calculados. Há uma subtileza menos óbvia: os pseudo-compromissos também não entram em $h_0, h_1, h_2$. Em vez disso, a transcrição de cada CLSAG inclui explicitamente o seu próprio $\widetilde{C}^a_j$, juntamente com

$\mathfrak{m}$, os $P_{i,j}$, os $C^a_{i,j}$, $I_j$ e $D_j$. Logo, mudar um pseudo-compromisso invalida a assinatura correspondente; não é, porém, verdade literal que exista um único *hash* de todos os bytes excepto as assinaturas.[7]

Esta disposição prende os destinatários, os anéis, as imagens de chave, a taxa, `extra`, os montantes em código, os compromissos de saída, a prova de domínio e cada pseudo-compromisso à autorização do dono. Uma alteração deixa de fechar pelo menos um ciclo CLSAG.

Cada camada verifica apenas a sua propriedade. Bulletproofs+ não prova propriedade, destinatário, ausência de gasto duplo nem validade completa da transacção. CLSAG não prova que os novos montantes estão no domínio. A equação de equilíbrio não autoriza uma entrada. A transacção só é válida quando estas respostas, a serialização e as regras exteriores de consenso passam em conjunto.

### 7.1.7 Verificação

A implementação divide a verificação RingCT em duas partes com nomes um pouco enganadores:

1. a verificação *semântica* confirma a forma dos vectores, a correspondência entre saídas e montantes em código, a existência de uma única prova Bulletproofs+ com a capacidade exacta, a equação (7.2) e a prova de domínio;
2. a verificação *não semântica* reconstrói os anéis a partir da cadeia e verifica uma CLSAG por entrada contra o seu pseudo-compromisso.

Em redor destas duas partes, o núcleo verifica ainda a versão e o tipo, os alvos das saídas, o número e a igualdade dos tamanhos dos anéis, os *offsets*, a existência e maturidade das referências, as imagens de chave e o limite de peso. "Semântica" não quer dizer aqui "tudo o que a transacção significa"; é o nome histórico da metade que pode ser verificada sem procurar os membros dos anéis.

Os escalares de resposta e desafio CLSAG têm de estar em forma canónica. Os pontos têm de descodificar, $I_j$ não pode ser a identidade e tem de satisfazer $lI_j = 0$; depois de recuperar $D_j = 8\bar{D}_j$, a verificação rejeita também a identidade. Bulletproofs+ verifica os escalares canónicos, a forma dos vectores e as suas próprias equações depois das multiplicações por 8. É por isso demasiado forte dizer simplesmente que "todos os pontos da transacção são testados um a um no subgrupo primo": a implementação usa testes e limpezas de cofactor diferentes em fronteiras diferentes.

[7] A construção está em `src/ringct/rctSigs.cpp`, funções `get_pre_mlsag_hash()`, `genRctSimple()` e `CLSAG_Gen()`; o *hash* final dos três termos está em `src/device/device_default.cpp`.

Os testes locais tornam estas fronteiras visíveis. Os testes de núcleo para Bulletproofs+ rejeitam provas em falta, provas duplicadas e uma prova sobre o compromisso errado; o teste de maleabilidade CLSAG altera $\bar{D}$ por um ponto de torção e espera rejeição; os testes de serialização confirmam que $V_t$ e $I_j$ são reconstruídos, não repetidos no corpo das provas.[8]

### 7.1.8 Evitar o gasto duplo

A imagem de chave $I_j = p_j \mathcal{H}_p(P_{\pi_j,j})$ é determinística para a chave privada de utilização única, mas não revela qual dos dezasseis $P_{i,j}$ lhe corresponde. O núcleo rejeita imagens repetidas dentro da mesma transacção, entre transacções do mesmo bloco e contra o conjunto de imagens já gastas na cadeia. Sob a segurança da CLSAG e da função de *hash* para ponto, uma segunda assinatura com a mesma chave produz a mesma $I_j$: o marcador denuncia a repetição sem desmascarar o membro real.

A igualdade de uma imagem de chave não diz quanto foi gasto, para onde foi, nem qual membro do anel era verdadeiro. Diz apenas que duas tentativas não podem ambas entrar na história canónica. Antes disso, a *mempool* também evita aceitar conflitos que conhece; a regra decisiva é a validação do bloco.

### 7.1.9 Maturação e tempo de desbloqueio

Todas as saídas referidas por um anel v16, verdadeira e desvios, têm de ter pelo menos dez blocos. Uma saída criada no bloco de altura $h$ pode, por esta regra, aparecer pela primeira vez num anel de uma transacção incluída à altura $h + 10$. Não é apenas o comportamento por omissão da carteira: desde v12 o núcleo impõe-o ao conjunto inteiro de referências.

O prefixo contém ainda um único `unlock_time` para todas as saídas da transacção. Valores inferiores a 500 000 000 são interpretados como altura; valores a partir daí, como tempo Unix. Na versão 16, o teste temporal usa o tempo ajustado derivado dos blocos anteriores e uma tolerância de 120 segundos. A carteira de referência constrói transacções ordinárias com `unlock_time`=0, e os nós não propagam normalmente uma transacção ordinária com outro valor; isto é política de *mempool*. O consenso conserva a funcionalidade e verifica também o `unlock_time` original de cada membro do anel.

Uma transacção de mineiro criada à altura $h$ tem obrigatoriamente `unlock_time` $= h + 60$. A sua saída só pode portanto ser referida a partir dessa altura; o bloqueio de 60 domina a maturação ordinária de 10.[9]

[8] Veja-se `tests/core_tests/bulletproof_plus.cpp`, `tests/core_tests/rct2.cpp`, `tests/unit_tests/bulletproofs_plus.cpp` e `tests/unit_tests/serialization.cpp`.

[9] Veja-se `is_tx_spendtime_unlocked()` e a verificação das referências em `src/cryptonote_core/blockchain.cpp`, a política em `src/cryptonote_core/tx_pool.cpp` e a construção da transacção de mineiro em `src/cryptonote_core/cryptonote_tx_utils.cpp`.

### 7.1.10 A relação desconhecida $\gamma = \log_G H$

O ponto $H$ é público e, por pertencer ao subgrupo gerado por $G$, existe matematicamente um escalar $\gamma$ tal que

$$H = \gamma G.$$

A segurança exige que esse escalar seja desconhecido. A construção de $H$ por *hash* para ponto procura impedir que algum participante conheça a relação durante a escolha dos parâmetros. Daqui em diante, toda a aritmética escalar desta secção é módulo $l$.

Suponha-se, contudo, que o Óscar conhece $\gamma$. Compra ou recebe 0,1 XMR numa saída já gastável que possui. Conhece a chave de gasto, o montante $b$, a máscara $y$ e o compromisso histórico

$$C^a = yG + bH.$$

Não precisa de publicar um novo ponto $C^a_{\text{mau}}$. Escolhe um montante alternativo de 64 bits, por exemplo $b_{\text{mau}} = 100$ XMR, e calcula uma segunda abertura do mesmo ponto:

$$y_{\text{mau}} = y + (b - b_{\text{mau}})\gamma \pmod{l}. \tag{7.6}$$

Então

$$\begin{aligned} y_{\text{mau}}G + b_{\text{mau}}H &= \big(y + (b - b_{\text{mau}})\gamma\big)G + b_{\text{mau}}\gamma G \\ &= yG + b\gamma G \\ &= C^a. \end{aligned}$$

As duas aberturas representam montantes diferentes no mesmo ponto publicado na cadeia.

Ao gastar a saída, o Óscar trata $C^a$ como compromisso a $b_{\text{mau}}$. Escolhe uma pseudo-máscara $x'$ e publica

$$\widetilde{C}^a = x'G + b_{\text{mau}}H.$$

Na coluna real do anel,

$$C^a - \widetilde{C}^a = (y_{\text{mau}} - x')G,$$

portanto conhece o segundo testemunho CLSAG $z = y_{\text{mau}} - x'$. Continua a precisar de $p$, a chave de gasto da saída: conhecer $\gamma$ não rouba moedas alheias por si só. Mas permite reabrir as próprias moedas com montantes inventados.

O Óscar cria, por exemplo, duas saídas cuja soma, mais a taxa, é $b_{\text{mau}}$. Escolhe as máscaras de saída, satisfaz (7.2), produz uma Bulletproofs+ válida para esses dois novos montantes de 64 bits e assina cada entrada. Tudo passa: a CLSAG conhece $p$ e $z$, o equilíbrio usa a abertura falsa e a prova de domínio só pergunta se os *novos* montantes de saída estão no domínio. A prova de domínio que acompanhou $C^a$ quando este foi criado demonstrava que existia uma abertura antiga em 64 bits; não demonstrava que essa abertura era única, e o gasto não volta a provar o domínio das entradas.

Sem $\gamma$, obter uma abertura alternativa exigiria

$$y_{\text{mau}} = \log_G(C^a - b_{\text{mau}}H)\,,$$

o cálculo de um logaritmo discreto.

### Extracção a partir de duas aberturas

O ponto essencial é que não conseguimos calcular $\gamma$ olhando apenas para $C^a$. Precisamos de conhecer duas descrições completas do mesmo ponto:

$$C^a = yG + bH,$$
$$C^a = y_{\text{mau}}G + b_{\text{mau}}H.$$

Ou seja, precisamos dos quatro números

$$(y, b) \qquad \text{e} \qquad (y_{\text{mau}}, b_{\text{mau}}).$$

Como as duas expressões dão exactamente o mesmo compromisso, subtraímos uma da outra:

$$\begin{aligned} 0 &= (y - y_{\text{mau}})G + (b - b_{\text{mau}})H \\ &= \big(y - y_{\text{mau}} + (b - b_{\text{mau}})\gamma\big)G. \end{aligned}$$

Na segunda linha substituímos simplesmente $H = \gamma G$. Como $G$ tem ordem prima $l$, o escalar entre parênteses tem de ser zero módulo $l$. Podemos então isolar $\gamma$:

$$\gamma = (y_{\text{mau}} - y)(b - b_{\text{mau}})^{-1} \pmod{l}. \tag{7.7}$$

O inverso existe: dois montantes distintos de 64 bits têm diferença não nula módulo o primo $l$. Encontrar uma segunda abertura não trivial quebra, pois, a vinculação computacional e revela exactamente $\gamma = \log_G H$, a relação de logaritmo discreto entre os geradores.

### Exemplo num grupo de ordem 11

Considere-se um grupo de ordem 11 no qual a relação secreta é

$$H = 4G.$$

Neste exemplo, $\gamma = 4$. Uma primeira abertura é

$$y = 2, \qquad b = 3.$$

O compromisso correspondente é

$$\begin{aligned} C^a &= 2G + 3H \\ &= 2G + 3(4G) \\ &= 14G = 3G \qquad \big(\text{pois } 14 \equiv 3 \pmod{11}\big). \end{aligned}$$

Suponhamos que alguém encontra outra abertura do mesmo $C^a$:

$$y_{\text{mau}} = 5, \qquad b_{\text{mau}} = 5.$$

Com efeito,

$$\begin{aligned} 5G + 5H &= 5G + 5(4G) \\ &= 25G = 3G \qquad \big(\text{pois } 25 \equiv 3 \pmod{11}\big). \end{aligned}$$

As duas aberturas dão o mesmo ponto $C^a = 3G$. Agora usamos os quatro números na equação (7.7):

$$\gamma = (5 - 2)(3 - 5)^{-1} \pmod{11}.$$

Temos $3 - 5 = -2 \equiv 9 \pmod{11}$. O inverso de 9 módulo 11 é 5, porque $9 \cdot 5 = 45 \equiv 1 \pmod{11}$. Assim,

$$\gamma = 3 \cdot 5 = 15 \equiv 4 \pmod{11}.$$

Recuperámos precisamente a relação usada para construir o exemplo.

As duas implicações são:

- se o Óscar conhece $\gamma$, consegue fabricar uma segunda abertura;
- se alguém consegue fabricar uma segunda abertura diferente e conhecemos as duas aberturas, conseguimos extrair $\gamma$.

Um algoritmo que recebesse a primeira abertura e encontrasse eficientemente a segunda dar-nos-ia, pela equação (7.7), um algoritmo para calcular $\log_G H$. Portanto, encontrar uma segunda abertura é pelo menos tão difícil quanto resolver o PLD. $\gamma$ não pode ser extraído apenas de $C^a$; a extracção usa duas aberturas distintas do mesmo compromisso.

Os 64 bits dos montantes também não encolhem o problema para 64 bits. Depois de escolher um candidato $b_{\text{mau}}$, o Óscar consegue calcular o ponto

$$P = C^a - b_{\text{mau}} H,$$

mas continua a precisar do escalar $y_{\text{mau}}$ tal que $P = y_{\text{mau}} G$. Como $G$ gera o subgrupo, qualquer montante candidato produz um ponto que tem algum logaritmo; o compromisso não indica se esse montante corresponde à abertura original. O obstáculo é calcular o logaritmo correspondente, um PLD no grupo inteiro, cujo tamanho é $l$, não uma busca no intervalo dos montantes.

Os melhores ataques *genéricos* clássicos conhecidos, como o método rho de Pollard, precisam de uma ordem de grandeza de

$$\sqrt{l}$$

operações de grupo, devido à complexidade de colisão de ordem $\sqrt{l}$. Aqui $l$ está um pouco acima de $2^{252}$, logo $\sqrt{l}$ está perto de $2^{126}$, cerca de $8{,}5 \times 10^{37}$ operações. À taxa hipotética de $10^{18}$ operações de grupo por segundo, o cálculo levaria cerca de $2{,}7 \times 10^{12}$ anos. Uma operação de grupo real também custa mais do que uma instrução elementar de processador.

"Genérico" quer dizer que o ataque usa apenas as operações do grupo e não uma fraqueza especial da curva, da geração de $H$ ou da implementação. A estimativa $2^{126}$ não é uma prova de que nunca surgirá matemática melhor; é o nível de segurança clássico que resulta dos melhores métodos conhecidos. Um algoritmo assintoticamente ou estruturalmente melhor afectaria todas as construções que dependem do mesmo PLD; aumentar moderadamente os recursos computacionais não altera a ordem de grandeza deste custo.

O segredo de $\gamma$ é, pois, a hipótese de *vinculação* dos compromissos e da oferta monetária. Não é a hipótese de ocultação: com uma máscara uniforme, $yG + bH$ esconde perfeitamente $b$ mesmo de quem conhecesse $\gamma$. Os logaritmos que aparecem ao variar $b_{\text{mau}}$ não são segredos independentes; são transformações afins da mesma relação $\gamma$. Se a relação for conhecida, torna-se possível criar montantes; se permanecer desconhecida, cada transformação exige resolver um PLD.

Três utilizações da letra gamma designam objectos diferentes. Este $\gamma = \log_G H$ é a relação desconhecida entre geradores. A distribuição gamma da `gamma_picker` escolhe idades de desvios. E os símbolos $\gamma_j$ usados por vezes na literatura de Bulletproofs para máscaras de compromissos são apenas escalares escolhidos pelo provador.

### Provas de domínio (saídas)

Uma prova Bulletproofs+ agregada, com $M$ a menor potência de dois tal que $M \geq p$, requer

$$32\big(6 + 2\log_2(64M)\big) \text{ bytes}$$

para os seus elementos de 32 bytes, além dos prefixos de comprimento dos vectores. Para $p = 2$, são 640 bytes; para $p = 16$, 832 bytes.

## 7.2 Resumo conceptual

Para resumir, apresenta-se aqui o conteúdo principal de uma transacção v16, organizado para clareza conceptual. O apêndice A mostra bytes reais de uma transacção mais antiga; as diferenças estão assinaladas no próprio apêndice.

- Tipo: '6' é `RCTTypeBulletproofPlus`; '0' é `RCTTypeNull` para a transacção de mineiro.
- Entradas: para cada entrada $j$:
  - montante visível zero;
  - dezasseis *offsets* relativos que localizam o anel;
  - imagem de chave $I_j$;
  - pseudo-compromisso $\widetilde{C}_j^a$, na parte podável;
  - CLSAG com dezasseis $s_{i,j}$, $c_{1,j}$ e $\bar{D}_j$, também na parte podável.
- Saídas: para cada saída $t$:
  - montante visível zero;
  - alvo etiquetado com o endereço oculto $P_t^b$ e um *view tag* de um byte;
  - compromisso normal $C_t^b = y_t G + b_t H$ em `outPk`;
  - oito bytes $b_t$ cifrados por XOR em `ecdhInfo`.
- Base RingCT: tipo, taxa pública, `ecdhInfo` e `outPk`. Esta parte não é podável.
- Parte podável: uma prova Bulletproofs+, seguida das CLSAG e dos pseudo-compromissos.
- `extra`: bytes opacos no prefixo, onde as carteiras colocam normalmente a chave pública principal de transacção, eventuais chaves adicionais e, quando aplicável, um *nonce*. O consenso assina os bytes exactos, mas não prova que estes obedecem às relações secretas pretendidas pela carteira.

Numa transacção com uma entrada e duas saídas, a validade segundo o consenso e a utilização pelos destinatários dependem das condições seguintes:

- para que a transacção seja utilizável, as chaves públicas de transacção permitam aos destinatários reconhecer as respectivas saídas;
- a CLSAG prove o conhecimento, na mesma posição secreta do anel, da chave privada da saída e da diferença de máscaras entre o compromisso histórico e o pseudo-compromisso;
- a imagem de chave ainda não tenha aparecido num gasto aceite;
- cada saída inclua endereço oculto, etiqueta de vista, compromisso e montante cifrado;
- a Bulletproof+ prove que os dois montantes pertencem ao intervalo de 64 bits;
- a taxa $f$ seja pública e se verifique $\widetilde{C}^a = C_0^b + C_1^b + fH$; e
- a mensagem CLSAG vincule o prefixo, a base RingCT e a prova, enquanto a transcrição CLSAG vincula também o pseudo-compromisso.

Estas condições ocultam a posição real do anel e os montantes transferidos, sem impedir a verificação da autorização, da ausência de gasto duplo, do intervalo dos montantes e do equilíbrio.

### 7.2.1 Requisitos de espaço

Um anel v16 não repete no *blob* os 16 endereços ocultos e os 16 compromissos: os *offsets* apontam para esses dados na cadeia. Por entrada, os elementos fixos directamente ligados ao gasto são:

$$\underbrace{16 \cdot 32 + 32 + 32}_{\text{CLSAG}} + \underbrace{32}_{I_j} + \underbrace{32}_{\widetilde{C}_j^a} = 640 \text{ bytes},$$

antes dos dezasseis *varints* dos *offsets* e dos pequenos prefixos de serialização. A CLSAG propriamente dita ocupa 576 bytes: 512 nas respostas, 32 em $c_1$ e 32 em $\bar{D}$.

Para as saídas, cada $t$ acrescenta pelo menos os 32 bytes do endereço oculto, um byte de *view tag*, 32 bytes do compromisso e oito bytes do montante em código, mais as marcas e comprimentos da serialização. As chaves de transacção vivem em `extra`. A prova Bulletproofs+ custa

$$32\big(6 + 2\log_2(64M)\big), \qquad M = 2^{\lceil \log_2 p \rceil},$$

mais dois comprimentos de vector.

#### Peso, não apenas bytes

Para uma transacção actual, o peso é o tamanho serializado mais uma penalização de "recuperação" da economia obtida pela agregação de Bulletproofs+. Se $M \leq 2$, a penalização é zero

e peso e bytes coincidem. Para $M > 2$, a revisão congelada calcula

$$w = |\text{blob}| + \frac{4}{5}\left[320M - 32\big(6 + 2\log_2(64M)\big)\right],$$

com divisão inteira no último termo. Na versão 16 uma transacção não pode exceder

$$\frac{300\,000}{2} - 600 = 149\,400$$

unidades de peso. A reserva de 600 deixa espaço para a transacção de mineiro; a relação com o peso dinâmico do bloco é explicada na secção 8.3.2.[10]

### O campo `extra`: conteúdo, limite e fronteira

`extra` é um vector de bytes do prefixo. As convenções conhecidas começam cada campo por uma etiqueta: por exemplo, `0x01` para a chave pública principal, `0x02` para um *nonce* com comprimento e `0x04` para chaves públicas adicionais; `0x00` representa enchimento. Portanto, não é verdade que as etiquetas tenham "todos os bits a um".

A carteira de referência ordena campos que consegue analisar e recusa criar `extra` com mais de 1060 bytes. Um nó recusa normalmente propagar pela *mempool* uma transacção que exceda esse valor. O limite de 1060 é política, não uma regra de consenso do bloco: no consenso, `extra` continua limitado indirectamente pelo tamanho e pelo peso de toda a transacção.

Também aqui "não é verificado" precisa de tradução. O vector e o seu comprimento têm de ser serializáveis, e os bytes exactos entram em $h_0$, logo não podem ser alterados depois da assinatura. Mas a validação de um bloco não exige que esses bytes formem todos campos reconhecidos, que haja uma só chave de cada espécie, nem que uma chave de transacção corresponda a um segredo usado pelo remetente. Essas são convenções que permitem às carteiras encontrar e decifrar saídas. Bytes mal formados podem tornar uma saída invisível ou inutilizável sem, por essa razão isolada, tornar inválido o bloco.[11]

É uma fronteira característica de Monero: o consenso protege a integridade dos bytes e as equações monetárias; a carteira dá significado operacional a alguns desses bytes. Confundir as duas coisas faz o protocolo parecer ao mesmo tempo mais rígido e menos seguro do que realmente é.

[10] Veja-se `get_transaction_weight()` e `get_transaction_weight_clawback()` em `src/cryptonote_basic/cryptonote_format_utils.cpp`, e o limite em `src/cryptonote_core/tx_verification_utils.cpp`.

[11] As etiquetas e formatos estão em `src/cryptonote_basic/tx_extra.h`; a análise e ordenação, em `src/cryptonote_basic/cryptonote_format_utils.cpp`; os limites de construção e propagação, em `cryptonote_tx_utils.cpp` e `tx_pool.cpp`.

# CAPÍTULO 8

# A Blockchain

**Em português: a lista de blocos.**

**Nota de leitura.** Neste capítulo, "regra activa" significa uma regra da rede principal no estado técnico identificado no resumo inicial. As afirmações sobre a implementação foram conferidas na mesma revisão do código-fonte de Monero [89]; as notas marginais indicam os ficheiros e funções relevantes. Comportamentos da carteira ou do nó, regras históricas e código inactivo serão assinalados como tais. Esta nota vale para todo o capítulo.

A Internet permite comunicação e troca de informação entre participantes geograficamente distantes. A troca de bens e serviços requer meios para representar e transferir valor; num sistema digital, essas transferências também podem ocorrer electronicamente [75].

Os meios de troca (dinheiros) são essenciais: dão-nos um ponto de referência para uma imensa diversidade de bens económicos que, de outra forma, seriam impossíveis de comparar, e permitem interacções mutuamente benéficas entre pessoas que nada têm em comum [75]. Ao longo da história houve muitos tipos de dinheiro, de conchas a papel e ouro. Foram trocados à mão; agora, o dinheiro também pode ser trocado electronicamente.

No modelo actual, de longe o mais adoptado, as transacções electrónicas são controladas por instituições financeiras. Estas entidades terceiras guardam o dinheiro e recebem a confiança necessária para o transferir. Têm de mediar disputas; os pagamentos são reversíveis; e podem ser censuradas ou controladas por organizações poderosas [95].

Para aliviar estas inconveniências, surgiram moedas digitais descentralizadas.[1]

[1] Este capítulo inclui mais detalhes de implementação do que os anteriores, pois a natureza da lista de blocos depende muito da sua estrutura específica.

## 8.1 Moedas digitais

Considerem-se três modelos de moeda digital: pessoal, centralizada e distribuída. Uma moeda digital é uma colecção de mensagens; os "montantes" guardados nelas são interpretados como quantidades monetárias.

No **modelo sem registo comum**, qualquer pessoa pode declarar a criação de moedas e comunicar essa declaração aos restantes participantes. A quantidade total não é limitada e é possível gastar as mesmas moedas mais do que uma vez (gasto duplo).

No **modelo de base de dados centralizada**, as moedas são guardadas numa base de dados centralizada. Os utilizadores confiam na honestidade do guardião. Observadores externos não conseguem verificar a quantidade total; o guardião pode alterar as regras a qualquer altura ou ser censurado por terceiros poderosos.

### 8.1.1 Versão de eventos comuns e distribuídos

No dinheiro digital "comum", muitos computadores possuem o registo de cada transacção. Quando um deles recebe uma nova transacção, transmite-a à rede, que a aceita se ela seguir regras predefinidas.

Os utilizadores só beneficiam das moedas se outros as aceitarem em troca. Para maximizar a sua utilidade, existe uma tendência natural para adoptar um conjunto de regras comuns sem autoridade central.[2]

**Regra n.º 1**: Dinheiro só pode ser criado em cenários bem definidos.

**Regra n.º 2**: As transacções gastam dinheiro que já existe.

**Regra n.º 3**: Cada transacção só ocorre uma vez.

**Regra n.º 4**: Só o proprietário do montante o pode gastar.

**Regra n.º 5**: As saídas de transacção representam o dinheiro gasto.

**Regra n.º 6**: As transacções seguem a sintaxe correcta.

As regras 2–6 estão contidas no esquema do capítulo 7, que acrescenta fungibilidade e privacidade: signatário ambíguo, destinatário anónimo e montantes ocultos a terceiros. A primeira regra será explicada mais adiante. Como as transacções usam criptografia, trata-se de uma *criptomoeda*.

[2] Na ciência política diz-se a isto um contracto social.

Considere-se que dois computadores distantes recebem, quase ao mesmo tempo, transacções distintas que gastam a mesma saída. Pouco depois, ambos conhecem as duas: como decidem qual é válida? O histórico divide-se em duas cópias que, à primeira vista, seguem as mesmas regras — uma bifurcação (*fork*).

É necessário determinar qual transacção será canónica. Obter consenso sobre o histórico de transacções é a função principal da blockchain, isto é, da lista de blocos.

### 8.1.2 Lista de blocos simples

Primeiro é necessário que todos os computadores, doravante *nós*, concordem com a ordem das transacções.

Digamos que uma criptomoeda começa com uma declaração de génese: "A moeda exemplo começa!"

Esta mensagem é, neste caso, um "bloco", e o seu ID é:

$$BH_G = \mathcal{H}(\text{“A moeda exemplo começa!”})$$

Cada vez que um nó recebe transacções, calcula os seus IDs $TH$; cada bloco recebe também um ID $BH$. Como cada bloco aponta para o anterior, uma lista de blocos é um vector unidimensional cujo primeiro elemento é o bloco de génese:

$$BH_1 = \mathcal{H}(BH_G, TH_1, TH_2, ...)$$
$$BH_2 = \mathcal{H}(BH_1, TH_3, TH_4, ...)$$

Desta forma há uma ordem clara de eventos, do bloco actual até à génese. Com os seus ramos alternativos, a estrutura é um grafo acíclico dirigido; o ramo canónico de Monero é a sua variante unidimensional. As arestas são setas unidireccionais e nenhum caminho regressa ao nó de onde partiu [139].

Os mineiros incluem data e hora nos blocos. Se a maioria for razoavelmente honesta, a lista dá também uma aproximação da altura temporal de cada transacção; veremos depois os limites impostos ao carimbo. Se dois blocos referem o mesmo antecessor e se propagam ao mesmo tempo, metade da rede pode receber um primeiro e a outra metade o outro. Forma-se então uma bifurcação, cuja resolução é descrita a seguir.

## 8.2 Dificuldade

Se os nós pudessem publicar blocos quando quisessem, a rede poderia divergir em listas igualmente legítimas. E, se um bloco demora 30 segundos a propagar-se, o que aconteceria se

surgissem blocos a cada 31 segundos? Ou a cada 10? É possível controlar a velocidade a que a rede cria blocos. Se o tempo de geração for muito maior do que o tempo necessário para o bloco anterior atingir a maioria dos nós, a rede tenderá a permanecer unida.

### 8.2.1 Mineração de um bloco

O resultado de uma função hash criptográfica parece uniformemente distribuído e independente do argumento de entrada. Isto significa que, perante entradas novas, cada resultado possível parece igualmente provável. Além disso, cada tentativa exige algum tempo de cálculo. Considere-se uma função hash $\mathcal{H}_i(x)$ cujo resultado é um número de 1 a 100:[3]

$$\mathcal{H}_i(x) \in_R^D \{1, \ldots, 100\}$$

Para um dado $x$, $\mathcal{H}_i(x)$ devolve sempre o mesmo número pseudo-aleatório de $\{1, \ldots, 100\}$. Suponha-se que demora um minuto a calculá-lo. Seja agora uma mensagem $\mathfrak{m}$. O objectivo é encontrar um inteiro $n$ tal que $\mathcal{H}_i(\mathfrak{m}, n)$ não exceda o *alvo* $t = 5$:

$$\mathcal{H}_i(\mathfrak{m}, n) \in \{1, \ldots, 5\}$$

Como só 1/20 dos resultados cai no alvo, esperamos cerca de 20 tentativas para encontrar um $n$ que funcione: aproximadamente 20 minutos de cálculo. Procurar esse argumento, ou *nonce*, é *mineração*; publicar a mensagem com $n$ é uma *prova de trabalho*. Qualquer pessoa a verifica calculando uma única vez $\mathcal{H}_i(\mathfrak{m}, n)$.

Seja uma função hash usada para provas de trabalho:

$$\mathcal{H}_{PoW} \in_R^D \{0, \ldots, m\}.$$

Aqui, $m$ é o resultado máximo. Dada uma mensagem $\mathfrak{m}$, um argumento $n$ e um alvo $t$, podemos aproximar o número esperado de tentativas por

$$d = m/t.$$

Chamamos *dificuldade* a $d$.

Se $\mathcal{H}_{PoW}(\mathfrak{m}, n)d \leq m$, então $\mathcal{H}_{PoW}(\mathfrak{m}, n) \leq t$ e $n$ é aceitável.[4] Alvos menores aumentam a dificuldade e o número esperado de tentativas. Verificar continua a exigir uma avaliação da função de prova de trabalho, independentemente de quão afortunado foi o mineiro.

src/cryptonote_basic/difficulty.cpp `check_hash()`

### 8.2.2 Velocidade de mineração

Assuma-se que todos os nós estão a minerar ao mesmo tempo, mas param quando recebem um novo bloco. Começam imediatamente outro que referencia o último aceite.

Recolhamos $b$ blocos recentes, indexados por $u \in \{1, \ldots, b\}$, cada um com dificuldade $d_u$. Por enquanto, admitamos carimbos temporais honestos. O intervalo entre o mais antigo e o mais

[3] Usa-se $\in_R^D$ para dizer que o resultado se comporta como uma escolha uniforme no conjunto indicado.

[4] Monero guarda e calcula a dificuldade; testa directamente o produto, sem precisar de materializar $t$.

recente é

$$Tempo \quad total = TS_b - TS_1.$$

O trabalho esperado para os blocos no intervalo é [5]

$$Dificuldade\ total = \sum_{u=1}^{b} d_u.$$

Se a potência da rede não mudou muito durante a amostra, podemos aproximá-la por [6]

$$velocidade\ de\ hash \approx dificuldade\ total / Tempo\ total$$

Para obter, em média, um bloco por tempo alvo, escolhe-se

$$nova\ dificuldade = velocidade\ de\ hash \cdot Tempo\ alvo.$$

Arredonda-se para cima.

Nada garante que o próximo bloco exija exactamente esse número de tentativas. Ao longo de muitos blocos e reajustamentos, porém, a dificuldade acompanha a potência real e os blocos tendem a surgir no intervalo alvo. [7]

### 8.2.3 Consenso: maior dificuldade cumulativa

Agora é possível resolver conflitos entre garfos da rede.

Cada bloco tem uma dificuldade, e cada ramo tem a soma das dificuldades dos seus blocos: a *dificuldade cumulativa*. Esta soma é metadado calculado e guardado pelo nó; não é um campo serializado no bloco. A regra de consenso activa escolhe o ramo com a maior dificuldade cumulativa, isto é, aquele que representa mais trabalho. Um ramo alternativo só substitui o principal quando a sua soma é *estritamente* maior. Se houver igualdade, o nó conserva o ramo principal que já adoptara; não há uma segunda regra que prefira o ramo com mais blocos.

Uma alteração das regras de consenso produz uma bifurcação rígida (*hard fork*): software que aplica regras incompatíveis deixa de aceitar os mesmos blocos. Na rede principal, a história de versões chegou a v16:

src/cryptonote_core/blockchain.cpp `handle_alternative_block()`

[5] O carimbo é escolhido pelo mineiro e não prova o instante exacto em que começou ou terminou. Esta hipótese serve apenas para chegar à intuição; o algoritmo activo apara carimbos extremos.

[6] Dois mineiros podem experimentar o mesmo *nonce*, mas as suas transacções de mineiro contêm normalmente chaves públicas distintas; os objectos de hash são portanto diferentes, excepto com probabilidade negligenciável. Não estão, na realidade, a repetir a mesma tentativa.

[7] Se a potência de hash subir continuamente, as dificuldades calculadas com trabalho passado ficarão ligeiramente atrasadas e o intervalo médio poderá ser um pouco inferior ao alvo. A emissão real também depende das penalizações de peso exploradas na secção 8.3.3.

| versão | altura de activação | mudança principal |
|---|---|---|
| v1 | 0 | génese, 18 de Abril de 2014 |
| v2 | 1009827 | anéis $\geq 3$, alvo de 120 s, zona de 60 kB |
| v3 | 1141317 | decomposição das saídas da transacção de mineiro |
| v4 | 1220516 | transacções RingCT permitidas |
| v5 | 1288616 | zona mínima de 300 kB e novas taxas |
| v6 | 1400000 | RingCT obrigatório e anéis $\geq 5$ |
| v7 | 1546000 | variante de CryptoNight, anéis $\geq 7$ |
| v8 | 1685555 | Bulletproofs permitidas, anel fixo de 11 |
| v9 | 1686275 | Bulletproofs obrigatórias |
| v10 | 1788000 | CryptoNight-R e peso de longo termo |
| v11 | 1788720 | formato RingCT antigo proibido |
| v12 | 1978433 | RandomX e novas regras da transacção de mineiro |
| v13 | 2210000 | CLSAG permitido e recompensa exacta |
| v14 | 2210720 | MLSAG antigo proibido |
| v15 | 2688888 | anel 16; Bulletproof+ e etiquetas de vista permitidas |
| v16 | 2689608 | Bulletproof+ e etiquetas de vista obrigatórias |

src/hardforks/hardforks.cpp `mainnet_hard_ forks[]`

As datas de activação foram, pela mesma ordem: v2 em 22 de Março de 2016; v3 em 21 de Setembro de 2016; v4 em 5 de Janeiro de 2017; v5 em 15 de Abril de 2017; v6 em 16 de Setembro de 2017; v7 em 6 de Abril de 2018; v8 e v9 em 18 e 19 de Outubro de 2018; v10 e v11 em 9 e 10 de Março de 2019; v12 em 30 de Novembro de 2019; v13 e v14 em 17 e 18 de Outubro de 2020; e v15 e v16 em 13 e 14 de Agosto de 2022.

Esta tabela é histórica até v16. O conjunto activo acumula as mudanças até à última linha, excepto quando uma regra posterior substituiu outra mais antiga. O campo de versão menor conserva a função de voto do mecanismo de bifurcação; em v16 tem de ser pelo menos igual à versão activa.

Para que um atacante convença nós honestos a alterar a história de transacções, tem de construir um ramo válido cuja dificuldade cumulativa ultrapasse a do ramo principal, enquanto este continua a crescer. Com menos de metade da potência, ainda pode ocorrer uma reorganização curta, mas a probabilidade de recuperar uma distância crescente decai rapidamente. Manter uma vantagem em regime estável exige uma fracção maioritária da potência de mineração [95].

### 8.2.4 Minerar em Monero

Para garantir que os garfos da lista de blocos não geram conflitos com o cálculo da dificuldade, não se usam os blocos mais recentes para este cálculo. Por exemplo, se existem 29 blocos na lista, o conjunto tem $b = 10$ e o atraso é $l = 5$, usam-se os blocos 15–24 para calcular a dificuldade do bloco n.º 30.

Mineiros desonestos podem manipular carimbos temporais. Para limitar esse efeito, ordenam-se *apenas* os carimbos e cortam-se $o$ valores de cada extremo. Fica uma janela efectiva $w = b - 2o$. No exemplo, com $o = 3$, conservaríamos os quatro carimbos centrais.

As dificuldades cumulativas mantêm a ordem do ramo. É uma assimetria deliberada: os extremos temporais são aparados sem reordenar o trabalho.

src/cryptonote_core/blockchain.cpp `get_difficulty_for_next_block()`

Usando índices $i$ e $j$ nos extremos da janela conservada, define-se

$$\begin{aligned} \textit{Tempo total} &= TS_j - TS_i, \\ \textit{Dificuldade total} &= D_j^{\text{cum}} - D_i^{\text{cum}}, \\ d_{\text{seguinte}} &= \left\lceil \frac{\textit{Dificuldade total} \cdot 120}{\textit{Tempo total}} \right\rceil . \end{aligned}$$

Na regra activa de Monero, o alvo é 120 segundos, $l = 15$, $b = 720$ e $o = 60$. O nó recolhe até $b + l = 735$ blocos anteriores, ignora os 15 mais recentes quando a janela está cheia e mede o intervalo entre 600 carimbos centrais. Como vimos, não reordena com eles o trabalho cumulativo. Este continua a ser o algoritmo de dificuldade activo em v16; não é uma descrição histórica. Tanto a dificuldade como a sua soma cumulativa são inteiros sem sinal de 128 bits; o cálculo intermédio usa 256 bits e rejeita um resultado que não caiba novamente em 128. [8] [9]

src/cryptonote_basic/difficulty.cpp `next_difficulty()`

As dificuldades não são serializadas nos blocos. Quem verifica a lista tem de as recalcular a partir dos carimbos temporais e das dificuldades cumulativas já verificadas. Antes de haver 735 blocos disponíveis, a função reduz gradualmente o corte; o bloco de génese não entra na amostra e a dificuldade mínima inicial é 1. A formulação pormenorizada que se segue é histórica apenas quanto à fase inicial da rede, não quanto ao algoritmo:

> **Regra n.º 1**: o bloco de génese é ignorado; com uma amostra de zero ou um elemento, $d = 1$.
>
> **Regra n.º 2**: até 600 elementos, usa-se toda a amostra.
>
> **Regra n.º 3**: entre 601 e 720, o corte cresce simetricamente; se o excedente for ímpar, corta-se mais um valor no extremo inferior.
>
> **Regra n.º 4**: com mais de 720 elementos, conservam-se os primeiros 720, produzindo o atraso de 15 blocos quando há 735 disponíveis.

## Prova de trabalho em Monero

Monero usou historicamente CryptoNight, uma função concebida para ser relativamente ineficiente em GPU, FPGA e ASIC quando comparada com SHA-256 [122]. Seguiram-se variantes

[8] Em Março de 2016 (v2), Monero mudou de um para dois minutos entre blocos [77].

[9] O algoritmo talvez seja subóptimo quando comparado com propostas mais recentes [145]; tem, contudo, resistência útil à mineração egoísta [30].

em v7, CryptoNight V2 em v8 e CryptoNight-R em v10 [81, 78, 79]. A regra activa desde v12 é **RandomX** [48, 83]. Aqui basta saber que produz o hash de prova de trabalho de 256 bits verificado contra a dificuldade; a sua construção fica para o tratamento próprio da etapa seguinte.

## 8.3 Quantidade monetária

Há dois mecanismos básicos para criar dinheiro numa criptomoeda baseada numa lista de blocos. Primeiro, os criadores podem emitir moedas na génese e distribuí-las num *airdrop*, ou reservar para si um grande montante numa pré-mineração [26]. Segundo, a moeda pode ser distribuída automaticamente como recompensa por minerar blocos. No modelo de Bitcoin, a quantidade é limitada e as recompensas acabam por chegar a zero.

Monero deriva de Bytecoin, que teve uma pré-mineração substancial seguida de recompensas de bloco [82]. Monero não teve pré-mineração. A recompensa base declinou até 0,6 XMR e entrou depois numa cauda constante. A quantidade absoluta cresce indefinidamente, mas a inflação percentual tende para zero; a subvenção continua, entretanto, a incentivar os mineiros.

### 8.3.1 Recompensa de bloco

Antes de procurar o argumento, o mineiro constrói uma transacção de mineiro. Ela tem uma única entrada especial, que declara a altura, e pelo menos uma saída quando há recompensa a reclamar. [10]

Na regra de consenso activa, a soma dos montantes claros das saídas é *exactamente* a subvenção do bloco, depois da eventual penalização de peso, mais todas as taxas das transacções normais. Cada nó verifica essa igualdade. Assim, a emissão monetária de base e as taxas são directamente auditáveis, apesar de os montantes das transacções normais serem ocultos. Historicamente, v2–v12 permitiam ao mineiro reclamar menos do que o máximo; v13 tornou a igualdade obrigatória.

Para além de distribuir dinheiro, a recompensa incentiva a mineração. Sem recompensas nem taxas, a participação dependeria de incentivos externos ao protocolo. Com pouca potência honesta, torna-se mais barato reunir uma maioria e tentar reescrever blocos recentes.[11]

É também por isso que a recompensa base de Monero não chega a zero. A competição leva os mineiros a acrescentar potência até o custo marginal superar o rendimento marginal esperado. Se o valor protegido aumenta, há espaço económico para mais potência e torna-se mais caro dominar a maioria.

[10] As regras activas não exigem uma única saída. O construtor de modelos do nó limita normalmente a transacção de mineiro a uma saída; isto é comportamento implementado, não consenso. O peso desta transacção conta para o peso total do bloco.

[11] Num modelo simplificado de potências constantes, se o atacante tem $v > v_h$, um histórico com $x$ dias demora aproximadamente $y = xv_h/(v - v_h)$ dias a ultrapassar.

### Deslocamento de bits

O deslocamento de bits calcula a recompensa base (que a penalização da secção 8.3.3 ainda pode reduzir). Seja $A = 13$, ou [1101] em binário. Deslocar duas posições para a direita, $A >> 2$, daria [0011],01 = 3,25 se guardássemos a fracção. A operação descarta a parte fraccionária e conserva [0011] = 3. Em geral, $A >> n = \lfloor A/2^n \rfloor$.

### Calcular a recompensa base de bloco em Monero

Seja $M$ o contador interno de moeda já emitida e $L = 2^{64} - 1$ o alvo aritmético da emissão principal. $L$ não é um limite à quantidade que pode existir para sempre: depois da emissão principal, o contador é saturado em $L$ e a cauda continua. No início, a recompensa base era

$$B = (L - M) >> 20.$$

Com $M = 0$, em formato decimal,

$$L = 18\,446\,744\,073\,709\,551\,615$$
$$B_0 = (L - 0) >> 20 = 17\,592\,186\,044\,415.$$

Estes números estão em unidades atómicas, a menor divisão de Monero. O valor $L$ tem vinte algarismos e serve como limite aritmético interno. Dividir por $10^{12}$ leva-nos às unidades usuais de XMR:

$$\frac{L}{10^{12}} = 18\,446\,744{,}073709551615$$
$$B_0 = \frac{(L - 0) >> 20}{10^{12}} = 17{,}592186044415.$$

A primeira recompensa, atribuída ao pseudónimo thankful_for_today no bloco de génese [127], foi portanto cerca de 17,6 XMR (apêndice C)! Na lista, os montantes continuam em unidades atómicas. À medida que os blocos eram minerados, $M$ aumentava e a recompensa diminuía. Desde a génese até v2, o alvo era um minuto por bloco; em Março de 2016 passou a dois minutos [77]. Para conservar o ritmo de emissão, a recompensa base de bloco foi duplicada.[12] Isto só significa que, depois da mudança, se faz $(L - M) >> 19$ em vez de $(L - M) >> 20$. A curva da emissão principal passou a ser:

$$B = \frac{(L - M) >> 19}{10^{12}}.$$

Esta última expressão já não basta para calcular a regra activa. Em unidades atómicas, a recompensa base sem penalização é

$$B_0 = \max\left\{(L - M) >> 19,\ 600\,000\,000\,000\right\}.$$

O segundo termo é 0,6 XMR. A emissão de cauda já está activa; não é uma promessa para o futuro.

src/crypto-note_basic/cryptonote_basic_impl.cpp `get_block_reward()`

[12] Para uma comparação entre as emissões de Bitcoin e Monero, veja-se [80].

### 8.3.2 Peso dinâmico de bloco

Incluir imediatamente cada nova transacção permitiria a um atacante aumentar rapidamente a lista de blocos por meio de um grande volume de submissões. Um limite fixo por bloco trava o crescimento, mas, quando aumenta o volume honesto, os remetentes competem pelo pouco espaço oferecendo taxas maiores. Os pagamentos pequenos podem tornar-se proibitivos. Monero evita os dois extremos — tamanho rígido ou crescimento sem limite — com peso dinâmico e uma penalização económica.

#### Tamanho vs. peso

Desde que as Bulletproofs foram permitidas (v8), o tamanho serializado e o peso deixaram de ser sempre iguais. O algoritmo atribui a uma transacção sem Bulletproof nem Bulletproof+ exactamente o número de bytes do seu objecto serializado; num bloco v16, este caso abrange a transacção de mineiro `RCTTypeNull`, enquanto uma transacção normal tem de usar Bulletproof+. Para esta última, acrescenta-se uma recuperação (*clawback*) que cobra parte da economia obtida ao agregar muitas saídas.

src/cryptonote_basic/cryptonote_format_utils.cpp `get_transaction_weight()`

Se $p$ é a capacidade preenchida pela prova, arredondada a uma potência de dois até ao máximo de 16 saídas, então, para $p > 2$, a prova Bulletproof+ ocupa

$$S_{\mathrm{BP+}}(p) = 32\big(6 + 2(\log_2 p + 6)\big)$$

bytes, e o acréscimo de peso é

$$C(p) = \left\lfloor \frac{4}{5}\big(320p - S_{\mathrm{BP+}}(p)\big) \right\rfloor .$$

Para $p \leq 2$, $C(p) = 0$. Logo,

$$peso(tx) = bytes(tx) + C(p).$$

Embora a prova cresça logaritmicamente, a verificação continua a ter custo. O peso incorpora parte desse custo. A fórmula mais antiga, com a constante 9 das Bulletproofs originais, pertence a v8–v15; em v15 coexistiu transitoriamente com a constante 6 de Bulletproof+. V16 só permite esta última.

O peso de um bloco é a soma dos pesos da transacção de mineiro e de todas as transacções normais. O cabeçalho e o vector de identificadores não são somados separadamente. Peso é uma grandeza de consenso; tamanho é uma propriedade da serialização.

#### O peso de bloco a longo termo

Se os blocos dinâmicos pudessem crescer depressa sem memória, a lista de blocos poderia aumentar rapidamente [63]. A v10 introduziu por isso um peso de longo termo. Depois de aceitar um bloco, o nó calcula o valor que guardará para esse bloco e as medianas que regerão o *bloco seguinte*; estes números são metadados derivados, não campos do bloco.

Chamemos $W$ ao peso normal do bloco acabado de aceitar e $M_L$ à mediana efectiva anterior de longo termo. Na regra activa desde v15,

$$W_L = \min\left\{\max\left\{W, \frac{10}{17}M_L\right\}, \frac{17}{10}M_L\right\},$$

$$M_L = \max\left\{300\,000, \text{mediana}(W_L \text{ dos últimos } 100\,000 \text{ blocos})\right\}.$$

src/cryptonote_core/blockchain.cpp `get_next_long_term_block_weight()`

As divisões são inteiras no código. A versão histórica v10–v14 só limitava por cima, a $1{,}4M_L$; não se deve usar essa regra para blocos v16. [13]

São necessários cerca de 50 000 blocos, aproximadamente 69 dias ao alvo de dois minutos, para que uma mudança persistente atravesse a mediana. Mesmo então, cada nova amostra fica entre $M_L/1{,}7$ e $1{,}7M_L$: o crescimento e a contracção de longo prazo ficam ambos amarrados.

#### As medianas de curto e longo prazo

O volume de transacções pode mudar dramaticamente num curto período, especialmente à volta das férias [129]. Chamemos $M_S$ à mediana dos pesos normais dos 100 blocos *anteriores*. A mediana efectiva que determina a penalização do bloco seguinte é

$$M_N = \min\left\{\max\{M_L, M_S\},\ 50M_L\right\}.$$

Daí resulta a regra de consenso activa

$$W_{\max} = 2M_N.$$

src/cryptonote_core/blockchain.cpp `update_next_cumulative_weight_limit()`

Um bloco com $W > 2M_N$ é inválido. O limite pode reagir depressa até $100M_L$, mas $M_L$ só se altera lentamente. A mediana "cumulativa" das edições antigas corresponde aqui a $M_N$; chamar-lhe simplesmente mediana de 100 blocos esconde metade do mecanismo. [14]

### 8.3.3 Recompensa de bloco com multa

Para minerar blocos maiores do que $M_N$, os mineiros pagam uma penalização na forma de uma subvenção reduzida. Existem portanto duas zonas: até $M_N$, a zona sem penalização; de $M_N$ até $2M_N$, a zona penalizada.

Se o peso $W$ excede $M_N$, dada a recompensa base $B_0$, a penalização idealizada em aritmética real é

$$P = B_0\left(\frac{W}{M_N} - 1\right)^2.$$

A regra de consenso faz as multiplicações com precisão alargada e as divisões inteiras. A recompensa de bloco é portanto:[15]

src/cryptonote_basic/cryptonote_basic_impl.cpp `get_block_reward()`

[13] A zona mínima foi 20 kB em v1, 60 kB em v2 e é 300 kB desde v5 [77, 94].

[14] Antes de v12, a penalização usava variantes da mediana de curto prazo; v12 tornou activa a mediana efectiva. V15 alterou os limites de longo termo para a escala 1,7.

[15] Antes da activação das transacções confidenciais em v4, os montantes eram comunicados em texto claro e decompostos, por exemplo $1244 \to 1000 + 200 + 40 + 4$. Em v2–v3, o construtor da transacção de mineiro arredondava ainda a recompensa para baixo, retirando os algarismos menos significativos segundo `BASE_REWARD_CLAMP_THRESHOLD`. O resto não desaparecia: ficava disponível para recompensas posteriores.

$$B(W) = \left\lfloor B_0 \frac{W(2M_N - W)}{M_N^2} \right\rfloor, \qquad M_N < W \leq 2M_N.$$

Para $W \leq M_N$, $B(W) = B_0$. Em $W = 2M_N$, a subvenção é zero; para $W > 2M_N$, o bloco é inválido. As taxas não são penalizadas: somam-se depois a $B(W)$ na transacção de mineiro.

O quadrado faz a penalização aumentar com a distância a $M_N$. Um peso 10% acima de $M_N$ perde aproximadamente 1% da subvenção; 50% acima perde 25%; 90% acima perde 81% [80]. Estes exemplos usam a mediana efectiva de curto prazo definida acima.

É de esperar que os mineiros entrem na zona penalizada quando as taxas adicionais compensarem a perda marginal de subvenção. Isto é economia do modelo de bloco, não uma condição de validade.

### 8.3.4 Taxas dinâmicas: política, não consenso

Actores maliciosos podem inundar a rede com transacções e fazer crescer a lista de blocos. Para desencorajar esse comportamento, os nós aplicam uma taxa mínima por unidade de *peso* ao admitir uma transacção na memória de transacções (*txpool*) e ao propagá-la.

src/cryptonote_core/blockchain.cpp `check_fee()`

**Isto não é uma regra de consenso.** Um bloco pode conter uma transacção sem taxa, ou com taxa inferior ao mínimo local, se satisfizer todas as restantes regras. Ao processar transacções vindas de um bloco, o nó ignora o teste de taxa da txpool. É por isso que carteiras e nós podem actualizar o algoritmo sem uma bifurcação da rede. Historicamente, a taxa era 0,01 XMR/KiB em v1 e 0,002 XMR/KiB em v3 [61].[16] V4 introduziu uma taxa dinâmica por KiB; v8 passou a cobrar por byte; v15 activou no software a escala de taxas de 2021. O objectivo mantém-se: relacionar o preço do espaço com a recompensa e com a capacidade que a rede consegue oferecer [21, 20, 19]. [17]

src/cryptonote_core/tx_pool.cpp `add_tx()`

#### Derivação histórica da penalização marginal

Para não perdermos a ideia que deu origem ao mecanismo, regressemos à transacção de referência de peso 3000 e à zona mínima de 300 000. Uma taxa natural é aquela que compensa a penalização marginal causada ao acrescentar a transacção [20, 55]. Esta é a derivação histórica do algoritmo anterior a v15; já a seguir chegaremos à fórmula que o nó v16 executa.

Primeiro, a taxa $T$ que iguala a penalização marginal de acrescentar uma transacção de peso $P_T$ a um bloco de peso $P_B$ é

$$T = B_0 \left[ \left( \frac{P_B + P_T}{M_N} - 1 \right)^2 - \left( \frac{P_B}{M_N} - 1 \right)^2 \right].$$

[16] A unidade KiB (kibibyte, 1 KiB = 1024 bytes) é diferente de kB (kilobyte, 1 kB = 1000 bytes).

[17] Os conceitos apresentados nesta secção devem muito a Francisco Cabañas, também conhecido por "ArticMine", arquitecto dos sistemas de bloco e de taxa dinâmicos [21, 20, 19].

Define-se o factor de peso do bloco:

$$F_b = \frac{P_B}{M_N} - 1$$

e o factor de peso de uma transacção:

$$F_t = \frac{P_T}{M_N}$$

De forma simples:

$$T = B_0(2F_bF_t + F_t^2).$$

No ponto de referência, $P_B = M_N = 300\,000$ e $P_T = 3000$. Logo $F_b = 0$ e

$$\begin{aligned} T_{\text{ref}} &= B_0(2 \cdot 0 \cdot F_t + F_t^2) \\ &= B_0 \left( \frac{3000}{300\,000} \right)^2 . \end{aligned}$$

Esses 3000 representam 1% da zona. Para uma mediana geral $M$, escolhemos uma transacção de referência $P_T^*$ que ocupe a mesma fracção:

$$\frac{P_T^*}{M} = \frac{3000}{300\,000}.$$

Escalamos depois linearmente para uma transacção real de peso $P_T$:

$$\begin{aligned} T_{\text{hist}} &= B_0 \left( \frac{P_T^*}{M} \right) \left( \frac{3000}{300\,000} \right) \frac{P_T}{P_T^*} \\ &= B_0 \frac{P_T}{M} \frac{3000}{300\,000}. \end{aligned}$$

Dividindo pelo peso, obtemos a antiga taxa padrão por unidade de peso:

$$\begin{aligned} f_{\text{standard}}^{\text{hist}} &= \frac{B_0}{M} \frac{3000}{300\,000}, \\ f_{\text{min}}^{\text{hist}} &= \frac{1}{5} f_{\text{standard}}^{\text{hist}} = \frac{0{,}002 B_0}{M}. \end{aligned}$$

O último factor 1/5 reflectia a ideia de cobrar menos abaixo da mediana [55]. A política activa conserva a relação marginal, mas usa uma potência diferente de $M$.

### A regra activa da txpool

Acontece que usar $M_N$ directamente para a taxa permitiria um ataque de *spam*: elevar $M_S$ reduziria o preço precisamente quando a rede estivesse mais ocupada. Por isso o nó usa a menor entre a mediana de penalização e a mediana efectiva de longo termo. Em v16, como $M_N \geq M_L$, a mediana de taxa é simplesmente $M_L$.[18] Se $B_0$ é a recompensa base sem penalização e $w$ o peso da transacção, o preço local por unidade de peso é calculado, com aritmética inteira, em torno de

$$f_{\text{pool}} \simeq \max \left\{ 1,\ 0{,}95 \frac{B_0 \cdot 3000}{M_L^2} \right\} \quad \text{unidades atómicas por unidade de peso.}$$

src/cryptonote_core/blockchain.cpp `get_dynamic_base_fee()`

[18] A fórmula anterior desta secção, proporcional a $B_0/M \cdot 0{,}002$, e os exemplos com recompensa de 2 XMR descreviam a política anterior a v15. A regra activa depende de $B_0/M_L^2$ e a cauda fixa $B_0$ em pelo menos 0,6 XMR.

O mínimo pedido é $wf_{\text{pool}}$, arredondado para cima ao múltiplo seguinte de 10 000 unidades atómicas (oito casas decimais de XMR). Para absorver diferenças de arredondamento, a txpool aceita até 2% abaixo desse total.

$$M_{\text{taxa}} = \min\{M_N, M_L\} = M_L.$$

O quadrado no denominador segue da derivação anterior: se a capacidade de longo termo duplica, o custo por unidade de peso cai quatro vezes. Se a recompensa base diminui, o preço cai com ela. Assim a política não se desliga da economia que limita o crescimento dos blocos.

src/cryptonote_core/blockchain.cpp `check_fee()`

A taxa da txpool é portanto um limiar de admissão local, não um valor exigido pelo consenso. Um operador pode modificar a sua política; não pode, por isso, tornar inválido para os outros nós um bloco que obedece ao consenso.

### Carteira e modelo de bloco

As taxas indicam ao mineiro quanto os remetentes estão dispostos a pagar por espaço na zona penalizada [20]. O construtor de modelos do nó ordena a txpool por taxa por unidade de peso, da maior para a menor (e pela antiguidade em caso de igualdade). Reserva 600 unidades de peso para a transacção de mineiro e experimenta acrescentar cada transacção válida e pronta, mas rejeita-a se ultrapassar o espaço restante até $2M_N$ ou se a nova soma

$$B(W_{\text{novo}}) + \sum taxas$$

ficar abaixo do melhor rendimento já obtido. Esta selecção é comportamento do mineiro; o consenso verifica apenas o bloco final, incluindo o peso real da transacção de mineiro. [19]

A carteira consulta ao nó quatro estimativas por unidade de peso: `unimportant`, `normal`, `elevated` e `priority`. Para as calcular, o nó projecta dez blocos de folga, obtém medianas projectadas e devolve quatro patamares da escala de 2021, arredondados para cima a dois algarismos significativos. Os antigos multiplicadores 1, 5, 25 e 1000 continuam em ramos históricos de compatibilidade do código, mas não descrevem a política activa v16. A carteira multiplica o patamar escolhido pelo peso estimado e volta a verificar o peso real ao construir a transacção.

src/cryptonote_core/tx_pool.cpp `fill_block_template()`

Uma consequência importante dos pesos dinâmicos é o laço de realimentação: transacções que competem com taxas maiores compram mais espaço, mas esse espaço faz crescer lentamente $M_L$ e reduz a taxa por unidade de peso. O mecanismo liga a oferta de espaço, a subvenção e o preço de propagação, sem fingir que a política da carteira é consenso. Este laço de realimentação é uma boa defesa contra a ameaça do "mineiro egoísta" [37].

src/cryptonote_core/blockchain.cpp `get_dynamic_base_fee_estimate_2021_scaling()`

[19] A intuição marginal permanece: uma transacção entra na zona penalizada quando a sua taxa compensa a subvenção que o mineiro perde ao acrescentar-lhe o peso. Não há, porém, uma regra de consenso que obrigue o mineiro a maximizar esse rendimento.

### 8.3.5 Cauda de emissão

Suponha-se uma criptomoeda com quantidade monetária fixa e peso de bloco dinâmico. Depois de algum tempo, as recompensas declinam para zero. Sem uma subvenção para penalizar o crescimento, os mineiros tenderiam a acrescentar qualquer transacção com taxa positiva.

O peso dos blocos estabilizaria à volta do volume submetido à rede e os remetentes procurariam a taxa mínima. Na nossa fórmula de política, uma recompensa que chegasse a zero arrastaria a taxa calculada para o piso de uma unidade atómica por unidade de peso.

Isto introduziria uma situação instável: com pouco incentivo para minerar, a potência de hash poderia cair. A dificuldade manteria aproximadamente o tempo entre blocos, mas o custo de atacar a rede diminuiria. Taxas por si só também podem agravar incentivos de mineração egoísta [23, 37]. [20]

Monero evita esse precipício: a recompensa base nunca desce abaixo de 0,6 XMR por bloco, isto é, 0,3 XMR por minuto ao alvo de dois minutos. A curva principal encontrou esse piso quando

src/cryptonote_basic/cryptonote_basic_impl.cpp `get_block_reward()`

$$0{,}6 > ((L - M) >> 19)/10^{12}$$
$$\mathrm{M} > \mathrm{L} - 0{,}6 \cdot 2^{19} \cdot 10^{12}$$
$$\mathrm{M}/10^{12} > \mathrm{L}/10^{12} - 0{,}6 \cdot 2^{19}$$
$$\mathrm{M}/10^{12} > 18\,132\,171{,}273709551615$$

A lista de blocos de Monero entrou então na chamada *cauda de emissão*, em Junho de 2022, no bloco 2 641 623 [84]. A regra activa conserva a recompensa base de 0,6 XMR para cada novo bloco. A emissão absoluta prossegue; a inflação percentual decresce à medida que a quantidade já emitida aumenta. [21]

### 8.3.6 Transacção de mineiro: `RCTTypeNull`

O mineiro reclama a subvenção e as taxas através da transacção de mineiro (também chamada *coinbase transaction*). Parece uma transacção normal à distância, mas as regras de consenso activas são muito específicas:

src/cryptonote_core/cryptonote_tx_utils.cpp `construct_miner_tx()`

- tem versão 2 e exactamente uma entrada `txin_gen`, cuja altura é a altura do bloco;
- usa `RCTTypeNull`: não contém CLSAG, Bulletproof+ nem pseudo-saídas;

[20] O caso de uma quantidade monetária limitada e um tamanho de bloco fixo, como em Bitcoin, também é considerado como instável [23].

[21] $L$ continua a ser o maior contador de emissão representável em 64 bits; a implementação satura o contador em $L$ durante a cauda para evitar um subfluxo em $L - M$. Não se deve interpretar isso como o fim da emissão. Cada montante individual continua limitado à gama de 64 bits demonstrada pelas provas de intervalo.

- o tempo de desbloqueio é exactamente $h + 60$;[22]
- em v16 todas as saídas são `txout_to_tagged_key`, com etiqueta de vista, chave pública válida e montante claro;
- a soma dos montantes é exactamente $B(W) + \sum taxas$.[23]

O construtor do nó cria normalmente uma única saída para o endereço do mineiro, inclui a chave pública da transacção no campo `extra` e pode incluir aí um argumento adicional fornecido pela reserva de mineração. Esta é uma escolha de modelo; o consenso permite várias saídas, desde que todas as condições acima e o peso máximo do bloco sejam satisfeitos.

Desde que RingCT foi implementado em Janeiro de 2017 (v4) [91], o nó que guarda uma saída de mineiro v2 fabrica também o compromisso de máscara nula

$$C = 0G + aH = aH.$$

Esse compromisso não está escondido na transacção de mineiro; é metadado derivado do montante claro. Permite, contudo, que a saída apareça mais tarde num anel CLSAG juntamente com saídas RingCT normais. A antiga expressão $G + aH$ estava errada: "máscara identidade" no comentário do código significa o escalar zero, não o escalar um.

src/cryptonote_core/blockchain.cpp `is_tx_spendtime_unlocked()` src/blockchain_db/blockchain_db.cpp `add_transaction()`

## 8.4 A estrutura da lista de blocos

O estilo da lista de blocos de Monero é simples.

Começa com o bloco de génese, neste caso essencialmente uma transacção de mineiro que dispersa a primeira recompensa (apêndice C). Cada bloco seguinte contém no cabeçalho o ID do bloco anterior.

Para calcular o ID, não se aplica a função hash à serialização completa do bloco. Primeiro constrói-se o *objecto de hash*: a serialização binária do cabeçalho, seguida pela raiz de Merkle dos IDs de transacção e pelo número total de transacções, incluindo a de mineiro. Depois aplica-se a função hash rápida de CryptoNote (Keccak):[24]

src/cryptonote_core/cryptonote_tx_utils.cpp `generate_genesis_block()`

$$ID_B = \mathcal{H}\big(\text{serializar}(\text{cabeçalho}) \,\|\, R_{\text{Merkle}} \,\|\, \text{varint}(1 + \#\text{txs normais})\big).$$

Para produzir um novo bloco, o mineiro altera o *nonce* de 4 bytes e, quando precisa de mais domínio, altera também a reserva no `extra` da transacção de mineiro. O mesmo objecto de

[22] Se esta transacção aparece no bloco 10, a altura de desbloqueio é 70 e a saída pode ser usada numa transacção incluída no bloco 70 ou posterior. Esta maturidade de 60 blocos [93] é distinta da idade mínima normal de 10 blocos para uma saída ser usada como membro real de uma entrada, activa desde v12 [89].

[23] V12 tornou histórica a possibilidade de usar uma transacção de mineiro v1 ou de incluir assinaturas RingCT: passou a exigir versão 2 e `RCTTypeNull` [45]. V13 tornou exacta a soma reclamada.

[24] Há uma excepção histórica de consenso para o bloco 202612, cujo ID é tratado especialmente. Não altera a construção dos blocos v16.

src/cryptonote_basic/cryptonote_format_utils.cpp; `get_block_hashing_blob()`;

hash alimenta a prova de trabalho, mas não a mesma função: em blocos v16 usa-se RandomX para obter $h_{\mathrm{PoW}}$; para o ID usa-se $\mathcal{H}$. Interpretando $h_{\mathrm{PoW}}$ como inteiro de 256 bits em ordem *little-endian*, o bloco é válido quando

$$h_{\mathrm{PoW}} \cdot d \leq 2^{256} - 1.$$

Enquanto o produto for maior, continua-se a procurar. A verificação faz uma avaliação do hash de prova de trabalho e uma multiplicação de precisão suficiente; a mineração repete tentativas até obter um resultado válido.

### 8.4.1 ID de transacção

As transacções activas têm versão 2. O seu ID não é simplesmente a hash de um único objecto monolítico: separa deliberadamente os dados que um nó podado tem de conservar daqueles que pode apagar.

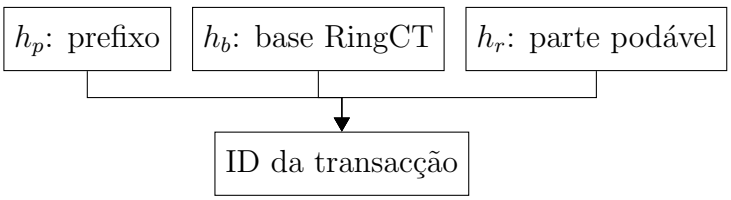
Calculam-se três hashes de 32 bytes:

- $h_p$: a hash do prefixo serializado: versão, desbloqueio, entradas (incluindo *offsets* e imagens de chave), saídas (chaves e etiquetas de vista) e `extra`;
- $h_b$: a hash da parte RingCT de base, não podável: tipo (`RCTTypeBulletproofPlus` nas transacções normais activas, ou `RCTTypeNull` na de mineiro), taxa, compromissos de saída e informação de montante;
- $h_r$: a hash da parte RingCT podável, que nas transacções activas normais inclui Bulletproof+, CLSAG e pseudo-saídas. Para `RCTTypeNull`, $h_r$ é a hash nula de 32 bytes.

Finalmente,

$$ID_{\mathrm{tx}} = \mathcal{H}(h_p \| h_b \| h_r).$$

Neste diagrama de árvore a seta preta indica uma hash de entradas.




Em vez de entradas que gastam saídas anteriores, a transacção de mineiro contém a altura do bloco actual. Assim, mesmo que o endereço e o montante se repetissem, a altura mudaria o prefixo e tornaria o ID distinto. Como `RCTTypeNull` não tem parte podável, usa-se $h_r = 0^{256}$; este valor não resulta de calcular o hash de uma lista vazia de assinaturas.

### 8.4.2 Árvore de Merkle

Uma raiz de Merkle [71] organiza os IDs das transacções do bloco numa árvore binária de hashes. Ela compromete o mineiro com a lista e com a ordem exactas dos IDs sem colocar a raiz como campo separado no bloco: qualquer alteração de uma transacção muda o seu ID, sobe pela árvore e muda o ID do bloco.

Monero usa a árvore CryptoNote, não a regra de Bitcoin de duplicar sempre a última folha. Para um número que não seja potência de dois, algumas folhas entram directamente no nível intermédio e as restantes são emparelhadas. A primeira folha é sempre o ID da transacção de mineiro; seguem-se, pela ordem serializada no bloco, os IDs das transacções normais.

src/crypto/tree-hash.c `tree_hash()`

Um exemplo com uma transacção de mineiro e quatro transacções normais aparece na Figura 8.1. [25]

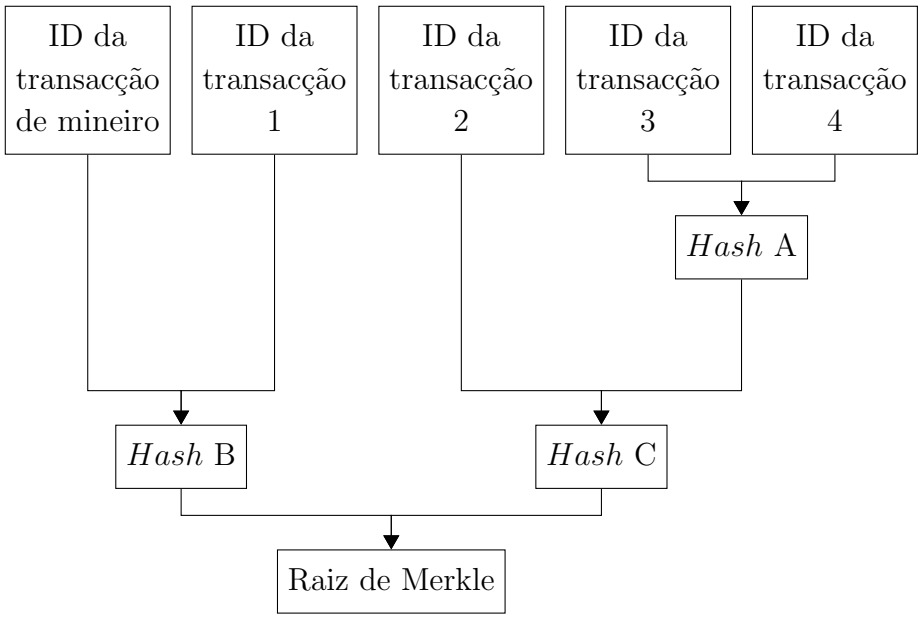



*Figura 8.1: árvore de Merkle*

Uma raiz de Merkle é, pois, uma referência compacta a todas as transacções incluídas. Mas há um pormenor: a poda real de Monero não corta ramos desta árvore. É a decomposição $h_p \| h_b \| h_r$ do ID da transacção que permite retirar dados e ainda reconstruir o mesmo ID.

#### Da decomposição à poda

A decomposição acima diz exactamente que material de uma transacção é podável. A política da base de dados, as oito faixas, a ponta conservada e as três formas de sincronizar um nó

[25] Um erro no código-fonte da raiz de Merkle deu origem a um ataque sério, embora aparentemente não crítico, em 4 de Setembro de 2014 [66].

podado pertencem ao capítulo 10, secção 10.8. Ali acompanharemos o que o nó verifica antes de apagar — e o que não pode reverificar se já receber a história sem as provas.

### 8.4.3 Blocos

Um bloco serializado é um cabeçalho, a transacção de mineiro completa e um vector de IDs de transacções normais. Os corpos destas últimas são obtidos da txpool ou fornecidos separadamente pelo par; não aparecem repetidos dentro do objecto `block`. Os nossos leitores podem encontrar um exemplo real no apêndice B.

- cabeçalho de bloco:
  - **versão maior**: versão das regras que construíram o bloco; na rede principal actual é 16;
  - **versão menor**: voto do mecanismo de bifurcação; a regra activa exige um valor pelo menos 16;
  - **carimbo temporal**: inteiro UTC escolhido pelo mineiro;
  - **ID do bloco anterior**: 32 bytes que encadeiam a lista;
  - **nonce**: inteiro sem sinal de 4 bytes usado na prova de trabalho.
- transacção de mineiro: objecto completo que cria a subvenção e reclama as taxas;
- IDs de transacção: vector ordenado de referências de 32 bytes às transacções normais.

Na serialização binária, as três primeiras grandezas do cabeçalho usam inteiros variáveis; o ID e o *nonce* têm tamanho fixo:

- versão maior e versão menor: inteiros de 8 bits serializados como *varints*, normalmente um byte cada;
- carimbo temporal: inteiro de 64 bits serializado como *varint*;
- ID do bloco anterior: 32 bytes;
- *nonce*: 4 bytes. O domínio pode ser estendido alterando a reserva do campo `extra` da transacção de mineiro [62];
- transacção de mineiro completa, com prefixo e base RingCT nula. A base de dados deriva ainda, para cada saída v2, o compromisso $C = aH$;
- contagem do vector como *varint* e IDs de transacção de 32 bytes cada. O desserializador impõe ainda um limite defensivo de $2^{28}$ IDs, muito acima do que o limite de peso permitiria.

#### O que um nó valida

Receber bytes bem formados ainda não é receber um bloco. Para o acrescentar ao ramo principal, um nó v16 aplica condições de consenso e uma condição de admissão dependente do seu relógio:

- que o ID anterior é a ponta actual e que as versões maior e menor satisfazem a programação da rede principal;

- como consenso, havendo 60 blocos anteriores, que o carimbo temporal não é menor do que a mediana dos seus carimbos; como admissão local, que não está mais de duas horas no futuro segundo o relógio do nó;
- que a dificuldade seguinte foi recalculada correctamente e que o hash RandomX satisfaz $h_{\mathrm{PoW}} d \leq 2^{256} - 1$;
- que cada corpo de transacção referido produz o ID anunciado, tem no máximo 1 000 000 bytes, satisfaz as regras v16 de entradas, saídas, imagens de chave, CLSAG e Bulletproof+, e não duplica gastos;
- que a raiz de Merkle, o número de transacções e o ID do bloco se recompõem; que o peso total não excede $2M_N$; e que a transacção de mineiro satisfaz altura, maturidade, tipo de saída e recompensa exacta.

Se o ID anterior não é a ponta, o bloco pode iniciar ou prolongar um ramo alternativo. O nó valida-o pelas regras da altura desse ramo e só reorganiza a lista quando a dificuldade cumulativa alternativa se torna estritamente maior. A txpool, a escolha das taxas e a poda da base de dados não mudam nenhum destes testes de consenso.

# CAPÍTULO 9

# RandomX: função de prova de trabalho

No capítulo anterior, a validade da prova de trabalho foi reduzida a uma condição curta. Dado o objecto de hash de um bloco, uma dificuldade $d$ e um resultado de prova de trabalho $h_{\mathrm{PoW}}$, o bloco passa quando

$$h_{\mathrm{PoW}}\, d \leq 2^{256} - 1. \tag{9.1}$$

Nada nessa desigualdade determina o custo de produzir $h_{\mathrm{PoW}}$. Uma função hash convencional executa sempre a mesma sequência de operações. Um circuito construído especificamente para essa sequência pode omitir recursos que ela não utiliza e superar a eficiência de um processador geral.

RandomX é a função de prova de trabalho activa em Monero desde a versão 12. Foi desenhada para estreitar a vantagem entre processadores de uso geral e máquinas especializadas. Em vez de repetir em cada tentativa uma sequência fixa, gera programas imprevisíveis, executa-os numa máquina virtual, movimenta um conjunto grande de memória e condensa o estado final num hash de 256 bits [126, 125]. O nome vem daí: execução aleatória.

RandomX não demonstra que um ASIC é impossível, nem define uma CPU por uma propriedade abstracta. Associa o trabalho a uma colecção ampla de recursos que os processadores correntes já possuem: aritmética inteira, vírgula flutuante, registos, vários níveis de memória, saltos, execução especulativa e largura para instruções independentes. Uma máquina especializada pode reproduzi-los, mas tem de implementar uma parte substancial dos recursos presentes num processador geral.

Este capítulo não reproduz programas, instruções de montagem ou funções do código. A análise usa entradas, estado, transformações e condições de segurança. A implementação consultada

fixa a versão 1.2.3 de RandomX como submódulo da revisão do código Monero consultada em Agosto de 2026 [89, 126]. Cada parâmetro numérico abaixo pertence a essa instanciação.

## 9.1 Especialização de hardware numa função fixa

Considere-se primeiro uma prova de trabalho definida por uma função fixa,

$$h = F(B, n),$$

onde $B$ representa a parte fixa do bloco e $n$ o argumento que o mineiro altera. Um processador geral interpreta instruções para avaliar $F$. Um circuito especializado pode implementar directamente $F$, sem suportar instruções ou estados que essa função nunca utiliza.

Se uma unidade de capital compra $r$ tentativas por segundo em equipamento geral e $\alpha r$ em equipamento especializado, $\alpha > 1$ é a vantagem de especialização. Um proprietário com fracção $x$ do investimento mineiro não fica necessariamente com fracção $x$ do trabalho. Num modelo simples, em que só ele possui a vantagem, a sua fracção da potência torna-se

$$q = \frac{\alpha x}{\alpha x + (1 - x)}. \tag{9.2}$$

Com $x = 1/4$ e $\alpha = 4$, por exemplo, obtém-se $q = 4/7$: um quarto do capital produz mais de metade das tentativas. O modelo ignora electricidade, preço do equipamento e concorrência entre fabricantes, mas mostra por que a eficiência do dispositivo acaba por ser uma questão de consenso económico.

RandomX procura aproximar $\alpha$ de 1 impondo a qualquer dispositivo os mesmos requisitos de memória, cálculo e imprevisibilidade. O protocolo não restringe a arquitectura usada para os satisfazer.

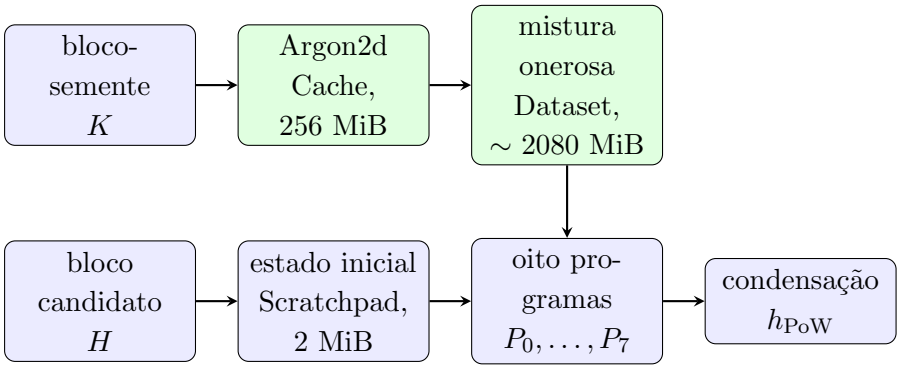
A mudança essencial é passar de uma função fixa para uma família de funções determinada por uma chave de época $K$:

$$R_K(H) \longrightarrow h_{\mathrm{PoW}}. \tag{9.3}$$

Aqui $H$ é o objecto de hash do bloco, incluindo o nonce, e $K$ deriva de um bloco anterior. Para cada tentativa, os bytes derivados de $H$ escolhem os programas; de época para época, $K$ muda a grande tabela de memória que esses programas consultam. Todos conseguem reconstruir ambas as coisas. O que um dispositivo não pode fazer é conservar uma única sequência curta de operações durante todas as épocas.

## 9.2 Estrutura completa de uma avaliação

O cálculo tem duas entradas principais. Um bloco anterior fornece $K$, da qual se derivam uma *Cache* de 256 MiB e um *Dataset* de aproximadamente 2080 MiB. Esta preparação é reutilizada em muitas tentativas da mesma época. O bloco candidato fornece $H$, que inicializa o espaço de trabalho e a sucessão de oito programas. O Dataset intervém em cada volta e o estado final determina o resultado.




Este desenho já separa três tempos que facilmente se confundem:

1. a **época** conserva a mesma chave e permite reutilizar Cache e Dataset;
2. a **tentativa** altera o bloco candidato e produz um novo hash;
3. dentro da tentativa, cada **programa** corre 2048 voltas e entrega ao seguinte uma nova semente.

O mineiro repete a avaliação dependente de $H$ muitas vezes. Um nó que recebe um bloco só precisa de a executar uma vez para verificar a prova. O Dataset torna essa avaliação rápida quando está materializado; a Cache permite obter exactamente o mesmo resultado com menos memória e mais tempo.

## 9.3 Derivação e atraso da chave de época

Se o próprio mineiro pudesse escolher $K$, procuraria uma chave que gerasse um Dataset particularmente favorável ao seu equipamento. A procura da chave introduziria uma etapa de selecção anterior à procura do nonce. A chave deve portanto derivar da cadeia e mudar, mas não pode derivar do bloco que está a ser minerado, pois nesse caso o nonce também influenciaria os dados usados na própria avaliação.

Monero divide as alturas em épocas de

$$E = 2048 \text{ blocos}$$

e introduz um atraso de

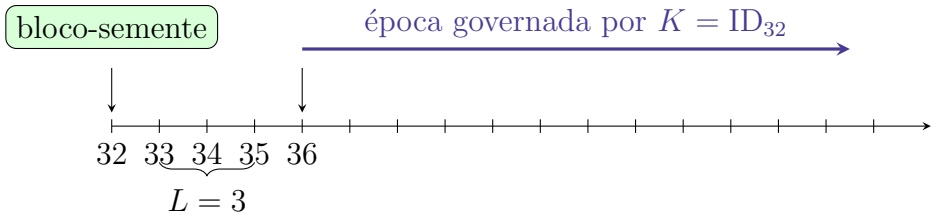
$$L = 64 \text{ blocos.}$$

Para uma altura candidata $h$, a função usada na revisão do código consultada escolhe a altura-semente

$$s(h) = \begin{cases} 0, & h \le E + L, \\ E \left\lfloor \dfrac{h - L - 1}{E} \right\rfloor, & h > E + L. \end{cases} \tag{9.4}$$

A chave é o ID do bloco à altura $s(h)$. A subtracção de 1 prende a fórmula à convenção exacta das alturas no nó. Em regime normal, a chave do bloco $jE$ começa a servir aos candidatos depois de existirem os 64 blocos intermédios $jE + 1, \ldots, jE + 64$.

Por exemplo, com $E = 16$ e $L = 3$, a semente da altura 32 fica conhecida quando esse bloco entra na cadeia, mas só governa a mineração a partir da altura 36:




O atraso não torna o bloco-semente impossível de influenciar. Um mineiro pode tentar reorganizar a cadeia ou reter um bloco, como em qualquer disputa de prova de trabalho. Torna, porém, dispendioso escolher entre muitas sementes: quando a chave entra em uso, sessenta e quatro sucessores já investiram trabalho por cima dela. Abandonar uma semente desfavorável significa abandonar também essa história.

A mudança a cada 2048 blocos equilibra dois custos. Se $K$ mudasse a cada bloco, os mineiros teriam de reconstruir gigabytes de Dataset antes de cada nova tentativa. Se nunca mudasse, um dispositivo poderia ser optimizado para uma tabela permanente. Com blocos de dois minutos, uma época dura cerca de

$$2048 \cdot 2 \text{ minutos} = 4096 \text{ minutos} \approx 2{,}84 \text{ dias}.$$

Este intervalo permite amortizar a preparação, mas exige a substituição periódica dos dados dependentes da cadeia.

## 9.4 Cache e Dataset

A chave $K$ alimenta Argon2d, uma função deliberadamente intensiva em memória [11]. Nesta instanciação, Argon2d preenche 262144 blocos de 1 KiB, numa só pista, e faz três passagens. O resultado final da fase de preenchimento — não o resumo curto normalmente devolvido por uma função de derivação de chave — é a Cache:

$$C_K = (C_K[0], C_K[1], \ldots, C_K[m-1]), \qquad |C_K| = 256 \text{ MiB}. \tag{9.5}$$

O Dataset deriva da Cache. Contém 34078719 itens de 64 bytes, pouco menos de 2080 MiB. Cada item pode ser calculado separadamente. Se $i$ for o seu índice, escrevemos de forma abstracta

$$D_K[i] = \Phi_K(i, C_K), \tag{9.6}$$

onde $\Phi_K$ faz oito acessos dependentes dos valores anteriores à Cache e, entre acessos, aplica oito funções de mistura geradas a partir de $K$. A especificação chama a essa mistura *SuperscalarHash*. Não é uma nona função hash criptográfica adicional: é uma sequência longa de aritmética inteira, sobretudo multiplicações de 64 bits, escolhida para ocupar as unidades de execução de um processador.

O índice $i$ entra no estado inicial de forma injectiva dentro do domínio do Dataset. Depois, cada ronda pode ser vista como

$$\begin{aligned} X_0 &= \mathrm{enc}(i), \\ X_{j+1} &= \mathrm{Super}_j(X_j) \,\mathsf{xor}\, C_K[a_j], \qquad j = 0, \ldots, 7, \\ D_K[i] &= X_8, \end{aligned}$$

em que $a_j$ deriva do próprio estado. Esta notação omite constantes, ordem de palavras e pormenores de serialização; conserva o facto essencial: pedir um item obriga a seguir uma cadeia de oito leituras imprevisíveis e oito misturas dependentes.

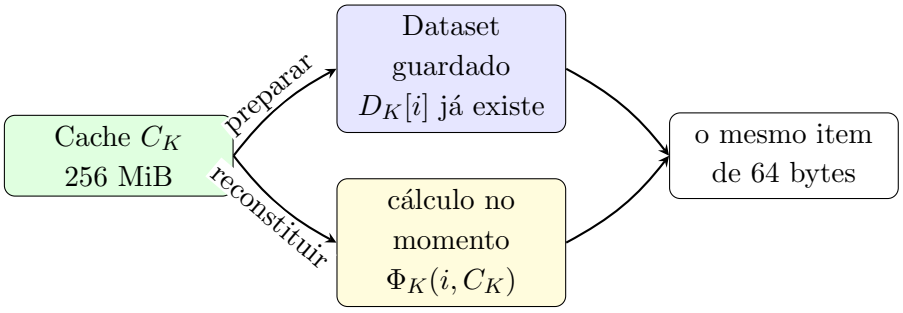
Como os itens são independentes entre si, os mineiros podem construir o Dataset em paralelo no início de uma época. Como todos derivam de $K$, não é preciso publicá-lo nem confiar em quem o preparou. A mudança de $K$ exige uma nova construção do Dataset em cada época.

## 9.5 Modos completo e leve

Há duas maneiras de responder quando a máquina virtual pede $D_K[i]$:

**modo completo:** calcular previamente todos os itens e ler o item pedido do Dataset em memória;

**modo leve:** guardar apenas $C_K$ e calcular $D_K[i]$ nesse momento por meio de $\Phi_K(i, C_K)$.




Os modos não são duas provas de trabalho. São duas avaliações da mesma função. Se a Cache, a chave e o índice forem iguais, a equação (9.6) determina os mesmos 64 bytes. Um nó leve pode portanto verificar um bloco minerado em modo completo; o bloco não anuncia qual modo foi usado.

O modo completo é apropriado à repetição. Paga uma vez a construção do Dataset e troca cada pedido futuro por uma leitura de DRAM. O modo leve poupa cerca de oito vezes a memória, mas em cada pedido repete as oito misturas e os oito acessos à Cache. Uma tentativa executa 16384 voltas da máquina virtual e faz uma leitura de Dataset por volta. Reconstituir cada item torna a mineração leve demasiado lenta; para um verificador ocasional, continua a ser uma saída praticável.

Esta assimetria é subtil. A prova de trabalho clássica é assimétrica entre *procurar* e *verificar*: o mineiro tenta muitos nonces, o verificador avalia um. Dentro de uma única avaliação RandomX existe outra escolha entre memória e tempo. O modo leve é mais lento do que uma avaliação com Dataset, mas verifica ainda uma vez; o mineiro teria de pagar essa lentidão milhares de vezes por segundo.

## 9.6 Compromissos entre memória e tempo

Exigir dois gigabytes sem mais cuidado não basta. Um adversário pode comprimir a tabela, guardar apenas uma parte ou construir memória especial junto dos circuitos de cálculo. Uma função é chamada intensiva em memória quando reduzir a memória obriga a pagar um aumento desproporcionado de tempo ou trabalho.

Para comparar dispositivos, o desenho de RandomX usa o modelo do produto área–tempo

$$AT = A \cdot T, \tag{9.7}$$

onde $A$ aproxima a área de circuito necessária e $T$ o tempo por resultado. Um dispositivo que usa metade da memória mas demora dez vezes mais tem, antes de outros custos, cerca de cinco vezes o produto $AT$. A medida não é uma lei física completa: energia, largura de banda, processo de fabrico, empacotamento e preço de mercado também importam. É uma maneira de impedir que «menos memória» seja confundido automaticamente com «mais eficiente».

A razão nominal entre os modos foi escolhida em torno de oito:

$$8 \cdot 256 \text{ MiB} = 2048 \text{ MiB}.$$

O Dataset acrescenta perto de 32 MiB porque o modo leve precisa também da área e energia das unidades que calculam SuperscalarHash. Assim se chega aos cerca de 2080 MiB. Os valores foram afinados por medições e modelos de hardware, não deduzidos de um axioma criptográfico [125].

Argon2d protege a própria Cache contra reduções mais agressivas. As referências de desenho estimam que, com três passagens, reduzir a memória de Argon2d para metade aumenta por um factor de 3423 o trabalho do melhor compromisso analisado [10, 125]. Esse número pertence a um modelo de ataque, não é uma garantia sobre todas as arquitecturas futuras. No modelo analisado, não há uma redução de baixo custo entre guardar a Cache inteira e recalculá-la.

Há, por fim, duas latências diferentes. O Dataset reside em DRAM e cada acesso tem latência relativamente alta. A Cache pode caber em memória mais próxima das unidades de execução

num ASIC. SuperscalarHash substitui parte da latência de memória evitada por cadeias longas de dependências e muitas multiplicações. Assim, colocar 256 MiB no circuito não elimina o custo computacional do modo leve.

RandomX não impede o investimento em memória mais eficiente. O objectivo é incorporar esse investimento no custo do dispositivo e alterar o compromisso entre memória e tempo.

## 9.7 Máquina virtual RandomX

Uma máquina virtual especifica um estado e a transformação causada por cada instrução, independentemente de o nó real usar arquitectura x86, ARM ou RISC-V. Um participante pode interpretar as instruções uma a uma ou traduzi-las para instruções nativas antes da execução. Se a tradução for correcta, o estado final é o mesmo.

Podemos condensar o estado relevante de RandomX numa tupla

$$\mathcal{S} = (\mathbf{r}, \mathbf{f}, \mathbf{e}, \mathbf{a}, M_1, M_2, M_3, m_a, m_x, \rho), \tag{9.8}$$

onde:

- $\mathbf{r} = (r_0, \ldots, r_7)$ contém oito palavras inteiras de 64 bits;
- $\mathbf{f}$ e $\mathbf{e}$ são dois grupos de quatro registos de vírgula flutuante, usados respectivamente por operações aditivas e multiplicativas;
- $\mathbf{a}$ contém quatro valores de vírgula flutuante fixos para o programa;
- $M_1 \subset M_2 \subset M_3$ são três vistas do mesmo Scratchpad;
- $m_a, m_x$ ajudam a escolher os próximos endereços no Dataset; e
- $\rho$ é o modo de arredondamento corrente.

Cada membro de $\mathbf{f}, \mathbf{e}, \mathbf{a}$ contém duas faixas *binary64*; as operações de vírgula flutuante actuam portanto sobre pares de números em 128 bits.

Os registos juntos ocupam 256 bytes. O Scratchpad ocupa 2 MiB e permite leitura e escrita. O Dataset é uma estrutura externa apenas de leitura, comum às tentativas da época e consultada uma vez por volta.

Um programa $P$ contém 256 instruções e uma configuração. Executar uma volta é aplicar uma função de transição

$$\mathcal{S}_{t+1} = P(\mathcal{S}_t, D_K),$$

e cada programa repete-a 2048 vezes:

$$\mathcal{S}_{2048} = P^{2048}(\mathcal{S}_0, D_K). \tag{9.9}$$

O expoente aqui significa composição repetida, não elevar bytes a uma potência. Durante cada volta, endereços derivados do estado seleccionam o que ler e escrever; por isso, não se pode em geral saltar directamente para $\mathcal{S}_{2048}$.

### 9.7.1 Codificação total das instruções

Cada instrução ocupa oito bytes. A codificação foi definida de modo que *qualquer* palavra de oito bytes seja uma instrução válida e qualquer sequência dessas palavras seja um programa válido. Os bytes pseudo-aleatórios podem portanto ser interpretados sem uma etapa de rejeição sintáctica.

O programa só pode alterar o estado da equação (9.8); não tem acesso ao sistema operativo. Todos os 64 bits de cada palavra recebem uma interpretação válida dentro dessa máquina virtual.

Os oito bytes escolhem uma operação, registos de origem e destino, modificadores e uma constante. Há 256 valores possíveis para a parte que escolhe a operação, distribuídos por 29 espécies de instruções. Dar vários valores à mesma espécie fixa a sua frequência sem construir uma tabela de probabilidades separada. Se 16 dos 256 valores representassem uma operação, por exemplo, ela apareceria em média uma vez por cada 16 instruções.

Para a análise de custo, basta agrupar as 29 espécies pelo recurso utilizado:

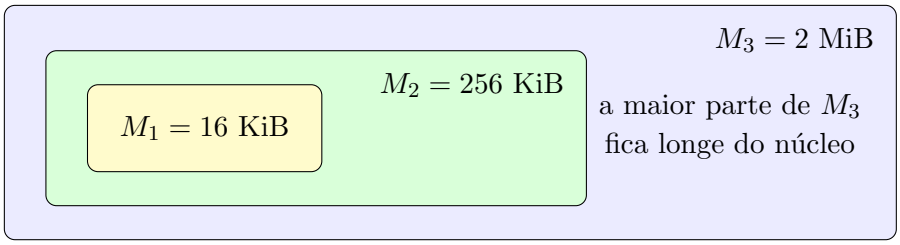
| **família** | **transformações** | **recurso que põe a trabalhar** |
|---|---|---|
| inteira | somas, diferenças, produtos, rotações e XOR | unidades aritméticas, dependências e cálculo de endereços |
| vírgula flutuante | soma, diferença, produto, divisão e raiz quadrada | unidade de vírgula flutuante e arredondamento IEEE 754 |
| memória | leituras e escritas dependentes do estado | caches, largura de banda e latência |
| controlo | saltos condicionais raros | execução especulativa e custo de previsões falhadas |

O repertório é suficientemente próximo das operações nativas para permitir tradução rápida, mas suficientemente amplo para impedir que um circuito selectivo conserve apenas multiplicadores ou apenas somadores. Os programas não são optimizados para uma tarefa externa; servem para distribuir de modo reprodutível o trabalho pelas unidades do processador.

## 9.8 Hierarquia do Scratchpad

O Scratchpad de 2 MiB é uma só sequência de bytes, vista através de três regiões aninhadas:

$$|M_1| = 16 \text{ KiB},$$
$$|M_2| = 256 \text{ KiB},$$
$$|M_3| = 2 \text{ MiB}.$$




Os nomes correspondem aproximadamente às caches físicas L1, L2 e L3 de um processador, mas não especificam uma correspondência exacta. A maior parte das instruções de memória acede às duas regiões pequenas; as leituras dirigidas a $M_1$ e $M_2$ aparecem numa razão de aproximadamente $3:1$, acompanhando a razão inversa das latências típicas. Há também dois acessos aleatórios à região grande por volta, e certas leituras repetidas de endereço fixo acabam naturalmente promovidas pela cache física.

No fim de cada volta, o estado de registos é misturado com dois blocos de 64 bytes do Scratchpad e regressa às mesmas zonas. Além disso, instruções de armazenamento escrevem palavras de oito bytes. A uma volta média correspondem, segundo a análise de desenho, 504 bytes lidos e 256 escritos, contando acessos explícitos, movimentos de estado e a linha de Dataset [125].

Como há $8 \cdot 2048 = 16384$ voltas numa tentativa, isto dá aproximadamente

$$16384 \cdot 504 = 8257536 \text{ bytes} = 7{,}875 \text{ MiB lidos},$$

$$16384 \cdot 256 = 4194304 \text{ bytes} = 4 \text{ MiB escritos}.$$

Dentro das leituras há precisamente 16384 itens de Dataset de 64 bytes, isto é, 1 MiB de dados pedidos à memória exterior. O custo decorre de os endereços serem dependentes do estado e de cada pedido trazer uma linha curta depois de uma latência longa.

Um acesso sequencial pode mover muitos gigabytes por segundo porque o endereço seguinte é conhecido antecipadamente. Em RandomX, cada pedido de 64 bytes pode ter um endereço dependente do estado anterior. A largura de banda continua a importar, mas a latência e o número de bancos de memória também limitam o paralelismo.

## 9.9 Paralelismo ao nível de instruções

Considere-se uma sequência abstracta de quatro operações:

$$u = a + b, \qquad v = c \cdot d, \qquad w = u \operatorname{xor} e, \qquad z = v + w.$$

As duas primeiras não dependem uma da outra e podem avançar ao mesmo tempo. A terceira espera por $u$; a quarta espera por $v$ e $w$. Um processador *superscalar* possui várias unidades capazes de executar operações no mesmo ciclo. Um processador *fora de ordem* pode adiantar operações cujos operandos estejam prontos enquanto aguarda por $w$.

RandomX gera instruções com dependências e independências misturadas. O processador corrente dedica uma área substancial a descobrir estas últimas, renomear registos e manter muitas

operações em voo. Um circuito simples que execute pela ordem escrita desperdiça essa estrutura. Um ASIC sofisticado pode reproduzi-la, mas paga pela sofisticação que pretendia evitar.

Os saltos condicionais acrescentam outro custo. Cada salto depende de um valor obtido em execução e tem probabilidade aproximada de 1/256 de ser tomado. Prever «não salta» é quase sempre correcto; quando falha, o trabalho especulativo é abandonado. As regras de destino impedem ciclos infinitos: o registo que controla o salto não é alterado no trecho para onde ele volta. Nesta construção, um salto condicional pode voltar para trás no máximo duas vezes seguidas.

Uma simulação simplificada publicada com o desenho mediu o programa de 256 instruções em dez processadores imaginários. O modelo mais simples, com uma unidade aritmética, uma unidade de memória, ordem fixa e sem especulação, precisou em média de 293 ciclos. Um modelo com quatro unidades aritméticas, duas de memória, execução fora de ordem e especulação precisou de 64 [125]. Os números não prevêem um processador comercial; mostram que os programas utilizam mecanismos já presentes nos processadores gerais.

Uma função hash fixa costuma ter uma estrutura regular e fácil de implementar em paralelo. RandomX usa primitivas criptográficas regulares nas fronteiras e uma sequência irregular de instruções na máquina virtual. Essa irregularidade aumenta o custo de especialização do hardware.

## 9.10 Encadeamento de oito programas

Esta corrente resolve o *problema do programa fácil*. O tempo $T$ de um programa aleatório não é constante. Programas com mais dependências, divisões ou acessos infelizes podem demorar mais; as medições de RandomX mostram uma distribuição próxima de log-normal, com uma cauda de programas lentos. Se cada programa produzisse sozinho um hash, um mineiro poderia examiná-lo rapidamente e executar apenas os que favorecessem a sua máquina.

Há duas formas de selecção:

1. rejeitar programas previstos como lentos, mesmo numa CPU completa;
2. construir hardware que execute muito depressa uma fracção da linguagem e rejeite tudo o que exige as unidades omitidas.

RandomX encadeia $N = 8$ programas. O primeiro deriva do bloco candidato. O estado resultante é condensado numa semente de 512 bits que gera o segundo; e assim sucessivamente:

$$H \longrightarrow P_0 \longrightarrow S_1 \longrightarrow P_1 \longrightarrow \cdots \longrightarrow S_7 \longrightarrow P_7 \longrightarrow h_{\mathrm{PoW}}. \tag{9.10}$$

Antes de executar $P_j$, o mineiro não conhece o estado que gerará $P_{j+1}$. Pode abandonar a corrente quando vê um sucessor indesejável, mas nesse caso perde todos os programas já executados.

### 9.10.1 Custo da filtragem de programas

Suponhamos que uma máquina aceita apenas uma fracção $Q$ dos programas e detecta gratuitamente os restantes. Para terminar uma corrente de $N$ programas, precisa de obter $N$ aceitáveis consecutivos. O número médio de programas efectivamente executados por resultado é

$$E_N(Q) = \frac{1 - Q^N}{(1 - Q)Q^{N-1}} = 1 + \frac{1}{Q} + \frac{1}{Q^2} + \cdots + \frac{1}{Q^{N-1}}. \tag{9.11}$$

Quando $Q \to 1$, o limite é $N$, como deve ser para uma máquina que aceita tudo. Para $Q = 3/4$, obtemos:

| $N$ | programas médios por resultado $E_N(3/4)$ | custo honesto $N$ |
|---|---|---|
| 1 | 1,0 | 1 |
| 2 | 2,3 | 2 |
| 4 | 6,5 | 4 |
| 8 | 27,0 | 8 |

Considere-se agora um dispositivo que, nos programas aceites, é 50% mais rápido do que uma CPU. A sua velocidade relativa, ignorando todos os outros custos, é

$$\frac{1{,}5N}{E_N(Q)}.$$

Com um só programa e $Q = 3/4$, conserva a vantagem de 1,5. Com oito, fica em cerca de

$$\frac{12}{27} \approx 0{,}44.$$

O dispositivo optimizado para três quartos da linguagem fica com menos de metade da velocidade da CPU por resultado. A corrente não impede a rejeição, mas aumenta o custo da selectividade.

### 9.10.2 Variância do tempo acumulado

Se os tempos $T_1, \ldots, T_N$ fossem independentes, com média $\mu$ e variância $\sigma^2$, o tempo total teria

$$\mathrm{E}\left[\sum_{j=1}^{N} T_j\right] = N\mu,$$

$$\mathrm{Var}\left(\sum_{j=1}^{N} T_j\right) = N\sigma^2.$$

O desvio relativo, ou coeficiente de variação, cairia como

$$\frac{\sqrt{N}\sigma}{N\mu} = \frac{1}{\sqrt{N}}\frac{\sigma}{\mu}. \tag{9.12}$$

Os programas reais ligam-se pelas sementes e não são independentes. Ainda assim, o cálculo mostra por que a soma de oito durações tem menor variação relativa do que uma duração isolada, reduzindo a vantagem disponível para um mecanismo de selecção baseado no tempo previsto.

## 9.11 Aritmética de vírgula flutuante

Uma regra de consenso deve dar o mesmo resultado em máquinas diferentes. Operações de vírgula flutuante podem divergir quando compiladores trocam a ordem das operações, guardam precisão adicional, contraem duas operações numa só ou tratam valores excepcionais de maneira distinta. RandomX usa essas operações porque a respectiva unidade é dispendiosa de reproduzir, mas fixa regras destinadas a conservar o consenso.

Um número normal de dupla precisão IEEE 754 tem um bit de sinal $s$, onze bits de expoente e 52 bits de fracção. O seu valor é

$$(-1)^s \left(1 + \frac{m}{2^{52}}\right) 2^{e-1023}, \tag{9.13}$$

para os expoentes normais. A recta real é muito mais densa do que o conjunto destes números. Depois de uma operação, o resultado exacto tem em geral de ser arredondado para um vizinho representável.

RandomX usa soma, diferença, produto, divisão e raiz quadrada: operações para as quais o padrão exige arredondamento correcto. Se $\circ$ representar uma delas e $\rho$ o modo de arredondamento, a máquina calcula

$$x \circ_\rho y = \mathrm{round}_\rho(x \circ y).$$

Os quatro modos aparecem nos programas:

| $\rho$ | destino do arredondamento |
|---|---|
| 0 | valor representável mais próximo, com desempate para o par |
| 1 | em direcção a $-\infty$ |
| 2 | em direcção a $+\infty$ |
| 3 | em direcção a zero |

Não basta ordenar «use IEEE 754». É preciso afastar zonas onde plataformas e ambientes de execução historicamente discordam. Os operandos introduzidos nas operações do grupo multiplicativo são forçados a ser positivos e a ter expoentes entre $-255$ e 0; assim se evitam raízes de números negativos, divisão por zero e resultados sem número. Valores subnormais são evitados. O infinito pode surgir, mas surge como uma representação definida e é depois misturado pelos seus bits.

Os quatro registos aditivos **f** ficam num domínio em que somar um operando de módulo maior do que 1 ainda altera bits significativos. Caso contrário, uma máquina especializada poderia substituir grande parte da vírgula flutuante por aritmética fixa mais estreita: a adição de um operando pequeno a um valor de grande módulo poderia ser sempre nula. Outra transformação muda bits do expoente para tornar essa representação simplificada menos eficiente.

Um compilador que resolva associar

$$(x + y) + z \quad \text{como} \quad x + (y + z)$$

pode obter outro resultado, pois cada parêntesis contém um arredondamento. Por isso, a tradução para código nativo deve preservar a ordem e o modo prescritos, sem aplicar reassociações algébricas. O intérprete e o tradutor podem ter desempenhos distintos, mas devem produzir exactamente os mesmos bits.

Esta parte de RandomX é matematicamente relevante justamente porque não há uma prova abstracta sobre reais. Há uma função finita sobre padrões de 64 bits, com arredondamento como parte da função. O consenso não pergunta pelo número ideal $x \circ y$; pergunta pelos bits de $\mathrm{round}_{\rho}(x \circ y)$.

## 9.12 Derivação completa do hash

Podemos agora descrever uma tentativa completa sem uma única linha de programa. Chamemos $\mathcal{B}_{512}$ e $\mathcal{B}_{256}$ às versões de BLAKE2b com saídas de 512 e 256 bits [3]. Dado o objecto de hash $H$, começa-se por

$$S_0 = \mathcal{B}_{512}(H). \tag{9.14}$$

Um gerador rápido baseado em rondas de AES expande $S_0$ para encher os 2 MiB do Scratchpad. O seu estado final inicializa outro gerador, com mais rondas por bloco, que produz a configuração e as 256 instruções do primeiro programa. As rondas de AES aqui são blocos de construção acelerados por processadores; não estão a cifrar uma mensagem secreta.

Se $\mathrm{RF}(\mathcal{S}_j)$ for o ficheiro de registos de 256 bytes depois de executar o programa $P_j$, os sete estados intermédios satisfazem

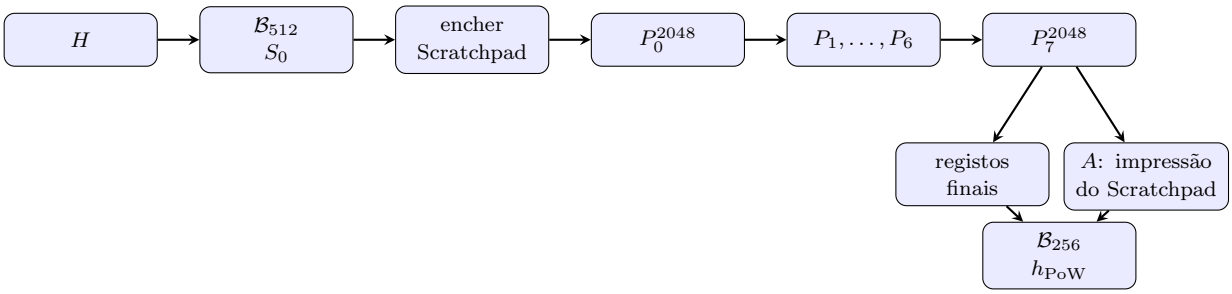
$$S_{j+1} = \mathcal{B}_{512}\big(\mathrm{RF}(\mathcal{S}_j)\big), \qquad j = 0, \ldots, 6. \tag{9.15}$$

Cada $S_{j+1}$ gera a configuração e o programa seguintes. O Scratchpad não é reiniciado entre programas e conserva as alterações acumuladas na tentativa.

Depois de $P_7$, uma função baseada em rondas de AES percorre o Scratchpad inteiro e produz uma impressão de 64 bytes $A$. Essa impressão substitui o último quarto do ficheiro de registos antes da condensação final:

$$h_{\mathrm{PoW}} = \mathcal{B}_{256}\big(\mathbf{r} \,\|\, \mathbf{f} \,\|\, \mathbf{e} \,\|\, A\big). \tag{9.16}$$

O símbolo $\|$ indica concatenação nas representações fixadas pela especificação.



As funções rápidas de uma ou quatro rondas de AES não são usadas como se fossem, isoladamente, oráculos criptográficos ideais. A primeira recebe uma semente produzida por BLAKE2b; a impressão do Scratchpad não recebe entrada livremente escolhida e entra por fim em BLAKE2b. A fronteira criptográfica forte continua a ser a função hash; as rondas de AES servem para mover e misturar muitos bytes depressa em hardware comum.

Esta composição também explica por que não se pode calcular apenas os registos e esquecer a memória. O hash final inclui $A$, que depende do Scratchpad inteiro, e os registos dependem de leituras do Dataset e do Scratchpad ao longo de 16384 voltas. Qualquer atalho deve reproduzir não uma saída isolada, mas a rede de dependências que a alimentou.

## 9.13 Condição de dificuldade e probabilidade de êxito

RandomX termina onde começou o capítulo: num inteiro de 256 bits. Se tratarmos $h_{\mathrm{PoW}}$ como aproximadamente uniforme em $\{0, 1, \ldots, 2^{256} - 1\}$, a condição (9.1) equivale a

$$h_{\mathrm{PoW}} \leq \left\lfloor \frac{2^{256} - 1}{d} \right\rfloor .$$

Logo, a probabilidade de uma tentativa passar é

$$p_d = \frac{\lfloor (2^{256} - 1)/d \rfloor + 1}{2^{256}} \approx \frac{1}{d}. \tag{9.17}$$

Se $N$ for o número de tentativas até ao primeiro êxito, então, no modelo de tentativas independentes,

$$\Pr[N = k] = (1 - p_d)^{k-1} p_d,$$

$$\mathrm{E}[N] = \frac{1}{p_d} \approx d,$$

$$\mathrm{Var}(N) = \frac{1 - p_d}{p_d^2}.$$

O mineiro não prova que executou exactamente $d$ hashes. Pode obter êxito na primeira tentativa ou exceder a média. Prova apenas que apresentou um resultado cuja probabilidade corresponde à dificuldade.

Esta condição não depende da marca do processador. Se o mineiro $i$ produz $r_i$ tentativas por segundo e a rede produz

$$R = \sum_i r_i,$$

a probabilidade aproximada de ele encontrar o próximo bloco é $r_i/R$. RandomX não altera essa proporcionalidade; tenta alterar quanto custa comprar cada unidade de $r_i$ fora de um processador acessível.

O trabalho esperado por bloco continua a ser determinado por $d$, enquanto a rede ajusta $d$ para conservar o intervalo de dois minutos. Uma melhoria de hardware não faz sair blocos permanentemente mais depressa: aumenta transitoriamente $R$, e a dificuldade acompanha-a. A

vantagem duradoura do mineiro é económica, uma fracção maior das recompensas por unidade de custo.

## 9.14 Economia da produção de hashes

Se um bloco paga recompensa base $B$ e taxas $F$, o rendimento bruto esperado do mineiro $i$ por bloco é aproximadamente

$$\frac{r_i}{R}(B + F). \tag{9.18}$$

Do outro lado estão electricidade, capital, memória, refrigeração, risco e o uso alternativo da máquina. A concorrência tende a acrescentar potência enquanto o rendimento marginal esperado justificar o custo marginal. Não há garantia de equilíbrio instantâneo, nem de que todos paguem a mesma electricidade; há um incentivo persistente para procurar eficiência.

Um algoritmo que favoreça fortemente um dispositivo escasso pode concentrar essa procura em fabricantes e operadores com acesso privilegiado. Um algoritmo favorável a CPUs não garante mineração individual nem distribuição uniforme. Explorações profissionais continuam a beneficiar de escala, electricidade barata, memória bem configurada e refrigeração. A afirmação mais estreita é que o principal bem de capital também tem mercados amplos e usos alternativos.

Isto muda o custo de entrada e o custo de saída. Um ASIC para uma função particular pode tornar-se sucata se a moeda mudar de algoritmo; uma CPU pode ser revendida ou aplicada a outra tarefa. Chamemos $P$ ao preço de compra, $V$ ao valor alternativo depois da mineração e $L = P - V$ ao capital efectivamente preso. Para equipamento reaproveitável, $V$ tende a ser maior e $L$ menor. A decisão de entrada fica menos dependente da continuidade de uma função de prova de trabalho específica.

CPUs disseminadas também podem ser abusadas por *malware* ou redes de máquinas comprometidas. RandomX exige memória substancial para mineração eficiente e torna a actividade intensiva mais visível, mas não impede o uso não autorizado de computadores. O algoritmo altera custos técnicos; não resolve o problema de autorização do equipamento.

Por isso, «mais igualitário» designa um objectivo relativo de desenho, não uma garantia de distribuição. Se muitos participantes delegarem a mineração em *pools*, se a energia for concentrada ou se aparecer hardware superior, a potência pode concentrar-se mesmo com RandomX.

## 9.15 Limites da resistência a ASIC

Há três afirmações de força crescente que não devemos confundir:

1. RandomX **funciona bem em CPUs**: isto pode ser medido;

2. RandomX **usa recursos caros de omitir**: isto pode ser modelado, comparado e atacado;

3. RandomX **impede para sempre qualquer ASIC vantajoso**: isto não foi demonstrado.

O termo mais exacto usado no desenho é *vinculação ao dispositivo*. A função escolhe como referência a classe de processadores gerais existente e tenta ocupar uma secção transversal da sua arquitectura. Um ASIC pode retirar funções administrativas, interfaces, compatibilidade e margens que RandomX não usa. Pode especializar a circulação de dados, escolher tecnologia de memória diferente e explorar regularidades ainda desconhecidas. Há portanto espaço para alguma vantagem.

O objectivo é impedir que a vantagem resulte apenas da implementação directa de uma longa função fixa. O dispositivo precisa de:

- executar todas as famílias de instruções, sob pena do custo de filtragem da equação (9.11);
- manter muitos registos e explorar instruções independentes;
- respeitar vírgula flutuante e quatro arredondamentos;
- suportar saltos e estados de memória dependentes;
- fornecer cerca de dois gigabytes de Dataset por grupo de trabalho eficiente, ou pagar a reconstituição leve;
- acompanhar uma chave de Dataset que muda com a cadeia.

Uma FPGA enfrenta dificuldade semelhante. Reconfigurar fisicamente o circuito para cada programa demoraria incomparavelmente mais do que a tentativa; manter um processador virtual dentro da FPGA aproxima-a justamente da máquina geral que pretendia superar. O espaço de programas deriva de sementes de 512 bits e contém até $2^{512}$ possibilidades, o que impede a preparação antecipada de um circuito distinto para cada programa.

Nem todos estes argumentos têm o mesmo estatuto. A igualdade dos modos leve e completo e a transição da máquina virtual pertencem à especificação. A uniformidade do resultado depende das propriedades criptográficas das funções e dos testes. A vantagem futura de hardware depende de tecnologia e economia. Uma boa exposição deve dizer onde acaba a função e começa a previsão.

## 9.16 Auditorias, versão e fronteira de consenso

RandomX foi submetido em 2019 a quatro auditorias independentes, por Trail of Bits, X41 D-Sec, Kudelski Security e QuarksLab. Os relatórios não registaram vulnerabilidades críticas; encontraram questões e recomendações que levaram a alterações no algoritmo e na implementação

[111]. As auditorias constituem evidência limitada às revisões, plataformas e âmbitos analisados; não testam hardware futuro.

Também há duas espécies de actualização. Uma implementação pode corrigir a tradução para uma arquitectura sem mudar a função abstracta: os hashes correctos permanecem os mesmos. Uma versão *algorítmica* muda a função e portanto exige coordenação de consenso. Se dois nós calcularem funções RandomX diferentes para o mesmo $(K, H)$, podem discordar sobre (9.1).

Na revisão consultada para esta edição, o código principal de Monero fixa o compromisso RandomX 1.2.3, da linha algorítmica 1 [126, 89]. O projecto RandomX publicou entretanto a versão algorítmica 2 [112]. Ela acrescenta trabalho na mistura dos registos de vírgula flutuante, entre outras alterações, mas não é por isso uma regra activa de Monero. Substituir a biblioteca por iniciativa individual não seria uma optimização: seria calcular outra prova de trabalho.

Uma versão mais recente pode corrigir uma falha de implementação sem alterar o consenso, mas não deve substituir automaticamente a versão algorítmica activa. A regra de consenso é a versão que todos os nós devem aplicar àquela altura; uma proposta posterior só se torna activa por coordenação de consenso.

## 9.17 Resumo da avaliação RandomX

Para minerar um bloco à altura $h$:

1. calcula-se $s(h)$ pela equação (9.4) e toma-se como chave $K$ o ID desse bloco-semente;
2. de $K$, Argon2d produz a Cache; dela pode materializar-se o Dataset, ou cada item pode ser reconstituído quando pedido;
3. o objecto de hash do bloco candidato, incluindo o nonce, fornece $H$ e a semente inicial $S_0 = \mathcal{B}_{512}(H)$;
4. $S_0$ enche o Scratchpad e gera o primeiro programa;
5. oito programas de 256 instruções fazem 2048 voltas cada, lendo o Dataset e transformando registos e Scratchpad;
6. o estado de cada um gera o programa seguinte; depois do oitavo, a impressão do Scratchpad e os registos entram em BLAKE2b-256;
7. o inteiro $h_{\mathrm{PoW}}$ é aceite exactamente quando $h_{\mathrm{PoW}} d \le 2^{256} - 1$.

O verificador executa estes passos uma vez. O mineiro altera o nonce e repete a tentativa até obter um resultado abaixo do alvo. A dificuldade determina a probabilidade de êxito;

RandomX procura reduzir a vantagem económica de hardware especializado na produção de cada tentativa.

A análise usa uma função determinística composta, uma variável geométrica, cadeias de programas, produtos área–tempo, estados finitos, arredondamentos e modelos de custo. A Cache e o Dataset dependem da época; o Scratchpad e os oito programas dependem da tentativa; BLAKE2b produz o resultado submetido à condição de dificuldade.

# CAPÍTULO 10

# Dandelion++, propagação de transacções e poda de nós

Quando uma carteira entrega uma transacção ao seu daemon, a transacção ainda não pertence a um bloco. Tem primeiro de ser validada localmente e propagada pela rede P2P. Os instantes e as direcções dos primeiros envios constituem metadados que podem permitir a um observador estimar o nó de origem, sem que seja necessário violar qualquer primitiva criptográfica da transacção.

Este capítulo separa três funções do nó:

1. Dandelion++ controla a *propagação inicial de transacções*;
2. a sincronização obtém e valida a lista de blocos;
3. a poda determina que parte dos dados históricos já validados permanece disponível localmente.

As três funções usam ligações P2P e políticas locais, mas têm objectivos e garantias diferentes. Dandelion++ não sincroniza blocos, e a poda não protege a origem de uma transacção.

Salvo indicação em contrário, toda a descrição da implementação Monero neste capítulo foi conferida na revisão `3646f648db57`, de 14 de Agosto de 2026 [89]. Estes comportamentos são política de rede e de armazenamento. Não são regras de consenso.

## 10.1 Propagação por inundação e exposição temporal

Uma maneira simples de propagar uma transacção é a inundação (*flooding*). O nó que a recebe pela primeira vez envia-a a muitos pares; cada par faz o mesmo, omitindo em geral a ligação por onde ela chegou. A cobertura cresce depressa. Se cada nó alcançar em média $d$ vizinhos novos, uma árvore idealizada teria aproximadamente $d^r$ receptores à distância $r$. A realidade contém ciclos, ligações lentas e nós que já conhecem a transacção, mas a principal vantagem continua a ser a velocidade.

Esta velocidade produz informação temporal. Seja $t_v$ o instante em que um observador vê a transacção chegar pelo nó $v$. Uma estimativa simples da origem é

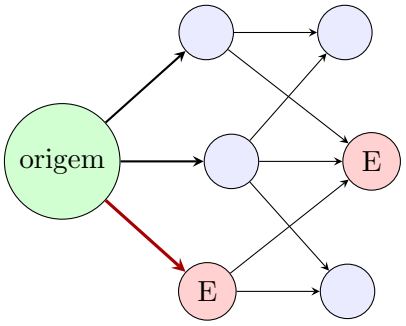
$$\widehat{v} = \operatorname*{argmin}_{v} \big(t_v - \widehat{\delta}_v\big), \tag{10.1}$$

onde $\widehat{\delta}_v$ é a latência que o observador atribui ao caminho até $v$. Não é necessário estimar todos os atrasos com exactidão. Se o adversário mantiver muitas ligações, o primeiro par honesto que lhe entregar a transacção é frequentemente um bom candidato à origem. Esta é a estimativa do *primeiro espião* estudada no artigo Dandelion++ [38].

Num modelo simplificado, cada um dos $d$ primeiros destinatários é controlado pelo adversário com probabilidade independente $a$. A probabilidade de a origem contactar logo pelo menos um espião é

$$P(\text{espião na primeira ronda}) = 1 - (1 - a)^d. \tag{10.2}$$

Com $d = 12$ e $a = 0{,}1$, o valor já é aproximadamente 0,718. Os pares da Internet não são amostras independentes e um contacto não é uma prova de autoria. O cálculo mostra apenas que aumentar o número de destinatários iniciais aumenta a probabilidade de contacto directo com o adversário.




inundação: muitos primeiros contactos;
o espião $E$ pode observar directamente a origem

Aleatorizar pequenos atrasos antes de cada envio dificulta a estimativa em (10.1), mas não reduz o número de destinatários do primeiro salto. Dandelion++ começa por encaminhar a transacção para um único sucessor antes de iniciar a difusão ampla.

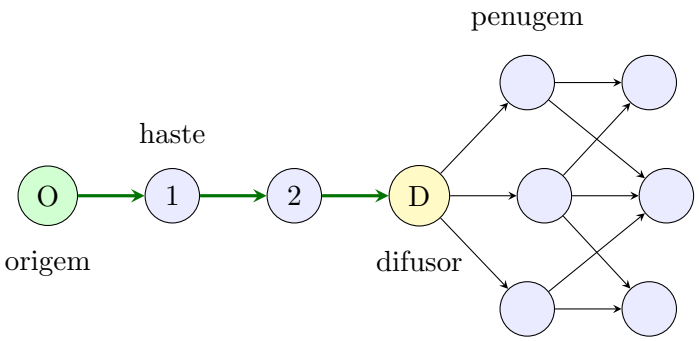
## 10.2 Fases de encaminhamento e difusão

Dandelion++ divide a propagação em duas fases [38]. Os nomes *stem* e *fluff* pertencem à terminologia do protocolo:

**haste (*stem*)** cada nó envia a transacção a um só sucessor num subgrafo temporário de anonimato;

**penugem (*fluff*)** um nó difusor muda o estado da transacção e a envia largamente; os restantes continuam essa difusão.

O nó que inicia a fase de penugem não tem de ser o nó de origem. Um adversário que observa primeiro a difusão pública sabe que houve uma fase de haste anterior, mas essa observação não identifica qual dos nós desse percurso criou a transacção.




Suponhamos que, em cada nó de retransmissão, a probabilidade de ser difusor é $q$, independentemente dos anteriores. A origem faz sempre pelo menos um salto de haste. Se $K$ contar os saltos de haste até à transição, então

$$P(K = k) = (1-q)^{k-1}q, \qquad k = 1, 2, \ldots, \tag{10.3}$$

$$P(K > k) = (1-q)^{k}, \qquad \mathbb{E}[K] = \frac{1}{q}.$$

Na implementação consultada, $q = 0{,}20$, pelo que $\mathbb{E}[K] = 5$. Este valor esperado não fixa uma distância de cinco nós para cada transacção. Cerca de 32,8% das hastes continuam activas após cinco decisões de retransmissão, pois $0{,}8^5 \approx 0{,}328$.

As transições de estado são

$$\mathsf{S} \xrightarrow[\text{um sucessor}]{1-q} \mathsf{S}, \qquad \mathsf{S} \xrightarrow[\text{muitos pares}]{q} \mathsf{F}, \qquad \mathsf{F} \xrightarrow[\text{muitos pares}]{} \mathsf{F}. \tag{10.4}$$

Existe também uma transição de segurança que impede a retenção indefinida da transacção por um nó malicioso ou indisponível:

$$\mathsf{S} \xrightarrow[\text{expirou o embargo}]{\text{falha segura}} \mathsf{F}. \tag{10.5}$$

## 10.3 Subgrafo de anonimato por época

O artigo não encaminha a haste no grafo inteiro de ligações. Em cada época, cada nó escolhe uniformemente, sem reposição, até duas das ligações que ele próprio iniciou. A união destas escolhas forma o subgrafo de anonimato $G_{\mathrm{s}} = (V, E_{\mathrm{s}})$. Cada nó tem grau de saída no máximo dois. Se as escolhas forem aproximadamente uniformes, o grau de entrada médio também é dois; daí a expressão «aproximadamente 4-regular». Não se exige que cada nó receba exactamente duas arestas nem que o grafo seja regular.

Para impedir que várias transacções vindas da mesma direcção revelem caminhos independentes por intersecção, o artigo fixa, durante a época, uma aplicação

$$f_e : \; E_{\mathrm{s}}^{\mathrm{in}}(v) \longrightarrow E_{\mathrm{s}}^{\mathrm{out}}(v). \tag{10.6}$$

Todas as hastes que chegam pela mesma aresta seguem pela mesma saída. A transacção criada no próprio nó recebe igualmente uma saída persistente. No artigo, as imagens de $f_e$ são escolhidas pseudo-aleatoriamente com reposição; cada nó é, durante toda a época, retransmissor ou difusor para todas as transacções alheias. As épocas avançam assincronamente, da ordem de dez minutos, para evitar que todos os nós alterem o mapa simultaneamente [38].

Monero conserva a estrutura, mas não copia cada detalhe. Na revisão consultada:

- há até duas hastes de saída por época, escolhidas aleatoriamente apenas entre ligações de saída cujo par não esteja mais de dois blocos atrasado em relação à altura de referência do nó;
- cada ligação de entrada é associada de modo persistente à haste menos usada; os empates são aleatórios. A origem local é representada por uma entrada própria e recebe a mesma espécie de associação;
- a época dura
$$T_{\mathrm{ep}} = 600\ \mathrm{s} + U[0, 30\ \mathrm{s}], \tag{10.7}$$
após o que se renovam hastes, associações e o papel de difusor;
- se uma haste se desligar, o mapa tenta substituí-la por uma ligação elegível sem esperar pela época seguinte.

A escolha «menos usada» aproxima as duas cargas; não é a amostragem com reposição do artigo. A elegibilidade pela altura é uma necessidade específica do nó Monero, não uma propriedade matemática de Dandelion++: encaminhar por um par significativamente dessincronizado prejudicaria a propagação sem melhorar a garantia formal de anonimato.

`src/net/dandelionpp.cpp`; `src/cryptonote_protocol/levin_notify.cpp`

## 10.4 O comportamento implementado na rede pública

Um nó sincronizado que recebe uma nova transacção valida-a antes de a retransmitir. Enquanto ainda sincroniza, ignora estas notificações: não tem estado suficiente para as tratar com

utilidade. Na rede pública, uma notificação sem a marca de penugem entra como haste; uma marcada como penugem entra como difusão. Estes são estados da txpool e da propagação, não campos da transacção que um mineiro compromete num bloco [89].

No começo de cada época, o nó torna-se difusor com probabilidade 0,20, ou retransmissor com probabilidade 0,80. Uma transacção recebida em haste é portanto enviada a uma só haste se o nó for retransmissor e passa à penugem se ele for difusor. Há uma excepção deliberada: uma transacção criada localmente é sempre tratada primeiro como haste e tenta esse primeiro salto, mesmo durante uma época em que o nó difunde transacções alheias. Esta excepção procura realizar a primeira seta de (10.3); a falha de ambas as tentativas conduz à promoção descrita a seguir.

O envio por haste tenta obter e usar um destino válido. Se falhar, actualiza as ligações e tenta uma segunda vez. Se também falhar, promove a transacção a penugem. Se a mesma transacção chegar duas vezes como haste, o ciclo é detectado e ela é igualmente promovida. São saídas de robustez: diminuem a probabilidade de desaparecimento, embora uma promoção prematura encurte o caminho de anonimato.

Na penugem, o nó prepara a transacção para todas as ligações públicas salvo a que a forneceu. Não envia tudo no mesmo instante. Cada ligação tem uma fila e um atraso aleatório; antes do envio, os corpos são ordenados e os duplicados removidos, para não publicar a ordem de chegada. O protocolo corrente envia o corpo completo da transacção. A proposta aberta #9334 substituiria, na penugem, estes envios redundantes por anúncios de hashes e pedidos posteriores do corpo; em Agosto de 2026 isso continua a ser uma proposta, não comportamento activo nem mudança de consenso [16].

O operador pode pedir enchimento das mensagens de transacções públicas até ao múltiplo seguinte de 1024 bytes. Essa opção não está activa por omissão. O enchimento reduz a resolução de uma análise por volume; não esconde um fluxo a um observador capaz de correlacionar horários e direcções.

### 10.4.1 Temporizadores de difusão e embargo

Os atrasos da penugem são amostras de Poisson numa grelha de um quarto de segundo. Para uma ligação que entrou no nó, a média é cinco segundos; para uma ligação que o nó iniciou, é metade:

$$\frac{D_{\text{in}}}{0{,}25\ \text{s}} \sim \text{Pois}(20), \qquad \mathbb{E}[D_{\text{in}}] = 5\ \text{s}, \tag{10.8}$$

$$\frac{D_{\text{out}}}{0{,}25\ \text{s}} \sim \text{Pois}(10), \qquad \mathbb{E}[D_{\text{out}}] = 2{,}5\ \text{s}. \tag{10.9}$$

Assim, a difusão mantém um atraso médio pequeno sem usar intervalos determinísticos por ligação.

Cada nó que guardou ou encaminhou uma transacção em haste inicia também um temporizador de embargo:

$$\frac{T_{\text{emb}}}{1\text{ s}} \sim \text{Pois}(39), \qquad \mathbb{E}[T_{\text{emb}}] = 39\text{ s}. \tag{10.10}$$

Se entretanto observar a transacção em penugem, o nó cancela a necessidade de promoção local. Se o prazo chegar primeiro, a rotina de reenvio promove-a a difusão pública. O prazo é local e aleatório: vários nós da haste podem manter temporizadores, e a primeira rotina executada inicia a difusão. A carga do processo e o agendamento podem atrasar a execução relativamente ao valor amostrado.

O comentário que acompanha esta constante dá como motivação

$$\overline{T} = -\frac{k(k-1)\overline{h}}{2\log(1-\varepsilon)}. \tag{10.11}$$

A equação documenta uma motivação de desenho, não uma regra de consenso. A substituição dos valores ali impressos, $k = 5$, $\varepsilon = 0{,}1$ e $\overline{h} = 0{,}175$ s, dá aproximadamente 16,6 s, não 39 s. O comportamento exacto da revisão é o de (10.10); a conta no comentário não altera a constante compilada.

Depois de pública, uma transacção obedece ainda à política normal da txpool: um reenvio periódico não começa antes de cinco minutos e o intervalo cresce até ao máximo de quatro horas. Esse reenvio combate perdas comuns e é distinto da transição de segurança Dandelion++ após o embargo de 39 segundos.

## 10.5 Relação com Tor e I2P

Quando o operador configura uma rede de anonimato para transacções locais, Monero trata-a como uma zona própria. A integração corrente suporta Tor e I2P e encaminha por elas transacções, não a sincronização da lista de blocos. A sincronização pode atravessar um *proxy* para endereços públicos, mas os serviços ocultos não fornecem o protocolo completo de sincronização [89].

Por omissão, a zona Tor/I2P usa ruído. Escolhe até dois canais de saída por épocas de

$$T_{\text{ruído}} = 300\text{ s} + U[0, 30\text{ s}] \tag{10.12}$$

e, em cada canal ligado, tenta enviar um fragmento de 3 KiB — real ou fictício — após

$$D_{\text{ruído}} = 10\text{ s} + U[0, 5\text{ s}]. \tag{10.13}$$

Uma mensagem real pode ocupar no máximo vinte fragmentos, cerca de 60 KiB. Como o tamanho e a periodicidade dos fragmentos já fornecem cobertura, o enchimento público de 1024 bytes não intervém.

O modo de ruído tem precedência sobre Dandelion++: não há haste Dandelion++ através desses canais. Uma transacção é posta nas filas cobertas dos canais ligados, até duas, e sai ao ritmo de (10.13); na ausência de mensagem, sai ruído. Isto procura esconder a ocasião do envio a um observador do acesso à Internet, enquanto Tor ou I2P escondem a origem da ligação ao

par remoto. Se o ruído for explicitamente desactivado, a zona usa a rotina de penugem apenas para ligações de saída. Uma transacção recebida de Tor/I2P sem marca de penugem recebe, antes do reencaminhamento, um atraso de Poisson de média 22 segundos.

As garantias de Dandelion++ e das redes de anonimato não são aditivas. Tor é vulnerável a correlação de tráfego; I2P tem uma topologia e uma população diferentes; e um serviço oculto malicioso continua a observar a ligação que lhe entregou a transacção. A reutilização de canais, a execução do daemon apenas durante o pagamento ou a manipulação adversarial da largura de banda produzem correlações que Dandelion++ não elimina. A integração oficial é classificada como experimental, pelo que essas limitações fazem parte do seu modelo operacional.

## 10.6 Modelo do adversário e limites

Considere-se um adversário honesto-curioso que controla uma fracção $a$ dos nós candidatos, segue o protocolo e partilha observações. Condicionada a uma haste de $k$ saltos, a probabilidade idealizada de nenhum desses destinos ser adversarial é

$$P(\text{nenhum espião} \mid K = k) = (1 - a)^k. \tag{10.14}$$

Se em cada nó honesto há probabilidade $q$ de começar a penugem, a probabilidade de a haste alcançar um espião antes da difusão satisfaz

$$r = a + (1 - a)(1 - q)r,$$

e portanto

$$r = \frac{a}{a + q - aq}. \tag{10.15}$$

Para $a = 0{,}1$ e $q = 0{,}2$, o resultado é cerca de 0,357. O contacto com um nó adversarial não identifica automaticamente a origem: esse nó observa apenas o predecessor imediato, que pode ter criado ou retransmitido a transacção. Dandelion++ aumenta o conjunto de predecessores plausíveis e resiste à aprendizagem do grafo e à intersecção de vários caminhos [38].

O modelo independente deixa de ser adequado quando o adversário influencia a selecção de pares. Muitas identidades Sybil, endereços concentrados, conhecimento do grafo ou controlo dos pares de saída aumentam a probabilidade efectiva $a$. Um adversário activo pode ainda:

- suprimir transacções recebidas em haste, forçando o embargo e observando qual nó acaba por difundir;
- encaminhar ou marcar mensagens de modo diferente para aprender o subgrafo;
- comparar várias transacções da mesma origem ao longo de épocas;
- combinar horários P2P com informação do fornecedor de acesso, da carteira, de uma bolsa ou de um destinatário.

O mapa persistente por ligação reduz o ataque de intersecção dentro de uma época, mas não torna secreto para sempre o grafo subjacente. A renovação reduz o valor de observações antigas,

não as revoga. Um observador global passivo que veja simultaneamente as ligações de grande parte da rede pode correlacionar a primeira haste com a penugem posterior. Dandelion++ foi desenhado contra desanonimização de rede em grande escala sob modelos declarados; não oferece anonimato contra um observador global passivo.

Há duas limitações práticas importantes. Primeiro, uma carteira que usa o daemon remoto de terceiro revela a esse daemon a ligação por onde submeteu a transacção. Dandelion++ actua apenas depois dessa submissão e não oculta essa ligação inicial. Segundo, a implementação corrente só escolhe hastes de saída. Um nó inalcançável, que apenas estabelece ligações de saída, pode enviar uma haste a um par que não consegue reenviá-la de volta para a população de nós inalcançáveis. A proposta aberta do Monero Research Lab #160 procura permitir uma vaga de haste por ligações de entrada acompanhadas de uma prova de limite de ligações. É investigação proposta, não código activo [17].

## 10.7 Separação entre anonimato de origem, privacidade e consenso

**Anonimato da origem.** Dandelion++, Tor/I2P e os seus temporizadores tentam dificultar a ligação entre a primeira aparição P2P de uma transacção e o endereço de rede do nó que a submeteu.

**Privacidade da transacção.** Endereços de utilização única, compromissos de montante e provas de gasto escondem destinatário, montantes e origem na própria transacção. Foram construídos nos capítulos 5–7.

**Consenso.** As regras de validação decidem se uma transacção pode entrar num bloco e qual cadeia possui trabalho suficiente. Dandelion++ não muda os bytes assinados, a imagem de chave, a prova de intervalo, a dificuldade ou a escolha da cadeia.

A privacidade dos dados na cadeia não implica anonimato do endereço IP. Do mesmo modo, ocultar a origem de rede não impede que a própria transacção seja reconhecível nem garante a sua validade. Estas propriedades são distintas. A falha de Dandelion++ pode revelar metadados ou atrasar a propagação, mas não cria moeda nem valida um gasto duplo. Inversamente, o consenso não garante que uma transacção ainda fora de um bloco seja propagada rapidamente aos mineiros.

## 10.8 Poda da blockchain e sincronização de nós podados

Dandelion++ propaga uma transacção recente da txpool; a sincronização transfere blocos históricos e constrói o estado necessário para validar transacções futuras. Ambas as funções

consomem ligações, largura de banda, memória e disco, mas não partilham as fases de haste e penugem. A poda não oculta a origem de uma transacção, e Dandelion++ não determina que dados históricos são conservados.

O capítulo 8, na secção 8.4.1, já separou exactamente o material podável do indispensável e mostrou como o hash permite reconstruir o mesmo ID. Não repetiremos aqui a lista de campos. Interessa-nos agora a política aplicada a essa decomposição depois da validação.

### 10.8.1 Dimensão da base podada

Sejam $S_{\text{indispensable}}$ os dados que todos os nós podados conservam, $S_{\text{old,prunable}}$ a parte podável dos blocos antigos e $S_{\text{recent}-\text{tip}}$ a parte podável da ponta recente, que também se conserva inteira. Para uma faixa $s$ escolhida uniformemente, a conta central é uma esperança:

$$\mathbb{E}_s[S_{\text{pruned}}] = S_{\text{indispensable}} + \frac{1}{8} S_{\text{old,prunable}} + S_{\text{recent}-\text{tip}}, \tag{10.16}$$

$$\mathbb{E}_s[S_{\text{discarded}}] = \frac{7}{8} S_{\text{old,prunable}}. \tag{10.17}$$

Um nó concreto conserva uma faixa inteira, não exactamente um oitavo dos bytes: o código distribui troços de alturas pelas oito faixas, e os troços têm tamanhos variáveis. O oitavo esperado refere-se apenas à *parte antiga podável*, não ao conjunto da blockchain. Cabeçalhos, prefixos, dados necessários ao estado, índices e a ponta recente não são multiplicados por 1/8. Por isso uma base podada não tem um oitavo do tamanho da base completa. A documentação oficial resume a poupança como cerca de dois terços [90]; os valores publicados no repositório Monero em Junho de 2026 eram aproximadamente 280 GB para a base completa e 95 GB para a podada, isto é,

$$\frac{95}{280} \approx 0{,}339.$$

O resultado é próximo de um terço, em conformidade com (10.16), e não de um oitavo.

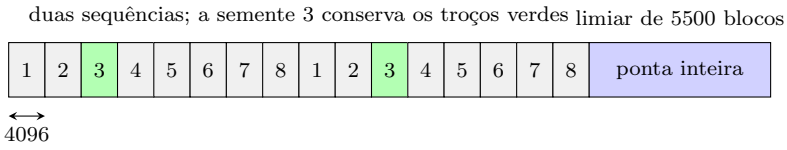
### 10.8.2 Faixas, semente e janela recente

Fora da ponta recente, a revisão consultada divide as alturas em troços de 4096 blocos e faz rodar oito faixas. Para uma altura $h$ e altura corrente $H$, a função é

$$\sigma(h, H) = \begin{cases} 0, & h + 5500 \geq H, \\ 1 + \left( \left\lfloor \dfrac{h}{4096} \right\rfloor \bmod 8 \right), & h + 5500 < H. \end{cases} \tag{10.18}$$

A faixa zero é uma convenção para «ponta não podada». Um nó podado recebe aleatoriamente uma faixa $s \in \{1, \ldots, 8\}$, conserva os dados podáveis antigos dos troços com $\sigma(h, H) = s$ e elimina esses dados nos outros sete troços. A escolha e o expoente $\log_2 8 = 3$ são codificados na *semente de poda*. A semente identifica a política de retenção, mas não permite reconstruir os dados eliminados.

`src/common/pruning.cpp`; `src/cryptonote_config.h`




À medida que $H$ avança, a janela recente avança com ele. Em operações de manutenção, o nó examina os dados que acabaram de sair da condição $h + 5500 \geq H$: conserva-os se pertencem à sua faixa e liberta-os se não pertencem. A ponta inteira ajuda a servir os blocos recentes e as reorganizações usuais; 5500 não é uma promessa de finalidade. O consenso continua a poder escolher uma reorganização válida mais profunda.

### 10.8.3 Dados fornecidos por um nó podado

Um nó podado pode fornecer a qualquer par os blocos e as partes indispensáveis da história, os dados podáveis completos da sua faixa e todos os dados da ponta recente. Não pode servir uma prova histórica que apagou. Um nó completo, representado por semente zero no protocolo, pode servir qualquer faixa.

A escolha aleatória de $s$ encoraja cobertura distribuída. Se $n$ nós podados escolhessem faixas independentemente e não houvesse nós completos, a probabilidade de uma faixa fixada não aparecer seria

$$\left(\frac{7}{8}\right)^n, \tag{10.19}$$

e, pela união dos oito acontecimentos, a probabilidade de faltar pelo menos uma faixa seria no máximo

$$8\left(\frac{7}{8}\right)^n. \tag{10.20}$$

É um incentivo estatístico, não uma obrigação. Operadores podem copiar bases, escolher a mesma faixa, ficar desligados ou desaparecer. As regras de consenso validam blocos, mas não impõem a distribuição das faixas entre discos privados. Logo, a cobertura de todas as faixas é desejada pela política de rede, mas não garantida pelo consenso.

### 10.8.4 Modos de sincronização

A expressão «sincronização podada» pode designar três procedimentos distintos.

**Validar primeiro, podar depois.** Um nó pode sincronizar uma base completa, receber cada prova histórica, validar blocos e transacções e só depois executar a poda local. Nesse percurso, a decisão de apagar vem depois da verificação independente. Podar uma LMDB já existente marca páginas como livres para reutilização e pode não encolher logo o ficheiro; a ferramenta de cópia podada cria uma base fisicamente menor.

**Começar do zero com poda activada.** Com `--prune-blockchain`, o daemon cria desde o início uma semente e mantém a base segundo (10.18). Na revisão consultada, esta opção, por si só, não activa `--sync-pruned-blocks`. O nó pede material completo a pares capazes de servir cada faixa, valida-o e vai libertando localmente os sete oitavos antigos que não lhe cabem. Poupa disco durante o crescimento, mas não activa automaticamente a aceitação de respostas sem as provas podáveis.

**Sincronizar a partir de respostas podadas.** A opção adicional `--sync-pruned-blocks` autoriza pedidos de respostas podadas quando a base local também tem uma semente. O pedido só é feito se todas estas condições se verificarem na revisão de Agosto de 2026:

- o intervalo não pertence à faixa local nem à ponta recente;
- começa pelo menos na altura ideal da versão 11, porque antes dela o nó não reconstrói por esta via o peso necessário;
- o último bloco do intervalo está dentro da zona de hashes de blocos compilados no programa;
- os pesos de todos os blocos pedidos já constam dessa informação compilada.

Se uma condição falhar, o nó pede dados completos. Assim se compreende a frase documental «sincronizar só cerca de um terço»: descreve o efeito do percurso que aceita história podada onde ele é permitido, não uma identidade universal entre `--prune-blockchain` e uma descarga de 1/8.

`src/cryptonote_protocol/cryptonote_protocol_handler.inl`

### 10.8.5 Verificar um hash não verifica uma prova ausente

Numa resposta podada, o par fornece o bloco, a base de cada transacção, o hash da parte podável e o peso necessário. O nó consegue reconstruir o ID da transacção, confirmar que esse ID aparece no bloco, encadear cabeçalhos e construir o estado indispensável. Recusa uma resposta podada quando tinha pedido uma completa e recusa pesos nulos onde esperava dados podados.

O que ele não consegue fazer é recalcular uma prova criptográfica cujos bytes não recebeu. Para os intervalos aceites por `--sync-pruned-blocks`, a correspondência com os hashes e pesos compilados vincula os dados recebidos aos valores de referência distribuídos com o programa. Um par não pode substituir a prova ausente por dados incompatíveis sem mudar os IDs; mas o nó também não obtém, nessa sincronização, uma reverificação independente das assinaturas e provas removidas. O procedimento pressupõe que a história comprometida pelos pontos de controlo foi validada antes da sua compilação.

Numa sincronização completa seguida de poda, as provas são verificadas antes de serem eliminadas. Numa sincronização baseada em respostas podadas, o nó verifica os dados recebidos e

a sua correspondência com os compromissos distribuídos, mas não recebe as provas históricas removidas. Ambos os modos aplicam normalmente as regras aos blocos novos e conservam a mesma capacidade funcional corrente, mas têm dependências históricas de confiança diferentes.

### 10.8.6 Selecção de pares por faixa

Durante a apresentação P2P, cada par anuncia a sua semente de poda. O receptor aceita a semente zero, que significa base completa, ou uma semente com $\log_2 8 = 3$ e faixa entre 1 e 8; valores incompatíveis levam ao fecho da ligação. A fila de blocos calcula a faixa da próxima altura necessária.

Sem `--sync-pruned-blocks`, o nó procura um par completo ou um par cuja faixa contenha material integral para esse troço. Pode reservar desde já o troço da faixa seguinte e liberta ligações pouco úteis para abrir lugar à faixa necessária. Com a opção activa, qualquer par que conserve a base indispensável pode responder de forma podada aos troços que o nó local não pretende conservar; para a faixa local e para a ponta, continuam necessários dados completos. A semente anunciada permite seleccionar apenas pares que declararam conservar os dados exigidos pelo pedido.

Perto do topo, $\sigma = 0$ para todos. Os pares podados conservam os dados completos da janela e podem servi-los; a validação corrente não depende de hashes históricos compilados. Numa reorganização normal dentro dessa janela, o nó possui ou descarrega as transacções e provas completas da nova rama. Se uma reorganização excepcional exigir uma rama profunda fora da janela, um nó podado tem de encontrar pares que forneçam o material integral que falta; uma rama alternativa nova não pode ser justificada por um hash compilado da antiga cadeia principal. A poda não transforma 5500 blocos em finalidade e a falha de cobertura pode atrasar ou impedir aquele nó de acompanhar a reorganização.

## 10.9 Garantias e limites

Dandelion++ encaminha a transacção por um subgrafo temporário antes da difusão pública. As hastes disponíveis, no máximo duas, o mapa persistente por entrada, o papel de difusor e o embargo reduzem a fiabilidade de estimativas baseadas no primeiro nó observado. A implementação Monero acrescenta filtros de altura, equilíbrio das hastes, difusão com atrasos, detecção de ciclos e uma transição de segurança concreta. Tor/I2P, quando configurados, usam canais e ruído próprios; hastes de entrada e anúncios por inventário continuam a ser propostas.

Nada disto oculta uma carteira de um daemon remoto, derrota um observador global, garante propagação contra todos os nós maliciosos ou altera consenso. É uma defesa probabilística de metadados, com hipóteses e falhas observáveis.

A poda reduz o armazenamento local. Conserva os dados indispensáveis ao estado corrente, em média um oitavo da parte antiga podável e a ponta recente inteira. As oito faixas aumentam a probabilidade de a rede reter cópias, mas nenhuma regra de consenso garante a disponibilidade contínua dos nós que as armazenam. Sincronizar dados completos e eliminá-los depois não é equivalente a aceitar uma história já podada, porque apenas o primeiro procedimento verifica localmente todas as provas históricas.

Dandelion++ protege metadados da propagação inicial; a poda reduz o conjunto de dados históricos conservados por cada nó. As duas funções são independentes e devem ser avaliadas segundo modelos de segurança distintos.

# CAPÍTULO 11

# Provas de pertença à cadeia completa (FCMP++)

O nó não precisa de ouvir «a minha saída está algures naquela árvore». Precisa de ficar convencido de uma frase muito mais apertada:

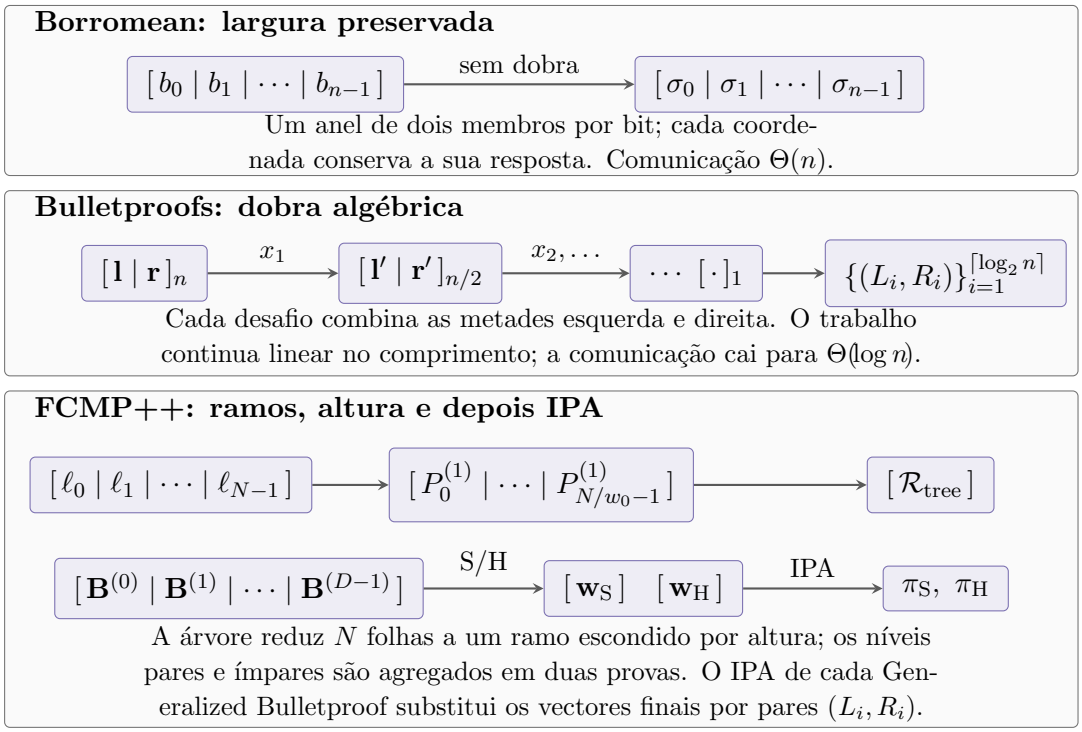
> Existe uma folha escondida sob a raiz pública, essa folha contém uma chave de saída $O$, a base $H_p(O)$ e um compromisso de montante $C$; o provador conhece a chave privada de utilização única $p_k$ que controla $O$; e a imagem de chave publicada é precisamente $I = p_k H_p(O)$.

É esta a declaração que construiremos. A árvore responderá «a folha está aqui», sem dizer onde. Uma prova de autorização responderá «conheço $p_k$», sem o publicar. Entre as duas ficará uma só entrada pública, re-randomizada, que impedirá as respostas de falarem de saídas diferentes. Começaremos pela costura inteira; as duas curvas e as optimizações terão a gentileza de esperar pela sua vez. Os certificados de divisores do capítulo 3 serão aplicados às multiplicações ocultas do caminho.

Curve Trees, de Matteo Campanelli, Mathias Hall-Andersen e Simon Holmgaard Kamp, fornece o acumulador e o caminho oculto [22]. Liam Eagen fornece a maquinaria por divisores usada nas multiplicações de pontos dentro do circuito [33]. Luke Parker adapta e compõe estas peças para as saídas do Monero e para a prova SAL [108]. FCMP++ é, portanto, uma composição, não uma alcunha para uma única equação.

## 11.1 Borromean, Bulletproofs e FCMP++

As provas de intervalo Borromean e Bulletproofs do Capítulo 6 fornecem a escala certa para ler FCMP++. Nas Borromean, as coordenadas continuam individualizadas na transcrição. Nas Bulletproofs, o argumento de produto interno (IPA) reduz os vectores a metade em cada uma das $\lceil \log_2 n \rceil$ rondas. FCMP++ acrescenta uma compressão estrutural anterior: as $N$ folhas tornam-se ramos de uma árvore, o caminho conserva um ramo por altura e Generalized Bulletproofs dobra os vectores agregados desse caminho [69, 18, 22, 29].




Se $D$ é a altura e $w_d$ a largura do ramo no nível $d$, então

$$N \le \prod_{d=0}^{D-1} w_d, \qquad m_{\text{ramos}} = 6w_0 + \sum_{d=1}^{D-1} w_d. \tag{11.1}$$

O primeiro número mede o universo coberto; o segundo conta, antes das variáveis auxiliares, os coeficientes do caminho. Restam ainda os $D-1$ pontos intermédios re-randomizados. Ramos e altura reduzem o conjunto inteiro a um caminho escondido; o IPA comprime a comunicação dos vectores que o certificam.

SAL contém outro IPA, o argumento ponderado de Bulletproofs+. Este certifica em $P$ o termo $p_{k_P} r_{I_P}$; não o acrescenta depois da extracção. A pertença e SAL usam relações diferentes, que o ponto público $R$ fará coincidir [28].

## 11.2 A declaração, antes da maquinaria

Comecemos pelo caso histórico que já conhecemos. Ao criar uma saída, a transacção publica uma chave de utilização única

$$O = p_k G. \tag{11.2}$$

O ponto $O$ não é «assinado quando nasce». É simplesmente criado como parte da saída. Mais tarde, o conhecimento do escalar $p_k$ autoriza o seu gasto. $p_k$ não é a chave de gasto permanente da carteira; é a chave privada de utilização única que controla aquela saída.

Da chave pública deriva-se uma base de imagem de chave, que escreveremos por agora como $H_p(O)$. A imagem de chave efectiva só aparece no gasto:

$$I = p_k H_p(O). \tag{11.3}$$

A mesma saída e a mesma chave $p_k$ produzem o mesmo $I$, mesmo quando a prova recebe aleatoriedade inteiramente nova. É esse ponto público que permite ao nó reconhecer um segundo gasto.

O terceiro ponto da saída é o compromisso de montante

$$C = aG + vH, \tag{11.4}$$

onde $a$ é a máscara e $v$ o montante. Os geradores $G$ e $H$ têm uma relação de logaritmo discreto que ninguém deve conhecer.

Assim, a folha conceptual do acumulador é a tupla ordenada

$$(O, H_p(O), C). \tag{11.5}$$

Os três termos são pontos, mas têm empregos diferentes:

| **ponto** | **quando se torna público** | **papel** |
|---|---|---|
| $O$ | quando a saída é criada | chave pública de utilização única; conhecer $p_k$ autoriza o gasto |
| $H_p(O)$ | pode ser calculado a partir da saída | base que torna determinística a imagem de chave |
| $C$ | quando a saída é criada | compromisso de montante $v$ com máscara $a$ |
| $I$ | quando a saída é gasta | imagem de chave efectiva $p_k H_p(O)$ |

**Tradução da notação da fonte.** Na revisão fixada de `monero-oxide`, o campo `I` de `Output` é a *base* da imagem de chave e a imagem efectiva chama-se `L`. Neste capítulo, para que a letra diga sempre a mesma coisa, escrevemos $H_p(O)$ para a base e $I$ para a imagem efectiva. O campo fonte `I_tilde` é, portanto, o nosso $\widetilde{H}_O$, não uma imagem de chave re-randomizada [109].

Vale a pena ler as três primeiras igualdades como o verificador as lerá.

**Público.** A raiz do acumulador, a imagem de chave $I$, os geradores e a transacção a autorizar. A cadeia também contém todos os pontos de saída, mas a prova não revela qual deles escolheu.

**Segredo conhecido pelo provador.** $p_k$, a folha escolhida, o seu caminho e as máscaras que introduziremos. A carteira conhece ainda o montante e a máscara de $C$, embora a relação FCMP não precise de os abrir numericamente.

**Igualdade demonstrada.** A composição convencerá o verificador de que existe uma folha (11.5) sob a raiz, que o mesmo $p_k$ abre a sua componente $O$ e que (11.3) determina a imagem de chave publicada.

**Ligação à igualdade anterior.** Não virá de uma equação «o compromisso do ramo implica conhecer $p_k$»: essa equação não existe. Virá de duas provas sobre a mesma entrada pública. Construiremos primeiro a metade da árvore, depois essa entrada, e por fim a metade da autorização.

## 11.3 Um compromisso não é uma prova de conhecimento

Suponhamos que um ramo contém escalares $b_0, \ldots, b_{w-1}$. Um compromisso vectorial da prova pode ter a forma

$$K = r_K H_K + \sum_{j=0}^{w-1} b_j G_j. \tag{11.6}$$

As bases são públicas e a máscara $r_K$ oculta o vector. Sob a hipótese de relações de logaritmo discreto desconhecidas entre as bases, a propriedade de *vinculação* diz que é computacionalmente inviável abrir o mesmo $K$ como dois vectores diferentes.

Isso ainda não diz que alguém conhece sequer uma abertura. Um ponto pode ser copiado, recebido ou calculado sem que venha acompanhado de uma testemunha. A prova precisa de acrescentar quatro ingredientes distintos:

1. um argumento de conhecimento demonstra que o provador conhece os coeficientes e a máscara que abrem $K$;
2. as restrições do circuito impõem igualdades entre esses coeficientes;
3. quando duas restrições recebem a mesma variável comprometida, não há duas testemunhas apenas baptizadas com o mesmo nome: há uma única posição do compromisso reutilizada;
4. quando duas provas recebem o mesmo ponto público, a composição refere-se àquela entrada comum, desde que cada prova seja sólida e os compromissos sejam vinculantes.

**Público.** Em (11.6), $K$ e as bases.

**Segredo conhecido pelo provador.** O vector $(b_0, \ldots, b_{w-1})$ e $r_K$.

**Igualdade demonstrada.** O argumento de conhecimento prova a existência de uma abertura que satisfaz simultaneamente (11.6) e as restrições do circuito.

**Ligação à igualdade anterior.** A vinculação fixa o vector dentro de $K$; a reutilização das variáveis fixa qual coeficiente desse vector entra na restrição seguinte. Nem a vinculação substitui o argumento de conhecimento, nem o argumento de conhecimento escolhe sozinho um caminho na árvore.

## 11.4 A metade da árvore: conhecer uma folha e um caminho

Reservaremos $x$ e $y$ exclusivamente às coordenadas de pontos. O nível da árvore fica no canto superior esquerdo e a posição local no canto inferior direito:

$$ {}^{n}P_j = ({}^{n}x_j, {}^{n}y_j). \tag{11.7}$$

Para extrair a posição $j$ do ramo usado no nível $n$, escreveremos sempre

$$ {}^{n}B[j]. $$

Isto extrai um *slot*, não promete que o slot seja sempre um único escalar. No nível das folhas, ${}^{0}B[j]$ designa a tupla conceptual naquela posição; nas camadas superiores, designa o escalar que guarda a coordenada $x$ de um ponto filho.

Para escrever uma só equação bem tipada, cada nível tem uma função pública de codificação em escalares

$$ \mathrm{enc}_n({}^{n}B[j]) = \left({}^{n}b_{j,0}, \ldots, {}^{n}b_{j,s_n-1}\right) \tag{11.8}$$

e um bloco igualmente comprido de pontos públicos ${}^{n}\mathbf{G}_j = ({}^{n}G_{j,0}, \ldots, {}^{n}G_{j,s_n-1})$. Definamos o emparelhamento do slot com esse bloco por

$$ \langle {}^{n}B[j], {}^{n}\mathbf{G}_j \rangle_n := \sum_{k=0}^{s_n-1} {}^{n}b_{j,k}\, {}^{n}G_{j,k}. \tag{11.9}$$

Nas camadas superiores, $s_n = 1$, e esta expressão reduz-se à multiplicação escalar habitual. Na camada de folhas, a codificação transforma os três pontos do slot em escalares; só depois de fecharmos a ligação à chave veremos quais e quantos.

Um ramo determinístico transforma os seus slots num ponto:

$$ {}^{n+1}P_{j_{n+1}} = D_n + \sum_{j=0}^{w_n-1} \langle {}^{n}B[j], {}^{n}\mathbf{G}_j \rangle_n \,. \tag{11.10}$$

$D_n$ é um ponto inicializador fixo e os blocos ${}^{n}\mathbf{G}_j$ contêm as bases públicas. O índice inferior $j_{n+1}$ diz em que posição do ramo seguinte este ponto aparecerá; não é uma coordenada. O emparelhamento (11.9) impede precisamente que uma tupla de pontos seja tratada como se fosse um escalar.

O ramo seguinte guarda apenas a coordenada $x$ do ponto completo:

$$ {}^{n+1}B[j_{n+1}] = {}^{n+1}x_{j_{n+1}}, \qquad {}^{n+1}P_{j_{n+1}} = ({}^{n+1}x_{j_{n+1}}, {}^{n+1}y_{j_{n+1}}). \tag{11.11} $$

A coordenada $y$ não desaparece da testemunha: o circuito usa ambas para verificar que o ponto está na curva. Só o rótulo transportado ao pai é mais curto.

Durante a prova, o ramo não aparece em claro. Os seus coeficientes entram num compromisso vectorial

$$ K_n = r_n H_n + \sum_{j=0}^{w_n-1} \langle {}^{n}B[j], {}^{n}\mathbf{G}_j \rangle_n \,. \tag{11.12} $$

O argumento aritmético prova conhecimento da abertura de $K_n$. Um dispositivo de logaritmo discreto prova ainda que, ao retirar $r_n H_n$ e somar o inicializador $D_n$, se obtém o ponto completo da Equação (11.10).

**Público.** $K_n$, os geradores, o inicializador e os compromissos que formam a prova.

**Segredo conhecido pelo provador.** O ramo ${}^{n}B[0], \ldots, {}^{n}B[w_n - 1]$, a máscara $r_n$ e qual posição continua o caminho.

**Igualdade demonstrada.** A prova demonstra que o provador conhece uma abertura de $K_n$ para o ramo; pela vinculação, $K_n$ fixa esses coeficientes; o ramo produz o ponto (11.10); e a coordenada ${}^{n+1}x_{j_{n+1}}$ desse mesmo ponto ocupa uma posição no ramo pai.

**Ligação à igualdade anterior.** As variáveis que abrem $K_n$ são as mesmas que entram na soma do ramo. As variáveis do ponto resultante são as mesmas que entram na equação de curva e no teste de pertença do nível seguinte. Esta continuidade de variáveis, e não a mera presença de $K_n$, constrói o caminho.

### 11.4.1 Uma folha dentro de um ramo

Tomemos um único ramo pequeno com quatro folhas, apenas para ver a engrenagem. Escrevamos

$$ {}^{0}B[0], \quad {}^{0}B[1], \quad {}^{0}B[2], \quad {}^{0}B[3]. $$

Se a folha procurada ocupa a terceira posição, uma escrita explícita é

$$ {}^{0}B[2] = (O, H_p(O), C). \tag{11.13} $$

O índice 2 ajuda-nos a acompanhar o raciocínio, mas não entra na declaração pública. Para uma posição secreta $j_0$, o argumento impõe

$$ {}^{0}B[j_0] = (O, H_p(O), C), \qquad j_0 \in \{0, 1, 2, 3\}. \tag{11.14} $$

Só depois de a ligação à chave estar completa veremos como a implementação substitui este extractor explícito por uma igualdade comprimida, sem publicar $j_0$.

**Público.** O compromisso vectorial do ramo e a descrição da relação de pertença. A árvore inteira pode ser reconstruída publicamente; o que a prova oculta é qual ramo e qual folha estão a ser abertos.

**Segredo conhecido pelo provador.** A abertura do compromisso do ramo, a posição conceptual $j_0$ e a tupla da folha.

**Igualdade demonstrada.** A tupla $(O, H_p(O), C)$ coincide com uma posição do ramo comprometido, como em (11.14), sem que essa posição seja revelada.

**Ligação à igualdade anterior.** A posição ${}^{0}B[j_0]$ pertence à mesma abertura que entra na soma do ramo (11.10). A tupla não é uma segunda folha livre: será ligada, componente a componente, à entrada pública que definiremos adiante.

### 11.4.2 Do ramo ao pai, e do pai à raiz

Nas camadas superiores, o membro procurado é a coordenada $x$ do ponto produzido pelo ramo anterior. Na escrita explícita, a restrição é

$$ {}^{n}B[j_n] = {}^{n}x_{j_n}, \qquad 0 \le j_n < w_n. \tag{11.15}$$

O circuito verifica ao mesmo tempo que ${}^{n}P_{j_n} = ({}^{n}x_{j_n}, {}^{n}y_{j_n})$ satisfaz a equação da curva e que, ao retirar a parcela de máscara do compromisso do ramo anterior e somar o inicializador, se obtém esse mesmo ponto. Portanto, a saída de um nível é a entrada do seguinte:

$$\begin{aligned} {}^{n-1}B[0], \ldots, {}^{n-1}B[w_{n-1}-1] &\longrightarrow {}^{n}P_{j_n} = ({}^{n}x_{j_n}, {}^{n}y_{j_n}) \\ &\longrightarrow {}^{n}B[j_n] = {}^{n}x_{j_n}. \end{aligned} \tag{11.16}$$

No topo, a prova contém um compromisso do ramo da raiz, ofuscado por uma máscara. Chamemos $\widehat{\mathcal{R}}$ ao ponto que esse compromisso produz depois de somar o inicializador, e $\mathcal{R}_{\text{tree}}$ à raiz pública canónica. Uma pequena prova de Schnorr demonstra conhecimento da diferença de máscara:

$$N_{\text{raiz}} + c_{\text{raiz}}(\widehat{\mathcal{R}} - \mathcal{R}_{\text{tree}}) = s_{\text{raiz}} H_{\text{raiz}}. \tag{11.17}$$

O desafio $c_{\text{raiz}}$ é derivado depois de a raiz, a entrada, $N_{\text{raiz}}$ e os dois conjuntos de compromissos vectoriais da prova entrarem na transcrição.

**Público.** A raiz $\mathcal{R}_{\text{tree}}$, o compromisso que determina $\widehat{\mathcal{R}}$, $N_{\text{raiz}}$, o desafio e a resposta.

**Segredo conhecido pelo provador.** Todos os ramos e aberturas do caminho, incluindo a máscara que separa $\widehat{\mathcal{R}}$ da raiz canónica.

**Igualdade demonstrada.** As igualdades de pertença coincidem; cada ponto intermédio é válido e é produzido pelos coeficientes de uma abertura conhecida do compromisso anterior; e (11.17) demonstra que o último ramo termina na raiz pública, salvo a máscara conhecida.

**Ligação à igualdade anterior.** A coordenada que entra em (11.15) pertence ao mesmo ponto que sai de (11.16). No topo, a transcrição inclui a raiz pública e o mesmo compromisso de ramo que o circuito utilizou. É esta cadeia de igualdades que faz um caminho; os compromissos isolados apenas fixariam vectores escondidos.

Chegámos à garantia da primeira metade: o argumento FCMP prova conhecimento de uma folha, dos ramos, das máscaras e de aberturas que satisfazem um caminho até $\mathcal{R}_{\text{tree}}$. Ainda não apareceu $p_k$. Isso é deliberado.

## 11.5 A ponte pública $Q$

Revelar a folha permitiria procurá-la na árvore. A carteira escolhe quatro escalares novos

$$r_O, \quad r_I, \quad r_R, \quad r_C$$

e publica exactamente uma tupla de quatro pontos:

$$Q = (\widetilde{O}, \widetilde{H}_O, R, \widetilde{C}), \tag{11.18}$$

com

$$\widetilde{O} = O + r_O T, \tag{11.19}$$

$$\widetilde{H}_O = H_p(O) + r_I U, \tag{11.20}$$

$$R = r_I V + r_R T, \tag{11.21}$$

$$\widetilde{C} = C + r_C G. \tag{11.22}$$

$T, U, V$ são pontos fixos com relações de logaritmo discreto desconhecidas. O ponto $R$ não é a raiz e não é um nonce de Schnorr: é o compromisso que transportará $r_I$ entre as duas metades da prova.

O argumento FCMP recebe $Q$ e prova que o provador conhece valores que recuperam a folha:

$$O = \widetilde{O} - r_O T, \qquad H_p(O) = \widetilde{H}_O - r_I U,$$

$$C = \widetilde{C} - r_C G, \qquad R = r_I V + r_R T. \tag{11.23}$$

Na segunda e na quarta igualdade, a relação reutiliza a mesma testemunha comprometida $r_I$. Não se trata de dois escalares independentes acompanhados da mesma legenda.

**Público.** $Q$, $\mathcal{R}_{\text{tree}}$, os geradores e a prova.

**Segredo conhecido pelo provador.** A folha $(O, H_p(O), C)$, $r_O, r_I, r_R, r_C$, os compromissos de ramo, as respectivas aberturas e o caminho.

**Igualdade demonstrada.** FCMP demonstra que as quatro relações (11.19)–(11.22) se verificam e que a tupla recuperada ocupa uma folha sob a raiz pública.

**Ligação à igualdade anterior.** Os pontos recuperados em (11.23) são os mesmos pontos que satisfazem as restrições de pertença da folha. O mesmo $r_I$ abre $\widetilde{H}_O - H_p(O)$ na base $U$ e o coeficiente em $V$ do ponto $R$. Assim FCMP liga a folha escondida a uma única apresentação pública $Q$.

Agora podemos dizer exactamente o que FCMP ainda *não* provou. Não provou que alguém conhece uma abertura de $O$ nas bases $G, T$, não provou a Equação (11.3) e não provou o valor numérico de $v$. Provou pertença, conhecimento do caminho e consistência entre a folha e $Q$.

## 11.6 A metade SAL: a chave que autoriza o gasto

No caso histórico $O = p_k G$, a primeira componente de $Q$ torna-se

$$\widetilde{O} = p_k G + r_O T. \tag{11.24}$$

Na geração honesta, SAL usa $p_k$ para abrir directamente o ponto público $\widetilde{O}$ nas bases $G, T$, sem revelar $O$.

Para abranger a abertura geral suportada pela composição, introduzimos agora o segundo coeficiente:

$$O = p_k G + p_a T, \qquad \widetilde{O} = p_k G + (p_a + r_O)T. \tag{11.25}$$

Nas saídas históricas, $p_a = 0$. $p_a$ é um escalar de abertura; não é uma coordenada de $O$. Chamemos

$$\widetilde{p}_a = p_a + r_O.$$

O provador honesto usa $p_k$ e $\widetilde{p}_a$ nesta abertura. A análise de conhecimento exigirá um pouco mais de cuidado: a relação SAL isolada extrai um candidato que abre $\widetilde{O}$, mas só a composição provará que ele coincide com o coeficiente de $G$ usado noutro ponto da SAL.

Para retirar da base de imagem o termo que a esconde, o provador calcula o produto secreto

$$z = p_k r_I. \tag{11.26}$$

Então a imagem de chave pública pode ser calculada a partir da componente re-randomizada:

$$\begin{aligned} I &= p_k \widetilde{H}_O - zU && (11.27)\\ &= p_k\big(H_p(O) + r_I U\big) - (p_k r_I)U \\ &= p_k H_p(O). && (11.28) \end{aligned}$$

Esta é a conta do provador honesto. A segunda linha substitui $\widetilde{H}_O = H_p(O) + r_I U$ e $z = p_k r_I$; na terceira, os termos $p_k r_I U$ cancelam. Ainda precisamos de demonstrar que a relação SAL não aceita um candidato $z$ diferente do produto escondido noutro ponto. É esse o emprego exacto de $R$.

### 11.6.1 Como $R$ identifica os candidatos extraídos

A carteira constrói

$$R = r_I V + r_R T \in \mathbb{G}_{\mathrm{Ed}}. \tag{11.29}$$

Este compromisso de duas bases transporta, na sua componente em $V$, a mesma máscara que abre $\widetilde{H}_O = H_p(O) + r_I U$ [108, 109]. O termo $r_R T$ oculta essa componente e $R$ integra a entrada pública $Q = (\widetilde{O}, \widetilde{H}_O, R, \widetilde{C})$.

Esta reutilização tem de ser lida com cuidado. Dentro da relação FCMP, a mesma variável de testemunha $r_I$ ocorre nas bases $U$ e $V$; o circuito impõe essa identidade directamente. Entre o extractor de FCMP e o extractor de SAL não existe memória privada partilhada. Aí, é o ponto público $R$, juntamente com $P$ e $\widetilde{O}$, que permite comparar coeficientes.

#### Convenção de proveniência e mapa de nomes

Na notação abaixo, $H_p(O)$ designa a base da imagem e $I$, a imagem efectiva; na fonte da prova, os mesmos objectos aparecem como $I$ e $L$, respectivamente:

| **fonte da prova** | **notação aqui** | **tipo e papel** |
|---|---|---|
| $I$, $L$ | $H_p(O)$, $I$ | pontos em $\mathbb{G}_{\mathrm{Ed}}$: base derivada da saída e imagem de chave efectiva |
| $r_o, r_i, r_j, r_c$ | $r_O^{(\mathrm{FCMP})}, r_I^{(\mathrm{FCMP})}, r_R^{(\mathrm{FCMP})}, r_C^{(\mathrm{FCMP})}$ | escalares extraídos da relação de pertença; $r_j$ é a máscara de $R$ na base $T$ |
| $x', r_i', r_p'$ | $p_{k_P}, r_{I_P}, r_P$ | escalares extraídos da abertura de produto de $P$ |
| $x'', y'', z'', r_p''$ | $p_{k_O}, \widetilde{p}_a, z_Q, r_{\mathrm{dif}}$ | escalares extraídos das três equações de Schnorr; $Q$ em $z_Q$ recorda a costura à entrada pública |

Nos candidatos extraídos por SAL, a origem fica num subíndice de segundo nível: $p_{k_P}$ é o candidato para $p_k$ obtido da abertura de $P$, enquanto $p_{k_O}$ vem da abertura de $\widetilde{O}$. Do mesmo modo, $r_{I_P}$ é o candidato para $r_I$ obtido de $P$. A letra de proveniência aparece deliberadamente menor que o subíndice semântico.

### As quatro aberturas que se encontram

Na geração honesta, SAL introduz o ponto auxiliar

$$P = p_k G + r_I V + zU + r_P T, \qquad z = p_k r_I, \tag{11.30}$$

onde $r_P$ é uma máscara nova. Na construção honesta, estes valores satisfazem

$$\widetilde{O} = p_k G + \widetilde{p}_a T, \tag{11.31}$$

$$P - \widetilde{O} - R = zU + (r_P - \widetilde{p}_a - r_R)T. \tag{11.32}$$

Para testemunhas honestas, a última linha expande-se termo a termo:

$$\begin{aligned} P - \widetilde{O} - R &= (p_k G + r_I V + zU + r_P T) \\ &\quad - (p_k G + \widetilde{p}_a T) - (r_I V + r_R T) \\ &= zU + (r_P - \widetilde{p}_a - r_R)T. \end{aligned}$$

O cancelamento de $p_k G$ e $r_I V$ explica a forma da diferença, mas é apenas uma verificação de correcção para uma testemunha já coerente. Na prova de conhecimento temos de começar com candidatos possivelmente distintos.

O extractor do argumento de produto interno ponderado de Bulletproofs+ usado por SAL devolve uma abertura multiplicativa de $P$; o extractor de Schnorr devolve aberturas de $\widetilde{O}$, da diferença e da imagem. Obtém-se

$$P = p_{k_P} G + r_{I_P} V + (p_{k_P} r_{I_P})U + r_P T, \tag{11.33}$$

$$\widetilde{O} = p_{k_O} G + \widetilde{p}_a T, \tag{11.34}$$

$$P - \widetilde{O} - R = z_Q U + r_{\mathrm{dif}} T, \tag{11.35}$$

$$I = p_{k_O} \widetilde{H}_O - z_Q U. \tag{11.36}$$

O produto $p_{k_P} r_{I_P}$ já pertence à relação certificada pelo IPA ponderado: é o coeficiente de $U$ que o extractor devolve juntamente com os coeficientes de $G$ e $V$. Não é formado a posteriori a partir de dois escalares extraídos independentemente [28]. Nas duas linhas seguintes, $z_Q$ é por enquanto um escalar independente: o extractor de Schnorr apenas sabe que é o coeficiente de $U$ reutilizado na diferença e na imagem. SAL isolada ainda não identificou $p_{k_P}$ com $p_{k_O}$, nem $r_{I_P}$ com a máscara da folha, nem $z_Q$ com o produto. Também não extrai uma abertura autónoma de $R$. O valor $r_{\text{dif}}$ é só o coeficiente combinado de $T$.

FCMP fornece a quarta abertura. A extracção da prova de pertença produz um caminho válido, a folha escondida e os escalares de re-randomização. Para essa folha, as relações extraídas são

$$\widetilde{O} = O + r_O^{(\text{FCMP})} T, \tag{11.37}$$

$$\widetilde{H}_O = H_p(O) + r_I^{(\text{FCMP})} U,$$

$$R = r_I^{(\text{FCMP})} V + r_R^{(\text{FCMP})} T. \tag{11.38}$$

Esta é a definição operacional de $r_I^{(\text{FCMP})}$: é o escalar que o extractor de pertença devolve na abertura simultânea de $\widetilde{H}_O$ e $R$. Não se pressupõe que seja igual a $r_{I_P}$; é isso que falta provar.

Façamos agora a substituição sem omitir passos. Pela abertura extraída de $P$, de $\widetilde{O}$ e de $R$, o lado esquerdo da terceira equação SAL é

$$\begin{aligned} P - \widetilde{O} - R = {} & \left[p_{k_P} G + r_{I_P} V + (p_{k_P} r_{I_P}) U + r_P T\right] \\ & - \left[p_{k_O} G + \widetilde{p}_a T\right] - \left[r_I^{(\text{FCMP})} V + r_R^{(\text{FCMP})} T\right]. \end{aligned}$$

Mas o extractor de Schnorr abre esse mesmo ponto público como $z_Q U + r_{\text{dif}} T$. Passando essa abertura para a esquerda e reunindo as quatro bases, fica

$$\begin{aligned} \mathbf{0} = {} & (p_{k_P} - p_{k_O}) G + (r_{I_P} - r_I^{(\text{FCMP})}) V \\ & + (p_{k_P} r_{I_P} - z_Q) U + (r_P - \widetilde{p}_a - r_R^{(\text{FCMP})} - r_{\text{dif}}) T. \end{aligned} \tag{11.39}$$

Se os quatro coeficientes não fossem todos nulos, o extractor teria produzido uma relação não trivial entre $G, V, U, T$. Sob a hipótese de logaritmos discretos multibase desconhecidos, a vinculação computacional força-os a zero. Os coeficientes de $G, V, U$ impõem, por esta ordem,

$$p_{k_P} = p_{k_O} =: p_k, \qquad r_{I_P} = r_I^{(\text{FCMP})} =: r_I, \qquad z_Q = p_{k_P} r_{I_P} = p_k r_I. \tag{11.40}$$

O coeficiente de $T$ fixa apenas a combinação $r_P - \widetilde{p}_a - r_R^{(\text{FCMP})} - r_{\text{dif}} = 0$; SAL não extrai $r_R^{(\text{FCMP})}$ de forma independente.

No relatório de composição, os mesmos valores chamam-se $z''$, $x'$ e $r_i'$. A extracção do IPA ponderado fornece o termo $x' r_i'$ na abertura de $P$; a comparação multibase impõe depois $z'' = x' r_i'$ [28].

As duas aberturas extraídas de $\widetilde{O}$, juntamente com $p_{k_O} = p_k$, fecham também a propriedade de controlo da própria saída:

$$\begin{aligned} O &= \widetilde{O} - r_O^{(\text{FCMP})} T \\ &= p_{k_O} G + (\widetilde{p}_a - r_O^{(\text{FCMP})}) T \\ &= p_k G + p_a T, \qquad p_a := \widetilde{p}_a - r_O^{(\text{FCMP})}. \end{aligned} \tag{11.41}$$

Agora, e só agora, podemos substituir (11.40) e (11.38) em (11.36):

$$\begin{aligned} I &= p_k \widetilde{H}_O - (p_k r_I) U \\ &= p_k \big( H_p(O) + r_I U \big) - (p_k r_I) U \\ &= p_k H_p(O). \end{aligned}$$

Esta derivação transforma a conta honesta (11.27)–(11.28) numa conclusão de conhecimento da composição.

**O que cada interveniente conhece e verifica**

| **classe** | **objectos** | **emprego na composição** |
|---|---|---|
| públicos | $Q, I, P$, raiz, mensagem, $G, T, U, V$ | são transcritos por ambos os verificadores; a identidade byte a byte de $Q$ impede trocar a entrada entre provas |
| privados | folha, caminho, $p_k, p_a$, máscaras e nonces | constituem a testemunha do provador; não são enviados ao verificador |
| dados da árvore | folhas ordenadas, ramos, raiz e fotografia de referência | os dados são públicos e reconstruíveis; só a escolha do caminho entra como testemunha privada |
| extraídos | candidatos com rótulos $P, O, Q$, (FCMP) | existem na demonstração de conhecimento; os rótulos mantêm-nos distintos até a vinculação justificar as igualdades |
| compromissos | $\widetilde{O}, \widetilde{H}_O, R, \widetilde{C}, P$ e os compromissos dos ramos | fixam aberturas; por si sós não demonstram que uma abertura é conhecida |
| verificações | caminho FCMP, produto ponderado, três equações de Schnorr e equação multibase | recuperam pertença, controlo da saída, produto $z = p_k r_I$ e imagem $I = p_k H_p(O)$ para a mesma folha |

**Público.** $Q$, $I$, $P$, as duas provas, a raiz, a mensagem da transacção e os geradores.

**Segredo conhecido pelo provador honesto.** $p_k, p_a, \widetilde{p}_a, r_O, r_I, r_C, r_R, z, r_P$, as restantes aberturas FCMP e os nonces.

**Igualdade demonstrada.** SAL fornece as relações extraíveis (11.33)–(11.36); FCMP fornece (11.37) e (11.38); a vinculação aplicada a (11.39) identifica os candidatos e demonstra $O = p_k G + p_a T$, $z = p_k r_I$ e $I = p_k H_p(O)$.

**Ligação à igualdade anterior.** FCMP reutiliza literalmente uma variável para $r_I^{(\mathrm{FCMP})}$ nas bases $U, V$. Entre protocolos não há uma variável privada partilhada: o mesmo $R$, dentro do mesmo $Q$, produz a equação multibase que obriga as extracções a concordar.

Esta é também a resposta à pergunta «porque é que não se pode provar a pertença de uma saída e a autorização de outra?». FCMP e SAL recebem, byte por byte, o mesmo $Q = (\widetilde{O}, \widetilde{H}_O, R, \widetilde{C})$. FCMP vincula esse $Q$ a uma folha sob a raiz. A relação SAL, composta com a abertura FCMP de $R$, vincula $p_k$, $r_I$ e $I$ a esse $Q$. Sob a solidez das provas e a vinculação dos compromissos, não sobra uma segunda entrada pública onde trocar a identidade da saída.

## 11.7 O compromisso de montante atravessa a ponte

Retomemos

$$C = aG + vH.$$

A quarta componente de $Q$ é

$$\widetilde{C} = C + r_C G = (a + r_C)G + vH. \tag{11.42}$$

Somar $r_C G$ muda a máscara e conserva o coeficiente $v$ de $H$. No formato da transacção, $\widetilde{C}$ desempenha o papel de pseudo-saída da entrada.

FCMP prova uma igualdade precisa: o provador conhece $r_C$ e um ponto $C$ da folha escondida tais que

$$C = \widetilde{C} - r_C G, \tag{11.43}$$

e esse mesmo $C$ entra na tupla seleccionada pelo produto de folhas. A prova não abre $C$ como $aG + vH$, não demonstra conhecimento do número $v$ e não transforma o compromisso do ramo numa prova do montante.

As obrigações monetárias continuam em RingCT. Com a convenção deste livro, o verificador impõe para as entradas $i$, as novas saídas $o$ e a taxa $v_{\mathrm{taxa}}$ uma igualdade da forma

$$\sum_i \widetilde{C}_i = \sum_o C_o^{\mathrm{novo}} + v_{\mathrm{taxa}} H, \tag{11.44}$$

e verifica as provas de intervalo dos montantes das novas saídas. O compromisso histórico $C$ foi sujeito às regras vigentes quando a saída foi criada. Dada essa abertura válida, a igualdade FCMP $\widetilde{C} = C + r_C G$ preserva o coeficiente em $H$; FCMP não o extrai numericamente. SAL não usa $r_C$ na sua álgebra, embora a sua transcrição inclua todo o $Q$, também $\widetilde{C}$.

**Público.** $\widetilde{C}$, os compromissos das novas saídas, $v_{\mathrm{taxa}}$ e as provas RingCT.

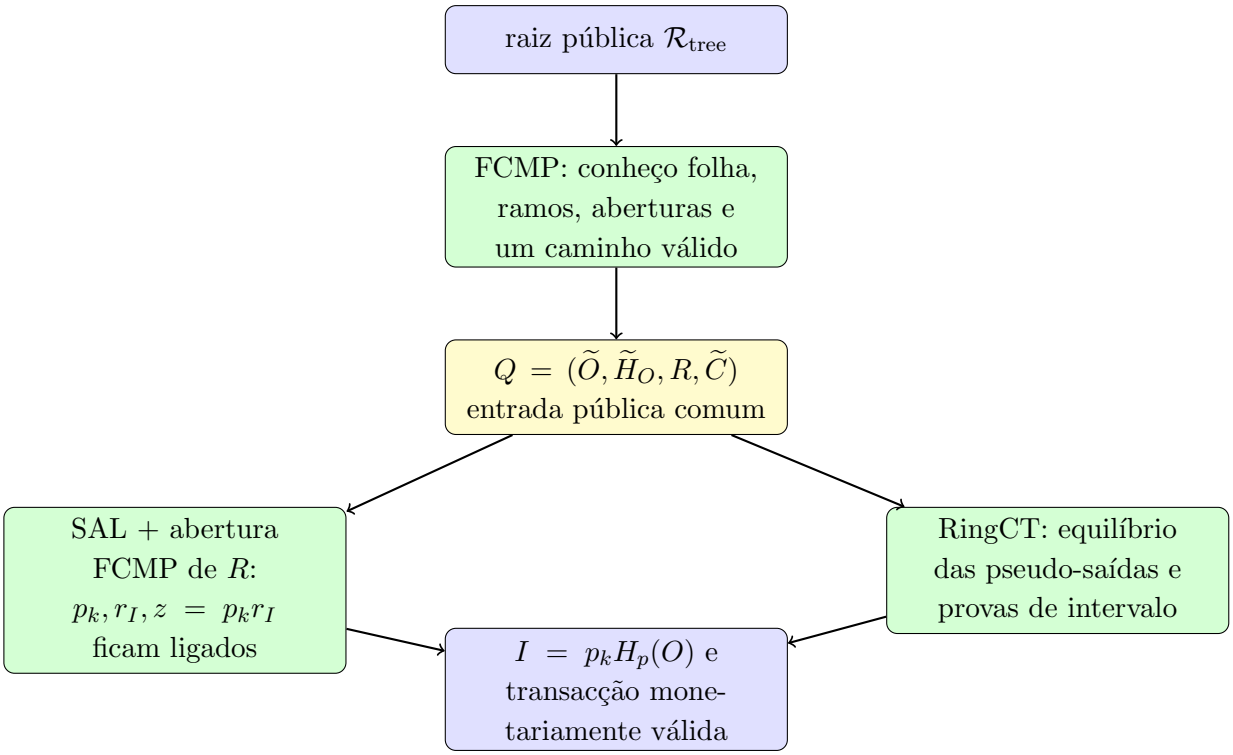
**Segredo conhecido pelo provador.** Para a relação FCMP, $C$ e $r_C$. A carteira conhece normalmente $a$ e $v$, mas esses valores não são testemunhas exigidas pela igualdade (11.43).

**Igualdade demonstrada.** FCMP demonstra (11.43) e a pertença de $C$; RingCT demonstra (11.44) e as obrigações de intervalo aplicáveis.

**Ligação à igualdade anterior.** A mesma componente pública $\widetilde{C}$ pertence a $Q$. Por isso a entrada cuja autorização SAL transcreve é também a entrada cujo montante FCMP liga ao acumulador e cuja pseudo-saída RingCT usa no equilíbrio.

## 11.8 A composição completa

Podemos agora seguir uma única linha desde a raiz até à imagem de chave:




A conclusão conceptual tem cinco passos, todos agora com uma igualdade atrás:

1. os compromissos de ramo vinculam os coeficientes escondidos;
2. o argumento FCMP prova conhecimento de aberturas que satisfazem as restrições de pertença e formam um caminho até à raiz pública;
3. FCMP liga a folha escondida à apresentação pública $Q$;
4. a relação SAL, comparada com a abertura FCMP de $R$, identifica $p_k, r_I, z$ e liga a imagem de chave $I$ a esse exacto $Q$;
5. a composição estabelece conhecimento da chave de gasto de uma saída escondida sob a raiz, enquanto RingCT conserva as obrigações monetárias.

Não há, entre o primeiro e o quarto passo, uma seta secreta chamada «o ramo conhece $p_k$». Há compromissos vinculantes, um argumento de conhecimento do caminho, restrições que reutilizam testemunhos, uma entrada pública comum e um segundo argumento de conhecimento da chave. A costura inteira é precisamente essa sequência.

**Público.** $\mathcal{R}_{\text{tree}}$, $Q$, $I$, a transacção e as duas provas.

**Segredo conhecido pelo provador.** Uma só folha, o seu caminho, as aberturas de todos os compromissos, $p_k, p_a$ e as máscaras de re-randomização.

**Igualdade demonstrada.** A extracção conjunta produz uma testemunha FCMP que liga $Q$ a uma folha sob a raiz e valores principais $p_k, p_a, r_I, z$ que, por (11.40), satisfazem (11.25), (11.26) e (11.27) para o mesmo $Q$.

**Ligação à igualdade anterior.** O verificador da implementação entrega a SAL uma referência à estrutura pública `Input`; depois constrói a entrada FCMP directamente dos mesmos quatro campos. A composição acontece por identidade dos pontos públicos, reforçada pela equação multibase de $R$, e não por uma semelhança informal entre duas transcrições.

## 11.9 Construção da árvore pública

Até aqui recebemos $\mathcal{R}_{\text{tree}}$ como dado. Falta dizer de onde vem essa raiz, o que um nó guarda e o que a carteira precisa de obter. A árvore pública é um acumulador determinístico das saídas elegíveis: transforma uma sequência acordada de tuplas $(O, H_p(O), C)$ num ponto-raiz compacto. A prova FCMP esconde a posição e o caminho; não esconde a regra de construção nem impede qualquer nó de recalcular a árvore [22, 108].

### 11.9.1 Da saída aos escalares da primeira camada

Cada folha lógica é a tupla ordenada

$$\ell_i = (O_i, H_p(O_i), C_i) \in \mathbb{G}^3_{\text{Ed}}. \tag{11.45}$$

Antes de a inserir, a implementação valida e converte os três pontos para a representação Wei25519 usada pelo circuito. A folha passa então a seis coordenadas no campo escalar da primeira curva:

$$\text{flat}(\ell_i) = \big(x(O_i), y(O_i), x(H_p(O_i)), y(H_p(O_i)), x(C_i), y(C_i)\big) \in \mathbb{F}^6_{\text{S}}. \tag{11.46}$$

A ordem é parte da codificação. Trocar $O_i$ com $C_i$, ou trocar $x$ com $y$, muda os coeficientes associados aos geradores e, portanto, muda a raiz. Na fotografia Rust fixada, 38 folhas formam o primeiro ramo: são $38 \times 6 = 228$ escalares. Ramos acima dele têm 18 ou 38 escalares, alternadamente [109].

Para uma curva $\mathbb{E}_c$, escrevamos uma compressão de ramo como

$$\mathcal{H}_c(b_0, \ldots, b_{w_c-1}) = D_c + \sum_{j=0}^{w_c-1} b_j G_{c,j} \in \mathbb{E}_c, \tag{11.47}$$

onde $D_c$ é um inicializador público e $(G_{c,0}, \ldots, G_{c,w_c-1})$ é uma família pública de geradores. Estes parâmetros são derivados deterministicamente com domínios distintos para as duas curvas e para os empregos previstos. A palavra «hash» designa aqui esta compressão vectorial vinculante e aditivamente homomórfica, não uma chamada a uma função de dispersão convencional sobre bytes.

O ramo incompleto é completado com o escalar reservado $Z = 0$. A construção escolhe as curvas de modo que $x = 0$ não represente um ponto admissível no emprego correspondente; por isso um slot vazio não pode ser confundido com a coordenada $x$ de um nó válido. Na camada de folhas, o preenchimento acontece depois do achatamento até perfazer 228 escalares. Nas camadas seguintes, completa-se cada ramo até à largura 18 ou 38. Isto é preenchimento de slots, não a criação de saídas fictícias.

### 11.9.2 Subir, ordenar e escolher uma raiz

Uma aplicação abstracta da construção procede assim:

1. fixa a sequência de folhas elegíveis numa ordem determinística;
2. divide-a em grupos de 38, completa o último grupo com $Z$, achata cada grupo e aplica $\mathcal{H}_{\mathrm{S}}$;
3. conserva apenas a coordenada $x$ de cada ponto resultante como membro da camada seguinte;
4. agrupa essas coordenadas segundo a largura da curva seguinte, completa o último ramo com $Z$, aplica a compressão dessa curva e repete;
5. termina quando resta uma só compressão: o ponto completo resultante é $\mathcal{R}_{\mathrm{tree}}$, do tipo Selene ou Helios conforme a paridade da altura.

A sequência é necessária porque $\mathcal{H}_c$ associa cada posição a um gerador diferente. A especificação de integração descreve as saídas da cadeia elegíveis e operações Grow/Trim; a revisão mais recente do código C++ consultada introduz ainda um identificador unificado e ordena cada lote por esse identificador. O próprio comentário do código avisa que lotes tardios, por exemplo saídas cujo bloqueio termina depois, podem não ficar globalmente ordenados pelo instante de entrada na cadeia. Logo, «ordem de bloco/transacção/saída» não deve ser promovida aqui a regra final sem a camada de integração que define elegibilidade e lotes [85].

Há igualmente duas noções que não devemos misturar. A árvore conceptual pode ser vista como infinita e preenchida por zeros acima da parte ocupada; a raiz usada por uma transacção pertence a uma fotografia finita e identificada. A especificação propõe que uma transacção indique um bloco de referência já elegível e que o nó procure a raiz depois desse bloco. Isso é uma regra de integração proposta, não uma consequência matemática de Curve Trees nem uma afirmação de activação na rede.

Um operador pode chamar «época» ao intervalo entre fotografias guardadas ou entre pontos de reconstrução, e pode manter uma cache de raízes e ramos recentes. Curve Trees não fixa, por si, uma duração de época. A identidade consensual relevante é a sequência de folhas e a fotografia de raiz escolhida; uma cache pode ser descartada e reconstruída sem mudar esses valores.

### 11.9.3 Guardar, reconstruir e actualizar

Um nó não é obrigado pela matemática a conservar toda a árvore em memória. Há três opções de engenharia compatíveis com a mesma raiz: recalcular tudo a partir das saídas da cadeia; guardar cada ramo e nó para responder depressa a pedidos de caminhos; ou guardar uma fronteira parcial e materializar ramos antigos numa base de dados. A escolha altera tempo, espaço e serviço RPC, não a declaração provada.

A homomorfia de (11.47) torna incrementais as actualizações. Se o slot $k$, que continha $b_k$, passa a conter $b'_k$, então

$$\mathcal{H}_c(\mathbf{b}') = \mathcal{H}_c(\mathbf{b}) + (b'_k - b_k) G_{c,k}. \tag{11.48}$$

A rotina `hash_grow` da revisão Rust aplica precisamente este delta ao primeiro slot alterado e soma os novos filhos seguintes. A rotina `hash_trim` subtrai as contribuições removidas e volta a colocar o filho que deve permanecer no primeiro slot afectado. Uma reorganização pode, portanto, desfazer as folhas mais recentes e propagar apenas os pontos alterados até à raiz; uma reconstrução completa continua a ser a referência simples para conferir o resultado.

A separação entre protocolo e implementação fica assim:

| **aspecto** | **regra da relação/protocolo** | **fotografia de implementação** |
|---|---|---|
| folha | três pontos ordenados e uma codificação canónica | seis coordenadas Wei25519; conversão e validação em lote |
| forma | ramos alternados, preenchimento reservado e parâmetros separados | 38 folhas, depois larguras 18/38; tipos de raiz `C1`/`C2` |
| estado | todos os participantes têm de obter a mesma sequência elegível e a mesma raiz de referência | o código Rust oferece prova, `hash_grow` e `hash_trim`; o C++ actual prepara folhas e endurece tipos/larguras |
| armazenamento | não faz parte da afirmação de conhecimento | árvore inteira, fronteira, base de dados ou reconstrução são escolhas do nó |
| caminho | ramo completo de cada camada e posição oculta | a estrutura `Path` exige que o chamador complete os ramos não-folha |

A revisão Monero fixada para este livro, `3646f648`, continha a formação da tupla de saída, mas não a classe C++ de conversão em lote. Na revisão corrente consultada, `3d3920d7`, existe `CurveTrees::outputs_to_leaves` e o tipo `UnifiedOutput`; a classe ainda não constitui, só por si, o gestor completo de crescimento, persistência, RPC e selecção de raízes descrito na especificação. Esta diferença temporal impede que uma intenção de protocolo seja apresentada como comportamento já fixado do nó.

### 11.9.4 O caminho que a carteira entrega à prova

Para provar uma folha, a carteira precisa da própria tupla e do ramo completo que a contém; para cada camada superior, precisa do vector de filhos do ramo e da posição conceptual por onde o caminho continua. Não é apenas a lista de um «irmão» por nível de uma árvore binária. Na API fixada, `Path` separa `leaves`, `curve_1_layers` e `curve_2_layers`; o chamador tem de completar os ramos não-folha às larguras esperadas. Estes ramos e posições são testemunhas privadas da prova, embora provenham de dados públicos. A raiz é pública.

Considere, só para visualizar a mecânica, larguras fictícias 2/2 e três folhas $\ell_0, \ell_1, \ell_2$:

$$N_0 = \mathcal{H}_{\mathrm{S}}\big(\mathrm{flat}(\ell_0) \parallel \mathrm{flat}(\ell_1)\big),$$
$$N_1 = \mathcal{H}_{\mathrm{S}}\big(\mathrm{flat}(\ell_2) \parallel \mathbf{0}_6\big),$$
$$\mathcal{R}_{\mathrm{toy}} = \mathcal{H}_{\mathrm{H}}(x(N_0), x(N_1)).$$

Para esconder $\ell_2$, a testemunha contém o ramo com $\mathrm{flat}(\ell_2)$ seguido do slot nulo $\mathbf{0}_6$, depois $(x(N_0), x(N_1))$, e as posições correspondentes. O verificador recebe $\mathcal{R}_{\mathrm{toy}}$, não aprende que o

primeiro factor escolhido foi o de $\ell_2$, e verifica no circuito que os pontos intermédios estão nas curvas certas. As larguras 2/2 são apenas deste exemplo; a fotografia fixada usa 38/18.

Finalmente, $Q$ e $R$ não são folhas da árvore. A árvore contém a folha não re-randomizada $(O, H_p(O), C)$. Dentro de FCMP, as igualdades de abertura recuperam essa folha a partir de $Q$, e $R = r_I^{(\mathrm{FCMP})} V + r_R^{(\mathrm{FCMP})} T$ leva a mesma máscara $r_I^{(\mathrm{FCMP})}$ até à costura SAL. Construir nós e calcular a raiz acontece fora do circuito; demonstrar secretamente que uma dessas folhas leva à raiz, e que é a folha aberta por $Q$, acontece dentro da relação FCMP.

## 11.10 Só agora: porque é que a árvore alterna duas curvas

Até aqui tratámos cada ramo como uma caixa que recebe escalares e devolve um ponto. Podemos agora abrir essa caixa. As coordenadas de um ponto pertencem ao campo base da sua curva; os coeficientes de um compromisso pertencem ao campo escalar da curva onde o compromisso vive. Curve Trees escolhe duas curvas para que um campo encaixe no outro:

$$\mathbb{E}_{\mathrm{S}}/\mathbb{F}_{q_{\mathrm{S}}} : \quad \#\mathbb{E}_{\mathrm{S}} = q_{\mathrm{H}}, \qquad\qquad \mathbb{E}_{\mathrm{H}}/\mathbb{F}_{q_{\mathrm{H}}} : \quad \#\mathbb{E}_{\mathrm{H}} = q_{\mathrm{S}}. \tag{11.49}$$

Na instanciação usada por FCMP++, $\mathbb{E}_{\mathrm{S}}$ chama-se Selene e $\mathbb{E}_{\mathrm{H}}$, Helios. Uma coordenada de Selene é um escalar nativo de Helios; uma coordenada de Helios é um escalar nativo de Selene. Por isso o caminho pode continuar sem simular, dentro de um campo, a aritmética dispendiosa de outro:

$$\mathrm{Ed25519} \longrightarrow \mathbb{E}_{\mathrm{S}} \longrightarrow \mathbb{E}_{\mathrm{H}} \longrightarrow \mathbb{E}_{\mathrm{S}} \longrightarrow \cdots . \tag{11.50}$$

Os três pontos da folha vivem no grupo Ed25519. As suas seis coordenadas afins, já na representação Wei25519 usada pelo circuito, alimentam o primeiro nível, que produz um ponto de Selene. A coordenada $x$ desse ponto cabe como escalar no circuito de Helios; no nível seguinte, uma coordenada de Helios volta a caber como escalar de Selene.

Quando for preciso marcar a curva, o rótulo fica no canto inferior esquerdo, sem deslocar o nível nem a posição:

$$\overset{n}{\mathrm{S}}P_j = (\overset{n}{\mathrm{S}}x_j, \overset{n}{\mathrm{S}}y_j), \qquad\qquad \overset{n+1}{\mathrm{H}}P_j = (\overset{n+1}{\mathrm{H}}x_j, \overset{n+1}{\mathrm{H}}y_j). \tag{11.51}$$

A notação de ramo não muda: ${}^{n}B[j]$ continua a extrair conceptualmente a posição $j$ do ramo do nível $n$. O rótulo inferior esquerdo pertence ao ponto, não ao índice.

**Público.** As equações das duas curvas, os seus campos, os geradores determinísticos e os compromissos de cada nível.

**Segredo conhecido pelo provador.** As coordenadas dos pontos do caminho e as aberturas dos compromissos que as transportam.

**Igualdade demonstrada.** Cada $\overset{n}{\mathrm{S}}P_j$ ou $\overset{n}{\mathrm{H}}P_j$ satisfaz a equação da curva indicada; a sua coordenada $x$ é a variável que entra no produto de pertença da curva seguinte.

**Ligação à igualdade anterior.** O ciclo não acrescenta uma nova declaração sobre a saída. Torna nativa a mesma igualdade de encaixe (11.16) que já ligava um ramo ao seguinte.

## 11.11 Generalized Bulletproofs conserva as referências

Um circuito aritmético poderia copiar todos os ramos para um só vector gigante. Essa cópia apagaria a estrutura útil: cada posição já pertence a um compromisso vectorial com identidade própria. Generalized Bulletproofs permite que uma restrição se refira directamente à posição comprometida. Se

$$K^{(a)} = r_a H_a + \sum_j b_j^{(a)} G_j^{(a)}, \qquad K^{(b)} = r_b H_b + \sum_j b_j^{(b)} G_j^{(b)}, \tag{11.52}$$

o circuito pode reutilizar, por exemplo, $b_7^{(a)}$ numa equação de curva e $b_3^{(b)}$ num produto, sem publicar nenhum deles. A referência contém o compromisso e a posição; não é uma cópia solta do valor.

Os níveis que vivem sobre Selene agregam-se num argumento, e os que vivem sobre Helios noutro:

$$\mathsf{Prova}_{\mathrm{S}}: \quad n = 0, 2, 4, \ldots, \qquad \mathsf{Prova}_{\mathrm{H}}: \quad n = 1, 3, 5, \ldots. \tag{11.53}$$

Cada argumento prova que o provador conhece as aberturas e que todas as restrições que as reutilizam são satisfeitas simultaneamente. É aqui, e não no compromisso isolado, que entram a propriedade de conhecimento e a correcção do circuito.

Os parâmetros são transparentes: inicializadores, bases de máscara e vectores de geradores são obtidos por derivações públicas com separação de domínio. ≪Transparente≫ significa que qualquer pessoa os pode reproduzir; não significa que alguém conheça relações lineares entre eles. A vinculação continua a depender de tais relações serem computacionalmente desconhecidas.

**Público.** $K^{(a)}, K^{(b)}$, as bases derivadas, as duas provas agregadas e a descrição das restrições.

**Segredo conhecido pelo provador.** Os vectores comprometidos, as máscaras e as variáveis auxiliares do circuito.

**Igualdade demonstrada.** As aberturas dos compromissos satisfazem, numa só conjunção, os produtos de pertença, as equações de curva, as relações de multiplicação de pontos e os encaixes entre níveis.

**Ligação à igualdade anterior.** As referências garantem que a coordenada usada em (11.15) vem do compromisso que também produz o ponto do nível anterior. Os dois argumentos agregados preservam essa identidade ao longo da alternância (11.50).

### 11.11.1 Da escrita explícita ao produto comprimido

Podemos agora traduzir ${}^{n}B[j]$ para o circuito fixado. O código não compromete um índice secreto $j$ nem um vector de selecção. Na primeira camada, a codificação prometida em (11.8) é

$$\mathrm{enc}_0\big({}^{0}B[j]\big) = (b_{j,0}, \ldots, b_{j,5}), \tag{11.54}$$

as seis coordenadas dos três pontos da folha. Chamemos $q_0, \ldots, q_5$ às coordenadas da folha escondida. Depois de os compromissos entrarem na transcrição, seis desafios $c_0, \ldots, c_5$ comprimem cada tupla:

$$t_\star = \sum_{k=0}^{5} c_k q_k, \qquad t_j = \sum_{k=0}^{5} c_k b_{j,k}. \tag{11.55}$$

No ramo pequeno de quatro folhas, a leitura ${}^0B[2] = (O, H_p(O), C)$ torna-se

$$(t_0 - t_\star)(t_1 - t_\star)(t_2 - t_\star)(t_3 - t_\star) = 0. \tag{11.56}$$

Como $t_2 = t_\star$, o terceiro factor anula-se. O verificador aprende que algum factor se anulou, não qual deles. Salvo uma colisão da combinação aleatória, igualdade comprimida significa igualdade das seis coordenadas.

Na instanciação corrente, o ramo inicial tem 38 folhas:

$$\prod_{j=0}^{37} (t_j - t_\star) = 0. \tag{11.57}$$

O compromisso achatado contém, portanto, seis escalares por slot:

$$K_0 = r_0 H_0 + \sum_{j=0}^{37} \sum_{k=0}^{5} b_{j,k}\, {}^0G_{j,k}. \tag{11.58}$$

São 38 slots e 228 escalares, sem mudar o significado conceptual de ${}^0B[j]$. Na indexação linear do código, o par $(j, k)$ ocupa a posição $6j + k$ do vector comprometido. Nas camadas superiores há um só escalar por posição, e (11.15) torna-se

$$\prod_{j=0}^{w_n - 1} ({}^nB[j] - {}^nx_{j_n}) = 0. \tag{11.59}$$

**Público.** Os compromissos vectoriais e os desafios $c_k$, derivados depois desses compromissos.

**Segredo conhecido pelo provador.** As aberturas dos ramos, as seis coordenadas $q_k$ da folha ligada a $Q$, e a posição conceptual em que a igualdade ocorre.

**Igualdade demonstrada.** O produto (11.57), ou (11.59) numa camada superior, é zero para variáveis referidas aos compromissos da prova.

**Ligação à igualdade anterior.** Os $q_k$ vêm das mesmas variáveis $O, H_p(O), C$ que FCMP recupera de $Q$; os coeficientes dos factores vêm dos mesmos compromissos que produzem os pontos de ramo. A compressão muda o custo da igualdade conceptual, não os objectos que ela liga.

## 11.12 Aplicação dos certificados de divisores

O capítulo 3 estabeleceu o certificado de $W = s_A A$, o divisor associado à decomposição do escalar e a verificação por uma recta desafiada. Aqui aplicamos essa construção aos pontos já comprometidos em (11.52). Os coeficientes da função, a representação escalar e as coordenadas usadas nos testes referem-se às mesmas variáveis que aparecem nas aberturas e no caminho.

Na revisão fixada, os denominadores públicos são testados e invertidos fora do circuito. O denominador secreto do ponto resultante usa a restrição (3.15); as razões das avaliações da função usam (3.14). O provador honesto interrompe a execução se uma dessas razões não estiver definida. Os casos degenerados da multiplicação cruzada pertencem à análise de solidez do argumento, com desafios posteriores aos compromissos e graus limitados [33, 109].

A implementação admite uma representação escalar mais flexível que a exposição binária de (3.7), impondo a consistência da decomposição reutilizada. O ponto $W$ é normalmente comprometido. O dispositivo certifica a multiplicação desse ponto, já usado na equação de curva e nas aberturas do caminho.

### 11.12.1 Quantas funções racionais são construídas, e onde

Não existe um único «divisor da prova». A revisão fixada prepara um representante de função — no código, um polinómio normalizado cujo divisor tem a forma exigida — para cada alegação de multiplicação escalar oculta. Na primeira camada, por entrada, a contagem exacta é cinco:

| **objecto** | **multiplicação certificada** | **funções** | **observação** |
|---|---|---|---|
| abertura de $\widetilde{O}$ | $r_O T$ | 1 | recupera o ponto $O$ da folha |
| abertura de $\widetilde{H}_O$ | $r_I U$ | 1 | usa a decomposição do escalar $r_I$ |
| componente $V$ de $R$ | $r_I V$ | 1 | reutiliza a mesma decomposição de $r_I$, mas a base e a função são distintas |
| máscara de $R$ | $r_R T$ | 1 | chamada `IBlindBlind` na revisão Rust |
| abertura de $\widetilde{C}$ | $r_C G$ | 1 | recupera o compromisso $C$ da folha |

Há quatro escalares de máscara, mas cinco construções de função porque $r_I$ tem de ser multiplicado por duas bases diferentes, $U$ e $V$. O código torna esta excepção explícita: `IBlind::new` calcula dois divisores, enquanto `OBlind`, `IBlindBlind` e `CBlind` calculam um cada um. No verificador, a alegação sobre $r_I V$ reutiliza os dígitos/decomposição do logaritmo discreto de $r_I U$, mas conserva um divisor próprio para a nova tabela de pontos [109].

Cada transição do caminho abaixo da raiz acrescenta ainda uma máscara de ramo $r_n H_n$ e, portanto, uma função. Se $L \geq 1$ é o número total de camadas, incluindo o ramo-raiz, existem $L-1$ ramos não-raiz por entrada. A contagem da revisão fixada é então

$$\underbrace{5}_{\text{aberturas de } Q} + \underbrace{(L-1)}_{\text{máscaras de ramos não-raiz}} = L+4 \quad \text{funções por entrada.} \tag{11.60}$$

Para $m$ entradas de igual profundidade, são $m(L + 4)$ construções de testemunha por divisor. A prova de Schnorr que abre a diferença de máscara na raiz não usa este dispositivo, e SAL usa os seus argumentos de produto ponderado e Schnorr: nenhuma delas acrescenta uma função racional a esta contagem. Se uma futura implementação mudar a relação, a profundidade ou o modo de abrir os ramos, a fórmula tem de ser recalculada; não é uma constante universal do nome FCMP++.

Também não se deve confundir o número de funções com o número de pontos onde são avaliadas. O agrupamento do argumento compromete muitos coeficientes, mas a identidade de reciprocidade desafia cada representante nos três pontos de intersecção da linha aleatória. Partilhar a transcrição e reutilizar a decomposição de $r_I$ reduz a prova e o trabalho de verificação; não funde as cinco relações da primeira camada numa única função.

O lugar de cada objecto no sistema é o seguinte:

| | |
|---|---|
| **fora da prova, árvore pública** | tuplas de pontos, seis coordenadas por folha, inicializadores, geradores, somas multiescalares, nós e raiz. A construção pública da árvore não interpola funções racionais. |
| **preparação do provador** | escalares secretos, decomposições, tabelas públicas $2^i A$, pontos resultantes e coeficientes normalizados das funções que representam os divisores. Esta é a parte computacionalmente dispendiosa assinalada pelos construtores `*Blind::new`. |
| **dentro do circuito/ transcrição** | aberturas comprometidas, consistência das decomposições, equações de curva, coeficientes das funções, avaliações nos pontos desafiados, derivadas e verificações dos denominadores. O circuito liga estes valores aos mesmos pontos usados no caminho. |
| **presente nos dois lados** | os pontos e coordenadas das alegações aparecem na preparação e como variáveis comprometidas; isso não torna a função racional um nó da árvore nem torna público o caminho. |

Consequentemente, «os divisores pertencem à árvore» é uma descrição enganadora. Eles pertencem ao mecanismo que prova, em conhecimento zero, as multiplicações escondidas necessárias para abrir $Q$ e deslocar a abertura dos compromissos de ramo. A árvore pública só fornece os pontos e os ramos sobre os quais essas alegações são feitas.

### 11.12.2 A rara falha de completude

Para excluir a função inteiramente nula, o dispositivo normaliza um dos seus coeficientes. Há divisores legítimos para os quais esse coeficiente se anula; nesse caso raro, uma testemunha

verdadeira pode não admitir aquela forma normalizada [146]. É uma questão de *completude*: um provador robusto deve reiniciar e reamostrar as máscaras antes de publicar a prova. Não é uma abertura para o verificador aceitar uma igualdade falsa.

A revisão fixada ainda não trata essa excepção como uma falha recuperável: inverte o coeficiente e exige que a inversão exista, podendo abortar a geração da prova no caso raro em vez de reamostrar [109]. A distinção é importante para disponibilidade e robustez do provador, não para a solidez da equação verificada.

## 11.13 As quatro equações exactas do verificador SAL

A derivação conceptual já mostrou porque é que SAL funciona. Podemos agora olhar para a forma que o verificador realmente acumula. Além de $P$, a prova publica os compromissos $A_{\mathrm{SAL}}, B_{\mathrm{SAL}}, N_O, N_P, N_I$ e respostas $s_1, s_2, s_T, s_a, s_z, s_r$. Os índices $1, 2$ são deliberadamente neutros: uma resposta partilhada não nos autoriza a pressupor, antes da extracção composta, que há um único escalar $p_k$ por trás de todas as suas ocorrências. Um único desafio $c$ é obtido por Fiat–Shamir depois de entrarem na transcrição o hash da transacção, todo o $Q$, a imagem de chave $I$, $P$ e aqueles compromissos. O verificador impõe

$$c^2 P + cA_{\mathrm{SAL}} + B_{\mathrm{SAL}} = cs_1 G + cs_2 V + s_1 s_2 U + s_T T, \tag{11.61}$$

$$N_O + c\widetilde{O} = s_1 G + s_a T, \tag{11.62}$$

$$N_P + c(P - \widetilde{O} - R) = s_z U + s_r T, \tag{11.63}$$

$$N_I + cI = s_1 \widetilde{H}_O - s_z U. \tag{11.64}$$

A primeira linha é a equação de verificação do IPA ponderado. É quadrática em $c$: o produto $s_1 s_2$ recompõe os termos dos nonces e a abertura multiplicativa de $P$, produzindo os candidatos $p_{k_P}, r_{I_P}$ e o coeficiente $p_{k_P} r_{I_P}$ da Equação (11.33). A segunda linha fornece o candidato $p_{k_O}$ que abre $\widetilde{O}$. A terceira fornece $z_Q$ na diferença $P - \widetilde{O} - R$; a quarta reutiliza esse candidato e $p_{k_O}$ na relação da imagem. Há uma transcrição e respostas partilhadas, mas a relação SAL isolada ainda permite que $p_{k_P} \neq p_{k_O}$ ou $p_{k_P} r_{I_P} \neq z_Q$. A Equação (11.39), que inclui a abertura FCMP de $R$, é o passo que exclui essas diferenças.

**Público.** $Q, I, P, A_{\mathrm{SAL}}, B_{\mathrm{SAL}}, N_O, N_P, N_I$, o desafio $c$ e todas as respostas.

**Segredo conhecido pelo provador honesto.** $p_k, \widetilde{p}_a, r_I, z, r_P, r_R$, a combinação de máscara em $T$ necessária para (11.32), e os nonces que formam os compromissos.

**Igualdade demonstrada.** As quatro linhas SAL produzem as relações extraíveis (11.33)–(11.36), com candidatos possivelmente distintos. A composição com FCMP produz depois (11.40), isto é, $z = p_k r_I$ e $I = p_k \widetilde{H}_O - zU$ para os valores identificados.

**Ligação à igualdade anterior.** O campo $\widetilde{C}$ não entra na álgebra das quatro linhas, mas entra na transcrição como parte de $Q$. Sob a segurança de Fiat–Shamir e a vinculação multibase, a

SAL fica presa à mesma tupla completa que FCMP abriu para a folha. A SAL não oferece, isoladamente, uma abertura independente de $R$ em $r_I, r_R$; a concordância de $r_I$ resulta da relação composta, da extracção e da vinculação computacional.

## 11.14 Da exposição ao código fixado

As equações criptográficas acima foram conferidas na revisão `31c26d96eaadbba910ffe3613ad8b4cf9c598a93` do ramo `fcmp++` de `monero-oxide`, datada de 18 de Agosto de 2026 [109]. A preparação C++ das folhas foi conferida na revisão Monero `3646f648db57f60cca86430e25a635d19fa9b92a`, de 14 de Agosto de 2026 [89]. São duas fotografias identificadas; os seus bytes de geradores não devem ser misturados.

### 11.14.1 A folha codificada, incluindo saídas históricas

A tupla $(O, H_p(O), C)$ é a história conceptual correcta quando os pontos estão no subgrupo esperado. A migração de saídas históricas acrescenta uma precisão importante. Se $O_{\text{enc}}$ e $C_{\text{enc}}$ forem as codificações originais, a preparação legada segue a ordem

$$H_O = H_{\text{legado}}(O_{\text{enc}}), \qquad O_{\text{folha}} = \text{clr}(O_{\text{enc}}), \qquad C_{\text{folha}} = \text{clr}(C_{\text{enc}}). \tag{11.65}$$

A base é derivada da codificação pública original *antes* da limpeza de torção. Isto conserva a mesma definição de imagem de chave para uma saída antiga que contenha uma componente de torção. A folha armazenada usa $(O_{\text{folha}}, H_O, C_{\text{folha}})$ e rejeita identidades.

Para saídas Carrot, os pontos já são validados sem torção pelo novo formato e a preparação paralela é

$$H_O = H_{\text{Carrot}}(O_{\text{enc}}), \qquad O_{\text{folha}} = \text{dec}(O_{\text{enc}}), \qquad C_{\text{folha}} = \text{dec}(C_{\text{enc}}), \tag{11.66}$$

onde dec é a descodificação validada, e $H_{\text{Carrot}}$ usa o mapeamento não enviesado correspondente [52]. Em nenhum dos casos o circuito FCMP recalcula uma função hash para ponto: recebe a base preparada como parte da folha e prova a sua pertença. Portanto, $H_p(O)$ é a notação semântica deste capítulo, enquanto (11.65) fixa a compatibilidade exacta da codificação histórica.

**Público.** As codificações da cadeia, as regras de classificação da saída e a raiz obtida das folhas preparadas.

**Segredo conhecido pelo provador.** Nenhum segredo adicional é criado pela preparação; a carteira continua a usar $p_k$ e as testemunhas já descritas.

**Igualdade demonstrada.** A prova de pertença refere-se à tupla efectivamente inserida no acumulador; SAL refere-se à base cuja definição histórica determina a imagem $I$.

**Ligação à igualdade anterior.** A regra de preparação garante que o $\widetilde{H}_O$ público aberto por FCMP e usado em (11.64) ofusca exactamente a base associada à saída original.

### 11.14.2 Achatamento, larguras e sinais

O primeiro ramo não guarda três símbolos abstractos por posição. Guarda, por folha, as seis coordenadas afins da representação Wei25519 dos três pontos do grupo Ed25519:

$$(x_O, y_O, x_{H_O}, y_{H_O}, x_C, y_C). \tag{11.67}$$

Há 38 posições, portanto 228 escalares comprometidos. Os seis desafios da compressão produzem um valor por posição e conduzem ao produto de 38 factores de (11.57). As camadas superiores alternam ramos de 18 e 38 escalares e verificam a coordenada $x$ de um ponto completo. A infra-estrutura fixada reserva até 12 camadas e agrega até 128 entradas; são limites desta implementação, não do conceito Curve Trees.

Na criação de $Q$, os quatro termos usam os sinais positivos de (11.19)–(11.22). Internamente, o construtor do circuito recebe $-r_O, -r_I, r_R, -r_C$, com $r_R$ positivo, porque recupera a folha somando essas máscaras à entrada. Esta convenção de API não altera qualquer igualdade pública.

### 11.14.3 Geradores e ordem de verificação

$G, T, U, V$ são pontos distintos. $T$ é o gerador de ofuscação definido para a integração; $U$ e $V$ são derivados deterministicamente, com separação de domínio e amostragem não enviesada, para transportar $r_I$. As famílias de Generalized Bulletproofs e os inicializadores de Selene e Helios são igualmente derivados por domínios separados. O gerador $H$ do montante pertence à camada RingCT e não é um quinto parâmetro SAL.

O verificador da composição invoca SAL com uma referência à estrutura `Input`; depois lê dela os mesmos quatro campos e constrói uma `fcmps::Input` com $(\widetilde{O}, \widetilde{H}_O, R, \widetilde{C})$ para verificar a pertença. As rotinas criptográficas colocam relações numa verificação em lote; a aceitação só fica completa quando o chamador verifica o lote. Dizer que uma função «devolve sucesso» antes desse passo omitiria uma obrigação da interface.

## 11.15 Hipóteses, auditorias e estado de integração

A garantia composta depende da dificuldade de encontrar logaritmos discretos e relações lineares multibase; da ocultação e vinculação dos compromissos; da solidez, extracção e conhecimento zero dos argumentos; do modelo de oráculo aleatório de Fiat–Shamir; da correcção dos dispositivos por divisores; da validação de curvas, subgrupos e casos excepcionais; e de aleatoriedade e separação de domínio adequadas. A identidade pública de $Q$ compõe declarações, mas não cura uma falha numa delas.

Os âmbitos de revisão também são distintos. A Cypher Stack analisou Generalized Bulletproofs e a composição FCMP++/SAL [29, 28]. A Veridise auditou os circuitos e aplicou Picus a dispositivos não interactivos [134]. O trabalho SLVer Bullet forneceu evidência independente para

as equações por divisores [44]; a revisão da zkSecurity documentou a questão rara de completude já descrita [146]. A Trail of Bits publicou em Agosto de 2026 uma auditoria da implementação criptográfica incremental: seis achados informativos, nenhum classificado como de gravidade alta, média ou baixa; cinco resolvidos e um parcialmente resolvido, relativo à fragilidade da disposição de dados na fronteira Rust–C++ [128]. Cada relatório constitui evidência para a revisão e o âmbito examinados, não uma activação de consenso.

Em 6 de Setembro de 2026, o marco oficial de integração permanecia aberto, sem data limite, e mostrava 69%, 27 itens fechados e 12 abertos [86]. Essa percentagem conta itens do marco; não mede, por si, prontidão criptográfica ou operacional. O pedido de integração #9436 continuava em rascunho/WIP [50]. Na fotografia consultada em 31 de Agosto, a proposta pública de auditoria da integração mostrava zero de três marcos concluídos [51].

Na mesma data, a tabela de bifurcações rígidas da revisão principal consultada continuava a terminar na versão 16 e os tipos RingCT activos continuavam a terminar em Bulletproofs+ [88]. FCMP++ permanecia, portanto, em desenvolvimento e integração; não era uma regra de consenso activa da rede principal. Componentes criptográficos e de suporte já estavam no ramo principal, mas presença de código não equivale a activação do protocolo.

A revisão Monero fixada neste capítulo antecede a correcção dos valores derivados de $T, U, V$, integrada no ramo principal em 23 de Agosto de 2026 [87]. Por isso usamos a revisão `monero-oxide` fixada como autoridade para as relações e famílias criptográficas correntes, e a revisão Monero fixada como fotografia da integração C++; não transplantamos constantes entre as duas.

## 11.16 A frase inicial, agora demonstrada

Podemos enfim reler a declaração que abriu o capítulo. O produto de pertença e os encaixes de Curve Trees demonstram conhecimento de uma folha e de um caminho até à raiz. FCMP prova que o provador conhece uma abertura do mesmo $Q$ para essa folha. A extracção conjunta FCMP+SAL usa esse $Q$ e a equação multibase de $R$ para identificar os candidatos $p_k, r_I, z$ e obter

$$I = p_k \widetilde{H}_O - (p_k r_I) U = p_k H_p(O).$$

RingCT usa $\widetilde{C}$ como pseudo-saída e conserva, separadamente, o equilíbrio e os intervalos. A árvore diz «uma folha sob esta raiz»; FCMP diz «é esta folha escondida por $Q$»; SAL fornece a relação de autorização; a comparação em $R$ diz «é a chave desta mesma folha». A imagem $I$ deixa a marca determinística de um gasto.

# FCMP++: QUEM SABE O QUÊ, E QUE EQUAÇÃO LIGA AS PEÇAS?

*A árvore fixa uma folha; $Q$ torna-a pública sem a identificar; FCMP e SAL provam relações complementares sobre os mesmos bytes.*

**PÚBLICO** bytes visíveis · **PROVADOR** só a carteira · **VERIFICADO** relação aceite · **DETERMINÍSTICO** todos recalculam · **FIXADO** fotografia de código datada · **CONTINGENTE** depende da integração, cadeia ou transacção

## 1 · CARTEIRA / PROVADOR — testemunha que nunca é publicada

### FOLHA ESCOLHIDA + CAMINHO

$\ell_i = (O_i, H_p(O_i), C_i)$, a posição conceptual $j_n$, todos os ramos, coordenadas e aberturas até à raiz.
A árvore é pública; **a escolha** da folha e o caminho usado como testemunha é que ficam secretos.

### CHAVES + MÁSCARAS FRESCAS

Chaves que controlam a saída: $p_{k,i}, p_{a,i}$. Máscaras amostradas pela carteira: $r_{O,i}, r_{I,i}, r_{R,i}, r_{C,i}$ e as dos ramos.

$$z_i = p_{k,i} r_{I,i} \quad \text{em } \mathbb{F}_\ell.$$

### A CARTEIRA CONSTRÓI A PONTE

$$\widetilde{O}_i = O_i + r_{O,i} T, \qquad \widetilde{H}_{O,i} = H_p(O_i) + r_{I,i} U,$$
$$R_i = r_{I,i} V + r_{R,i} T, \qquad \widetilde{C}_i = C_i + r_{C,i} G.$$

$R_i$ transporta a mesma máscara $r_{I,i}$ nas bases $U, V$.

### TRABALHO DE PROVA — FORA DA ÁRVORE PÚBLICA

FCMP recebe a folha/caminho e prepara compromissos, decomposições e funções de divisor. Por input e $L$ camadas:

$$\underbrace{5}_{r_O T,\ r_I U,\ r_I V,\ r_R T,\ r_C G} + \underbrace{(L-1)}_{\text{máscaras de ramo}} = L + 4.$$

SAL prepara $P_i = p_{k,i} G + r_{I,i} V + z_i U + r_{P,i} T$ e nonces. As funções racionais são testemunhas do provador; **não** são nós da árvore.

caminho privado · re-randomiza · publica quatro pontos · prova SAL · publica

## 2 · TRANSCRIÇÃO PÚBLICA — pontos que nó e provas recebem byte por byte

### ÁRVORE / RAIZ

38 folhas $\times 6$ coordenadas; acima, ramos $18/38$; $Z = 0$.
$\mathcal{R}_{\text{tree}}$

raiz escolhida

### A PONTE $Q_i$

$$Q_i \quad = \quad (\widetilde{O}_i, \widetilde{H}_{O,i}, R_i, \widetilde{C}_i)$$

FCMP e SAL recebem **o mesmo objecto serializado**.
A folha continua oculta; as quatro componentes são públicas.

terceira componente

### O ELO $R_i$

$R_i = r_{I,i} V + r_{R,i} T$
não é raiz, nonce nem divisor

entra na diferença

### AUXILIAR SAL

$P_i$
fixa produto em $U$

produto ligado

### MARCA DO GASTO

$I_i = p_{k,i} H_p(O_i)$
repete num gasto duplo

### PSEUDO-SAÍDA

$\widetilde{C}_i$
conserva o montante

$C_i$

### mensagem / transacção

fecha o caminho · abre a folha · mesmos bytes · obriga concordância · abre · verifica · equilíbrio

## 3 · VERIFICADOR — relações provadas, não segredos revelados

### FCMP — «UMA FOLHA SOB ESTA RAIZ ABRE ESTE $Q_i$»

O circuito verifica um caminho até $\mathcal{R}_{\text{tree}}$ e recupera a mesma folha das variáveis de $Q_i$:

$$O_i = \widetilde{O}_i - r_{O,i}^{(\text{FCMP})} T, \quad H_p(O_i) = \widetilde{H}_{O,i} - r_{I,i}^{(\text{FCMP})} U,$$
$$C_i = \widetilde{C}_i - r_{C,i}^{(\text{FCMP})} G, \qquad R_i = r_{I,i}^{(\text{FCMP})} V + r_{R,i}^{(\text{FCMP})} T.$$

**VERIFICADO** pertença + abertura de $Q_i$. Não prova ainda $p_k$, a imagem de chave ou o valor numérico do montante.

### COSTURA MULTIBASE — AS EXTRACÇÕES TÊM DE COINCIDIR

SAL extrai candidatos com proveniência $P, O, Q$; FCMP extrai os candidatos marcados $(\text{FCMP})$.
Comparar duas aberturas do **mesmo** ponto $P_i - \widetilde{O}_i - R_i$ dá

$$\mathbf{0} = (p_{k,P} - p_{k,O}) G + (r_{I,P} - r_I^{(\text{FCMP})}) V + (p_{k,P} r_{I,P} - z_Q) U + (r_P - \widetilde{p}_a - r_R^{(\text{FCMP})} - r_{\text{dif}}) T.$$

Vinculação força $p_{k,P} = p_{k,O} = p_k$, $r_{I,P} = r_I^{(\text{FCMP})} = r_I$ e $z_Q = p_{k,P} r_{I,P} = p_k r_I$.

Os sobrescritos e subscritos são **rótulos de proveniência**, não potências nem posições da árvore.

### SAL + RINGCT — AUTORIZAÇÃO, DUPLO GASTO E EQUILÍBRIO

SAL abre $P_i$, $\widetilde{O}_i$, a diferença e $I_i$ com quatro equações de verificação. Depois da costura:

$$I_i = p_{k,i} \widetilde{H}_{O,i} - z_i U = p_{k,i} H_p(O_i).$$

**VERIFICADO** o mesmo $p_{k,i}$ controla a folha; a mesma saída volta a produzir $I_i$, permitindo detectar um segundo gasto.

RingCT usa a componente pública $\widetilde{C}_i$:

$$\sum_i \widetilde{C}_i = \sum_o C_o^{\text{novo}} + v_{\text{taxa}} H.$$

FCMP liga $C_i$ à folha; RingCT prova equilíbrio e intervalos. FCMP não extrai $v_i$, e SAL não usa $r_{C,i}$ na sua álgebra.

**FIXADO revisões consultadas:** Monero `3646f648`; `monero-oxide 31c26d96`. $38/18$, até 12 camadas e até 128 inputs descrevem essa fotografia; não são leis matemáticas de Curve Trees.

**CONTINGENTE** activação, raiz/bloco de referência, ordenação final de lotes, $m$ inputs, outputs novos, taxa e ocupação da árvore dependem da integração ou da transacção. Presença de código não significa regra de consenso activa.

# CAPÍTULO 12

# Relações de logaritmo discreto entre geradores

Estas relações são por vezes designadas «chaves privadas dos geradores». É necessário distinguir dois casos. Uma chave privada vulgar é intencional:

$$P = xG,$$

e o dono deve conhecer $x$. Uma armadilha de parâmetros é antes uma relação que o protocolo precisava que ninguém soubesse, por exemplo

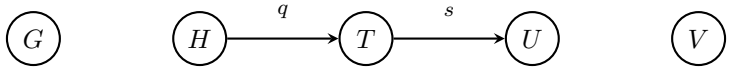
$$B = qA,$$

ou, com mais bases,

$$a_1 J_1 + \cdots + a_n J_n = \mathcal{O}.$$

A segunda relação pode ser conhecida sem fornecer o logaritmo entre um par escolhido, porque os restantes termos continuam presentes. A hipótese certa é, pois, a *vinculação por logaritmo discreto generalizado*, não uma colecção de «três gammas secretos».

Também importa *quem* conhece a relação. Uma relação deliberadamente pública pode ser parte do protocolo; uma relação conservada em segredo pelo criador dos parâmetros é uma armadilha; uma descoberta posterior torna a relação pública daquele momento em diante. Esta última não extrai retroactivamente todas as chaves ordinárias. Para calcular uma abertura alternativa de um ponto continua geralmente a ser necessária uma abertura inicial.




$$H \xrightarrow{q} T \xrightarrow{s} U \Longrightarrow U = (sq)H.$$

Este é o *grafo de relações*. Os vértices são geradores; uma aresta $A \xrightarrow{q} B$ significa $B = qA$. Como os escalares não nulos têm inverso, o caminho pode ser percorrido para trás, e os rótulos dos caminhos são os produtos dos escalares das arestas. Cada componente ligada entrega todos os logaritmos aos pares *dentro* dela. No desenho, porém, $G$ e $V$ continuam em componentes desligadas: conhecer a aresta $H - T$ não revela $\log_G H$. Uma relação de aridade superior é representada por uma hiper-aresta.

Convém ainda separar as consequências. Neste capítulo usaremos cinco classificações: *exploração algébrica directa*; *exploração provável que exige composição*; *falha só de privacidade*; *hipótese de segurança invalidada sem exploração concreta demonstrada*; e *relação inofensiva ou deliberadamente conhecida*. Roubo, gasto duplo, pertença falsa, prova de intervalo falsa e inflação são resultados distintos. A invalidação de uma hipótese de segurança não demonstra automaticamente uma exploração concreta.

As equações seguintes foram confrontadas com os geradores do Monero e com a SAL corrente nas revisões fixadas nas notas do capítulo anterior [89, 109], e com a especificação e a análise de composição FCMP++ [108, 28].

## 12.1 Relações que envolvem montantes e saídas

**Se $H = \gamma G$**

Um compromisso de montante é

$$C = aG + bH = (a + b\gamma)G.$$

Quem conhece uma abertura $(a, b)$ e a armadilha escolhe $b'$ e calcula

$$\boxed{b \longmapsto b', \qquad a' = a + (b - b')\gamma}$$

porque $a'G + b'H = aG + bH$. Perdeu-se a vinculação do montante, não a sua ocultação. Um estranho que só vê $C$ ainda precisa de $\log_G C = a + b\gamma$; o dono de uma saída, pelo contrário, já conhece $(a, b)$. Pode reabri-la como uma entrada maior, criar novas saídas com montantes honestamente dentro de 64 bits, produzir as provas de intervalo e fechar a equação de balanço com a máscara $a'$. A autorização continua a ser da sua moeda; o valor inventado é que não era dela.

**Conclusão — exploração algébrica directa:** $G - H$ destrói a vinculação de compromissos existentes cujas aberturas o atacante conhece e, composta com o balanço e as provas de intervalo normais, permite inflação oculta. Não dá ao portador de $\gamma$ a chave de gasto de uma saída alheia. O capítulo 7 deriva esta transacção passo a passo.

**Se $T = \tau G$**

A chave de saída compatível com formatos antigos e futuros é

$$O = xG + yT = (x + y\tau)G.$$

De uma abertura sai uma família inteira:

$$y \longmapsto y', \qquad x' = x + (y - y')\tau, \qquad O = x'G + y'T.$$

Mas um estranho diante de um $O$ arbitrário ainda não conhece $k = \log_G O = x + y\tau$. Portanto $\tau$, sozinho, não rouba essa saída. O que desaparece é o significado independente das duas coordenadas: migrações e formatos futuros não podem atribuir poderes diferentes a $x$ e $y$, e uma prova SAL de «conheço $x, y$» deixa de provar a abertura pretendida.

Há uma consequência concreta para o próprio dono. Numa saída histórica $O = xG$, a árvore conserva também $I = \mathcal{H}_p(O_{\text{original}})$, onde $O_{\text{original}}$ é a chave publicada antes da eventual limpeza de torsão, e a SAL publica $L = xI$. Com $\tau$, o dono escolhe aberturas alternativas $x'$ do mesmo $O$ e obtém marcadores $L' = x'I$ diferentes. A autorização não passou para um estranho; a detecção do segundo gasto deixou de reconhecer o primeiro.

**Conclusão — exploração algébrica directa:** $G - T$ permite ao dono de uma saída afectada quebrar a ligação SAL e tentar o gasto duplo. O roubo por um observador não foi demonstrado; falta-lhe uma abertura de $O$.

**Relações de $H$ com $T$, $U$ e $V$**

Se $H = qT$, $H = qU$ ou $H = qV$, a aresta liga a família de montantes à família FCMP++, mas $H$ e a outra base não aparecem juntas num compromisso de montante com coeficientes variáveis. A aresta isolada não dá $\log_G H$, não reabre $C = aG + bH$ e não é inflação. Se outra aresta ligar essa componente a $G$, o caminho compõe-se: por exemplo, $H = qU$, $U = uG$ dá $H = (qu)G$, e regressamos ao ataque de $\gamma$.

**Conclusão — hipótese invalidada sem exploração concreta demonstrada:** cada uma de $H - T, H - U, H - V$, sozinha. Com um caminho adicional até $G$, a inflação passa a ser a exploração directa anterior.

**A base que muda com a saída**

$\mathcal{H}_p(O_{\text{original}})$ não é um sexto parâmetro estático. Se, para uma saída histórica, alguém descobre

$$\mathcal{H}_p(O_{\text{original}}) = \eta_O G, \qquad O = xG, \qquad L = x\mathcal{H}_p(O_{\text{original}}),$$

então

$$\boxed{L = \eta_O O.}$$

O marcador da saída passa a ser calculável a partir de dados públicos. Conhecer $\eta_O$ para uma saída permite reconhecê-la quando $L$ aparece; conhecer as relações para todas permite testar todas as saídas da cadeia. Não revela $x$, não assina uma transacção e não faz $L$ deixar de repetir.

**Conclusão — falha só de privacidade:** identificação retroactiva da saída gasta; sem roubo, quebra da autorização ou inflação demonstrada.

## 12.2 Consequências de relações entre as bases da SAL

O código corrente não se limita a $\widetilde{I}$ e $R_{\text{fcmp}}$. Para uma abertura $\widetilde{O} = xG + yT$, escreve, com $z = xr_i$,

$$\begin{aligned} \widetilde{I} &= I + r_i U, & R_{\text{fcmp}} &= r_i V + r_j T, \\ P &= xG + r_i V + zU + r_p T, & z &= xr_i, \\ P - \widetilde{O} - R_{\text{fcmp}} &= zU + r'_p T, & L &= x\widetilde{I} - zU. \end{aligned}$$

Aqui $r'_p = r_p - y - r_j$. A prova de produto vincula $z = xr_i$ a $P$; três provas de representação vinculam $x$ a $\widetilde{O}$, $z$ à terceira linha e $(x, z)$ a $L$. A pertença FCMP vincula o mesmo $r_i$ a $\widetilde{I}$ e $R_{\text{fcmp}}$. Cada ponto público impõe a igualdade das variáveis partilhadas entre as provas.

### $U = qT$: muda o coeficiente de $U$

Para quaisquer $a, b, \Delta$,

$$aU + bT = (a + \Delta)U + (b - q\Delta)T.$$

Assim, $zU + r'_p T$ aceita um $z_{\text{lig}}$ diferente do $z_{\text{prod}} = xr_i$ provado dentro de $P$, ajustando a máscara em $T$. A última equação publica então

$$L = x\widetilde{I} - z_{\text{lig}} U,$$

que o dono pode variar entre gastos da mesma folha. A relação conhecida não muda $r_i$ em $\widetilde{I} = I + r_i U$; muda a interpretação do coeficiente de $U$ na representação conjunta nas bases $U, T$.

**Conclusão — exploração algébrica directa:** falha de ligação SAL e gasto duplo pelo dono; não roubo de uma saída arbitrária.

### $V = qT$: $R_{\text{fcmp}}$ já não se compromete com $r_i$

Agora

$$r_i V + r_j T = (r_i + \Delta)V + (r_j - q\Delta)T.$$

A prova FCMP pode usar o $r_i$ que abre $\widetilde{I}$, enquanto a SAL usa $r'_i = r_i + \Delta$ no produto $z' = xr'_i$; o mesmo ponto $R_{\text{fcmp}}$ admite as duas aberturas. O termo extra em $V$ é absorvido pela máscara em $T$ de $P$. Resulta $L = x\widetilde{I} - xr'_i U$, variável com $\Delta$.

**Conclusão — exploração algébrica directa:** perdeu-se a vinculação do compromisso a $r_i$ que compunha pertença e SAL; o dono pode produzir marcadores diferentes.

### $V = qU$: inconsistência entre produto e ligação

No ponto $P$,

$$r_i V + zU = (r_i + \Delta)V + (z - q\Delta)U.$$

Mais explicitamente, deixe a pertença usar $r$, a prova de produto usar $r'$ e $z' = xr'$. Depois de subtrair $R_{\text{fcmp}}$, a prova de representação lê

$$z_{\text{lig}} = xr' + q(r' - r),$$

logo

$$L = xI + (x + q)(r - r')U.$$

Escolher $r'$ muda $L$, salvo o caso degenerado $x = -q$.

**Conclusão — exploração algébrica directa:** novamente gasto duplo por quebra da ligação; ainda exige conhecer uma abertura autorizada da saída.

### $U = qG$ e $V = qG$ consideradas isoladamente

Se $U = qG$, $xG + zU$ admite $(x, z) \mapsto (x - q\Delta, z + \Delta)$. Se $V = qG$, $xG + r_i V$ admite o mesmo deslocamento em $(x, r_i)$. A vinculação interna de $P$ desaparece. Isoladamente, porém, $\widetilde{O} = xG + yT$ fixa $x$, $R_{\text{fcmp}} = r_i V + r_j T$ fixa $r_i$, e a equação em $U, T$ fixa $z$, sob as relações restantes desconhecidas. Não obtivemos uma transacção falsa.

**Conclusão — hipótese de segurança invalidada sem exploração concreta demonstrada:** $G - U$ e $G - V$ isoladas; outra aresta que permita absorver uma destas diferenças pode completar o ataque.

### Relação multibase geral

Se for conhecida

$$aG + bT + cU + dV = \mathcal{O},$$

então qualquer compromisso multibase

$$M = xG + pT + zU + rV$$

conserva o ponto sob a transformação

$$(x, p, z, r) \longmapsto (x, p, z, r) + \lambda(a, b, c, d).$$

Isto é perda directa de vinculação do vector-testemunho. A consequência final depende dos coeficientes que as equações externas voltam a fixar; os três ataques acima mostram relações cujo suporte elimina a vinculação necessária. Uma relação de aridade superior não deve ser promovida automaticamente a inflação, nem ignorada por não fornecer isoladamente $\log_G T$.

## 12.3 Análise das dez relações por pares

| Relação conhecida | Equação pública e abertura alternativa | Quem a explora; saídas já existentes | Resultado isolado | Outra armadilha? |
|---|---|---|---|---|
| $H = \gamma G$ | $C = aG + bH$; $b'$, $a' = a + (b - b')\gamma$. | Quem conhece a abertura; sim, compromissos próprios antigos continuam reabríveis. | Montante não vinculante; inflação oculta pela transacção normal. | Não; requer apenas autorização de uma entrada e as provas normais. |
| $T = \tau G$ | $O = xG + yT$; $y'$, $x' = x + (y - y')\tau$. | Só quem conhece uma abertura; sim, saídas históricas próprias migradas. | Autorização ambígua; $L$ variável, gasto duplo. Não rouba $O$ arbitrário. | Não para o dono; ao estranho falta $\log_G O$. |
| $U = qG$ | $P$ contém $xG + zU$; $(x, z) \mapsto (x - q\Delta, z + \Delta)$. | Provador com uma abertura; pontos existentes têm duas descrições locais. | Vinculação de $P$ perdida, mas $\widetilde{O}$ e a equação $U, T$ bloqueiam a falsificação completa. | Sim, uma relação que absorva a diferença externa. |
| $V = qG$ | $P$ contém $xG + r_i V$; $(x, r_i) \mapsto (x - q\Delta, r_i + \Delta)$. | Provador com uma abertura; a descrição local de $P$ muda. | Redução SAL invalidada; $\widetilde{O}$ e $R_{\text{fcmp}}$ fixam os coeficientes em isolamento. | Sim. |
| $H = qT$ | $C$ usa $G, H$; saída/SAL usa $G, T$. Não há compromisso variável comum a $H, T$. | Ninguém obtém nova abertura só com a aresta; saídas existentes inalteradas. | Famílias ligadas; sem inflação ou roubo demonstrado. | Sim: caminho de $T$ até $G$ dá $\log_G H$. |
| $H = qU$ | $C$ usa $G, H$; SAL usa $G, T, U, V$. Aresta não basta para deslocar $a, b$. | Idem; sem efeito concreto retroactivo isolado. | Hipótese global invalidada, sem exploração concreta. | Sim: caminho de $U$ até $G$. |
| $H = qV$ | Mesma separação: $H$ não aparece nas equações SAL. | Idem. | Sem exploração concreta demonstrada. | Sim: caminho de $V$ até $G$. |
| $U = qT$ | $zU + pT = (z + \Delta)U + (p - q\Delta)T$. | Dono/provador autorizado; sim, pode gastar a mesma folha existente com outro $L$. | Coeficiente $z$ da ligação difere do produto; gasto duplo. | Não. |
| $V = qT$ | $R = r_i V + r_j T = (r_i + \Delta)V + (r_j - q\Delta)T$. | Dono/provador; sim, FCMP e SAL podem abrir o mesmo $R$ com $r_i$ diferentes. | Compromisso a $r_i$ perdido; $L$ variável, gasto duplo. | Não. |
| $V = qU$ | $r_i V + zU = (r_i + \Delta)V + (z - q\Delta)U$. | Dono/provador; sim, uma folha autorizada basta. | Produto e ligação recebem aberturas distintas; gasto duplo. | Não. |

≪Existentes≫ não quer dizer que um estranho possa escolher uma saída de terceiro. Quer dizer que, depois da descoberta, o titular que ainda conserva a abertura consegue aplicá-la ao ponto já publicado. O criador malicioso dos parâmetros teve essa possibilidade desde o princípio; uma descoberta pública dá-a a todos os titulares e simultaneamente dá aos verificadores a oportunidade de parar, actualizar regras ou rejeitar a família comprometida. As provas antigas permanecem registadas.

## 12.4 Consequências para as provas de intervalo

As bases indexadas das Bulletproofs e Bulletproof+ formam compromissos do género

$$Q = \rho H + \langle \boldsymbol{a}, \boldsymbol{G} \rangle + \langle \boldsymbol{b}, \boldsymbol{H} \rangle.$$

Na notação concreta do Monero, $G$ desempenha também o papel de base de algumas máscaras internas e $H$ leva o montante; a troca de nomes não muda o argumento. Suponhamos conhecida uma relação

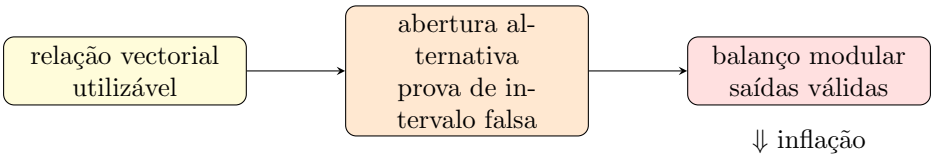
$$\rho_0 H + \langle \boldsymbol{u}, \boldsymbol{G} \rangle + \langle \boldsymbol{v}, \boldsymbol{H} \rangle = \mathcal{O}.$$

Então, sem mudar $Q$, existe a seta

$$(\rho, \boldsymbol{a}, \boldsymbol{b}) \longmapsto (\rho, \boldsymbol{a}, \boldsymbol{b}) + \lambda(\rho_0, \boldsymbol{u}, \boldsymbol{v}).$$

Uma abertura vectorial deixa de fixar os bits, os vectores de produtos ou as máscaras vinculadas pelas equações de produto interno. É a perda de vinculação de Pedersen aplicada a um compromisso multibase [18].

Isto não implica que qualquer relação vectorial permita criar valor. Uma relação pode cair numa direcção que não altera a coordenada constrangida, ou invalidar apenas a redução sem oferecer uma transcrição falsa. Se a direcção permite trocar um vector inválido por uma abertura que satisfaz as equações da prova, porém, a solidez de intervalo desaparece: um compromisso fora de $[0, 2^{64} - 1]$ pode receber uma prova aceite.




A relação $G - H$ fornece o primeiro mecanismo directo. Uma prova de intervalo falsa dá outra: permite que a aritmética módulo $l$ esconda um valor negativo ou transbordado, enquanto as saídas recebidas parecem montantes canónicos. É uma *exploração provável que exige composição* com uma relação que atinja os testemunhos certos, o balanço da transacção e a criação de novos compromissos. Não rouba por si só uma chave de saída e não torna falsas, retroactivamente, todas as provas antigas.

**O controlo negativo**

As tabelas

$$J, 2J, 4J, \ldots, 2^{n-1}J$$

usadas pelo dispositivo de logaritmo discreto têm relações públicas por desenho: os dígitos recompõem um múltiplo. Também $P = xG$, um nonce $R = rH$ e qualquer ponto construído com testemunho conhecido são declarações, não falhas de parâmetros. A solidez vem da prova de conhecimento e das restantes restrições; as relações públicas entre os múltiplos fazem parte da declaração. **Conclusão — relação inofensiva ou deliberadamente conhecida.**

## 12.5 Consequências para Curve Trees, Selene e Helios

Cada curva tem a sua própria família

$$g, h, \boldsymbol{g}, \boldsymbol{h}, J_{\text{init}}.$$

Selene e Helios são grupos distintos: uma relação dentro de uma curva não se estende à outra. Em cada uma, os compromissos da prova têm a forma

$$Q = rh + \langle \boldsymbol{a}, \boldsymbol{g} \rangle + \langle \boldsymbol{b}, \boldsymbol{h} \rangle,$$

e um nó que compromete os filhos tem a forma

$$N = J_{\text{init}} + \sum_i v_i J_i.$$

As famílias são derivadas de rótulos separados, sem configuração confiada, na implementação consultada; «transparente» significa que todos reproduzem os pontos, não que uma relação linear conhecida fosse aceitável [22, 29].

Uma relação entre $h, \boldsymbol{g}, \boldsymbol{h}$ dá aberturas alternativas dos compromissos da prova, como na secção anterior. Uma relação $\sum_i d_i J_i = \mathcal{O}$ dá uma colisão explícita entre vectores de filhos:

$$(v_0, \ldots, v_m) \longmapsto (v_0, \ldots, v_m) + \lambda(d_0, \ldots, d_m), \qquad N \text{ inalterado.}$$

Se essa colisão troca um filho honesto por outro compromisso, pode produzir uma passagem falsa numa camada e propagá-la até à raiz. Relações que envolvem $g$ ou as bases vectoriais podem, em vez disso, falsificar as restrições de selecção e re-randomização. O resultado possível é uma prova de pertença de uma saída que nunca esteve na árvore.

O caso de $J_{\text{init}}$ deve ser tratado separadamente. Se o seu coeficiente é sempre exactamente um, ele cancela ao comparar dois nós bem formados; conhecer apenas $J_{\text{init}} = qJ_0$ não produz automaticamente dois vectores de filhos com o mesmo nó. A relação torna-se utilizável se o inicializador puder ser absorvido por uma máscara da prova, se colidir uma codificação inicializada com uma soma crua, ou se vier acompanhada de uma relação entre os $J_i$. Sem essa composição, temos hipótese invalidada, não uma árvore falsa demonstrada.

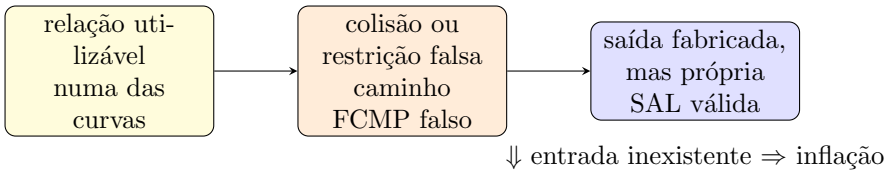
No topo há ainda a prova Schnorr da abertura da raiz. Se

$$X = \widehat{\mathcal{R}}_{\text{tree}} - \mathcal{R}_{\text{tree}} = \rho h,$$

o verificador exige

$$R_{\text{sig}} + cX = sh.$$

Uma armadilha que permita converter uma diferença de *conteúdo* falsa num múltiplo conhecido de $h$ faz esta equação aceitar a raiz comprometida. Conhecer apenas um caminho verdadeiro permite provar a pertença do membro correspondente, que é a propriedade pretendida.




Pertença falsa, isoladamente, não é autorização. Provar a pertença de uma saída alheia não revela $x$ e não produz a SAL. Para chegar à inflação, o atacante constrói uma tupla que ele próprio sabe abrir, atribui-lhe um compromisso de montante, usa a armadilha para provar um caminho inexistente e produz uma SAL válida. A entrada sem antepassado financia saídas novas. É uma *exploração provável que exige composição*: relação utilizável em Selene *ou* Helios, pertença falsa concreta, saída fabricada possuída e balanço válido. Roubo de uma saída alheia não resulta da pertença falsa.

Esta conclusão vale separadamente nas duas curvas. Uma colisão utilizável numa camada Selene pode bastar sem uma segunda relação Helios, e vice-versa; uma relação que apenas reabre um compromisso auxiliar mas não altera uma restrição relevante pode ficar no nível «solidez assumida, exploração não demonstrada».

## 12.6 Matriz de consequências

| Família | Armadilha conhecida | Vinculação perdida onde | Privacidade? | Roubo? | Pertença falsa? | Intervalo falso? | Detecção gasto duplo? | Inflação? | Hipóteses adicionais | Confiança / evidência |
|---|---|---|---|---|---|---|---|---|---|---|
| $G, H$ | $H = \gamma G$ | compromisso de montante | não | não | não | não | não | sim | abertura e transacção normal | alta: álgebra e código RingCT |
| $G, T$ | $T = \tau G$ | abertura $x, y$ da saída | não | não | não | não | sim | condicional | abertura própria; segundo gasto | alta: álgebra SAL/código |
| $G, U$ | $U = qG$ | $x, z$ dentro de $P$ | não | não | não | não | condicional | condicional | outra relação que vença $O, P'$ | média: álgebra; exploração não demonstrada |
| $G, V$ | $V = qG$ | $x, r_i$ dentro de $P$ | não | não | não | não | condicional | condicional | outra relação que vença $O, R$ | média: álgebra; exploração não demonstrada |
| $H, T$ | $H = qT$ | nenhuma equação isolada | não | não | não | não | não | condicional | caminho $T \rightsquigarrow G$ | alta: grafo e equações separadas |
| $H, U$ | $H = qU$ | nenhuma equação isolada | não | não | não | não | não | condicional | caminho $U \rightsquigarrow G$ | alta: grafo e equações separadas |
| $H, V$ | $H = qV$ | nenhuma equação isolada | não | não | não | não | não | condicional | caminho $V \rightsquigarrow G$ | alta: grafo e equações separadas |
| $T, U$ | $U = qT$ | $zU + rT$ | não | não | não | não | sim | condicional | abertura própria; segundo gasto | alta: deslocação SAL explícita |
| $T, V$ | $V = qT$ | $R_{\text{fcmp}}$ em $r_i$ | não | não | não | não | sim | condicional | abertura própria; segundo gasto | alta: deslocação SAL explícita |
| $U, V$ | $V = qU$ | $r_i V + zU$ | não | não | não | não | sim | condicional | abertura própria; segundo gasto | alta: deslocação SAL explícita |
| $G, T, U, V$ | $aG + bT + cU + dV = 0$ | compromisso multibase | condicional | não demonstrado | condicional | não | condicional | condicional | suporte deve vencer equações externas | média: vinculação certa; efeito depende do suporte |
| $I$ | $I = \eta_O G$ | bases da DLEQ tornam-se relacionadas | sim | não | não | não | não | não | relação para a saída observada | alta: $L = \eta_O O$ |
| BP/BP+ | relação em $H, \boldsymbol{G}, \boldsymbol{H}$ | compromisso vectorial | não | não | não | condicional | não | condicional | direcção altera testemunho; balanço | média: abertura explícita; falsificação depende da relação |
| Selene | relação em $g, h, \boldsymbol{g}, \boldsymbol{h}, J_{\text{init}}$ | nó ou compromisso da prova | não | não | condicional | não | não | condicional | colisão/restrição útil; saída própria e SAL | média: Curve Trees e composição |
| Helios | relação em $g, h, \boldsymbol{g}, \boldsymbol{h}, J_{\text{init}}$ | nó ou compromisso da prova | não | não | condicional | não | não | condicional | colisão/restrição útil; saída própria e SAL | média: Curve Trees e composição |
| Controlo | $J, 2J, \ldots$; $P = xG$; $R = rH$ | nenhuma | não | não | não | não | não | não | prova de conhecimento normal | alta: relação deliberada |

O único ≪sim≫ directo na coluna de inflação é $G - H$. As arestas SAL assinaladas quebram directamente a ligação, mas a inflação vem condicionalmente de um segundo gasto. Bulletproofs e Curve Trees exigem uma relação que atinja a restrição certa e a composição indicada. $\eta_O$ é privacidade sem autorização; uma hipótese invalidada continua a exigir correcção mesmo sem uma exploração concreta demonstrada.

# Parte II

# Extensões

# CAPÍTULO 13

# Provas sobre transacções

A lista de blocos não publica os endereços dos remetentes e destinatários, e os montantes RingCT estão cifrados. Uma carteira pode, porém, divulgar provas limitadas sobre uma transacção ou sobre um conjunto de saídas. Este capítulo define as afirmações que essas provas sustentam, os dados que revelam e as condições que não demonstram.

Salvo indicação em contrário, as observações sobre a implementação referem-se à revisão do código Monero datada de 14 de Agosto de 2026 [89]. Estas provas são artefactos da carteira. Não fazem parte de uma transacção, não são verificadas pelo consenso e não alteram o estado da lista de blocos.

## 13.1 Interface e alcance

A carteira consultada gera quatro formatos:

| **Formato** | **Segredo usado** | **Afirmação verificada** |
|---|---|---|
| `OutProofV2` | chave privada de transacção | O endereço indicado recebeu um montante determinado na transacção. |
| `InProofV2` | chave privada de vista | O endereço indicado, controlado para efeitos de vista pelo provador, recebeu um montante determinado. |
| `SpendProofV1` | chaves privadas das saídas gastas | O provador conhece a chave de gasto correspondente a uma posição desconhecida de cada anel de entrada. |
| `ReserveProofV2` | chaves privadas de vista e de gasto | O provador controla as saídas enumeradas e liga a prova a um endereço principal; o verificador calcula o total e a parte já gasta. |

Os verificadores ainda aceitam `InProofV1`, `OutProofV1` e `ReserveProofV1` para compatibilidade com artefactos antigos. A carteira gera as versões V2. A prova de gasto continua a usar `SpendProofV1`; não existe no código consultado uma `SpendProofV2`.

Na tabela, "recebeu" não significa "permanece não gasta". Uma prova de entrada ou de saída confirma a recepção e o montante, mas não determina se a saída foi gasta depois, não reconstrói a actividade de saída da carteira e não prova o saldo actual.

Na carteira de linha de comandos, as operações agrupam-se assim:

- `get_tx_proof` e `check_tx_proof`;
- `get_spend_proof` e `check_spend_proof`;
- `get_reserve_proof` e `check_reserve_proof`.

A interface RPC da carteira expõe métodos com os mesmos nomes. A geração e a verificação recebem uma mensagem opcional. As duas partes têm de usar exactamente os mesmos bytes.

Para as provas relativas a uma só transacção, a mensagem assinada é

$$\mathfrak{m} = \mathcal{H}(\texttt{txid} \,\|\, \texttt{message}).$$

O campo `message` deve conter o contexto da afirmação, por exemplo um identificador de factura e um desafio aleatório escolhido pelo verificador. Uma mensagem vazia produz uma prova transferível: qualquer pessoa que a receba pode apresentá-la de novo.

Os prefixos de formato ficam em texto simples. O corpo é serializado e codificado na variante Base58 de Monero [12]. Para verificar uma prova são ainda necessários o ID da transacção, o endereço e a mensagem correctos, bem como acesso a um nó que forneça a transacção e as saídas referidas.

### 13.1.1 A prova V2 com duas bases

`OutProofV2`, `InProofV2` e uma parte de `ReserveProofV2` usam a mesma prova de igualdade de logaritmos discretos. Sejam

$$P_1 = xJ_1, \qquad P_2 = xJ_2,$$

onde o provador conhece $x$. Definimos $J_1^* = 0^{32}$ quando $J_1 = G$, e $J_1^* = J_1$ quando $J_1 \neq G$. Esta convenção reproduz o último campo do traslado implementado. A separação de domínio é

$$T_{\text{tx}} = \mathcal{H}(\texttt{TXPROOF_V2}).$$

O provador escolhe $\alpha \in_R \mathbb{Z}_l$, calcula

$$\begin{aligned} L_1 &= \alpha J_1, & L_2 &= \alpha J_2, \\ c &= \mathcal{H}_n(\mathfrak{m}, P_2, L_1, L_2, T_{\text{tx}}, P_1, J_2, J_1^*), \\ z &= \alpha - cx \pmod{l}, \end{aligned}$$

e publica $(c, z)$. O verificador recompõe

$$L_1' = zJ_1 + cP_1, \qquad L_2' = zJ_2 + cP_2$$

e aceita quando o desafio recalculado com $L_1', L_2'$ é igual a $c$. Assim, a prova liga o mesmo escalar às duas relações sem o revelar.

## 13.2 Revelar a chave privada de transacção

Antes das provas codificadas, há uma opção mais simples. A carteira remetente pode obter a chave privada de transacção com `get_tx_key`; o verificador usa `check_tx_key` com o ID, a chave e o endereço. Para um endereço principal, a relação pública é

$$R = rG,$$

e o verificador calcula a derivação a partir de $rK^v$. Para uma transacção com chaves adicionais, o valor devolvido concatena a chave principal e todas as chaves adicionais. O verificador testa as derivações contra as saídas, descodifica os montantes que pertencem ao endereço e devolve o total recebido, o estado na memória temporária e o número de confirmações.

A chave de transacção não permite gastar as saídas nem obter as chaves privadas do destinatário. Contudo, permite testar endereços candidatos e identificar os que receberam fundos nessa transacção. Também permite produzir uma `OutProofV2`. Deve, por isso, ser entregue apenas quando essa divulgação for aceitável. Se a carteira remetente não conservou as chaves, não pode recuperá-las da lista de blocos.

## 13.3 Prova de saída: `OutProofV2`

Uma prova de saída demonstra a relação entre a chave pública de transacção, o endereço indicado e o segredo partilhado, sem revelar a chave privada de transacção. Para um pagamento a um

endereço principal $(K^s, K^v)$, a prova de duas bases usa

$$x = r, \qquad J_1 = G, \qquad J_2 = K^v,$$
$$P_1 = R = rG, \qquad P_2 = D = rK^v.$$

Para um sub-endereço $(K^{s,i}, K^{v,i})$, usa

$$x = r, \qquad J_1 = K^{s,i}, \qquad J_2 = K^{v,i},$$
$$P_1 = R = rK^{s,i}, \qquad P_2 = D = rK^{v,i}.$$

O corpo de `OutProofV2` contém, para a chave pública principal e para cada chave pública adicional da transacção, o ponto partilhado $D$ e a assinatura $(c, z)$. O verificador confirma as assinaturas, converte cada $D$ na derivação $8D$, testa as chaves de saída contra o endereço e descodifica os respectivos montantes RingCT. A prova só é gerada se a pesquisa encontrar um montante não nulo para o endereço apresentado.

O resultado prova que quem construiu o artefacto conhecia a chave privada de transacção correspondente e que o endereço recebeu o montante calculado. Não prova a identidade civil do remetente, não identifica as entradas usadas e não prova que o provador foi o único participante na construção da transacção. Quem obtiver a chave privada de transacção também consegue gerar a mesma classe de prova.

## 13.4 Prova de entrada: `InProofV2`

Quando o endereço apresentado pertence à carteira, `get_tx_proof` gera uma prova de entrada. Para um endereço principal, sejam $R = rG$ a chave pública de transacção e $k^v$ a chave privada de vista. A prova usa

$$x = k^v, \qquad J_1 = G, \qquad J_2 = R,$$
$$P_1 = K^v = k^v G, \qquad P_2 = D = k^v R.$$

Para um sub-endereço, substitui-se a primeira base por $K^{s,i}$ e a primeira chave pública por $K^{v,i} = k^v K^{s,i}$; a segunda relação continua a ser $D = k^v R$.

O formato serializado é paralelo ao de `OutProofV2`: o prefixo `InProofV2` é seguido por um segredo partilhado e uma assinatura para cada chave pública de transacção. O verificador confirma a igualdade das duas relações, deriva $8D$, encontra as saídas dirigidas ao endereço e soma os montantes.

Esta prova demonstra conhecimento da chave privada de vista associada ao endereço e a recepção do montante indicado. Não demonstra conhecimento da chave privada de gasto e, portanto, não prova por si só que o provador possa gastar a saída. Uma carteira só de vista pode gerar esta classe de prova. A divulgação de provas de entrada escolhidas não entrega $k^v$; a divulgação da própria chave de vista permitiria ao verificador detectar também as recepções futuras da conta.

### 13.4.1 O estado gasto e a actividade de saída

Na hierarquia actualmente usada, a chave privada de vista detecta uma saída recebida e descodifica o montante. Para decidir se essa saída foi gasta, a carteira tem de procurar na lista de blocos a sua imagem de chave. No caso principal,

$$k^o = \mathcal{H}_n\big(8k^v R, \mathrm{varint}(t)\big) + k^s \pmod{l},$$
$$I = k^o \mathcal{H}_p(K^o).$$

A chave de vista $k^v$ não fornece a chave de gasto $k^s$; por isso, não fornece $k^o$ nem a imagem $I$. A lista de blocos publica $I$ quando a saída é gasta, mas não publica uma ligação que a chave de vista consiga reconstruir sozinha.

Consequentemente, uma carteira restaurada apenas com o endereço e $k^v$ consegue somar recepções, mas não consegue, a partir desses dados e da lista de blocos, subtrair com segurança as saídas gastas, reconstruir as transacções de saída ou calcular o saldo actual. Pode detectar uma saída de troco como uma nova recepção, sem com isso identificar o gasto que a produziu nem o seu destinatário. Se uma carteira só de vista apresentar o estado gasto ou o histórico de saída, essa informação provém de imagens de chave ou de metadados fornecidos por uma carteira completa, não da chave de vista isolada.

`InProofV2` não elimina esta limitação: prova a relação de vista e o montante recebido, mas não fornece a imagem de chave. Em contraste, `ReserveProofV2` é gerada por uma carteira completa, inclui as imagens de chave das saídas declaradas e permite ao verificador consultar o estado de gasto dessas imagens.

### 13.4.2 Âmbitos de vista previstos no Carrot

O protocolo de endereçamento Carrot, previsto para a actualização FCMP++, separa capacidades que a hierarquia anterior concentra ou não oferece [52]. Na hierarquia recomendada:

- a chave de vista de entrada $k_v$, correspondente ao nível *View-Received*, detecta pagamentos externos e troco externo, mas não mostra onde as saídas foram gastas nem as saídas internas;
- o segredo de vista de saldo $s_{vb}$, correspondente ao nível *View-All*, permite derivar a informação necessária para reconhecer recepções externas e internas, gastos e o saldo, sem autorizar esses gastos;
- o segredo $s_{ga}$ permite gerar endereços sem conceder acesso à lista de blocos.

Assim, a capacidade de observar envios é fornecida pelo nível associado a $s_{vb}$, não pela chave privada de vista legada. Os formatos descritos neste capítulo são os formatos da carteira actual. A especificação Carrot ainda não define versões desses formatos para saídas Carrot nem afirma que possam ser reutilizados sem adaptação.

## 13.5 Prova de gasto: `SpendProofV1`

Uma prova de gasto contém uma nova assinatura em anel para cada entrada de gasto; no código, estas entradas têm o tipo `txin_to_key`. A carteira reutiliza exactamente o anel e a imagem de chave da entrada original. Reutilizar os mesmos membros evita uma intersecção entre dois anéis diferentes que pudesse reduzir o anonimato.

Embora as transacções activas usem CLSAG, esta prova auxiliar usa o esquema de assinatura em anel original de CryptoNote [133]. Para um anel $(K_1^o, \dots, K_n^o)$, a posição real $\pi$, a chave privada $k_\pi^o$ e a imagem de chave

$$\widetilde{K} = k_\pi^o \mathcal{H}_p(K_\pi^o),$$

o provador escolhe $\alpha \in_R \mathbb{Z}_l$ e pares aleatórios $(c_i, z_i)$ para $i \neq \pi$. Define

$$\begin{aligned} L_i &= z_i G + c_i K_i^o, & R_i &= z_i \mathcal{H}_p(K_i^o) + c_i \widetilde{K} \qquad (i \neq \pi), \\ L_\pi &= \alpha G, & R_\pi &= \alpha \mathcal{H}_p(K_\pi^o), \\ c_{\text{tot}} &= \mathcal{H}_n(\mathfrak{m}, L_1, R_1, \dots, L_n, R_n), \\ c_\pi &= c_{\text{tot}} - \sum_{i \neq \pi} c_i \pmod{l}, \\ z_\pi &= \alpha - c_\pi k_\pi^o \pmod{l}. \end{aligned}$$

Publica os pares $(c_1, z_1), \dots, (c_n, z_n)$. O verificador recompõe todos os $L_i, R_i$ e aceita se

$$\mathcal{H}_n(\mathfrak{m}, L_1, R_1, \dots, L_n, R_n) \stackrel{?}{=} \sum_{i=1}^{n} c_i \pmod{l}.$$

`SpendProofV1` concatena essas assinaturas codificadas em Base58. A carteira geradora tem de possuir as chaves correspondentes a todas as imagens de chave da transacção. Uma carteira só de vista e a interface de uma carteira de hardware não geram esta prova.

Uma prova válida demonstra o controlo de uma saída real em cada anel de entrada, sem revelar qual membro era o real. Não identifica o destinatário, não revela os montantes enviados e não demonstra, isoladamente, a identidade do autor. Numa transacção construída em colaboração, a ferramenta só funciona para uma carteira que disponha das chaves de todas as entradas.

## 13.6 Prova de reserva: `ReserveProofV2`

Uma prova de reserva associa um endereço principal a saídas controladas pela carteira. Pode abranger todas as saídas elegíveis ou apenas as saídas necessárias para ultrapassar um montante mínimo. No segundo caso, a carteira selecciona primeiro as saídas de maior valor e pode provar um total superior ao mínimo pedido. Saídas gastas ou congeladas não entram na selecção.

A geração exige uma carteira completa, não multi-assinatura, com as chaves privadas de vista e de gasto. A carteira de linha de comandos também recusa a operação através de uma carteira

de hardware. O verificador fornece o endereço principal; um sub-endereço não é aceite nesse campo.

Sejam $I_1, \ldots, I_q$ as imagens de chave das saídas seleccionadas. Todas as componentes assinam

$$\mathfrak{m}_{\text{res}} = \mathcal{H}(\texttt{message} \parallel \texttt{address} \parallel I_1 \parallel \cdots \parallel I_q).$$

Aqui `address` é a representação binária do par de chaves públicas, não o texto Base58. O corpo de `ReserveProofV2` contém duas colecções:

1. Para cada saída: o ID da transacção, o índice da saída, o segredo partilhado $D = k^v R$, a imagem de chave, uma prova V2 do segredo partilhado e uma assinatura que liga a imagem de chave à chave de saída.

2. Para cada chave pública de gasto principal ou de sub-endereço que controla alguma das saídas: uma assinatura de Schnorr com a chave privada de gasto correspondente.

Na verificação, a carteira rejeita entradas ou imagens de chave duplicadas, obtém as transacções confirmadas, verifica as provas do segredo partilhado e das imagens de chave, deriva o sub-endereço pertinente, descodifica cada montante e verifica as assinaturas das chaves de gasto. Consulta ainda o nó para classificar cada imagem de chave como gasta ou não gasta. O resultado é o par de montantes `total` e `spent`; a reserva não gasta no momento da consulta é

$$\texttt{total} - \texttt{spent}.$$

A prova revela as saídas seleccionadas, os seus montantes, as imagens de chave e as chaves públicas de gasto dos sub-endereços envolvidos. Uma imagem de chave ainda não vista permite reconhecer publicamente o gasto futuro da saída. As assinaturas demonstram o controlo separado dessas chaves de gasto, mas não provam que cada sub-endereço foi derivado canonicamente da chave de gasto principal.

A prova de um mínimo estabelece apenas um limite inferior; o provador pode omitir outras contas, outras carteiras e fundos não seleccionados. Mesmo a opção `all` cobre apenas as saídas elegíveis conhecidas pela carteira que produziu o artefacto. O estado gasto ou não gasto deve ser consultado de novo, porque pode mudar depois da geração.

## 13.7 Afirmações de pagamento e identidade

As provas anteriores tratam relações entre chaves. Não autenticam nomes, empresas, facturas ou contractos. Para ligar uma prova a uma identidade, o verificador deve fornecer uma mensagem específica e exigir, separadamente, uma assinatura autenticada dessa mensagem. A mensagem deve incluir pelo menos o ID da transacção, o endereço, a finalidade da prova e um desafio fresco.

As afirmações disponíveis podem resumir-se assim:

- uma chave de transacção ou `OutProofV2` demonstra que um endereço recebeu certo montante, mas não identifica quem controlava as entradas;
- `InProofV2` demonstra recepção e controlo da chave de vista, mas não autorização de gasto;
- `SpendProofV1` demonstra controlo das entradas, mas não o destino nem o montante líquido enviado a uma pessoa;
- `ReserveProofV2` demonstra controlo das saídas enumeradas e o seu estado observado, mas não a totalidade do património do provador.

Uma auditoria que precise de conhecer a origem declarada de um pagamento deve combinar estas provas com documentos externos e autenticação das partes. A lista de blocos e as provas da carteira, por si sós, não fornecem essa identidade.

## 13.8 Construções não implementadas

As duas construções seguintes descrevem relações úteis para auditoria, mas não existem nas interfaces da carteira consultada. Não são formatos de prova aceites pelas ferramentas actuais.

### 13.8.1 Provar que uma saída não corresponde a uma imagem de chave

Considere-se uma saída própria $K^o$, com

$$k^o = h + k^s \pmod{l}, \qquad h = \mathcal{H}_n\big(8rK^v, \mathrm{varint}(t)\big),$$

e uma imagem de chave $I_?$ publicada por uma entrada cujo anel contém $K^o$. O verificador calcula

$$Z_? = I_? - h\mathcal{H}_p(K^o).$$

Se essa entrada gastou $K^o$, então

$$Z_? = k^s\mathcal{H}_p(K^o).$$

A proposta de prova de não-gasto [130] evita publicar directamente esse último ponto. O provador demonstra duas relações:

1. com testemunha $k^s$,

$$\mathcal{J}_3 = \{G, K^s, Z_?\},$$
$$\mathcal{K}_3 = \{K^s, k^sK^s, k^sZ_?\};$$

2. com testemunha $(k^s)^2$,

$$\mathcal{J}_2 = \{G, \mathcal{H}_p(K^o)\},$$
$$\mathcal{K}_2 = \{k^s K^s, (k^s)^2 \mathcal{H}_p(K^o)\}.$$

O verificador confirma as duas provas e compara

$$k^s Z_? \stackrel{?}{=} (k^s)^2 \mathcal{H}_p(K^o).$$

A igualdade indica que a imagem testada gastou a saída; a desigualdade exclui essa imagem, sob as hipóteses das provas. Para afirmar que a saída continua não gasta seria necessário repetir o teste para todas as entradas em que $K^o$ aparece como membro do anel. Não há comando, formato serializado nem verificador desta construção no código Monero consultado.

### 13.8.2 Provar a relação entre um endereço e um sub-endereço

Para um endereço principal com $K^v = k^v G$ e um sub-endereço que satisfaz

$$K^{v,i} = k^v K^{s,i},$$

uma prova de igualdade de logaritmos poderia usar

$$x = k^v, \qquad J_1 = G, \qquad J_2 = K^{s,i},$$
$$P_1 = K^v, \qquad P_2 = K^{v,i}.$$

O traslado teria de incluir uma separação de domínio própria, os dois endereços e a mensagem do verificador. Uma prova válida ligaria o sub-endereço à chave de vista principal sem divulgar $k^v$. Esta `SubaddressProof` não está implementada no código consultado.

### 13.8.3 Limite de uma auditoria completa

Com provas de relação de sub-endereços e provas de não-gasto implementadas, um provador poderia apresentar uma auditoria mais selectiva:

1. declarar os endereços principais e sub-endereços abrangidos e provar as relações entre eles;
2. usar provas de entrada, ou entregar chaves privadas de vista, para identificar as saídas recebidas;
3. testar as imagens de chave de todas as entradas em cujos anéis essas saídas aparecem;
4. fornecer provas de saída para os destinatários cuja divulgação tenha sido autorizada.

Este procedimento não está disponível como sistema na carteira. Mesmo que fosse implementado, não provaria que a lista de contas declarada é completa: um provador pode controlar outras sementes, endereços ou carteiras. A completude exige controlos externos ao protocolo. A entrega de uma chave privada de vista simplifica parte da análise, mas permite observar todas as recepções passadas e futuras da conta e, sem imagens de chave importadas, não determina sozinha quais saídas foram gastas.

# CAPÍTULO 14

# Multi-assinaturas

Uma carteira Monero normal tem uma chave privada de vista e uma chave privada de gasto. Quem conhece a chave de gasto inteira pode autorizar uma transacção. Numa carteira de multi-assinatura, essa autoridade é repartida: escolhem-se $N$ participantes e um limiar $M$, e pelo menos $M$ deles têm de cooperar para gastar. Um grupo 2-de-3, por exemplo, continua a operar se um participante estiver indisponível, mas nenhum participante decide sozinho.

Este resultado não é obtido por um tipo especial de saída. Na lista de blocos, o endereço partilhado tem o formato de um endereço Monero e a transacção final tem o formato de uma transacção RingCT vulgar. A política $M$-de-$N$ é aplicada pelas chaves, pelos dados conservados nas carteiras e pelo protocolo interactivo que produz a assinatura. A lista de blocos não anuncia se o dinheiro pertence a uma pessoa, a uma empresa ou a um grupo de co-signatários.

As observações sobre a implementação neste capítulo referem-se à revisão do código Monero datada de 14 de Agosto de 2026 [89]. A descrição separa deliberadamente três coisas: o que essa revisão implementa, o que o consenso verifica e o que continua apenas conceptual. Quando dissermos que uma operação «existe», queremos dizer que há código e uma interface de carteira para a executar nessa revisão, não apenas uma construção plausível num artigo.

## 14.1 Estado da implementação

| Componente | Estado | Significado |
|---|---|---|
| Troca de chaves $M$-de-$N$ | Implementada, experimental | A carteira constrói uma chave de vista comum e uma chave de gasto repartida, com um máximo de 16 participantes. |
| Recepção e sincronização | Implementadas, experimentais | Os participantes detectam as mesmas saídas e trocam imagens de chave parciais e valores aleatórios públicos. |
| Assinatura de despesas | Implementada, experimental | A carteira constrói assinaturas CLSAG distribuídas e transacções `RCTTypeBulletproofPlus`. |
| Verificação pelo consenso | Implementada como RingCT normal | Os nós verificam a transacção final; não verificam uma política $M$-de-$N$ separada nem aprendem o número de signatários. |
| Sistema MMS | Implementado na carteira de linha de comandos | Organiza e transporta as mensagens do protocolo; não altera a criptografia nem o consenso. |
| Políticas em árvore | Não implementadas | São uma extensão conceptual conservada para explicar propostas de capítulos posteriores, não uma função da carteira Monero. |
| Multi-assinatura FCMP++ ou Carrot | Primitivas experimentais, sem integração | O ramo FCMP++ de `monero-oxide` contém algoritmos SAL com FROST; a carteira Monero examinada continua a assinar CLSAG e não oferece um fluxo FCMP++ multi-assinatura. |

A funcionalidade implementada continua marcada como *experimental* e está desactivada por omissão. A carteira de linha de comandos exige

```
set enable-multisig-experimental 1
```

antes das operações correspondentes. O aviso não é decorativo: um erro de protocolo, uma cópia de segurança inadequada ou um participante malicioso pode bloquear fundos; certas falhas podem ainda expor segredos. A multi-assinatura deve, por isso, ser tratada como um protocolo criptográfico interactivo, e não como autenticação convencional com várias palavras-passe.

## 14.2 Modelo e objectivos de segurança

Representaremos os participantes por $\mathcal{P} = \{P_1, \ldots, P_N\}$. A política é válida quando

$$1 \leq M \leq N.$$

Qualquer subconjunto com pelo menos $M$ participantes deve conseguir assinar; um subconjunto com menos de $M$ não deve conseguir reconstruir a chave de gasto nem produzir uma assinatura válida.

Há três propriedades diferentes que convém não confundir:

- *confidencialidade*: menos de $M$ participantes não aprendem a chave de gasto completa;
- *autorização*: menos de $M$ participantes não conseguem criar a resposta secreta necessária à assinatura;
- *disponibilidade*: a política tolera a ausência de até $N - M$ participantes.

A última propriedade não obriga ninguém a colaborar. Um co-signatário pode recusar uma proposta, apagar os seus dados ou abandonar a sessão do protocolo. Multi-assinatura reduz o número de participantes indispensáveis; não garante a cooperação dos participantes disponíveis.

Todos os membros aprendem a chave privada de vista comum. Conseguem, portanto, detectar as saídas recebidas pelo endereço, descodificar os respectivos montantes e observar as recepções da carteira. Depois de trocarem as imagens de chave parciais, conseguem também acompanhar quais dessas saídas foram gastas. O limiar protege a autoridade de *gasto*, não a privacidade interna entre os membros do grupo.

## 14.3 Comunicação entre co-signatários

A criação da carteira e a assinatura são protocolos com várias mensagens. Cada participante tem de saber quais mensagens pertencem à mesma sessão, quem as enviou e em que ronda devem ser usadas. As mensagens de troca de chaves da implementação têm assinaturas de autenticação, mas a primeira ronda contém um segredo auxiliar que será comum ao grupo. É, por isso, necessário um canal autenticado e confidencial para transportar os ficheiros ou cadeias de texto.

Autenticar apenas o nome do contacto não basta. Antes de receber fundos, os participantes devem comparar por um canal independente:

- a lista e a ordem canónica dos signatários;
- os valores $M$ e $N$;
- o endereço partilhado final;
- a chave pública de gasto e a chave pública de vista que lhe dão origem.

Esta confirmação posterior à troca de chaves detecta mensagens omitidas ou substituídas e carteiras que acabaram, por engano, em contas diferentes. A implementação inclui uma ronda final precisamente para assinar e comparar as chaves públicas resultantes.

Os formatos actuais começam por `MultisigxV2R1` na primeira ronda e `MultisigxV2Rn` nas rondas seguintes. O código consultado rejeita os formatos V1, considerados inseguros. Forçar a aceitação de um conjunto incompleto de mensagens pode produzir uma conta inconsistente ou tornar os restantes participantes dependentes de uma chave conhecida por um só membro.

## 14.4 Agregação das chaves de gasto

Antes de construir uma política $M$-de-$N$, vejamos um problema mais pequeno. Alice escolhe $a \in_R \mathbb{Z}_l$ e publica $A = aG$; Bob escolhe $b \in_R \mathbb{Z}_l$ e publica $B = bG$. A primeira ideia é somar as chaves:

$$K^s = A + B = (a + b)G.$$

Alice conhece uma parcela e Bob conhece a outra. Parece uma chave 2-de-2.

### 14.4.1 A soma ingénua

Suponhamos, porém, que Alice publica $A$ primeiro. Bob pode escolher uma chave pública $T = tG$ que controla sozinho e anunciar

$$B = T - A.$$

A soma anunciada é então

$$A + B = T,$$

cuja chave privada $t$ é conhecida por Bob. As assinaturas do traslado não corrigem esta relação algébrica. Este é um ataque de cancelamento de chaves.

### 14.4.2 Coeficientes de agregação

A defesa usada pela implementação associa um coeficiente a cada chave e ao conjunto completo de chaves [106]. Sejam

$$\mathbb{B} = (B_1, \ldots, B_t)$$

as chaves componentes distintas, em ordem canónica. Para cada $B_j$, calcula-se

$$\lambda_j = \mathcal{H}_n(B_j, B_1, \ldots, B_t, T_{\text{agg}}), \qquad T_{\text{agg}} = \texttt{Multisig_key_agg},$$

e a chave pública de gasto é

$$K^s = \sum_{j=1}^{t} \lambda_j B_j. \tag{14.1}$$

Quem conhece $b_j$, onde $B_j = b_j G$, conserva como parcela privada

$$x_j = \lambda_j b_j.$$

A dependência de $\lambda_j$ em todo o conjunto impede que o último participante escolha uma chave que cancele as anteriores sem resolver um problema criptográfico. A ordenação é essencial: sem ela, duas carteiras com as mesmas chaves poderiam calcular agregados diferentes.

## 14.5 Construção de uma política M-de-N

A implementação actual obtém o limiar através de trocas Diffie–Hellman. Definamos

$$q = N - M + 1. \tag{14.2}$$

Cada chave privada componente é conhecida por um subconjunto de $q$ participantes. Há

$$\binom{N}{q} = \binom{N}{M-1}$$

componentes distintas, e cada participante conserva

$$\binom{N-1}{q-1} = \binom{N-1}{N-M}$$

delas.

Por que funciona esta escolha? Se $N - M$ participantes faltarem, restam $M$. O complemento de qualquer subconjunto com $q = N - M + 1$ elementos tem apenas $M - 1$ elementos. Logo, os $M$ presentes intersectam todos os subconjuntos de $q$ membros e, em conjunto, possuem todas as parcelas da chave agregada. Com apenas $M - 1$ participantes pode faltar pelo menos uma parcela.

### 14.5.1 Chaves-base e chave de vista

Cada participante $P_i$ começa com dois escalares independentes:

$$a_i \in_R \mathbb{Z}_l, \qquad u_i \in_R \mathbb{Z}_l.$$

A chave-base pública $A_i = a_i G$ autentica as mensagens e entra nas trocas Diffie–Hellman. O valor auxiliar $u_i$ é enviado aos restantes membros na primeira ronda, através do canal confidencial. Depois de ordenar as contribuições, todos calculam a mesma chave privada de vista

$$k^v = \mathcal{H}_n(u_1, \ldots, u_N), \qquad K^v = k^v G.$$

A função hash inclui a serialização e a separação de domínio definidas pelo formato da implementação. A notação abrevia esses pormenores para mostrar a relação essencial.

Partilhar $k^v$ é deliberado. Cada carteira tem de reconhecer as mesmas saídas e calcular a parte comum das respectivas chaves privadas. Quem não deve ver os montantes e destinatários internos não deve pertencer a esta carteira.

### 14.5.2 Segredos partilhados por subconjuntos

Para um subconjunto $S \subseteq \mathcal{P}$ com $|S| = q$, as rondas Diffie–Hellman produzem conceptualmente

$$D_S = 8^{q-1}\left(\prod_{i \in S} a_i\right) G, \qquad b_S = \mathcal{H}_n(D_S).$$

O factor 8 em cada passo limpa o cofactor da curva. Cada membro de $S$ consegue chegar a $D_S$ a partir da sua chave privada e dos pontos recebidos; quem está fora de $S$ não aprende $b_S$.

Os pontos $B_S = b_S G$ formam a lista $\mathbb{B}$ da equação (14.1). Depois de calcular os coeficientes de agregação, cada membro substitui a parcela que conhece por

$$x_S = \lambda_S b_S.$$

A chave pública final é

$$K^s = \sum_{S:\,|S|=q} \lambda_S B_S.$$

Uma parcela conhecida por dois signatários continua a entrar uma única vez na soma. Durante a assinatura, a carteira conserva o conjunto das parcelas já incluídas para impedir duplicações.

### 14.5.3 Rondas e confirmação do endereço

O número de rondas criptográficas de troca de chaves é

$$N - M + 1 = q.$$

Segue-se uma ronda de confirmação, pelo que a configuração completa exige

$$N - M + 2$$

rondas. Uma política $N$-de-$N$ precisa de uma ronda de troca e uma de confirmação; baixar o limiar acrescenta trocas Diffie–Hellman.

Em cada ronda posterior à primeira, um participante aplica a sua chave-base a pontos cuja lista de origens ainda não o contém. A carteira acompanha essas origens para que cada segredo final corresponda a exactamente $q$ membros. Na última troca reúne os pontos públicos finais, elimina duplicados, calcula os coeficientes da agregação e actualiza as parcelas privadas locais. Na ronda de confirmação, todos assinam as mesmas chaves públicas de gasto e de vista.

Só depois dessa confirmação o endereço deve receber fundos. Uma diferença de um byte numa mensagem pode dar uma carteira válida mas diferente, o que é uma forma particularmente pouco interessante de descobrir que a verificação foi saltada.

### 14.5.4 Exemplo 2-de-3

Consideremos Alice, Bob e Carlos, com $M = 2$ e $N = 3$. Temos $q = 2$, pelo que são criados três segredos:

$$b_{AB} = \mathcal{H}_n(8a_A b_B G),$$
$$b_{AC} = \mathcal{H}_n(8a_A a_C G),$$
$$b_{BC} = \mathcal{H}_n(8a_B a_C G).$$

Depois da ponderação, Alice conhece $x_{AB}$ e $x_{AC}$, Bob conhece $x_{AB}$ e $x_{BC}$, e Carlos conhece $x_{AC}$ e $x_{BC}$. A chave de gasto é

$$K^s = (x_{AB} + x_{AC} + x_{BC})G.$$

Alice e Bob cobrem as três parcelas: Alice fornece $x_{AC}$, Bob fornece $x_{BC}$, e um deles inclui $x_{AB}$. O mesmo acontece com os pares $(A, C)$ e $(B, C)$. Um participante isolado conhece apenas duas parcelas e não consegue formar a chave completa.

Nos extremos, uma política $N$-de-$N$ tem $q = 1$: cada participante possui uma parcela exclusiva. Uma política 1-de-$N$ tem $q = N$: todos derivam a mesma parcela e qualquer um consegue usá-la. Portanto, a política 1-de-$N$ não oferece protecção contra um membro que decida gastar sozinho.

O número combinatório de parcelas cresce depressa. O código limita $N$ a 16 participantes, um limite prático e não uma propriedade matemática do protocolo. A própria implementação recomenda uma construção do género FROST se esse limite algum dia tiver de crescer.

## 14.6 Recepção de saídas e imagens de chave

Como todos conhecem $k^v$, todas as carteiras conseguem detectar uma saída recebida. Para uma saída pública $P$, escrevamos a respectiva chave privada sob a forma

$$x = d + \sum_{j=1}^{t} x_j, \tag{14.3}$$

onde $d$ reúne a derivação comum obtida com a chave de vista e, quando aplicável, o termo do sub-endereço; os $x_j$ são as parcelas distintas da chave de gasto. A imagem de chave é

$$I = x\mathcal{H}_p(P) = d\mathcal{H}_p(P) + \sum_{j=1}^{t} x_j \mathcal{H}_p(P).$$

Cada carteira calcula imagens parciais

$$I_j = x_j \mathcal{H}_p(P)$$

para as parcelas que possui. Depois de importar informação suficiente dos outros signatários, reúne as imagens parciais distintas e acrescenta uma só vez o termo comum $d\mathcal{H}_p(P)$. Assim obtém a mesma imagem $I$ que uma carteira de uma só chave produziria.

Este passo é necessário mesmo antes de gastar. Uma carteira precisa da imagem de chave completa para perguntar se a saída já apareceu como entrada na lista de blocos. Ver a saída com a chave de vista informa que ela foi recebida; não informa, por si só, se outro conjunto de $M$ signatários já a gastou.

A operação de exportação de informação multi-assinatura inclui, para cada saída, as imagens parciais e pares públicos de valores aleatórios que poderão ser usados numa tentativa de assinatura. O ficheiro é cifrado com material derivado da chave de vista comum e ligado à conta e ao signatário. Ao importar dados de pelo menos $M$ fontes contando a própria carteira, as imagens de chave podem ser reconstruídas e o estado de gasto actualizado.

Uma resposta de um nó remoto não fiável acerca do estado de gasto pode ser falsa. A criptografia valida a imagem de chave; não transforma o nó que responde numa testemunha honesta da lista de blocos.

## 14.7 Assinaturas de Schnorr com limiar

O mecanismo de resposta é mais fácil de ver numa assinatura de Schnorr sem anel. Suponhamos que a chave privada de uma saída está repartida como

$$x = \sum_{i \in \mathcal{A}} x_i, \qquad X = xG,$$

onde $\mathcal{A}$ contém exactamente as parcelas distintas cobertas pelos signatários escolhidos.

Cada participante escolhe um valor aleatório $\alpha_i \in_R \mathbb{Z}_l$ e publica $L_i = \alpha_i G$. O ponto agregado e o desafio são

$$L = \sum_{i \in \mathcal{A}} L_i,$$

$$c = \mathcal{H}_n(m, X, L).$$

Cada participante devolve

$$s_i = \alpha_i - c x_i,$$

e a resposta final é

$$s = \sum_{i \in \mathcal{A}} s_i.$$

O verificador recompõe

$$\begin{aligned} sG + cX &= \sum_i (\alpha_i - c x_i) G + c \sum_i x_i G \\ &= \sum_i \alpha_i G = L \end{aligned}$$

e confirma o desafio. Nenhum participante precisou de reconstruir $x$ num único lugar.

O perigo clássico está na reutilização de $\alpha_i$. Se o mesmo valor aparecer em desafios distintos $c$ e $c'$, observam-se

$$s_i = \alpha_i - c x_i, \qquad\qquad s_i' = \alpha_i - c' x_i,$$

e qualquer co-signatário calcula

$$x_i = (s_i - s'_i)(c' - c)^{-1}.$$

Um valor aleatório de assinatura é de uso único. Fazer uma cópia de segurança desse valor e reutilizá-la não é prudência; é guardar duas metades de uma equação que revela a chave.

Protocolos simples de duas rondas têm ainda ataques nos quais o último participante escolhe o seu ponto aleatório depois de observar os restantes e manipula muitos desafios relacionados [32]. Uma solução histórica é comprometer primeiro os pontos e revelá-los depois. A implementação actual segue uma solução do género MuSig2: usa dois valores aleatórios independentes por participante e liga a combinação ao traslado completo [98].

### 14.7.1 Dois valores aleatórios por signatário

Para cada entrada e tentativa, o signatário $i$ prepara

$$\alpha_{i,0}, \alpha_{i,1} \in_R \mathbb{Z}_l$$

e os pontos correspondentes necessários às duas bases da CLSAG. Seja $\tau$ o traslado que contém o anel, os compromissos, a compensação, a mensagem, as imagens, as respostas simuladas, a posição real e as dimensões do anel. Depois de somar separadamente os pontos públicos dos participantes, $(L_0, L_1)$ e $(R_0, R_1)$, a carteira calcula

$$b = \mathcal{H}_n(T_{\text{ms}}, \tau, L_0, L_1, R_0, R_1),$$

com separação de domínio própria. O valor secreto combinado é

$$\alpha_i = \alpha_{i,0} + b\alpha_{i,1}. \tag{14.4}$$

Mudar a transacção, o anel, uma resposta simulada ou os pontos de qualquer participante muda $b$. O último signatário deixa de poder escolher livremente o valor aleatório agregado depois de conhecer o traslado que o determina.

## 14.8 Assinatura CLSAG distribuída

Uma transacção corrente usa CLSAG para provar que a entrada real está numa posição desconhecida de um anel e que os compromissos de montante se equilibram. A prova de intervalo Bulletproof+ trata os montantes das saídas; ela não substitui a CLSAG que autoriza cada entrada.

Para a posição real, escrevamos $x$ para a chave privada da saída e $z$ para o segredo do compromisso à diferença entre a entrada real e o pseudo-compromisso. A CLSAG combina os dois testemunhos com coeficientes públicos $\mu_P$ e $\mu_C$:

$$w = \mu_P x + \mu_C z.$$

As imagens agregadas correspondentes são construídas a partir da imagem de chave $I = x\mathcal{H}_p(P)$ e da imagem de compromisso $D$. Os restantes membros do anel recebem respostas simuladas, como numa CLSAG normal.

Na versão distribuída, o primeiro signatário da tentativa inclui na sua resposta:

- as parcelas locais da chave de gasto ainda não usadas;
- o termo comum derivado da chave de vista e o ajuste de sub-endereço;
- o segredo do compromisso à diferença, ponderado por $\mu_C$;
- a sua componente aleatória combinada da equação (14.4).

Cada signatário posterior acrescenta apenas as parcelas de gasto que ainda não constam do conjunto `signing_keys`, além da sua componente aleatória. Quando todas as parcelas distintas estão cobertas, a resposta da posição real é a mesma que seria obtida com o testemunho completo $w$. O desafio inicial da volta CLSAG e a resposta real são então inseridos numa assinatura CLSAG ordinária.

O coeficiente $b$ dos dois valores aleatórios inclui o anel, os compromissos, a compensação, a mensagem, as imagens $I$ e $D$, todas as componentes públicas aleatórias, as respostas falsas, a posição real e as dimensões do anel. A finalidade desta lista extensa é simples: não deixar um pormenor relevante da tentativa fora do compromisso criptográfico.

Na revisão estudada, o construtor multi-assinatura aceita as versões RingCT activas que usam CLSAG. A saída produzida tem o tipo `RCTTypeBulletproofPlus`. O nome antigo `RCTType Bulletproof2`, conservado no rótulo desta secção para não quebrar referências de outras edições, já não descreve o resultado.

## 14.9 Construção e assinatura da transacção

Uma carteira individual pode escolher entradas, saídas, troco e taxa e assinar de imediato. Uma carteira $M$-de-$N$ tem de fazer os mesmos cálculos em várias máquinas e obter respostas compatíveis. O processo separa sincronização, proposta, assinatura e submissão.

### 14.9.1 Sincronização da carteira

Antes de propor uma transacção, os participantes trocam informação multi-assinatura. A exportação contém as imagens de chave parciais e, por saída, os pares públicos de valores aleatórios preparados para futuras tentativas. A importação combina imagens, actualiza o estado das saídas e regista quais signatários têm material disponível.

Exportar novamente substitui os valores aleatórios privados antigos. As tentativas que dependiam deles deixam de poder ser concluídas. Este comportamento ajuda a impor o uso único, mas significa que os ficheiros de uma mesma sessão não devem ser misturados ao acaso.

### 14.9.2 Proposta

O proponente escolhe as entradas e destinos, calcula os pseudo-compromissos e a prova Bulletproof+, e fixa a estrutura que todos irão verificar. Cria também tentativas para as combinações de signatários que podem atingir o limiar com o material importado.

Numa carteira 2-de-4, se Alice propõe e tem dados de Bob, Carlos e Diana, pode preparar tentativas $AB$, $AC$ e $AD$. Cada tentativa usa pontos aleatórios próprios. Assim, a indisponibilidade de Bob não obriga a reconstruir todo o pagamento, desde que outra tentativa ainda tenha valores privados válidos.

Dois proponentes não devem construir propostas concorrentes com os mesmos pares públicos. Uma resposta enviada para uma delas consome os respectivos segredos, mesmo que a transacção nunca seja submetida. Reutilizar o par na outra proposta abre a equação de extracção vista na secção anterior.

A chave privada da transacção é derivada de entropia nova e das imagens de chave das entradas. Isto evita que propostas repetidas criem as mesmas chaves de saída de uso único e tornem inacessíveis fundos destinados a transacções diferentes.

### 14.9.3 Respostas e submissão

Ao receber o conjunto de transacções, cada carteira reconstrói a proposta e verifica a estrutura, as entradas, os destinos, a taxa, o equilíbrio dos compromissos e a prova de intervalo. O utilizador deve ainda confirmar os destinos e montantes no seu próprio dispositivo. Uma estrutura criptograficamente válida pode conter um endereço de destino incorrecto.

Se aceitar, o signatário acrescenta as suas respostas parciais a todas as tentativas em que participa. Antes de expor essas respostas, a carteira apaga o material aleatório guardado para as tentativas e grava o estado actualizado. Esta ordem procura garantir que uma falha posterior ou uma segunda cópia do ficheiro não provoque reutilização.

Quando uma tentativa reúne todas as parcelas necessárias, a assinatura CLSAG fica completa. O último participante pode então importar o conjunto, verificar a transacção final e submetê-la à rede. Os nós e os mineiros verificam uma transacção RingCT normal; não conhecem os nomes, o limiar nem o número total de co-signatários.

### 14.9.4 Comunicação simplificada

Em termos operacionais, uma sessão correcta segue esta sequência:

1. todos sincronizam a carteira com a lista de blocos;

2. cada signatário exporta informação nova uma vez;
3. o proponente importa informação de um conjunto suficiente e cria a proposta;
4. os participantes verificam e assinam a mesma proposta, sem gerar uma nova exportação a meio;
5. um participante importa as respostas completas e submete a transacção;
6. todos voltam a sincronizar e trocam informação nova antes da próxima despesa.

O Sistema de Mensagens de Multi-assinatura, MMS, automatiza o transporte e o estado desta troca de protocolo na carteira de linha de comandos [113, 114]. Automatizar reduz erros de operação, mas não remove a necessidade de autenticar os contactos e confirmar a transacção.

## 14.10 Sequência mínima de comandos

Os comandos seguintes não constituem um guia de instalação; fixam apenas a correspondência entre o protocolo matemático e a interface que realmente o executa. Cada participante começa com uma carteira nova e nunca usada. Em cada carteira:

```
set enable-multisig-experimental 1
prepare_multisig
make_multisig <M> <R1-P1> [<R1-P2> ...]
exchange_multisig_keys <Rk-P1> [<Rk-P2> ...]
```

O resultado de cada comando de troca é enviado aos restantes participantes. Repete-se o último comando enquanto a carteira disser que falta outra ronda. Matematicamente, esta sequência constrói $k^v$, as parcelas $x_S$ e

$$K^s = \sum_{S:\,|S|=N-M+1} x_S G.$$

Todos confirmam que a carteira terminou com o mesmo endereço antes de esse endereço receber fundos.

Para gastar uma saída, os signatários escolhidos actualizam primeiro as imagens de chave e os pares públicos de valores aleatórios:

```
export_multisig_info info-Pi
import_multisig_info info-P1 [info-P2 ...]
```

Os ficheiros exportados são trocados entre eles. O proponente fixa a mensagem da transacção, os anéis, os compromissos, os destinos e a taxa:

```
transfer <endereco-destino> <montante>
```

Numa carteira multi-assinatura, `transfer` escreve a proposta no ficheiro `multisig_monero_tx`; não a difunde. Os co-signatários passam a versão actualizada desse mesmo ficheiro entre si e executam

```
sign_multisig multisig_monero_tx
```

até uma tentativa conter as $M$ contribuições necessárias. Cada chamada acrescenta as parcelas ainda ausentes à resposta CLSAG

$$s = \sum_i \alpha_i - c\left(\mu_P \left(d + \sum_i x_i\right) + \mu_C z\right).$$

Depois da última verificação, uma carteira difunde a transacção completa:

```
submit_multisig multisig_monero_tx
```

Assim, a sequência operacional corresponde às equações anteriores: trocar chaves para obter $K^s$, trocar dados para obter $I$ e os pontos aleatórios, fixar o traslado, somar respostas e submeter a CLSAG resultante. O MMS automatiza o transporte e o estado destas mensagens [113, 114]; não altera a matemática nem o consenso.

Na revisão consultada aplicam-se ainda os seguintes limites:

- a funcionalidade tem de ser activada explicitamente por ser experimental;
- o máximo é 16 signatários;
- as operações multi-assinatura indicadas não são suportadas por carteiras de hardware;
- mensagens de rondas antigas ou incompletas não devem ser forçadas;
- uma cópia de uma carteira já convertida não deve actuar como um novo participante independente.

Uma carteira clonada possui as mesmas parcelas e pode repetir material de assinatura que a original julgava consumido. As cópias de segurança devem ser entendidas como recuperação do mesmo participante, não como um participante adicional no limiar.

## 14.11 Políticas compostas não implementadas

A construção implementada é plana: $M$ de um conjunto fixo de $N$ participantes. Não implementa políticas como «Alice e um de três auditores» ou «dois departamentos, cada um representado por uma de duas pessoas». Essas expressões formam árvores de limiares e pertencem, nesta edição, à análise conceptual.

### 14.11.1 Agregação conceptual por conjuntos de chaves

Conceptualmente, cada folha de uma árvore teria uma chave pública. Um nó interno agregaria as chaves dos filhos segundo a sua própria regra de limiar, e a raiz forneceria a chave pública de gasto. Uma extensão possível permitiria que um «participante» de uma carteira exterior fosse ele próprio uma carteira interior.

Esta ideia é conservada porque capítulos posteriores a usam ao discutir serviços de garantia e grupos de moderadores. Não é, contudo, uma capacidade da implementação revista. A troca de chaves Monero aceita um único par $M, N$ e uma lista plana de signatários; a interface não serializa uma árvore arbitrária, não coordena os seus valores aleatórios e não prova que os ramos interiores executaram as políticas anunciadas.

Certas políticas podem ser simuladas distribuindo dispositivos ou escolhendo cuidadosamente as parcelas, mas isso altera os pressupostos de confiança e pode aumentar a probabilidade de reutilização de valores aleatórios. Uma implementação real de políticas compostas precisaria de um protocolo próprio, análise de segurança, formatos de mensagens e testes de recuperação. Nada disso está presente no código consultado.

## 14.12 Nota sobre multi-assinatura com FCMP++

**Resposta curta: ainda não está disponível na carteira Monero, mas já existem primitivas experimentais no ramo de referência consultado.** FCMP++ separa a prova de pertença à cadeia da prova SAL de autorização e ligação [108]. Um coordenador pode construir o caminho de pertença sem conhecer a chave de gasto inteira. A cooperação $M$-de-$N$ concentra-se na imagem de chave e na SAL.

Para uma saída de uma carteira de hierarquia antiga, escrevamos

$$x = d + \sum_{j \in \mathcal{A}} x_j, \qquad I = \mathcal{H}_{\mathrm{KI}}(K_o),$$

$$L = xI = dI + \sum_{j \in \mathcal{A}} x_j I.$$

$\mathcal{A}$ contém uma vez cada parcela coberta pelos signatários. A imagem $L$ é, portanto, a soma de imagens parciais; $\mathcal{H}_{\mathrm{KI}}$ denota a derivação de base exigida pela versão da saída. Não se inverte a chave repartida. Esta linearidade torna a multi-assinatura viável.

A SAL prova também conhecimento de $x$, da outra abertura $\widetilde{y}$ em $\widetilde{O} = xG + \widetilde{y}T$, e do produto $z = xr$. Fixado o mesmo re-randomizador $r$, podem formar

$$z = rd + \sum_{j \in \mathcal{A}} r x_j, \qquad s_x = \sum_j \alpha_j + ex, \qquad s_z = \sum_j \rho_j + ez.$$

Nenhum signatário pode enviar $rx_j$ sem máscara, pois $r$ revelaria $x_j$. Envia apenas pontos públicos e respostas ocultas pelos valores aleatórios $\alpha_j, \rho_j$, usados uma vez. Um protocolo completo tem ainda de construir conjuntamente $P, A, B, R_O, R_P, R_L$, tratar a abertura em $T$ e resistir a valores duplicados e abortos maliciosos. As somas acima são um esboço algébrico, não o protocolo distribuído da implementação de referência nem uma análise de segurança.

### 14.12.1 As duas primitivas no ramo de referência

Na revisão `31c26d96eaadbba910ffe3613ad8b4cf9c598a93` do ramo `fcmp++` de `monero-oxide`, o módulo SAL expõe, sob a funcionalidade opcional `multisig`, dois algoritmos baseados em FROST [109]:

- `SalLegacyAlgorithm` recebe uma chave $x$ repartida, como nas carteiras históricas, e uma abertura $y$ conhecida. Cada participante acrescenta parcelas interpoladas da imagem de chave e de $xU$; a resposta FROST comum produz simultaneamente as relações em $G$, $U$ e $\widetilde{I}$.

- `SalAlgorithm` destina-se à decomposição nova $O = xG + yT$. O escalar $x$, necessário para gerar a imagem de chave, é conhecido, enquanto $y$, a componente em $T$ que autoriza o gasto, é repartido com FROST sobre a base $T$. A resposta partilhada torna-se $s_y$; as restantes respostas SAL são construídas a partir dos dados comuns e de valores aleatórios ligados ao traslado.

Os dois algoritmos têm testes unitários que agregam parcelas e verificam a SAL resultante. Isto é implementação de primitivas criptográficas, não integração na carteira Monero nem activação pelo consenso: o módulo está condicionado por uma opção de compilação e não define por si só criação de contas, transporte de mensagens, recuperação, selecção de entradas ou comandos de utilizador.

Carrot acrescenta outra mudança. A hierarquia nova usa

$$K^s = k_{\mathrm{gi}} G + k_{\mathrm{ps}} T$$

e especifica que o segredo mestre não existe numa carteira multi-assinatura [52]. A primitiva nativa acima conserva $k_{\mathrm{gi}}$, necessário à imagem de chave, como componente conhecida e reparte $k_{\mathrm{ps}}$, a componente de autorização em $T$. Assim, uma vista completa pode actualizar o estado gasto e o saldo sem reconstruir a parcela privada que autoriza o gasto.

Na cadeia ver-se-iam apenas uma prova de pertença, uma SAL e $L$, nunca $M$, $N$ ou os signatários. No código Monero consultado, porém, o construtor multi-assinatura termina numa CLSAG: não há comandos nem mensagens que liguem a carteira às primitivas SAL distribuídas do ramo de referência. Logo, *as primitivas existem, mas a funcionalidade ainda não está disponível na carteira.*

## 14.13 Resumo

Uma carteira Monero $M$-de-$N$ transforma a chave de gasto numa soma ponderada de parcelas Diffie–Hellman. Cada parcela é conhecida por $N-M+1$ participantes; por construção, qualquer grupo de $M$ cobre todas as parcelas e um grupo menor não as cobre. Os coeficientes ligados ao conjunto completo de chaves impedem cancelamentos durante a agregação.

A chave de vista é comum a todos os membros. Para reconhecer despesas, as carteiras trocam imagens de chave parciais; para autorizar uma entrada, somam respostas parciais numa CLSAG. Dois valores aleatórios por signatário, ligados ao traslado completo, defendem a assinatura de ataques adaptativos, e cada par tem de ser usado uma única vez.

O resultado publicado é uma transacção RingCT normal. A complexidade fica nas carteiras: rondas autenticadas e confidenciais, confirmação do endereço, sincronização, verificação independente da proposta, destruição segura dos valores aleatórios e cópias de segurança coerentes. É precisamente por essa complexidade que a implementação consultada continua assinalada como experimental.

# CAPÍTULO 15

# Mercados garantidos por terceiros

Uma garantia 2-de-3 coloca comprador, vendedor e moderador no mesmo endereço multi-assinatura. Qualquer par pode autorizar um gasto; nenhum participante pode fazê-lo sozinho. Esta propriedade permite liquidar uma compra por acordo directo ou por uma decisão apoiada pelo moderador.

A garantia não é um contracto executado pelo consenso do Monero. O consenso verifica uma transacção RingCT e a respectiva assinatura. Não conhece a ordem de compra, a entrega, o prazo, as provas apresentadas numa disputa nem a identidade do moderador. Esses elementos pertencem ao software do mercado e ao protocolo de comunicação entre os participantes.

A análise histórica de integração com OpenBazaar identificou os problemas operacionais que uma aplicação deste tipo teria de resolver [115]. O código Monero consultado implementa as operações gerais de carteira multi-assinatura e o Sistema de Mensagens de Multi-assinatura, MMS [89, 113, 114]. Não implementa um mercado, um formato de ordem, um processo de arbitragem ou uma política automática de reembolso.

## 15.1 Estado da implementação

Na revisão de 14 de Agosto de 2026, a separação entre componentes implementados e componentes externos é a seguinte:

| **Componente** | **Estado** | **Consequência** |
|---|---|---|
| Carteira $M$-de-$N$, incluindo 2-de-3 | Implementada, experimental | Os três participantes podem criar um endereço comum e quaisquer dois podem produzir uma transacção válida. |
| Exportação e importação de informação multi-assinatura | Implementada, experimental | As imagens de chave parciais e o material necessário à assinatura podem ser sincronizados entre carteiras independentes. |
| Proposta, assinatura e submissão de transacções | Implementada, experimental | Uma proposta pode receber contribuições até atingir o limiar e ser difundida como uma transacção RingCT comum. |
| MMS | Implementado | Organiza mensagens e estados da carteira; não decide disputas nem substitui a autenticação dos participantes. |
| Ordem de compra, prova de entrega e decisão do moderador | Não implementadas pelo Monero | A aplicação tem de definir formatos, assinaturas, armazenamento e regras de validação. |
| Reembolso unilateral após um prazo | Não implementado | Uma saída Monero não contém um ramo de script que altere o conjunto de signatários depois de uma data. |
| Acesso do moderador apenas durante uma disputa | Não implementado pela carteira multi-assinatura | Os participantes partilham a chave privada de vista comum; o moderador pode observar a carteira desde a configuração. |
| Multi-assinatura com FCMP++ | Primitivas de referência, sem integração na carteira | O ramo FCMP++ de `monero-oxide` contém algoritmos SAL com FROST; a construção da carteira Monero termina em CLSAG distribuída, como indicado na secção 14.12. |

Consequentemente, este capítulo especifica uma utilização possível das operações implementadas. Não afirma que exista uma aplicação de mercado fornecida pelo projecto Monero.

## 15.2 Modelo formal

Sejam $B$, $V$ e $M$ o comprador, o vendedor e o moderador. A política de autorização é

$$\mathcal{A} = \big\{ S \subseteq \{B, V, M\} : |S| \geq 2 \big\}. \tag{15.1}$$

Para um conjunto de signatários $S$, uma transacção pode ser concluída quando

$$S \in \mathcal{A} \quad \Longleftrightarrow \quad |S| \geq 2. \tag{15.2}$$

O modelo fornece as propriedades seguintes, sob as hipóteses criptográficas da carteira multi-assinatura:

1. **Autorização:** um participante isolado não produz a assinatura necessária para gastar as saídas da carteira.

2. **Liquidação por acordo:** $B$ e $V$ podem pagar ou reembolsar sem intervenção de $M$.

3. **Liquidação de uma disputa:** $M$ pode cooperar com $B$ ou com $V$, segundo a decisão tomada fora da lista de blocos.

4. **Exclusão entre resultados:** duas transacções alternativas que gastam as mesmas saídas publicam as mesmas imagens de chave; no máximo uma pode ser confirmada.

5. **Ausência de disponibilidade garantida:** se menos de dois participantes fornecem material válido e actualizado, os fundos não podem ser gastos.

A política não garante que o moderador seja imparcial, que a mercadoria exista ou que as provas externas sejam verdadeiras. Também não impede uma coligação de dois participantes. Por exemplo, $\{V, M\}$ pode pagar ao vendedor sem o consentimento do comprador, porque esse resultado é precisamente autorizado por (15.1).

### 15.2.1 Fundos e resultados possíveis

Se o valor confirmado na carteira de garantia é $e$ e a taxa da transacção de liquidação é $f$, um resultado é o vector

$$\mathbf{v} = (v_B, v_V, v_M), \qquad v_B + v_V + v_M + f = e, \tag{15.3}$$

com $v_B, v_V, v_M \geq 0$. A componente $v_M$ pode ser zero ou pode conter uma taxa de moderação previamente acordada.

Três resultados comuns são:

$$\mathbf{v}_{\text{pagamento}} = (e - p - f,\ p,\ 0), \tag{15.4}$$

$$\mathbf{v}_{\text{reembolso}} = (e - f,\ 0,\ 0), \tag{15.5}$$

$$\mathbf{v}_{\text{div}} = (e - p' - m - f,\ p',\ m), \tag{15.6}$$

onde $p$ é o preço, $p'$ é o montante atribuído ao vendedor numa decisão parcial e $m$ é a taxa paga ao moderador. Cada vector tem de ser verificado pelos signatários antes da assinatura.

No nível RingCT, sejam $C_j^{\text{in}}$ os pseudo-compromissos das entradas e $C_k^{\text{out}}$ os compromissos das saídas. A transacção aceite satisfaz

$$\sum_j C_j^{\text{in}} = \sum_k C_k^{\text{out}} + fH. \tag{15.7}$$

A equação prova o equilíbrio dos compromissos; não prova que $\mathbf{v}$ corresponde ao contracto comercial.

### 15.2.2 Identidade de uma ordem

A aplicação deve fixar uma representação canónica da ordem. Um exemplo é

$$\mathsf{ord} = (\mathsf{version}, \mathsf{id}, B, V, M, \mathsf{description}, p, m_{\max}, \tag{15.8}$$

$$\mathsf{delivery_terms}, \mathsf{dispute_deadline}, \mathsf{evidence_policy}, \mathsf{refund_address}, \mathsf{payment_address}). \tag{15.9}$$

A identidade criptográfica da ordem é

$$h_{\mathrm{ord}} = \mathcal{H}(\texttt{monero-escrow-order-v1} \,\|\, \mathrm{enc}(\mathsf{ord})). \tag{15.10}$$

A função enc tem de ser determinística: a mesma ordem não pode admitir duas sequências de bytes. Comprador, vendedor e moderador confirmam $h_{\mathrm{ord}}$, as chaves de autenticação e o endereço multi-assinatura antes do financiamento.

As assinaturas da aplicação não devem reutilizar chaves ou valores aleatórios do protocolo de gasto. Cada participante dispõe de uma chave de autenticação separada e assina, pelo menos,

$$\sigma_i = \mathrm{Sign}_{a_i}(h_{\mathrm{ord}}, K^s_{\mathrm{escrow}}, K^v_{\mathrm{escrow}}, \mathsf{role}_i). \tag{15.11}$$

Esta assinatura liga o papel comercial à carteira concreta. A confirmação independente das impressões digitais das chaves continua necessária.

## 15.3 Configuração da carteira

Cada ordem usa uma carteira 2-de-3 nova. Os três participantes executam o protocolo interactivo da secção 14.5 e terminam com o mesmo par público

$$(K^s_{\mathrm{escrow}}, K^v_{\mathrm{escrow}}). \tag{15.12}$$

A construção requer as rondas de troca de chaves da implementação. Publicar uma chave-base de produto ou pré-seleccionar um moderador não permite ao comprador calcular unilateralmente o endereço multi-assinatura actual.

Cada participante deve:

1. criar uma carteira nova, sem histórico e sem reutilizar uma cópia de outra carteira convertida;
2. autenticar as mensagens de troca de chaves;
3. concluir todas as rondas exigidas pela carteira;
4. confirmar por um canal independente o endereço final e $h_{\mathrm{ord}}$;
5. conservar cópias seguras da sua própria carteira e do histórico de mensagens necessário à recuperação.

As três carteiras conhecem a chave privada de vista comum. Portanto, todos os participantes podem detectar as saídas recebidas e descodificar os montantes. Depois de trocarem as imagens

de chave parciais, podem também acompanhar quais dessas saídas foram gastas. A confidencialidade dos montantes perante observadores externos não implica confidencialidade entre $B$, $V$ e $M$.

## 15.4 Financiamento

O comprador financia o endereço apenas depois da confirmação da configuração. A mensagem de financiamento fixa

$$\mathsf{fund} = (h_{\mathrm{ord}}, \mathsf{txid}_{\mathrm{fund}}, e, n_{\mathrm{conf}}), \tag{15.13}$$

onde $n_{\mathrm{conf}}$ é a política de confirmações acordada. O vendedor não deve tratar uma transacção apenas observada na memória temporária como liquidação definitiva.

Depois das confirmações, cada carteira sincroniza a saída. Os participantes trocam informação multi-assinatura para construir as imagens de chave completas. Para uma saída com chave $P$ e chave privada de utilização única distribuída

$$x = d + \sum_{j \in \mathcal{I}} x_j,$$

as parcelas combinam-se em

$$I = x\mathcal{H}_p(P) = d\mathcal{H}_p(P) + \sum_{j \in \mathcal{I}} x_j \mathcal{H}_p(P). \tag{15.14}$$

Sem a importação suficiente, uma carteira pode não conhecer correctamente o estado gasto da saída e pode não dispor do material necessário à proposta.

O montante $e$ deve incluir a taxa prevista para a liquidação e uma margem definida pela aplicação. Uma estimativa insuficiente não pode ser corrigida por uma saída inexistente; pode exigir uma entrada adicional e, por consequência, nova sincronização e nova proposta.

## 15.5 Registo autenticado das mensagens

A decisão do moderador depende de dados que não estão na lista de blocos. A aplicação deve produzir um registo verificável. Seja a mensagem $j$

$$m_j = (h_{\mathrm{ord}}, j, \mathsf{type}_j, \mathsf{payload}_j, h_{j-1}), \qquad h_j = \mathcal{H}(\mathrm{enc}(m_j)), \tag{15.15}$$

e seja

$$\sigma_j = \mathrm{Sign}_{a_{\mathsf{sender}(j)}}(h_j). \tag{15.16}$$

O número de sequência e a hash anterior detectam omissões, reordenações e substituições dentro da transcrição apresentada. Não demonstram que um facto físico, como uma entrega, aconteceu; demonstram quem assinou determinada afirmação e qual era a sua posição no registo.

Os campos $\mathsf{type}_j$ devem distinguir pelo menos:

- aceitação ou rejeição da ordem;

- confirmação do financiamento;
- declaração de expedição ou prestação;
- confirmação de recepção;
- pedido de pagamento ou de reembolso;
- abertura de disputa;
- apresentação de prova;
- decisão do moderador;
- referência à transacção final.

A aplicação deve definir limites de tamanho, codificação, relógio, política de retenção e tratamento de mensagens duplicadas. A encriptação do canal protege o conteúdo em trânsito; as assinaturas autenticam o conteúdo depois de descodificado. São propriedades diferentes.

## 15.6 Liquidação sem disputa

Depois de o comprador aceitar o resultado, $B$ e $V$ escolhem um vector $\mathbf{v}$ de (15.6). Um deles constrói uma proposta que fixa:

$$\mathsf{proposal} = (h_{\mathrm{ord}}, \mathsf{txid}_{\mathrm{fund}}, \mathbf{v}, f, \mathsf{destinations}, \mathsf{change}, \mathsf{proposal_id}). \tag{15.17}$$

Antes de assinar, cada participante verifica directamente na proposta:

1. que as entradas pertencem à carteira da ordem;
2. que os destinos correspondem aos endereços autenticados;
3. que os montantes realizam $\mathbf{v}$;
4. que a taxa é aceitável;
5. que não existem saídas adicionais;
6. que a proposta ainda não foi substituída ou cancelada.

A sequência de comandos é a da secção 14.10: sincronizar informação multi-assinatura, criar uma proposta, adicionar duas contribuições e submeter a transacção completa. O ficheiro da proposta é parte da transcrição criptográfica de gasto, não uma descrição comercial da ordem.

Os dados exportados e os valores aleatórios de assinatura são de uso único. Uma nova proposta, uma alteração de taxa ou uma nova exportação exige que os participantes abandonem de forma explícita o material anterior. Duas propostas concorrentes não devem partilhar valores aleatórios privados.

## 15.7 Disputa e decisão

Uma disputa começa com uma mensagem assinada que contém $h_{\mathrm{ord}}$, o resultado pedido e a hash do registo apresentado. O moderador verifica:

1. as assinaturas e a continuidade do registo;
2. a correspondência entre a ordem e a carteira financiada;
3. a política de prova definida na ordem;
4. as afirmações de ambas as partes;
5. o saldo e o estado das saídas da carteira;
6. os endereços que receberão cada componente do resultado.

A decisão tem uma forma canónica, por exemplo

$$\mathsf{decision} = (h_{\mathrm{ord}}, h_{\log}, \mathbf{v}, \mathsf{reason}, \mathsf{decision_id}), \tag{15.18}$$

e o moderador publica

$$\sigma_M^{\mathrm{decision}} = \mathrm{Sign}_{a_M}(\mathcal{H}(\mathrm{enc}(\mathsf{decision}))). \tag{15.19}$$

A decisão não move fundos. Para a executar, o moderador e a parte que a aceita sincronizam as carteiras, verificam uma proposta que realiza $\mathbf{v}$ e fornecem as duas contribuições necessárias. Os conjuntos $\{B, M\}$ e $\{V, M\}$ têm exactamente o mesmo poder criptográfico; a diferença entre reembolso e pagamento está nos destinos e montantes da proposta.

Se o comprador e o vendedor resolverem a disputa antes da liquidação, podem assinar directamente uma proposta substituta. A aplicação deve marcar a decisão anterior como não executada e registar a transacção efectivamente confirmada.

### 15.7.1 Resultados concorrentes

Considere duas transacções válidas $T_1$ e $T_2$ que gastam pelo menos uma mesma saída $P_j$. A imagem de chave dessa entrada é determinística:

$$I_j = x_j \mathcal{H}_p(P_j). \tag{15.20}$$

A política de consenso rejeita a segunda ocorrência confirmada de $I_j$. Assim,

$$T_1 \text{ confirmada} \implies T_2 \text{ inválida relativamente ao estado resultante.} \tag{15.21}$$

Antes da confirmação, propostas incompatíveis podem circular simultaneamente. A aplicação não deve tratar uma assinatura parcial, uma transacção na memória temporária ou uma decisão do moderador como confirmação final. A referência final é a transacção aceite na lista de blocos com a profundidade exigida pela política da ordem.

## 15.8 Prazos e disponibilidade

Um prazo comercial não altera $\mathcal{A}$. Depois da data indicada na ordem, o comprador continua sem poder reembolsar sozinho e o vendedor continua sem poder pagar-se sozinho. O campo de bloqueio temporal de uma transacção Monero não implementa uma condição do tipo

$$\text{antes de } t:\ \text{2-de-3}, \qquad \text{depois de } t:\ B \text{ sozinho.}$$

Uma transacção de reembolso completamente assinada com antecedência é apenas um gasto alternativo já autorizado por dois participantes. Não constitui um ramo de consenso activado no futuro e pode competir com outras transacções que usem as mesmas entradas. Alterações de taxa, reorganizações, entradas adicionais ou perda do ficheiro também podem impedir o comportamento esperado.

A disponibilidade pode ser resumida por

$$\text{live}(S) = \begin{cases} 1, & |S \cap \{B, V, M\}| \geq 2 \text{ e os dados estão sincronizados,} \\ 0, & \text{caso contrário.} \end{cases} \tag{15.22}$$

O moderador é, portanto, parte da disponibilidade durante uma disputa. A aplicação deve definir substituição de moderadores antes do financiamento ou aceitar que a indisponibilidade permanente de $M$, combinada com desacordo entre $B$ e $V$, bloqueia os fundos.

## 15.9 Privacidade

Para um observador da lista de blocos, o financiamento e a liquidação têm a forma normal de transacções Monero. A política 2-de-3, os papéis dos participantes e o conteúdo da disputa não são campos públicos de consenso.

Entre os participantes, a privacidade é menor:

- todos conhecem o endereço multi-assinatura e a chave privada de vista comum;
- todos podem associar o financiamento a $h_{\mathrm{ord}}$;
- os signatários de uma proposta conhecem entradas, destinos, montantes e taxa;
- o moderador recebe o registo e as provas definidos pela política da aplicação;
- metadados de rede e de comunicação podem ligar identidades mesmo quando a lista de blocos não o faz.

A proposta histórica de entregar ao moderador a capacidade de vista apenas quando surge uma disputa não corresponde à carteira multi-assinatura implementada. Essa propriedade exigiria uma construção de chaves ou uma camada de custódia de dados diferente. Encriptar o registo comercial até à disputa pode limitar o acesso às mensagens, mas não retira ao moderador a chave de vista da carteira.

Cada ordem deve usar novas chaves e novos endereços de destino. A reutilização de uma carteira 2-de-3 entre ordens permite aos três participantes relacionar os respectivos financiamentos e liquidações.

## 15.10 Relação com FCMP++

A política comercial $\mathcal{A}$ não depende de CLSAG. Se a carteira multi-assinatura for adaptada a FCMP++, a configuração continua a distribuir o segredo de gasto e a liquidação continua a exigir duas contribuições. A diferença está na prova produzida para cada entrada:

$$\text{CLSAG distribuída} \quad \longrightarrow \quad \text{pertença FCMP} + \text{SAL distribuída.}$$

A lista de blocos continuaria sem publicar $B, V, M$ ou o limiar. Contudo, a implementação precisa de construir conjuntamente o marcador de ligação, o produto $z = xr_i$, os compromissos e as respostas SAL sem reconstruir a chave privada completa. O ramo de referência contém primitivas distribuídas para essa prova, mas a sua integração no fluxo da carteira Monero não existe na revisão consultada. A secção 14.12 distingue esses dois níveis.

## 15.11 Requisitos mínimos de uma aplicação

Uma aplicação que pretenda usar esta construção deve implementar e testar, pelo menos:

1. codificação canónica e assinatura de ordens, decisões e mensagens;

2. autenticação dos três participantes e confirmação independente do endereço;
3. uma carteira nova por ordem;
4. armazenamento cifrado e cópias de segurança de cada carteira;
5. sincronização de informação multi-assinatura antes de cada proposta;
6. validação local de entradas, destinos, montantes, troco e taxa;
7. controlo de uso único dos valores aleatórios e dos ficheiros de proposta;
8. uma política explícita de confirmações e reorganizações;
9. um formato verificável de prova e decisão;
10. tratamento de indisponibilidade e de fundos bloqueados;
11. protecção dos canais de comunicação e dos metadados;
12. testes contra propostas concorrentes, mensagens repetidas, estados divergentes e participantes maliciosos.

Estas funções não são acessórios de interface. São parte do modelo de segurança do mercado. A assinatura 2-de-3 limita quem pode autorizar a transacção; a aplicação determina qual transacção cada participante deve autorizar.

## 15.12 Resumo

Uma garantia Monero 2-de-3 usa a política

$$\mathcal{A} = \{\{B, V\}, \{B, M\}, \{V, M\}, \{B, V, M\}\}.$$

A carteira multi-assinatura implementa o limiar e produz uma transacção RingCT comum. O mercado tem de implementar a ordem, a autenticação, o registo de provas, a decisão, os prazos e a verificação da proposta.

Qualquer par pode liquidar; um participante isolado não pode. Duas liquidações incompatíveis que gastem as mesmas saídas têm imagens de chave iguais, pelo que no máximo uma pode ser confirmada. Não existe reembolso unilateral por prazo, acesso condicional nativo do moderador nem fluxo de mercado fornecido pelo Monero. A segurança operacional depende de carteiras independentes, mensagens autenticadas, sincronização correcta e valores aleatórios de uso único.

# CAPÍTULO 16

# Transacções conjuntas (`TxTangle`)

Uma transacção não conta histórias; deixa, contudo, relações. Mineiros, reservas de mineração, mercados com garantia e bolsas produzem ritmos e estruturas próprios, e esses hábitos podem alimentar heurísticas de grafos mesmo quando os montantes e os verdadeiros membros dos anéis permanecem ocultos (veja-se a secção 5.1 de [72]). A álgebra cala muito, mas não corrige por si só os costumes de quem a usa.

Este capítulo conserva a proposta *TxTangle*, inspirada no CoinJoin de Bitcoin [13]: vários participantes constroem uma única transacção e procuram esconder a partição das entradas e saídas entre si. Pretende-se que um observador não consiga decidir quais entradas financiaram quais saídas, nem quantos participantes reais estiveram presentes. Pretende-se ainda que cada participante aprenda tão pouco quanto possível acerca dos outros.

**Estado da proposta.** TxTangle não é uma funcionalidade do Monero, não é um tipo de transacção do consenso e não está implementado nas carteiras de referência. O que se segue é uma construção de investigação, originalmente escrita para a época de MLSAG e Bulletproofs. Uma proposta anterior, MoJoin, atribuía a coordenação a um administrador de confiança; essa confiança contrariava o objectivo de fungibilidade e o trabalho não prosseguiu. No fim do capítulo distinguiremos a ideia algébrica, que continua instrutiva, do protocolo histórico, que não pode ser apresentado como software disponível.

## 16.1 O núcleo algébrico

Numa transacção RingCT, os pseudo-compromissos das entradas e os compromissos das saídas satisfazem

$$\sum_{j} \widetilde{C}_{j}^{a} - \left( \sum_{t} C_{t}^{b} + fH \right) = 0, \tag{16.1}$$

onde $f$ é a taxa. Se cada participante construísse uma sub-transacção separadamente equilibrada e todas fossem depois coladas, qualquer observador poderia procurar subconjuntos que satisfizessem a mesma equação. O grande segredo acabaria reduzido a um exercício de somas.

TxTangle propõe quebrar esses balanços parciais sem quebrar o balanço global. Sejam $N$ participantes. Para cada par não ordenado $\{u, v\}$, escolhe-se um escalar secreto $s_{u,v} \in \mathbb{Z}_l$ e adopta-se a convenção antissimétrica

$$s_{v,u} = -s_{u,v}.$$

O participante $u$ recebe o *offset*

$$\delta_u = \sum_{v \neq u} s_{u,v}$$

e adiciona $\delta_u$ à máscara de um dos seus pseudo-compromissos; isto é, adiciona $\delta_u G$ ao próprio pseudo-compromisso. Então

$$\sum_{u=1}^{N} \delta_u = \sum_{\{u,v\}} (s_{u,v} + s_{v,u}) = 0. \tag{16.2}$$

Logo, a Equação (16.1) continua válida. Para um subconjunto próprio $S \subsetneq \{1, \ldots, N\}$, porém,

$$\sum_{u \in S} \delta_u = \sum_{\substack{u \in S \\ v \notin S}} s_{u,v}, \tag{16.3}$$

porque os termos internos de $S$ se cancelam aos pares. Sob a hipótese de que pelo menos um termo de corte é desconhecido e uniforme, o lado direito é uniforme em $\mathbb{Z}_l$. Um balanço parcial deixa, portanto, um resíduo aleatório; só a reunião de todos os participantes faz desaparecer a dívida.

É esta a parte verdadeiramente matemática de TxTangle. Os *offsets* alteram as máscaras, não os montantes, e cancelam-se apenas no agregado. Não demonstram, todavia, que o protocolo distribuído que os produz seja seguro: essa conclusão exige autenticação, vinculação do transcrito, resistência a abortos e um modelo explícito do adversário.

## 16.2 Construção descentralizada histórica

### 16.2.1 Canal de comunicação em grupo

O número de participantes reais não pode exceder o menor dos números de entradas e saídas, pois cada participante precisa de contribuir com pelo menos uma de cada. A proposta original

atribui, além disso, uma identidade pseudónima distinta a cada saída. Assim se forma um canal com $n$ identidades sem se publicar directamente quantas pessoas as controlam.

Uma sessão admite $2 \le n \le 16$, limite que coincide com as dezasseis saídas cobertas por uma Bulletproof+ nas regras actuais. A sala abre em $t_0$, fecha inscrições em $t_1$ e fixa previamente a prioridade da taxa, o método de estimação, a versão de transacção e os restantes parâmetros que poderiam criar uma impressão digital. Em $t_1$, cada identidade publica uma chave pública. Dessas chaves derivam-se os segredos de canal e os escalares $s_{u,v}$.

As mensagens relativas a entradas eram autenticadas, na proposta, por assinaturas SAG (secção 4.3); as relativas a saídas, por bLSAG (secção 4.4) sobre o conjunto de identidades. Reutilizar a mesma imagem de chave entre mensagens da mesma saída liga essas mensagens umas às outras sem revelar qual identidade as produziu. É uma separação delicada: ligar de menos permite substituições; ligar de mais recompõe o participante.

### 16.2.2 Cinco rondas de mensagens

O protocolo histórico divide a construção em cinco rondas. Cada ronda tem um intervalo próprio, e as publicações são espalhadas dentro dele para que a proximidade temporal não denuncie grupos de entradas ou saídas.

1. Cada identidade escolhe um escalar aleatório por saída, autentica-o com uma bLSAG e publica-o. A ordenação desses escalares determina os índices das saídas; o menor recebe $t = 0$. Publicam-se também, sob SAG, os tipos das entradas. Conhecidos os números e tipos de entradas e saídas, todos calculam o mesmo peso estimado e a mesma taxa.

   A divisão inteira da taxa é fixada por regra comum. O índice de saída mais baixo paga o resto. O campo *extra*, a ordenação das saídas e todos os valores opcionais têm igualmente de seguir uma política comum: uma transacção conjunta que ostente um uniforme especial anuncia aquilo que desejava esconder.

2. Para cada par de identidades deriva-se $s_{u,v}$; a ordem canónica das chaves decide o sinal. Cada participante constrói os compromissos das suas saídas, os montantes cifrados e os pseudo-compromissos, aplicando $\delta_u G$ a um destes últimos. A proposta original construía ainda a primeira parte de uma Bulletproof agregada. Todo o material de saída era autenticado por bLSAG; o material de entrada, por SAG. No fim da ronda, cada participante verifica o balanço global.

3. Se o balanço for correcto, cada identidade calcula o primeiro desafio comum da Bulletproof e publica a contribuição seguinte para a prova. Uma divergência no transcrito deve abortar a sessão: continuar com desafios diferentes não produz uma prova parcialmente válida, mas duas provas inválidas inteiras.

4. Fixam-se os membros dos anéis, as imagens de chave, os pseudo-compromissos, os endereços ocultos, as chaves públicas de transacção e a contribuição final para a prova de intervalo. A ordem e a autenticação das mensagens procuram impedir que o coordenador troque uma peça sem que o seu autor o detecte.

5. Com o transcrito completo, todos obtêm a mesma prova de intervalo e a mesma mensagem a assinar. Cada participante produz as assinaturas das suas entradas e publica-as no canal. Quem reunir todas as componentes pode serializar e difundir a transacção.

Estes passos descrevem MLSAG e Bulletproofs clássicos, não a construção activa com CLSAG e Bulletproof+. A diferença não é uma troca de nomes: mudam os desafios, os dados assinados, as condições de agregação e a gestão segura dos *nonces*.

### 16.2.3 Chaves públicas de transacção e ataque de Janus

Se todos conhecessem uma única chave privada de transacção $r$, qualquer participante poderia testar os endereços ocultos dos demais contra uma lista de endereços conhecidos. A proposta usa, por isso, uma chave pública de transacção distinta por saída, como acontece quando há destinatários em sub-endereços (secção 5.3).

O texto original considerava ainda uma chave pública 'base' destinada à mitigação de Janus [100]. Essa chave seria uma soma de contribuições aleatórias, de modo que ninguém conhecesse sozinho o escalar agregado. O cancelamento de chaves discutido na secção 14.4.1 não autoriza aqui um gasto, mas continua necessário vincular cada contribuição ao transcrito para impedir substituições.

Ainda assim, quem recebe num sub-endereço pode reconhecer padrões: muitas chaves públicas de transacção, muitas saídas ou uma chave base associável ao troco. Há duas obrigações: cifrar os dados e disfarçar o formato. Cumprir a primeira e esquecer a segunda seria uma economia bastante cara.

### 16.2.4 Fraquezas

Há dois ataques elementares. Primeiro, um adversário pode controlar muitas identidades e *poluir* a sessão, reduzindo o conjunto honesto a um grupo pequeno ou vazio [70]. Sem reputação, credenciais escassas ou outro custo de admissão, uma identidade não prova que atrás dela exista uma pessoa distinta.

Segundo, o adversário pode abortar repetidamente e comparar as tentativas seguintes. As chaves públicas de transacção, máscaras, escalares das provas e *nonces* de assinatura têm de ser regenerados; reutilizá-los pode revelar segredos ou ligar transcritos. Os membros fictícios

dos anéis, pelo contrário, devem permanecer fixos quando se reutiliza a mesma entrada, pois a intersecção de anéis entre tentativas pode denunciar o membro verdadeiro. A regra mais segura é não reutilizar a entrada após abortos, embora isso imponha custos de liquidez e selecção de moedas.

Acrescem a negação de serviço, a estimação inconsistente da taxa, mensagens equívocas, contribuições inválidas para a prova agregada e a fuga de metadados na rede. Nenhuma equação de cancelamento resolve esses problemas.

## 16.3 TxTangle servido

No modelo descentralizado resta saber quem abre as salas, faz avançar as rondas e distribui as mensagens. A solução mais directa introduz um anfitrião. Ele simplifica a disponibilidade e o relógio, mas ocupa uma cadeira demasiado boa: vê ligações, tempos e volumes, e pode apresentar listas de participantes diferentes a pessoas diferentes.

### 16.3.1 Comunicação através de I2P

A proposta sugere I2P para dissociar as mensagens das ligações de rede [36]. Túneis distintos transportariam pedidos de inscrição, mensagens de entrada e mensagens de saída. Essa disciplina reduz correlações triviais, mas não transforma o anfitrião em observador cego: a temporização, o tamanho, a reutilização de circuitos e a possibilidade de ele controlar pontos de entrada continuam no modelo de ameaça.

O serviço executaria quatro tarefas:

1. *Formar sessões.* Cada candidato declara o número de saídas, ou entrega as chaves das identidades pseudónimas, e o anfitrião reúne uma sala compatível. Aprende assim o volume declarado por ligação, embora não deva aprender a associação final entre entradas e saídas.

2. *Transportar rondas.* O anfitrião recolhe as mensagens durante o intervalo, publica o conjunto canónico no fim e aguarda uma margem antes da ronda seguinte. Todos devem comprometer-se com o mesmo *hash* do transcrito.

3. *Separar circuitos.* Mensagens ligadas à mesma saída podem usar o mesmo túnel; mensagens de saídas ou entradas diferentes não o devem fazer. Pedidos fictícios podem reduzir a correlação entre consultas e inscrições, mas aumentam custo e não constituem uma prova de anonimato.

4. *Vincular a vista da sessão.* Um anfitrião pode entregar listas diferentes a participantes diferentes. A defesa histórica vincula cada chave pública de transacção ao conjunto canónico de identidades $\mathbb{S}_{\mathrm{id}}$ e à lista canónica das chaves originais $\mathbb{S}_R^{(0)}$, com uma separação de domínio própria $T_{\mathrm{TT}}$:
$$R_t = \mathcal{H}_n(T_{\mathrm{TT}}, \mathbb{S}_{\mathrm{id}}, \mathbb{S}_R^{(0)}, R_t^{(0)})\, R_t^{(0)}, \qquad R_t^{(0)} = r_t G.$$
Participantes que receberam listas diferentes obtêm chaves diferentes e não conseguem concluir a mesma transacção. Isto detecta a equivocação entre participantes honestos; não impede o anfitrião de preencher todas as outras identidades de uma sala e executar um ataque de poluição. Numa implementação moderna, a vinculação teria de abranger ainda a versão, a sessão, todas as mensagens e a ordem canónica, e não apenas estas chaves.

### 16.3.2 Operação como serviço

Um anfitrião pode cobrar sem criar contas: acrescenta uma saída sua e exige que os participantes a financiem. A taxa de serviço e a taxa de mineiro são divididas por uma regra determinística; uma identidade canonicamente escolhida paga os restos das divisões inteiras.

Como o anfitrião não fornece uma entrada, não possui um pseudo-compromisso que cancele a máscara da sua saída. A proposta reparte essa máscara por escalares destinados aos participantes. Se $h$ é a máscara da saída do anfitrião, as contribuições $\eta_u$ têm de satisfazer
$$\sum_u \eta_u = h,$$
e cada participante incorpora a sua parcela num pseudo-compromisso. O sinal é positivo com a convenção da Equação (16.1); se se inverter essa convenção, invertem-se ambos os lados. O invariante indispensável é que a soma final das máscaras seja zero.

O anfitrião pode abortar perante mensagens inválidas ou falta de pagamento e, na ronda final, difundir a transacção e o seu identificador. Esta faculdade é útil e perigosa: oferece disponibilidade quando tudo corre bem e um oráculo de abortos quando não corre.

## 16.4 Construção com administrador de confiança

O modelo inspirado em MoJoin abandona a tentativa de esconder a partição do administrador. Este recebe os números e tipos de entradas e saídas, escolhe os índices, calcula a taxa e distribui os *offsets*. A execução histórica é:

1. Para cada par de participantes, o administrador escolhe um escalar com sinais opostos. Envia privadamente a cada participante a soma que lhe cabe, a fracção da taxa e os índices das suas saídas.

2. Cada participante constrói a sua sub-transacção: endereços ocultos, compromissos, montantes cifrados, pseudo-compromissos e dados dos anéis. Adiciona o *offset* recebido a um pseudo-compromisso e envia ao administrador a primeira contribuição para a prova de intervalo.

3. O administrador distribui o conjunto canónico dessas contribuições; cada participante calcula o desafio seguinte e devolve a segunda parte.

4. Depois da última contribuição, o administrador conclui a prova de intervalo e envia a mensagem final a assinar.

5. Cada participante assina as suas entradas. O administrador reúne as assinaturas, serializa a transacção e difunde-a.

O protocolo fica mais simples porque o administrador conhece aquilo que o modelo descentralizado tentava esconder. A privacidade passa então de uma propriedade criptográfica para uma promessa operacional.

## 16.5 Avaliação face ao Monero actual

O código-fonte de referência foi verificado na revisão de 14 de Agosto de 2026:

`3646f648db57f60cca86430e25a635d19fa9b92a`.

Não contém TxTangle, MoJoin nem uma interface CoinJoin para Monero. [1]

Há, pois, três afirmações que não devemos confundir:

1. *A álgebra é plausível.* Os *offsets* antissimétricos das Equações (16.2)–(16.3) preservam o balanço global e escondem balanços próprios de subconjuntos.

2. *Uma transacção final poderia usar o formato ordinário.* Se todos os compromissos, provas e assinaturas fossem válidos, o consenso não precisaria de uma etiqueta 'TxTangle': verificaria apenas uma transacção RingCT. Isto torna concebível uma implementação na camada das carteiras e da coordenação; não significa que ela exista.

[1] A construção ordinária está em `src/wallet/wallet2.cpp` e `src/ringct/rctSigs.cpp`; a construção distribuída existente pertence às contas de multi-assinatura e usa `src/multisig/multisig_tx_builder_ringct.cpp`. Nenhum destes caminhos implementa o protocolo deste capítulo.

3. *O transcrito histórico não é uma especificação actual.* O caminho activo usa CLSAG e Bulletproof+, com etiquetas de vista e regras modernas para o campo *extra*. FCMP++ e Carrot alteram novamente a autorização, a pertença e a construção das saídas, mas não estavam activos no consenso na revisão considerada (capítulo 11).

Uma implementação séria teria de fornecer, no mínimo, uma especificação canónica do transcrito; provas de posse das entradas; geração distribuída de Bulletproof+ ou outra forma segura de agregar provas; protecção dos *nonces*; validação de cada mensagem antes da ronda seguinte; regras de aborto e recuperação; defesa contra Sybil e poluição; testes diferenciais com o verificador do consenso; análise de metadados; vectores de teste; e uma auditoria independente. Teria ainda de provar que um participante malicioso não consegue roubar fundos, criar inflação, ligar honestos ou obter assinaturas sobre uma transacção diferente.

TxTangle é, em suma, uma proposta matemática real e uma implementação ausente. O cancelamento dos *offsets* merece ficar; a alegação de que o protocolo vive hoje em XMR, essa deve ficar de fora. Entre uma equação elegante e uma carteira segura há sempre um corredor comprido — e, neste caso, ainda não foi construída a porta.

# Bibliografia


[1] Kurt M. Alonso and Jordi Herrera Joancomartí. Monero - Privacy in the Blockchain. Cryptology ePrint Archive, Report 2018/535, 2018. https://eprint.iacr.org/2018/535.

[2] Kurt M. Alonso and Koe. Zero to Monero - First Edition, June 2018. https://web.getmonero.org/library/Zero-to-Monero-1-0-0.pdf [Online; accessed 01/15/2020].

[3] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O'Hearn, and Christian Winnerlein. BLAKE2: Simpler, Smaller, Fast as MD5. In *Applied Cryptography and Network Security*, pages 119–135, 2013. https://blake2.net/blake2.pdf.

[4] Adam Back. Ring signature efficiency. BitcoinTalk, 2015. https://bitcointalk.org/index.php?topic=972541.msg10619684#msg10619684 [Online; accessed 04/04/2018].

[5] Daniel J. Bernstein. Curve25519: New Diffie-Hellman Speed Records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *Public Key Cryptography - PKC 2006*, pages 207–228, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg. https://cr.yp.to/ecdh/curve25519-20060209.pdf [Online; accessed 03/04/2020].

[6] Daniel J. Bernstein. ChaCha, a variant of Salsa20, January 2008. https://cr.yp.to/chacha/chacha-20080120.pdf [Online; accessed 03/04/2020].

[7] Daniel J. Bernstein, Peter Birkner, Marc Joye, Tanja Lange, and Christiane Peters. Twisted Edwards Curves. In Serge Vaudenay, editor, *Progress in Cryptology – AFRICACRYPT 2008: First International Conference on Cryptology in Africa, Casablanca, Morocco, June 11-14, 2008. Proceedings*, pages 389–405. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008. https://eprint.iacr.org/2008/013.pdf [Online; accessed 02/13/2020].

[8] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, Sep 2012. https://ed25519.cr.yp.to/ed25519-20110705.pdf [Online; accessed 03/04/2020].

[9] Daniel J. Bernstein and Tanja Lange. Faster addition and doubling on elliptic curves. In Kaoru Kurosawa, editor, *Advances in Cryptology – ASIACRYPT 2007: 13th International Conference on the Theory and Application of Cryptology and Information Security, Kuching, Malaysia, December 2-6, 2007. Proceedings*, pages 29–50. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. https://eprint.iacr.org/2007/286.pdf [Online; accessed 03/04/2020].

[10] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Fast and Tradeoff-Resilient Memory-Hard Functions for Cryptocurrencies and Password Hashing. Cryptology ePrint Archive, Report 2015/430, 2015. https://eprint.iacr.org/2015/430.

[11] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. In *2016 IEEE European Symposium on Security and Privacy*, pages 292–302, 2016. https://www.password-hashing.net/argon2-specs.pdf.

[12] Base58Check encoding, November 2017. https://en.bitcoin.it/wiki/Base58Check_encoding [Online; accessed 02/20/2020].

[13] CoinJoin. https://en.bitcoin.it/wiki/CoinJoin [Online; accessed 01/26/2020].

[14] Karina Bjørnholdt. Dansk politi har knækket bitcoin-koden, May 2017. http://www.dansk-politi.dk/artikler/2017/maj/dansk-politi-har-knaekket-bitcoin-koden [Online; accessed 04/04/2018].

[15] Dan Boneh and Victor Shoup. A Graduate Course in Applied Cryptography. https://crypto.stanford.edu/~dabo/cryptobook/ [Online; accessed 12/30/2019].

[16] Boog900. Change how transactions are broadcasted to significantly reduce P2P bandwidth usage. Monero, proposta #9334, May 2024. Proposta aberta, não implementada na revisão consultada, https://github.com/monero-project/monero/issues/9334.

[17] Boog900. Proving maximum outbound connection count to allow selecting inbound peers as a Dandelion++ stem. Monero Research Lab, proposta #160, May 2026. Proposta aberta, não implementada na revisão consultada, https://github.com/monero-project/research-lab/issues/160.

[18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short Proofs for Confidential Transactions and More. Cryptology ePrint Archive, Report 2017/1066, 2017. https://eprint.iacr.org/2017/1066.pdf [Online; accessed 10/28/2018].

[19] Francisco Cabañas. Francisco Cabanas - Critical Role of Min Block Reward Trail Emission - DEF CON 27 Monero Village, December 2019. https://www.youtube.com/watch?v=IlghysBBuyU [Online; accessed 01/15/2020].

[20] Francisco Cabañas. Lightning talk: An Overview of Monero's Adaptive Blockweight Approach to Scaling, December 2019. https://frab.riat.at/en/36C3/public/events/125.html [Online; accessed 01/12/2020].

[21] Francisco Cabañas. MoneroKon 2019 - Spam Mitigation and Size Control in Permissionless Blockchains, June 2019. https://www.youtube.com/watch?v=Hbm0ub3qWw4&list=LL2HXH-vq_sTPXMNP4fZNKRw&index=10&t=0s [Online; accessed 01/10/2020].

[22] Matteo Campanelli, Mathias Hall-Andersen, and Simon Holmgaard Kamp. Curve Trees: Practical and Transparent Zero-Knowledge Accumulators. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4391–4408. USENIX Association, August 2023. Versão revista de Cryptology ePrint Archive, Report 2022/756, https://www.usenix.org/conference/usenixsecurity23/presentation/campanelli.

[23] Miles Carlsten, Harry Kalodner, Matthew Weinberg, and Arvind Narayanan. On the Instability of Bitcoin Without the Block Reward, 2016. http://randomwalker.info/publications/mining_CCS.pdf [Online; accessed 01/12/2020].

[24] Chainalysis. THE 2020 STATE OF CRYPTO CRIME, January 2020. https://go.chainalysis.com/rs/503-FAP-074/images/2020-Crypto-Crime-Report.pdf [Online; accessed 02/11/2020].

[25] David Chaum and Eugène Van Heyst. Group Signatures. In *Proceedings of the 10th Annual International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'91, pages 257–265, Berlin, Heidelberg, 1991. Springer-Verlag. https://chaum.com/publications/Group_Signatures.pdf [Online; accessed 03/04/2020].

[26] What is a premine? https://www.cryptocompare.com/coins/guides/what-is-a-premine/ [Online; accessed 06/11/2018].

[27] Cryptonote Address Tests. https://xmr.llcoins.net/addresstests.html [Online; accessed 04/19/2018].

[28] Cypher Stack. FCMP++ Composition Security Analysis, June 2024. Relatório e provas de segurança para a composição e a prova de autorização de gasto e ligação, revisão 8f6ac85ee5c5de8dc38a87788cc4ed4a9549108a, consultada em 6 de Setembro de 2026, https://github.com/cypherstack/fcmp-review/tree/8f6ac85ee5c5de8dc38a87788cc4ed4a9549108a.

[29] Cypher Stack. Generalized Bulletproofs Security Analysis, May 2024. Relatório e materiais em https://github.com/cypherstack/generalized-bulletproofs.

[30] Michael Davidson and Tyler Diamond. On the Profitability of Selfish Mining Against Multiple Difficulty Adjustment Algorithms. Cryptology ePrint Archive, Report 2020/094, 2020. https://eprint.iacr.org/2020/094 [Online; accessed 03/26/2020].

[31] W. Diffie and M. Hellman. New Directions in Cryptography. *IEEE Trans. Inf. Theor.*, 22(6):644–654, November 1976. https://ee.stanford.edu/~hellman/publications/24.pdf [Online; accessed 03/04/2020].

[32] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the Security of Two-Round Multi-Signatures. Cryptology ePrint Archive, Report 2018/417, 2018. https://eprint.iacr.org/2018/417 [Online; accessed 02/07/2020].

[33] Liam Eagen. Zero Knowledge Proofs of Elliptic Curve Inner Products from Principal Divisors and Weil Reciprocity. Cryptology ePrint Archive, Report 2022/596, 2022. https://eprint.iacr.org/2022/596.

[34] Justin Ehrenhofer. Justin Ehrenhofer - Improving Monero Release Schedule - DEF CON 27 Monero Village, December 2019. https://www.youtube.com/watch?v=MjNXmJUk2Jo timestamp 22:15 [Online; accessed 03/20/2020].

[35] Justin Ehrenhofer and knacc. Advisory note for users making use of subaddresses, October 2019. https://web.getmonero.org/2019/10/18/subaddress-janus.html [Online; accessed 01/02/2020].

[36] Exantech. Methods of anonymous blockchain analysis: an overview, November 2019. https://medium.com/@exantech/methods-of-anonymous-blockchain-analysis-an-overview-d700e27ea98c [Online; accessed 01/26/2020].

[37] Ittay Eyal and Emin Gün Sirer. Majority is not Enough: Bitcoin Mining is Vulnerable. *CoRR*, abs/1311.0243, 2013. https://arxiv.org/pdf/1311.0243.pdf [Online; accessed 03/04/2020].

[38] Giulia Fanti, Shaileshh Bojja Venkatakrishnan, Surya Bakshi, Bradley Denby, Shruti Bhargava, Andrew Miller, and Pramod Viswanath. Dandelion++: Lightweight Cryptocurrency Networking with Formal Anonymity Guarantees. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 2(2):29:1–29:34, June 2018. https://arxiv.org/abs/1805.11060.

[39] Amos Fiat and Adi Shamir. How To Prove Yourself: Practical Solutions to Identification and Signature Problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO' 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg. https://link.springer.com/content/pdf/10.1007%2F3-540-47721-7_12.pdf [Online; accessed 03/04/2020].

[40] Ryo "fireice_uk" Cryptocurrency. On-chain tracking of Monero and other Cryptonotes, April 2019. https://medium.com/@crypto_ryo/on-chain-tracking-of-monero-and-other-cryptonotes-e0afc6752527 [Online; accessed 03/25/2020].

[41] Riccardo "fluffypony" Spagni and luigi1111. Disclosure of a Major Bug in Cryptonote Based Currencies, May 2017. https://getmonero.org/2017/05/17/disclosure-of-a-major-bug-in-cryptonote-based-currencies.html [Online; accessed 04/10/2018].

[42] Eiichiro Fujisaki and Koutarou Suzuki. Traceable ring signature. In Tatsuaki Okamoto and Xiaoyun Wang, editors, *Public Key Cryptography – PKC 2007: 10th International Conference on Practice and Theory in Public-Key Cryptography Beijing, China, April 16-20, 2007. Proceedings*, pages 181–200. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. https://link.springer.com/content/pdf/10.1007%2F978-3-540-71677-8_13.pdf [Online; accessed 03/04/2020].

[43] Brandon Goodell, Sarang Noether, and Arthur Blue. Concise linkable ring signatures and applications, MRL-0011, September 2019. https://web.getmonero.org/resources/research-lab/pubs/MRL-0011.pdf [Online; accessed 02/02/2020].

[44] Brandon Goodell, Rigo Salazar, Freeman Slaughter, and Luke Szramowski. SLVer Bullet: Straight-Line Verification for Bulletproofs. Cryptology ePrint Archive, Report 2025/1345, 2025. https://eprint.iacr.org/2025/1345.

[45] hackerone #501585. https://hackerone.com/reports/501585 [Online; accessed 12/28/2019].

[46] Thomas C Hales. The NSA back door to NIST. *Notices of the AMS*, 61(2):190–192, 2014. https://www.ams.org/notices/201402/rnoti-p190.pdf [Online; accessed 03/04/2020].

[47] Darrel Hankerson, Alfred J. Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2003. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.394.3037&rep=rep1&type=pdf [Online; accessed 03/04/2020].

[48] Howard "hyc" Chu. RandomX, Pull Request #5549, May 2019. https://github.com/monero-project/monero/pull/5549 [Online; accessed 03/03/2020].

[49] cryptography - What is a cryptographic oracle? https://security.stackexchange.com/questions/10617/what-is-a-cryptographic-oracle [Online; accessed 04/22/2018].

[50] j-berman. [WIP] Full-Chain Membership Proof++ Integration, Pull Request #9436, August 2024. Pedido de integração em rascunho consultado em 6 de Setembro de 2026, https://github.com/monero-project/monero/pull/9436.

[51] j-berman. FCMP++ Integration Audit, April 2026. Proposta e estado dos marcos consultados em 31 de Agosto de 2026, https://ccs.getmonero.org/proposals/fcmp%2B%2B-integration-audit.html.

[52] Jeffro256. Carrot: Cryptonote Address on Rerandomizable-RingCT-Output Transactions. Especificação em desenvolvimento, 2026. Hierarquia de chaves, níveis de acesso e protocolo de endereçamento na revisão `c65a418c3860e28afcc8b8f75b3127069d830750`, consultada em 26 de Agosto de 2026, https://github.com/jeffro256/carrot/blob/c65a418c3860e28afcc8b8f75b3127069d830750/carrot.md.

[53] Aram Jivanyan. Lelantus: Towards Confidentiality and Anonymity of Blockchain Transactions from Standard Assumptions. Cryptology ePrint Archive, Report 2019/373, 2019. https://eprint.iacr.org/2019/373.pdf [Online; accessed 03/04/2020].

[54] Don Johnson, Alfred Menezes, and Scott A. Vanstone. The Elliptic Curve Digital Signature Algorithm (ECDSA). Technical Report CORR 99-34, Dept. of C&O, University of Waterloo, Canada, 1999. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.472.9475&rep=rep1&type=pdf [Online; accessed 04/04/2018].

[55] JollyMort. Monero Dynamic Block Size and Dynamic Minimum Fee, March 2017. https://github.com/JollyMort/monero-research/blob/master/Monero%20Dynamic%20Block%20Size%20and%20Dynamic%20Minimum%20Fee/Monero%20Dynamic%20Block%20Size%20and%20Dynamic%20Minimum%20Fee%20-%20DRAFT.md [Online; accessed 01/12/2020].

[56] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, January 2017. https://rfc-editor.org/rfc/rfc8032.txt [Online; accessed 03/04/2020].

[57] Simon Josefsson and Niels Möller. EdDSA and Ed25519. Internet-Draft draft-josefsson-eddsa-ed25519-03, May 2015. https://datatracker.ietf.org/doc/html/draft-josefsson-eddsa-ed25519-03 [Online; accessed 05/11/2018].

[58] kenshi84. Subaddresses, Pull Request #2056, May 2017. https://github.com/monero-project/monero/pull/2056 [Online; accessed 02/16/2020].

[59] Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal Security Proofs for Signatures from Identification Schemes. In *Proceedings, Part II, of the 36th Annual International Cryptology Conference on Advances in Cryptology — CRYPTO 2016 - Volume 9815*, pages 33–61, Berlin, Heidelberg, 2016. Springer-Verlag. https://eprint.iacr.org/2016/191.pdf [Online; accessed 03/04/2020].

[60] Alexander Klimov. ECC Patents?, October 2005. http://article.gmane.org/gmane.comp.encryption.general/7522 [Online; accessed 04/04/2018].

[61] koe and jtgrassie. Historical significance of FEE_PER_KB_OLD, December 2019. https://monero.stackexchange.com/questions/11864/historical-significance-of-fee-per-kb-old [Online; accessed 01/02/2020].

[62] koe and jtgrassie. Complete extra field structure (standard interpretation), January 2020. https://monero.stackexchange.com/questions/11888/complete-extra-field-structure-standard-interpretation [Online; accessed 01/05/2020].

[63] Mitchell Krawiec-Thayer. Numerical simulation for upper bound on dynamic blocksize expansion, January 2019. https://github.com/noncesense-research-lab/Blockchain_big_bang/blob/master/models/Isthmus_Bx_big_bang_model.ipynb [Online; accessed 01/08/2020].

[64] Russell W. F. Lai, Viktoria Ronge, Tim Ruffing, Dominique Schröder, Sri Thyagarajan, and Jiafan Wang. Omniring: Scaling Private Payments Without Trusted Setup. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 31–48, November 2019. https://eprint.iacr.org/2019/580.pdf [Online; accessed 03/04/2020].

[65] Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. Linkable spontaneous anonymous group signature for ad hoc groups. In Huaxiong Wang, Josef Pieprzyk, and Vijay Varadharajan, editors, *Information Security and Privacy: 9th Australasian Conference, ACISP 2004, Sydney, Australia, July 13-15, 2004. Proceedings*, pages 325–335. Springer Berlin Heidelberg, Berlin, Heidelberg, 2004. https://eprint.iacr.org/2004/027.pdf [Online; accessed 03/04/2020].

[66] Jan Macheta, Sarang Noether, Surae Noether, and Javier Smooth. Counterfeiting via Merkle Tree Exploits within Virtual Currencies Employing the CryptoNote Protocol, MRL-0002, September 2014. https://web.getmonero.org/resources/research-lab/pubs/MRL-0002.pdf [Online; accessed 05/27/2018].

[67] Ueli Maurer. Unifying Zero-Knowledge Proofs of Knowledge. In Bart Preneel, editor, *Progress in Cryptology – AFRICACRYPT 2009*, pages 272–286, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. https://www.crypto.ethz.ch/publications/files/Maurer09.pdf [Online; accessed 03/04/2020].

[68] Greg Maxwell. Confidential Transactions. https://people.xiph.org/~greg/confidential_values.txt [Online; accessed 04/04/2018].

[69] Gregory Maxwell and Andrew Poelstra. Borromean Ring Signatures. Blockstream technical draft, 2015. https://github.com/Blockstream/borromean_paper/blob/master/borromean_draft_0.01_9ade1e49.pdf [Online; accessed 04/04/2018].

[70] Sarah Meiklejohn and Claudio Orlandi. Privacy-Enhancing Overlays in Bitcoin. https://fc15.ifca.ai/preproceedings/bitcoin/paper_5.pdf [Online; accessed 01/26/2020].

[71] R. C. Merkle. Protocols for Public Key Cryptosystems. In *1980 IEEE Symposium on Security and Privacy*, pages 122–122, April 1980. http://www.merkle.com/papers/Protocols.pdf [Online; accessed 03/04/2020].

[72] Andrew Miller, Malte Möser, Kevin Lee, and Arvind Narayanan. An Empirical Analysis of Linkability in the Monero Blockchain. *CoRR*, abs/1704.04299, 2017. https://arxiv.org/pdf/1704.04299.pdf [Online; accessed 03/04/2020].

[73] Victor S Miller. Use of Elliptic Curves in Cryptography. In *Lecture Notes in Computer Sciences; 218 on Advances in cryptology—CRYPTO 85*, pages 417–426, Berlin, Heidelberg, 1986. Springer-Verlag. https://link.springer.com/content/pdf/10.1007/3-540-39799-X_31.pdf [Online; accessed 03/04/2020].

[74] Nicola Minichiello. The Bitcoin Big Bang: Tracking Tainted Bitcoins, June 2015. https://bravenewcoin.com/insights/the-bitcoin-big-bang-tracking-tainted-bitcoins [Online; accessed 03/31/2020].

[75] Ludwig von Mises. *Human Action: A Treatise on Economics*. Ludwig von Mises Institute, Auburn, Alabama, scholar's edition, 1998. https://cdn.mises.org/Human%20Action_3.pdf [Online; accessed 03/05/2020].

[76] Fungible, July 2014. https://wiki.mises.org/wiki/Fungible [Online; accessed 03/31/2020].

[77] Monero v0.9.3 - Hydrogen Helix - released! https://www.reddit.com/r/Monero/comments/4bgw4z/monero_v093_hydrogen_helix_released_urgent_and/ [Online; accessed 05/24/2018].

[78] Monero 0.13.0 "Beryllium Bullet" Release. https://www.getmonero.org/2018/10/11/monero-0.13.0-released.html [Online; accessed 10/12/2019].

[79] Monero 0.14.0 "Boron Butterfly" Release. https://web.getmonero.org/2019/02/25/monero-0.14.0-released.html [Online; accessed 10/12/2019].

[80] Useful for learning about Monero coin emission. https://www.reddit.com/r/Monero/comments/512kwh/useful_for_learning_about_monero_coin_emission/d78tpgi/ [Online; accessed 05/25/2018].

[81] Monero cryptonight variants, and add one for v7, April 2018. https://github.com/monero-project/monero/pull/3253 [Online; accessed 05/23/2018].

[82] Monero inception and history. https://monero.stackexchange.com/questions/475/monero-inception-and-history-how-did-monero-get-started-what-are-its-origins-a/476#476 [Online; accessed 05/23/2018].

[83] Randomx. https://www.monerooutreach.org/stories/RandomX.php [Online; accessed 10/12/2019].

[84] Monero Project. Tail emission, 2022. https://www.getmonero.org/get-started/faq/ [Online; accessed 08/29/2026].

[85] Monero Project. Curve Trees leaf preparation and unified output ordering, 2026. Fontes `src/fcmp_pp/curve_trees.*` e `fcmp_pp_types.h`, revisão `3d3920d7487b5df7ac388b6b8577fd04d505885f`, consultada em 6 de Setembro de 2026, https://github.com/monero-project/monero/tree/3d3920d7487b5df7ac388b6b8577fd04d505885f/src/fcmp_pp.

[86] Monero Project. FCMP++ Integration Milestone, 2026. Estado de integração consultado em 6 de Setembro de 2026, https://github.com/monero-project/monero/milestone/1.

[87] Monero Project. Fix FCMP++ generator definitions, Pull Request #10963, August 2026. Correcção integrada em 23 de Agosto de 2026, https://github.com/monero-project/monero/pull/10963.

[88] Monero Project. Mainnet hard-fork schedule and RingCT type definitions, 2026. Fontes do ramo principal na revisão `3d3920d7487b5df7ac388b6b8577fd04d505885f`, consultadas em 6 de Setembro de 2026: https://github.com/monero-project/monero/tree/3d3920d7487b5df7ac388b6b8577fd04d505885f.

[89] Monero Project. Monero source code, August 2026. Revisão 3646f648db57f60cca86430e25a635d19fa9b92a, datada de 14 de Agosto de 2026, https://github.com/monero-project/monero/tree/3646f648db57f60cca86430e25a635d19fa9b92a.

[90] Monero Project. Pruning — Moneropedia, 2026. Documentação oficial consultada em Agosto de 2026, https://www.getmonero.org/resources/moneropedia/pruning.html.

[91] Ring CT; Moneropedia. https://getmonero.org/resources/moneropedia/ringCT.html [Online; accessed 06/05/2018].

[92] How do payment ids work? https://monero.stackexchange.com/questions/1910/how-do-payment-ids-work [Online; accessed 04/21/2018].

[93] What is the block maturity value seen in many pool interfaces? https://monero.stackexchange.com/questions/2251/what-is-the-block-maturity-value-seen-in-many-pool-interfaces [Online; accessed 05/26/2018].

[94] Add intervening v5 fork for increased min block size. https://github.com/monero-project/monero/pull/1869 [Online; accessed 05/24/2018].

[95] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. http://bitcoin.org/bitcoin.pdf [Online; accessed 03/04/2020].

[96] Arvind Narayanan and Malte Möser. Obfuscation in Bitcoin: Techniques and Politics. *CoRR*, abs/1706.05432, 2017. https://arxiv.org/ftp/arxiv/papers/1706/1706.05432.pdf [Online; accessed 03/04/2020].

[97] National Institute of Standards and Technology. NIST Releases SHA-3 Cryptographic Hash Standard, August 2015. https://www.nist.gov/news-events/news/2015/08/nist-releases-sha-3-cryptographic-hash-standard [Online; accessed 06/02/2018].

[98] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple Two-Round Schnorr Multi-Signatures. In *Advances in Cryptology – CRYPTO 2021*, volume 12825 of *Lecture Notes in Computer Science*, pages 189–221. Springer, 2021. Cryptology ePrint Archive, Report 2020/1261, https://eprint.iacr.org/2020/1261.

[99] Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 7539, May 2015. https://www.rfc-editor.org/info/rfc7539 [Online; accessed 05/11/2018].

[100] Sarang Noether. Janus mitigation, Issue #62, January 2020. https://github.com/monero-project/research-lab/issues/62 [Online; accessed 02/17/2020].

[101] Sarang Noether and Brandon Goodell. An efficient implementation of Monero subaddresses, MRL-0006, October 2017. https://web.getmonero.org/resources/research-lab/pubs/MRL-0006.pdf [Online; accessed 04/04/2018].

[102] Sarang Noether and Brandon Goodell. Triptych: logarithmic-sized linkable ring signatures with applications. Cryptology ePrint Archive, Report 2020/018, 2020. https://eprint.iacr.org/2020/018.pdf [Online; accessed 03/04/2020].

[103] Shen Noether. Understanding ge_fromfe_frombytes_vartime. https://web.getmonero.org/resources/research-lab/pubs/ge_fromfe.pdf [Online; accessed 01/20/2020].

[104] Shen Noether, Adam Mackenzie, and Monero Core Team. Ring Confidential Transactions, MRL-0005, February 2016. https://web.getmonero.org/resources/research-lab/pubs/MRL-0005.pdf [Online; accessed 06/15/2018].

[105] Shen Noether and Sarang Noether. Monero is Not That Mysterious, MRL-0003, September 2014. https://web.getmonero.org/resources/research-lab/pubs/MRL-0003.pdf [Online; accessed 06/15/2018].

[106] Shen Noether and Sarang Noether. Thring Signatures and their Applications to Spender-Ambiguous Digital Currencies, MRL-0009, November 2018. https://web.getmonero.org/resources/research-lab/pubs/MRL-0009.pdf [Online; accessed 01/15/2020].

[107] Michael Padilla. Beating Bitcoin bad guys, August 2016. http://www.sandia.gov/news/publications/labnews/articles/2016/19-08/bitcoin.html [Online; accessed 04/04/2018].

[108] Luke Parker. FCMP++: Full-Chain Membership Proofs for Monero. Especificação em desenvolvimento, 2024. Ramo `develop`, revisão `6f01fd3296b17f90b5c97c738cc3320e3369ad49`, consultada em 6 de Setembro de 2026, https://github.com/kayabaNerve/fcmp-plus-plus-paper/tree/6f01fd3296b17f90b5c97c738cc3320e3369ad49.

[109] Luke Parker and contributors. monero-oxide FCMP++ implementation, 2026. Fotografia do ramo `fcmp++`, revisão `31c26d96eaadbba910ffe3613ad8b4cf9c598a93`, 18 de Agosto de 2026, https://github.com/monero-oxide/monero-oxide/tree/31c26d96eaadbba910ffe3613ad8b4cf9c598a93.

[110] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91: Proceedings*, pages 129–140. Springer Berlin Heidelberg, Berlin, Heidelberg, 1992. https://www.cs.cornell.edu/courses/cs754/2001fa/129.PDF [Online; accessed 03/04/2020].

[111] RandomX contributors. RandomX Security Audits, 2019. Relatórios de Trail of Bits, X41 D-Sec, Kudelski Security e QuarksLab, https://github.com/tevador/RandomX/tree/12f2c2ffe2108d6cf54c391fee33c8bc3646cdab/audits.

[112] RandomX contributors. RandomX version 2 releases, 2026. Versão algorítmica posterior, publicada separadamente e não incorporada na fotografia de consenso Monero descrita neste livro, https://github.com/tevador/RandomX/releases/tag/v2.0.1.

[113] René "rbrunner7" Brunner. Multisig transactions with MMS and CLI wallet. https://web.getmonero.org/resources/user-guides/multisig-messaging-system.html [Online; accessed 01/21/2020].

[114] René "rbrunner7" Brunner. Project Rationale From the Initiator, January 2018. https://taiga.getmonero.org/project/rbrunner7-really-simple-multisig-transactions/wiki/home [Online; accessed 01/21/2020].

[115] René "rbrunner7" Brunner. Basic Monero support ready for assessment, Issue #1638, June 2019. https://github.com/OpenBazaar/openbazaar-go/issues/1638 [Online; accessed 02/11/2020].

[116] Jamie Redman. Industry Execs Claim Freshly Minted 'Virgin Bitcoins' Fetch 20% Premium, March 2020. https://news.bitcoin.com/industry-execs-freshly-minted-virgin-bitcoins/ [Online; accessed 03/31/2020].

[117] Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to Leak a Secret. C. Boyd (Ed.): ASIACRYPT 2001, LNCS 2248, pp. 552-565, 2001. https://people.csail.mit.edu/rivest/pubs/RST01.pdf [Online; accessed 04/04/2018].

[118] SafeCurves. SafeCurves: choosing safe curves for elliptic-curve cryptography, 2013. https://safecurves.cr.yp.to/rigid.html [Online; accessed 03/25/2020].

[119] C. P. Schnorr. Efficient Identification and Signatures for Smart Cards. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 239–252, New York, NY, 1990. Springer New York. https://link.springer.com/content/pdf/10.1007%2F0-387-34805-0_22.pdf [Online; accessed 03/04/2020].

[120] Paola Scozzafava. Uniform distribution and sum modulo m of independent random variables. *Statistics & Probability Letters*, 18(4):313 – 314, 1993. https://doi.org/10.1016/0167-7152(93)90021-A.

[121] Bassam El Khoury Seguias. Monero Building Blocks, 2018. https://delfr.com/category/monero/ [Online; accessed 10/28/2018].

[122] Seigen, Max Jameson, Tuomo Nieminen, Neocortex, and Antonio M. Juarez. CryptoNight Hash Function. CryptoNote, March 2013. https://cryptonote.org/cns/cns008.txt [Online; accessed 04/04/2018].

[123] SerHack and Monero Community. *Mastering Monero: The Future of Private Transactions*. LernoLibro LLC, December 2018. https://masteringmonero.com/ [Online; accessed 01/10/2020].

[124] QingChun ShenTu and Jianping Yu. Research on Anonymization and De-anonymization in the Bitcoin System. *CoRR*, abs/1510.07782, 2015. https://arxiv.org/ftp/arxiv/papers/1510/1510.07782.pdf [Online; accessed 03/04/2020].

[125] Tevador and RandomX contributors. RandomX Design, 2026. Descrição, análise e resultados empíricos da versão 1, na revisão incorporada pelo Monero, https://github.com/tevador/RandomX/blob/12f2c2ffe2108d6cf54c391fee33c8bc3646cdab/doc/design.md.

[126] Tevador and RandomX contributors. RandomX Specification, version 1, 2026. Especificação na revisão `12f2c2ffe2108d6cf54c391fee33c8bc3646cdab` (versão 1.2.3), incorporada como submódulo pelo código Monero consultado, https://github.com/tevador/RandomX/blob/12f2c2ffe2108d6cf54c391fee33c8bc3646cdab/doc/specs.md.

[127] thankful_for_today. [ANN][BMR] Bitmonero - a new coin based on CryptoNote technology - LAUNCHED, April 2014. Monero's actual launch date was April 18$^{th}$, 2014. https://bitcointalk.org/index.php?topic=563821.0 [Online; accessed 05/24/2018].

[128] Trail of Bits. Monero FCMP++ Cryptography Implementation Audited by Trail of Bits, August 2026. Resumo e ligação para o relatório de auditoria em https://magicgrants.org/2026/08/17/Monero-FCMP-Cryptography-Implementation-ToB.

[129] Manny Trillo. Visa Transactions Hit Peak on Dec. 23, January 2011. https://www.visa.com/blogarchives/us/2011/01/12/visa-transactions-hit-peak-on-dec-23/index.html [Online; accessed 01/10/2020].

[130] UkoeHB. Proof an output has not been spent, Issue #68, January 2020. https://github.com/monero-project/research-lab/issues/68 [Online; accessed 02/19/2020].

[131] Maciej Ulas. Rational points on certain hyperelliptic curves over finite fields, 2007. https://arxiv.org/pdf/0706.1448.pdf [Online; accessed 03/03/2020].

[132] user36100 and cooldude45. Which entities are related to Bytecoin and Minergate?, December 2016. https://monero.stackexchange.com/questions/2930/which-entities-are-related-to-bytecoin-and-minergate [Online; accessed 01/17/2020].

[133] Nicolas van Saberhagen. CryptoNote V2.0, 2013. https://cryptonote.org/whitepaper.pdf [Online; accessed 04/04/2018].

[134] Veridise. Full-Chain Membership Proofs Audit, June 2025. Auditoria do circuito, da aritmética e da implementação, incluindo verificação formal de dispositivos não interactivos com Picus, https://veridise.com/audits-archive/company/monero-research-lab/magic-grants-monero-fcmp-2025-06-03.

[135] Adam “waxwing” Gibson. From Zero (Knowledge) To Bulletproofs, March 2018. https://github.com/AdamISZ/from0k2bp/blob/master/from0k2bp.pdf [Online; accessed 03/01/2020].

[136] Albert Werner, Montag, Prometheus, and Tereno. CryptoNote Transaction Extra Field. CryptoNote, October 2012. https://cryptonote.org/cns/cns005.txt [Online; accessed 04/04/2018].

[137] Wikibooks. Cryptography/Prime Curve/Standard Projective Coordinates, March 2011. https://en.wikibooks.org/wiki/Cryptography/Prime_Curve/Standard_Projective_Coordinates [Online; accessed 03/03/2020].

[138] Diffie–Hellman problem, December 2019. https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_problem [Online; accessed 03/31/2020].

[139] Directed acyclic graph. https://en.wikipedia.org/wiki/Directed_acyclic_graph [Online; accessed 05/27/2018].

[140] Extended Euclidean algorithm. https://en.wikipedia.org/wiki/Extended_Euclidean_algorithm [Online; accessed 03/19/2020].

[141] Modular arithmetic. https://en.wikipedia.org/wiki/Modular_arithmetic [Online; accessed 04/16/2018].

[142] Peter Woit. Trust the math? An Update, December 2013. https://www.math.columbia.edu/~woit/wordpress/?p=6522 [Online; accessed 04/04/2018].

[143] XOR – from Wolfram Mathworld. http://mathworld.wolfram.com/XOR.html [Online; accessed 04/21/2018].

[144] Tsz Hon Yuen, Shi-feng Sun, Joseph K. Liu, Man Ho Au, Muhammed F. Esgin, Qingzhao Zhang, and Dawu Gu. RingCT 3.0 for Blockchain Confidential Transaction: Shorter Size and Stronger Security. Cryptology ePrint Archive, Report 2019/508, 2019. https://eprint.iacr.org/2019/508.pdf [Online; accessed 03/04/2020].

[145] zawy12. Summary of Difficulty Algorithms, Issue #50, December 2019. https://github.com/zawy12/difficulty-algorithms/issues/50 [Online; accessed 02/11/2020].

[146] zkSecurity. Notes on Divisor Techniques for Elliptic-Curve Proofs, 2026. Revisão técnica em https://blog.zksecurity.xyz/posts/divisor-notes/divisor-techniques.pdf.

# Appendices

# APÊNDICE A

# Exemplo histórico: estrutura de transacção `RCTTypeBulletproof2`

Apresenta-se neste apêndice uma transacção de Monero real do tipo `RCTTypeBulletproof2`, juntamente com explicações dos seus campos relevantes. É um exemplo histórico, incluído no bloco 2 045 821, em 2020, quando a versão 12 estava activa. Os seus anéis de onze membros, as assinaturas MLSAG, os Bulletproofs clássicos e os alvos sem etiquetas de visualização não descrevem uma nova transacção v16. O formato activo é `RCTTypeBulletproofPlus`, explicado no capítulo 7.

Esta transacção foi obtida do explorador de blocos `https://xmrchain.net`. Também se pode obter ao executar o comando `print_tx <TransactionID> +hex +json` no *daemon* `monerod` (sem o parâmetro *detached*). Aqui, `<TransactionID>` é o *hash* da transacção (secção 8.4.1). A primeira linha mostra o comando executado. Para replicar estes resultados, o leitor faz o seguinte:

1. É preciso o programa de linha de comandos de Monero, disponível no sítio oficial de descargas.[1] Descarregue a versão para o seu sistema operativo e extraia o ficheiro.
2. Abra um terminal e navegue para o directório criado pela extracção.
3. Corra `monerod` com `./monerod`. O nó descarrega e valida a lista de blocos na máquina local; o comando abaixo só poderá encontrar a transacção depois de a sincronização alcançar a altura necessária.

[1] `https://web.getmonero.org/downloads/`

4. Depois de o processo de sincronização estar concluído, é possível executar comandos como `print_tx`. Use `help` para conhecer outros comandos.

O componente `rctsig_prunable` é, como o nome indica, podável. Um nó podado pode omitir estes dados antigos e conservar a parte não podável e o *hash* da parte podável necessário para reconstruir o identificador da transacção. A raiz de Merkle do bloco compromete-se com os identificadores das transacções; não se substitui literalmente este campo pela raiz da árvore.

As imagens de chave ficam nas entradas do prefixo, na área não podável. São cruciais para detectar gastos duplos e não podem ser podadas.

Esta transacção exemplo tem duas entradas e duas saídas, e foi adicionada à lista de blocos com o carimbo temporal 2020-03-02 19:01:10 UTC.

```
print_tx 84799c2fc4c18188102041a74cef79486181df96478b717e8703512c7f7f3349
Found in blockchain at height 2045821
{
  "version": 2,
  "unlock_time": 0,
  "vin": [ {
      "key": {
        "amount": 0,
        "key_offsets": [ 14401866, 142824, 615514, 18703, 5949, 22840, 5572, 16439,
        983, 4050, 320
        ],
        "k_image": "c439b9f0da76ca0bb17920ca1f1f3f1d216090751752b091bef9006918cb3db4"
      }
    }, {
      "key": {
        "amount": 0,
        "key_offsets": [ 14515357, 640505, 8794, 1246, 20300, 18577, 17108, 9824, 581,
        637, 1023
        ],
        "k_image": "03750c4b23e5be486e62608443151fa63992236910c41fa0c4a0a938bc6f5a37"
      }
    }
  ],
  "vout": [ {
      "amount": 0,
      "target": {
        "key": "d890ba9ebfa1b44d0bd945126ad29a29d8975e7247189e5076c19fa7e3a8cb00"
      }
```

```
    }, {
      "amount": 0,
      "target": {
        "key": "dbec330f8a67124860a9bfb86b66db18854986bd540e710365ad6079c8a1c7b0"
      }
    }
  ],
  "extra": [ 1, 3, 39, 58, 185, 169, 82, 229, 226, 22, 101, 230, 254, 20, 143,
  37, 139, 28, 114, 77, 160, 229, 250, 107, 73, 105, 64, 208, 154, 182, 158, 200,
  73, 2, 9, 1, 12, 76, 161, 40, 250, 50, 135, 231
  ],
  "rct_signatures": {
    "type": 4,
    "txnFee": 32460000,
    "ecdhInfo": [ {
        "amount": "171f967524e29632"
      }, {
        "amount": "5c2a1a9f54ccf40b"
      }],
    "outPk": [ "fed8aded6914f789b63c37f9d2eb5ee77149e1aa4700a482aea53f82177b3b41",
    "670e086e40511a279e0e4be89c9417b4767251c5a68b4fc3deb80fdae7269c17"]
  },
  "rctsig_prunable": {
    "nbp": 1,
    "bp": [ {
        "A": "98e5f23484e97bb5b2d453505db79caadf20dc2b69dd3f2b3dbf2a53ca280216",
        "S": "b791d4bc6a4d71de5a79673ed4a5487a184122321ede0b7341bc3fdc0915a796",
        "T1": "5d58cfa9b69ecdb2375647729e34e24ce5eb996b5275aa93f9871259f3a1aecd",
        "T2": "1101994fea209b71a2aa25586e429c4c0f440067e2b197469aa1a9a1512f84b7",
        "taux": "b0ad39da006404ccacee7f6d4658cf17e0f42419c284bdca03c0250303706c03",
        "mu": "cacd7ca5404afa28e7c39918d9f80b7fe5e572a92a10696186d029b4923fa200",
        "L": [ "d06404fc35a60c6c47a04e2e43435cb030267134847f7a49831a61f82307fc32",
        "c9a5932468839ee0cda1aa2815f156746d4dce79dab3013f4c9946fce6b69eff",
        "efdae043dcedb79512581480d80871c51e063fe9b7a5451829f7a7824bcc5a0b",
        "56fd2e74ac6e1766cfd56c8303a90c68165a6b0855fae1d5b51a2e035f333a1d",
        "81736ed768f57e7f8d440b4b18228d348dce1eca68f969e75fab458f44174c99",
        "695711950e076f54cf24ad4408d309c1873d0f4bf40c449ef28d577ba74dd86d",
        "4dc3147619a6c9401fec004652df290800069b776fe31b3c5cf98f64eb13ef2c"
        ],
        "R": [ "7650b8da45c705496c26136b4c1104a8da601ea761df8bba07f1249495d8f1ce",
        "e87789fbe99a44554871fcf811723ee350cba40276ad5f1696a62d91a4363fa6",
```

```
        "ebf663fe9bb580f0154d52ce2a6dae544e7f6fb2d3808531b0b0749f5152ddbf",
        "5a4152682a1e812b196a265a6ba02e3647a6bd456b7987adff288c5b0b556ec6",
        "dc0dcb2e696e11e4b62c20b6bfcb6182290748c5de254d64bf7f9e3c38fb46c9",
        "101e2271ced03b229b88228d74b36088b40c88f26db8b1f9935b85fb3ab96043",
        "b14aae1d35c9b176ac526c23f31b044559da75cf95bc640d1005bfcc6367040b"
        ],
        "a": "4809857de0bd6becdb64b85e9dfbf6085743a8496006b72ceb81e01080965003",
        "b": "791d8dc3a4ddde5ba2416546127eb194918839ced3dea7399f9c36a17f36150e",
        "t": "aace86a7a1cbdec3691859fa07fdc83eed9ca84b8a064ca3f0149e7d6b721c03"
      }
    ],
    "MGs": [ {
        "ss": [[ "d7a9b87cfa74ad5322eedd1bff4c4dca08bcff6f8578a29a8bc4ad6789dee106",
        "f08e5dfade29d2e60e981cb561d749ea96ddc7e6855f76f9b807842d1a17fe00"],
        ["de0a86d12be2426f605a5183446e3323275fe744f52fb439041ad2d59136ea0b",
        "0028f97976630406e12c54094cbbe23a23fe5098f43bcae37339bfc0c4c74c07"],
        ["f6eef1f99e605372cb1ec2b3dd4c6e56a550fec071c8b1d830b9fda34de5cc05",
        "cd98fc987374a0ac993edf4c9af0a6f2d5b054f2af601b612ea118f405303306"],
        ["5a8437575dae7e2183a1c620efbce655f3d6dc31e64c96276f04976243461e08",
        "5090103f7f73a33024fbda999cd841b99b87c45fa32c4097cdc222fa3d7e9502"],
        ["88d34246afbccbd24d2af2ba29d835813634e619912ea4ca194a32281ac14e0e",
        "eacdf59478f132dd8dbb9580546f96de194092558ffceeff410ee9eb30ce570e"],
        ["571dab8557921bbae30bda9b7e613c8a0cff378d1ec6413f59e4972f30f2470d",
        "5ca78da9a129619299304d9b03186233370023debfdaddcd49c1a338c1f0c50d"],
        ["ac8dbe6bb28839cf98f02908bd1451742a10c713fdd51319f2d42a58bf1d7507",
        "7347bf16cba5ee6a6f2d4f6a59d1ed0c1a43060c3a235531e7f1a75cd8c8530d"],
        ["b8876bd3a5766150f0fbc675ba9c774c2851c04afc4de0b17d3ac4b6de617402",
        "e39f1d2452d76521cbf02b85a6b626eeb5994f6f28ce5cf81adc0ff2b8adb907"],
        ["1309f8ead30b7be8d0c5932743b343ef6c0001cef3a4101eae98ffde53f46300",
        "370693fa86838984e9a7232bca42fd3d6c0c2119d44471d61eee5233ba53c20f"],
        ["80bc2da5fc5951f2c7406fce37a7aa72ffef9cfa21595b1b68dfab4b7b9f9f0c",
        "c37137898234f00bce00746b131790f3223f97960eefe67231eb001092f5510c"],
        ["01c89e07571fd365cac6744b34f1b44e06c1c31cbf3ee4156d08309345fdb20e",
        "a35c8786695a86c0a4e677b102197a11dadc7171dd8c2e1de90d828f050ec00f"]],
        "cc": "0d8b70c600c67714f3e9a0480f1ffc7477023c793752c1152d5df0813f75ff0f"
      }, {
        "ss": [[ "4536e585af58688b69d932ef3436947a13d2908755d1c644ca9d6a978f0f0206",
        "9aab6509f4650482529219a805ee09cd96bb439ee1766ced5d3877bf1518370b"],
        ["5849d6bf0f850fcee7acbef74bd7f02f77ecfaaa16a872f52479ebd27339760f",
        "96a9ec61486b04201313ac8687eaf281af59af9fd10cf450cb26e9dc8f1ce804"],
        ["7fe5dcc4d3eff02fca4fb4fa0a7299d212cd8cd43ec922d536f21f92c8f93f00",
```

```
111         "d306a62831b49700ae9daad44fcd00c541b959da32c4049d5bdd49be28d96701"],
112         ["2edb125a5670d30f6820c01b04b93dd8ff11f4d82d78e2379fe29d7a68d9c103",
113         "753ac25628c0dada7779c6f3f13980dfc5b7518fb5855fd0e7274e3075a3410c"],
114         ["264de632d9cb867e052f95007dfdf5a199975136c907f1d6ad73061938f49c01",
115         "dd7eb6028d0695411f647058f75c42c67660f10e265c83d024c4199bed073d01"],
116         ["b2ac07539336954f2e9b9cba298d4e1faa98e13e7039f7ae4234ac801641340f",
117         "69e130422516b82b456927b64fe02732a3f12b5ee00c7786fe2a381325bf3004"],
118         ["49ea699ca8cf2656d69020492cdfa69815fb69145e8f922bb32e358c23cebb0f",
119         "c5706f903c04c7bed9c74844f8e24521b01bc07b8dbf597621cceeeb3afc1d0c"],
120         ["a1faf85aa942ba30b9f0511141fcab3218c00953d046680d36e09c35c04be905",
121         "7b6b1b6fb23e0ee5ea43c2498ea60f4fcf62f70c7e0e905eb4d9afa1d0a18800"],
122         ["785d0993a70f1c2f0ac33c1f7632d64e34dd730d1d8a2fb0606f5770ed633506",
123         "e12777c49ffc3f6c35d27a9ccb3d9b8fed7f0864a880f7bae7399e334207280e"],
124         ["ab31972bf1d2f904d6b0bf18f4664fa2b16a1fb2644cd4e6278b63ade87b6d09",
125         "1efb04fe9a75c01a0fe291d0ae00c716e18c64199c1716a086dd6e32f63e0a07"],
126         ["a6f4e21a27bf8d28fc81c873f63f8d78e017666adbf038da0b83c2ad04ef6805",
127         "c02103455f93c2d7ec4b7152db7de00d1c9e806b1945426b6773026b4a85dd03"]],
128         "cc": "d5ac037bb78db41cf924af713b7379c39a4e13901d3eac017238550a1a3b910a"
129       }],
130     "pseudoOuts": [ "b313c1ae9ca06213684fbdefa9412f4966ad192bc0b2f74ed1731381adb7ab58",
131     "7148e7ef5cfd156c62a6e285e5712f8ef123575499ff9a11f838289870522423"]
132   }
133 }
```

## Componentes de transacção

As linhas 2–132 misturam três camadas: o prefixo da transacção, a parte RingCT não podável e a parte RingCT podável. A enumeração seguinte conserva os nomes do JSON, mas diz expressamente a que época pertencem.

- A linha 2, acrescentada por `print_tx`, indica a altura em que o nó encontrou a transacção. Não pertence à sua serialização.
- `version` (linha 4) é a versão de serialização. O valor ‘2’ abrange os formatos RingCT históricos e o formato activo.
- `unlock_time` (linha 5) impõe um limite comum às saídas. Um valor inferior a 500 000 000 representa uma altura; a partir daí, representa tempo Unix. Zero não acrescenta bloqueio.
- `vin` (linhas 6–23) contém as duas entradas. Em cada uma, `amount`=0 porque o montante RingCT não é publicado.

- `key_offsets` codifica os índices globais dos membros do anel. O primeiro é absoluto; os restantes são diferenças. Assim, $\{7, 11, 15, 20\}$ torna-se $\{7, 4, 4, 5\}$. Os onze elementos de cada lista pertencem à regra da época; uma transacção v16 usa dezasseis membros (secção 7.2.1).

- `k_image` é a imagem de chave da entrada. Aqui é ligada pela MLSAG; numa transacção activa, a mesma posição do prefixo contém a imagem ligada pela CLSAG.

- `vout` (linhas 24–35) contém duas saídas. O campo visível `amount` é zero e `key` é o endereço oculto da secção 5.2. O alvo histórico não tem a etiqueta de vista obrigatória nas saídas v16.

- `extra` (linhas 36–39) contém bytes interpretados por etiquetas. A etiqueta '1' introduz a chave pública de transacção $rG$, de 32 bytes e comprimento implícito. A etiqueta '2' introduz um *nonce* de nove bytes: o primeiro, '1', identifica um ID de pagamento cifrado; os oito seguintes contêm-no. Estas são convenções da carteira sujeitas aos limites de consenso explicados no capítulo 7; vejam-se [62, 136]. `src/cryptonote_basic/tx_extra.h`

- `rct_signatures` (linhas 40–50) é a parte RingCT não podável. O campo `type` vale '4', isto é, o tipo histórico `RCTTypeBulletproof2`. O caminho v16 usa o valor '6', correspondente a `RCTTypeBulletproofPlus`. Finalmente, `txnFee`=32 460 000 unidades atómicas, ou 0,00003246 XMR.

- `ecdhInfo` contém, para cada saída, o montante cifrado segundo a construção da secção 6.3. `outPk` contém os dois compromissos de saída da secção 6.4.

- `rctsig_prunable` (linhas 51–132) é a parte podável. O campo `nbp`=1 anuncia uma única Bulletproof agregada para as duas saídas, com
$$\Pi_{\mathrm{BP}} = (A, S, T_1, T_2, \tau_x, \mu, \mathbb{L}, \mathbb{R}, a, b, t).$$

- `MGs` contém duas MLSAG, uma por entrada. Em cada assinatura, `ss` guarda as respostas escalares e `cc` o desafio inicial. Uma transacção v16 teria `CLSAGs`, com outra estrutura.

- `pseudoOuts` contém os pseudo-compromissos $\widetilde{C}^a_j$. A equação verificável é
$$\sum_j \widetilde{C}^a_j = \sum_t C^b_t + fH.$$
Nenhum dos lados revela os montantes individuais. Em conjunto com a vinculação dos compromissos, as provas de intervalo e as assinaturas das entradas, esta igualdade exclui criação ou destruição oculta de valor para além da taxa.

# APÊNDICE B

# Conteúdo de bloco

Este apêndice abre o bloco de altura 1 582 196. Além da transacção de mineiro, contém cinco transacções ordinárias. O mineiro declarou o instante 2018-05-27 21:56:01 UTC; como vimos no capítulo 8, um carimbo temporal de bloco é uma entrada limitada pelo consenso, não o testemunho infalível de um relógio.

```
print_block 1582196
timestamp: 1527458161
previous hash: 30bb9b475a08f2ea6fe07a1fd591ea209a7f633a400b2673b8835a975348b0eb
nonce: 2147489363
is orphan: 0
height: 1582196
depth: 2
hash: 50c8e5e51453c2ab85ef99d817e166540b40ef5fd2ed15ebc863091ca2a04594
difficulty: 51333809600
reward: 4634817937431
{
  "major_version": 7,
  "minor_version": 7,
  "timestamp": 1527458161,
  "prev_id": "30bb9b475a08f2ea6fe07a1fd591ea209a7f633a400b2673b8835a975348b0eb",
  "nonce": 2147489363,
  "miner_tx": {
```

```
18      "version": 2,
19      "unlock_time": 1582256,
20      "vin": [ {
21          "gen": {
22            "height": 1582196
23          }
24        }
25      ],
26      "vout": [ {
27          "amount": 4634817937431,
28          "target": {
29            "key": "39abd5f1c13dc6644d1c43db68691996bb3cd4a8619a37a227667cf2bf055401"
30          }
31        }
32      ],
33      "extra": [ 1, 89, 148, 148, 232, 110, 49, 77, 175, 158, 102, 45, 72, 201, 193,
34      18, 142, 202, 224, 47, 73, 31, 207, 236, 251, 94, 179, 190, 71, 72, 251, 110,
35      134, 2, 8, 1, 242, 62, 180, 82, 253, 252, 0
36      ],
37      "rct_signatures": {
38        "type": 0
39      }
40    },
41    "tx_hashes": [ "e9620db41b6b4e9ee675f7bfdeb9b9774b92aca0c53219247b8f8c7aecf773ae",
42                   "6d31593cd5680b849390f09d7ae70527653abb67d8e7fdca9e0154e5712591bf",
43                   "329e9c0caf6c32b0b7bf60d1c03655156bf33c0b09b6a39889c2ff9a24e94a54",
44                   "447c77a67adeb5dbf402183bc79201d6d6c2f65841ce95cf03621da5a6bffefc",
45                   "90a698b0db89bbb0704a4ffa4179dc149f8f8d01269a85f46ccd7f0007167ee4"
46    ]
47  }
```

## Componentes do bloco

- As linhas 2–10 são metadados calculados pelo nó para o comando `print_block`; não fazem parte da serialização do bloco. O valor `is orphan`=0 diz precisamente que o bloco *não* é órfão. A profundidade '2' é a distância ao topo que o nó conhecia nesse instante.
- `hash` é o identificador calculado do bloco. `difficulty` e `reward` são também resultados calculáveis: a dificuldade decorre do histórico e a recompensa, do peso, da emissão e das taxas. Não se devem confundir os dados impressos pelo cliente com os bytes assinados pelo mineiro.

- `major_version`=7 selecciona as regras de consenso desta altura. `minor_version` pertencia ao antigo mecanismo de sinalização dos mineiros; no exemplo repete a versão principal.
- `timestamp`=1 527 458 161 é o tempo Unix declarado pelo mineiro. `prev_id` compromete-se com o bloco anterior; é este elo, e não o aspecto tabular do JSON, que encadeia a lista.
- `nonce` é o valor variado pelo mineiro. Em conjunto com o resto do cabeçalho e a função de prova de trabalho da época, produz um *hash* que satisfaz a dificuldade exigida.
- `miner_tx` (linhas 17–40) é a transacção de mineiro, incluída integralmente no bloco. Tem versão 2 e uma única entrada `txin_gen`, representada por `gen`; esta não gasta uma saída anterior e declara a altura 1 582 196.
- `unlock_time`=1 582 256 é exactamente a altura do bloco mais 60. A saída de mineiro só pode ser gasta numa transacção incluída nessa altura ou depois dela.
- `vout` contém uma saída de 4 634 817 937 431 unidades atómicas. O montante reúne a subvenção e as taxas do bloco; `key` atribui-o a um endereço oculto. Ao contrário de uma saída RingCT ordinária, a saída de mineiro publica o montante.
- `extra` contém a chave pública da transacção e um *nonce* adicional. `RCTTypeNull`, valor ‘0’, confirma que a transacção de mineiro não transporta as provas RingCT de uma transacção ordinária.
- `tx_hashes` contém os cinco identificadores das transacções ordinárias incluídas. O identificador da transacção de mineiro não aparece neste vector. Para este bloco é, com uma quebra meramente tipográfica,

`06fb3e1cf889bb972774a8535208d98d`
`b164394ef2b14ecfe74814170557e6e9`.

# APÊNDICE C

# Bloco de génese

O bloco de génese é a excepção que permite todas as regras posteriores: não tem antecessor, e a sua identidade é fixada pela implementação. Não contém transacções ordinárias; contém apenas a transacção de mineiro que atribui a primeira subvenção a thankful_for_today [127]. O campo temporal vale zero. O código inicial do Monero derivou de Bytecoin, cuja história se encontra descrita em [82, 132].

O bloco 1 foi adicionado à rede com o carimbo temporal 2014-04-18 10:49:53 UTC. É plausível que o bloco 0 tenha sido minerado no mesmo dia, em concordância com a data de lançamento anunciada por thankful_for_today [127].

```
print_block 0
timestamp: 0
previous hash: 0000000000000000000000000000000000000000000000000000000000000000
nonce: 10000
is orphan: 0
height: 0
depth: 1580975
hash: 418015bb9ae982a1975da7d79277c2705727a56894ba0fb246adaabb1f4632e3
difficulty: 1
reward: 17592186044415
{
  "major_version": 1,
```

```
  "minor_version": 0,
  "timestamp": 0,
  "prev_id": "0000000000000000000000000000000000000000000000000000000000000000",
  "nonce": 10000,
  "miner_tx": {
    "version": 1,
    "unlock_time": 60,
    "vin": [ {
        "gen": {
          "height": 0
        }
      }
    ],
    "vout": [ {
        "amount": 17592186044415,
        "target": {
          "key": "9b2e4c0281c0b02e7c53291a94d1d0cbff8883f8024f5142ee494ffbbd088071"
        }
      }
    ],
    "extra": [ 1, 119, 103, 170, 252, 222, 155, 224, 13, 207, 208, 152, 113, 94, 188,
    247, 244, 16, 218, 235, 197, 130, 253, 166, 157, 36, 162, 142, 157, 11, 200, 144,
    209
    ],
    "signatures": [ ]
  },
  "tx_hashes": [ ]
}
```

## Componentes do bloco de génese

O mesmo comando imprimiu este bloco e o do apêndice B; por isso, a forma exterior é semelhante. As diferenças, porém, dizem exactamente onde começa a cadeia.

- A dificuldade calculada é '1', o mínimo. O *nonce* '10000' é o valor fixado para este bloco; não resulta de uma competição de mineração observável na cadeia.
- `timestamp`=0 não representa uma data civil significativa. O primeiro carimbo temporal posterior fornece apenas um limite plausível, não uma data criptograficamente provada para a criação do bloco 0.

- `prev_id` contém 32 bytes nulos. Não existe bloco anterior; a sequência de zeros é o sentinela convencional dessa ausência.
- O montante 17,592186044415 XMR coincide com a primeira subvenção derivada na secção 8.3.1. A chave de saída atribui esses primeiros moneroj a thankful_for_today.
- `extra` contém somente a etiqueta ‘1’ e a chave pública da transacção. Não há *nonce* adicional.
- Os vectores `signatures` e `tx_hashes` estão vazios: a transacção de mineiro v1 não tem assinatura de gasto e o bloco não inclui transacções ordinárias. O JSON mostra explicitamente o vazio; a cadeia, com uma sobriedade muito sua, nada acrescenta.

# APÊNDICE D

# Notação vectorial das Bulletproofs

As Bulletproofs alternam escalares e pontos com tal frequência que uma letra mal lida pode trocar um corpo por um grupo. Adoptamos a convenção usada no capítulo 6: minúsculas são escalares em $\mathbb{Z}_l$; maiúsculas são pontos do subgrupo de ordem prima. Assim, $a + b = c$ é uma igualdade de escalares, ao passo que $A + B = C$ é uma igualdade de pontos.

Para vectores de escalares $\underline{a} = (a_1, \ldots, a_n)$ e $\underline{b} = (b_1, \ldots, b_n)$, definimos

$$\begin{aligned}
\lambda\underline{a} &= (\lambda a_1, \ldots, \lambda a_n),\\
\underline{a} \circ \underline{b} &= (a_1 b_1, \ldots, a_n b_n),\\
\langle \underline{a}, \underline{b} \rangle &= \sum_{i=1}^{n} a_i b_i,\\
\underline{y}^n &= (1, y, y^2, \ldots, y^{n-1}),\\
\underline{1} &= (1, \ldots, 1).
\end{aligned}$$

O símbolo $\circ$ é o produto de Hadamard: multiplica coordenadas com o mesmo índice. Da definição seguem, por exemplo,

$$\begin{aligned}
\langle \underline{a}, \underline{b} \rangle &= \langle \underline{1}, \underline{a} \circ \underline{b} \rangle,\\
\langle \underline{a}, \underline{b} \rangle + \langle \underline{a}, \underline{c} \rangle &= \langle \underline{a}, \underline{b} + \underline{c} \rangle.
\end{aligned}$$

Para um vector de pontos $\underline{B} = (B_1, \ldots, B_n)$, a notação angular significa uma soma multiescalar:

$$\langle \underline{a}, \underline{B} \rangle = \sum_{i=1}^{n} a_i B_i.$$

O resultado é um ponto, não um escalar. As implementações podem calcular em conjunto várias somas destas por algoritmos de multiplicação multiescalar; a optimização não altera a igualdade matemática verificada.

# De onde vem $\delta(y, z)$

Tomemos os vectores de bits $\underline{a}_L$ e $\underline{a}_R = \underline{a}_L - \underline{1}$. Para cada coordenada, $a_{L,i} \in \{0, 1\}$, e portanto

$$\langle \underline{a}_L, \underline{2}^n \rangle = v, \tag{D.1}$$

$$\underline{a}_L - \underline{1} - \underline{a}_R = 0, \tag{D.2}$$

$$\underline{a}_L \circ \underline{a}_R = 0. \tag{D.3}$$

Na parte constante dos polinómios da Bulletproof aparecem

$$\underline{l}_0 = \underline{a}_L - z\underline{1},$$
$$\underline{r}_0 = \underline{y}^n \circ (\underline{a}_R + z\underline{1}) + z^2\underline{2}^n.$$

Expandimos o seu produto interno, sem omitir as duas igualdades que fazem o trabalho:

$$\begin{aligned}\langle \underline{l}_0, \underline{r}_0 \rangle &= \underbrace{\langle \underline{a}_L, \underline{y}^n \circ \underline{a}_R \rangle}_{=0} + z\langle \underline{a}_L, \underline{y}^n \rangle + z^2 \underbrace{\langle \underline{a}_L, \underline{2}^n \rangle}_{=v} \\ &\quad - z\langle \underline{1}, \underline{y}^n \circ \underline{a}_R \rangle - z^2\langle \underline{1}, \underline{y}^n \rangle - z^3\langle \underline{1}, \underline{2}^n \rangle \\ &= z^2 v + (z - z^2)\langle \underline{1}, \underline{y}^n \rangle - z^3\langle \underline{1}, \underline{2}^n \rangle.\end{aligned}$$

Na última passagem usámos $\underline{a}_L - \underline{a}_R = \underline{1}$. Define-se, por isso,

$$\delta(y, z) = (z - z^2)\langle \underline{1}, \underline{y}^n \rangle - z^3\langle \underline{1}, \underline{2}^n \rangle. \tag{D.4}$$

Obtemos a identidade limpa

$$\langle \underline{l}_0, \underline{r}_0 \rangle = z^2 v + \delta(y, z).$$

O termo $\delta(y, z)$ não é um remendo misterioso: é exactamente o resto conhecido produzido pelas deslocações em $z$. As componentes aleatórias dos polinómios acrescentam os coeficientes de grau um e dois; esta igualdade fixa o termo constante ao montante comprometido.


ZERO
TO
MONERO